

Bilgisayar Ağları ve Veri İletişimi Dersi 4. Bölüm

Bu ders notu, temel olarak **James F. Kurose** ve **Keith W. Ross** tarafından yazılan **Computer Networking: A Top-Down Approach** adlı kaynak eser esas alınarak hazırlanmıştır. İçerik, ilgili konu başlıklarının daha anlaşılır, düzenli ve ders akışına uygun biçimde sunulabilmesi amacıyla öğretim amaçlı olarak yeniden düzenlenmiştir.

Metnin oluşturulması ve düzenlenmesi sürecinde, içeriklerin yapılandırılması, sadeleştirilmesi ve açıklayıcı ders notu formatına dönüştürülmesi için **yapay zekâ destekli araçlardan** yararlanılmıştır.

Bununla birlikte, nihai içerik dersin kapsamı ve öğretim hedefleri doğrultusunda gözden geçirilerek düzenlenmiştir.

Ağ katmanı: “Veri Düzlemi” Yol Haritası

■ Ağ Katmanı: Genel Bakış

- veri düzlemi
- kontrol düzlemi

■ Bir yönlendiricinin içinde neler var?

- Giriş portları, anahtarlama, çıkış portları
- Tampon yönetimi, zamanlama

■ IP: the Internet Protocol

- Datagram Biçimi
- Adresleme
- Ağ Adresi Çevirisi
- IPv6

■ Genelleştirilmiş Yönlendirme, SDN

- Eşleşme+Eylem al
- OpenFlow: Eşleşme +Eylem içinde eylem

■ Middleboxes



Ders Notu: Ağ katmanı: “Veri Düzlemi” Yol Haritası

Bu slayt, ağ katmanı bölümünün veri düzlemi odağında nasıl ilerleyeceğini gösteren genel bir yol haritasıdır. Buradaki amaç, tek tek kavramlara geçmeden önce büyük resmi kurmaktır. Ağ katmanında iki temel bakış vardır: **veri düzlemi** ve **kontrol düzlemi**. Veri düzlemi, bir paketin yönlendiriciye geldiği anda hangi çıkıştan gönderileceğine ilişkin hızlı ve yerel kararlarla ilgilenir. Kontrol düzlemi ise yönlendirme mantığını, yani yolların nasıl belirlendiğini ve yönlendirme tablolarının nasıl oluştuğunu kapsar. Bu ayrım, ağ katmanındaki birçok konunun neden iki farklı seviyede ele alındığını anlamak için temeldir.

Slaytta ikinci büyük başlık olarak **bir yönlendiricinin içinde neler olduğu** sorusu yer almaktadır. Bu bölümde giriş portları, anahtarlama yapısı ve çıkış portları incelenecektir. Yani paket bir yönlendiriciye girdiğinde fiziksel olarak nasıl alınır, içeride nasıl taşınır ve uygun çıkış portuna nasıl ulaştırılır soruları açıklanacaktır. Buna ek olarak **tampon yönetimi** ve **zamanlama** konuları da ele alınacaktır. Bu başlıklar, yönlendiricinin yalnızca “paketi ileten” bir cihaz olmadığını; yoğunluk, kuyruklama, gecikme ve paket kaybı gibi performans konularını da yöneten bir sistem olduğunu gösterir.

Üçüncü ana başlık **IP: the Internet Protocol** bölümüdür. Burada önce **datagram biçimi** incelenecek, yani IP paketinin başlığında hangi alanların bulunduğu ve bunların ne işe yaradığı açıklanacaktır. Ardından **adresleme** konusu gelecektir. Bu bölümde IP adreslerinin yapısı, subnet mantığı, CIDR gösterimi ve hiyerarşik adresleme gibi kavramlar ağ katmanının temel çerçevesini oluşturur. Sonrasında **ağ adresi çevirisi (NAT)** konusu ele alınır; bu da gerçek ağlarda özel adres alanları ile genel internet arasındaki ilişkiyi açıklayan önemli bir mekanizmadır. Devamında **IPv6** ile daha geniş adres alanına sahip yeni nesil adresleme yapısına geçilir.

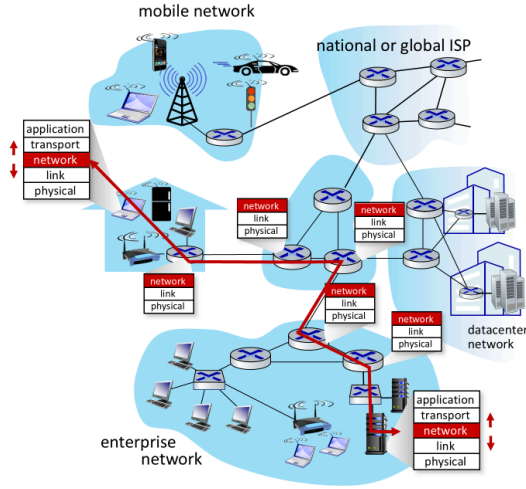
Slaytın sağ tarafında daha ileri düzey iki başlık daha görülmektedir: **Genelleştirilmiş Yönlendirme, SDN ve Middleboxes**. Genelleştirilmiş yönlendirme, klasik hedef tabanlı iletmenin ötesine geçerek başlıktaki çeşitli alanlara göre **eşleşme ve eylem** mantığını kullanır. Bu yaklaşım özellikle **SDN (Software-Defined Networking)** ile önem kazanır. OpenFlow gibi yapılar, bu mantığın ağ cihazlarında nasıl uygulandığını gösterir. Middlebox’lar ise yalnızca yönlendirme yapan klasik cihazlardan farklı olarak ağ trafiği üzerinde ek işlevler gerçekleştiren sistemleri ifade eder. Böylece ağ katmanı, yalnızca IP yönlendirme ile sınırlı olmayan daha geniş bir çerçeveye oturur.

Bu slaytın temel işlevi, konular arasındaki bağı görünür kılmaktır. Önce ağ katmanının genel yapısı, sonra yönlendirici iç yapısı, ardından IP’nin temel unsurları ve sonrasında daha gelişmiş veri düzlemi kavramları ele alınacaktır. Bu sıralama, konuların rastgele değil, birbirini tamamlayacak şekilde düzenlendiğini gösterir.

Özetle bu slaytın ana mesajı şudur: veri düzlemi bölümünde önce ağ katmanının temel ayrımları kurulacak, sonra yönlendirici iç yapısı incelenecek, ardından IP protokolünün temel kavramları ele alınacak ve daha sonra SDN ile genelleştirilmiş yönlendirme gibi ileri başlıklara geçilecektir. Bu yapı, ağ katmanını hem klasik internet mimarisi hem de modern ağ yaklaşımları açısından bütünlüklü biçimde anlamayı sağlar.

Ağ katmanı hizmetleri ve protokolleri

- Segment 'in gönderici hosttan alıcı hosta taşınması
 - **gönderici** : segmentleri datagramlara dönüştürür ve bağlantı katmanına iletir
 - **alıcı** : segmentleri aktarım katmanı protokolüne iletir
- Her İnternet cihazındaki ağ katmanı protokolleri: ana bilgisayarlar, yönlendiriciler
- **Yönlendiriciler** :
 - Kendisinden geçen tüm IP datagramlarındaki başlık alanlarını inceler
 - Datagramları uçtan uca yol boyunca aktarmak için giriş portlarından çıkış portlarına taşır



Network Layer: 4-4

Ders Notu: Ağ katmanı hizmetleri ve protokolleri

Bu slayt, ağ katmanının internet içindeki temel rolünü büyük resim üzerinden açıklamaktadır. Ana fikir şudur: Taşıma katmanından gelen bir **segment**, gönderici uç sistemden alıcı uç sisteme doğrudan tek adımda gitmez; bunun yerine ağ boyunca **IP datagramları** halinde taşınır. Ağ katmanı, bu taşıma işinin uçtan uca gerçekleşmesini sağlar. Buradaki “uçtan uca” ifadesi, tek bir fiziksel bağlantıyı değil, çok sayıda ağ ve yönlendirici üzerinden gerçekleşen bütün yolculuğu anlatır.

Slayttaki ilk madde bu süreci iki tarafta ayırarak verir. **Gönderici** tarafta ağ katmanı, taşıma katmanından aldığı segmentleri **datagramlara kapsüller** ve bunları bağlantı katmanına teslim eder. Yani taşıma katmanının ürettiği veri, ağ katmanında IP başlığı eklenerek yönlendirilebilir hale gelir. **Alıcı** tarafta ise bunun tersi yapılır: ağ katmanı datagramın içindeki segmenti çıkarır ve bunu tekrar taşıma katmanına verir. Böylece gönderici ve alıcı arasındaki ağ katmanı işlevi, segmentin paketlenmesi ve doğru yere ulaştığında tekrar üst katmana teslim edilmesi şeklinde düşünülebilir.

Slayttaki şekil de bu süreci somutlaştırmaktadır. Soldan başlayan kırmızı yol, verinin farklı ağ ortamlarından geçerek hedefe ulaştığını göstermektedir. Bu yol üzerinde mobil ağlar, kurumsal ağlar, servis sağlayıcı omurgaları ve veri merkezi ağları gibi farklı ağ türleri vardır. Bu, internetin tek parça bir ağ olmadığını; çok sayıda farklı alt ağın birbirine bağlanmasıyla oluştuğunu gösterir. Ağ katmanının önemi de tam burada ortaya çıkar: farklı fiziksel ve bağlantı katmanı teknolojileri olsa bile, IP datagramları bu farklı yapılar arasında ortak taşıma birimi gibi davranır.

Slayttaki ikinci önemli vurgu, ağ katmanı protokollerinin yalnızca uç sistemlerde değil, **internet üzerindeki her cihazda**, özellikle de **yönlendiricilerde** bulunmasıdır. Uç sistemlerde uygulama, taşıma, ağ, bağlantı ve fiziksel katmanlar birlikte bulunurken; yönlendiriciler esas olarak ağ katmanı ve altındaki katmanlarla çalışır. Şekilde de uç sistemlerde tüm katmanlar gösterilirken, yönlendiriciler üzerinde özellikle **network, link, physical** katmanlarının öne çıkarıldığı görülmektedir. Bu, yönlendiricilerin uygulama verisini yorumlamadığını; temel görevlerinin datagramı doğru yöne iletmek olduğunu anlatır.

Slayttaki **yönlendiriciler** başlığı bu nedenle çok önemlidir. Yönlendirici, kendisinden geçen IP datagramlarının başlık alanlarını inceler. Yani paketin içeriğindeki uygulama verisine değil, başlıktaki hedef adres gibi ağ katmanı bilgilerine bakar. Sonra bu datagramı uygun giriş portundan alıp doğru çıkış portuna aktarır. Bu işlem, veri düzlemi mantığının özüdür. Yönlendirici, paketi üretmez ya da tüketmez; onu ağ içinde bir sonraki adıma taşır.

Burada dikkat edilmesi gereken nokta, ağ katmanının taşıma katmanı gibi uçtan uca mantıkla ilgilenmesine rağmen, işlemin kendisinin hop-by-hop ilerlemesidir. Yani datagram alıcıya tek sıçramada ulaşmaz; bir yönlendiriciden diğerine aktarılır. Her yönlendirici kendi yerel kararını verir, ama toplamda bu kararlar birleşerek paketin uçtan uca taşınmasını sağlar. Bu nedenle ağ katmanı hem yerel iletmeye kararlarını hem de küresel ulaşımı birlikte düşündüren bir katmandır.

Şekildeki katman blokları da bunu destekler. Uç sistemde uygulama verisi aşağı katmanlara iner, ağ boyunca yönlendiricilerde ağ katmanı ve alt katmanlar arasında taşınır, hedefte ise tekrar yukarı katmanlara çıkar. Bu, katmanlı mimarinin internet içindeki hareketli bir görünümüdür. Ağ katmanı, bu akışın tam ortasında, farklı ağları birbirine bağlayan ortak işlevi yerine getirir.

Özetle bu slaytın ana mesajı şudur: ağ katmanı, taşıma katmanından gelen segmentleri IP datagramlarına dönüştürerek onları göndericiden alıcıya internet boyunca taşır. Uç sistemlerde segmentleri kapsüller ve açar; yönlendiricilerde ise datagram başlıklarını inceleyip paketleri giriş portundan uygun çıkış portuna aktarır. Böylece çok sayıda farklı ağdan oluşan internet, ortak bir ağ katmanı hizmeti üzerinden uçtan uca çalışabilir.

Ağ katmanındaki iki temel işlev

ağ katmanı işlevleri :

- **forwarding**: paketleri yönlendiricinin giriş bağlantısından uygun çıkış bağlantısına aktarmak
- **routing**: paketlerin kaynaktan hedefe kadar izlediği yolu belirlemek
 - *routing algorithms*

benzetme: seyahate çıkmak

- **forwarding**: tek bir kavşaktan geçme süreci
- **routing**: kalkış noktasından varış noktasına kadar seyahat planlama süreci



Network Layer: 4-5

Ders Notu: Ağ katmanındaki iki temel işlev

Bu slayt, ağ katmanının iki temel işlevini birbirinden ayırarak açıklamaktadır:

forwarding ve **routing**. Bu iki kavram çoğu zaman birlikte anılsa da, aynı şeyi ifade etmez. Ağ katmanını doğru anlamak için önce bu ayrımı netleştirmek gerekir. Çünkü biri **yerel ve anlık karar verme** ile ilgilidir, diğeri ise **uçtan uca yolun belirlenmesi** ile ilgilidir.

Forwarding, bir paketin yönlendiriciye girdikten sonra uygun çıkış bağlantısına aktarılması işlemidir. Başka bir deyişle, paket bir yönlendiricinin giriş portuna geldiğinde, bu yönlendirici “bu paketi hangi çıkış portundan göndermeliyim?” sorusuna cevap verir. Bu işlem çok hızlı yapılmalıdır; çünkü yönlendirici üzerinden geçen her paket için tekrar tekrar uygulanır. Bu nedenle forwarding, veri düzleminin temel işlevlerinden biridir. Burada karar, çoğu zaman paketin başlığındaki hedef adres bilgisine bakılarak ve yönlendirme tablosu kullanılarak verilir.

Routing ise daha geniş ölçekli bir işlevdir. Burada tek bir yönlendiricinin anlık kararı değil, paketin **kaynaktan hedefe kadar hangi yolu izleyeceği** belirlenir. Yani routing, internet üzerindeki olası yollar arasından uygun olanın seçilmesi sürecidir. Bu seçim, yönlendirme algoritmaları ve yönlendirme protokolleri ile yapılır. Elde edilen sonuç, yönlendiricilerin forwarding tablolarına yansır. Bu yüzden routing ile forwarding arasında doğrudan bir ilişki vardır: routing yolu belirler, forwarding ise bu yolun her adımda uygulanmasını sağlar.

Slayttaki benzetme bu ayrımı çok anlaşılır hale getirir. **Forwarding**, bir yolculuk sırasında tek bir kavşaktan geçerken hangi çıkıştan sapılacağına karar vermeye benzer. Bu, yerel ve anlık bir karardır. **Routing** ise yolculuğun tamamını planlamaya benzer; yani kalkış noktasından varış noktasına kadar hangi şehirlerden, hangi otoyollardan ve hangi bağlantılardan gidileceğini belirlemektir. Bu nedenle forwarding ile routing arasındaki fark, “tek adım” ile “bütün yol” arasındaki fark gibi düşünülebilir.

Burada önemli olan nokta, bu iki işlevin farklı düzeylerde çalışmasına rağmen birbirini tamamlamasıdır. Eğer yalnızca routing olsaydı ama her yönlendirici yerel aktarım kararını hızlı veremeseydi, paketler taşınmazdı. Eğer yalnızca forwarding olsaydı ama genel yol bilgisi oluşturulmasaydı, yönlendiriciler paketi hangi yöne doğru ilerletmeleri gerektiğini bilemezdi. İnternetin çalışması, bu iki işlevin birlikte işlemesine bağlıdır.

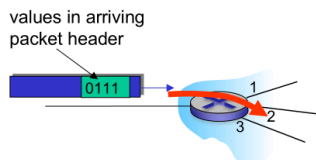
Bu slayt aynı zamanda veri düzlemi ve kontrol düzlemi ayrımına da hazırlık yapmaktadır. Forwarding daha çok **veri düzlemi** içinde düşünülür; çünkü gerçek paketler üzerinde, çok kısa zaman ölçeklerinde uygulanır. Routing ise daha çok **kontrol düzlemi** ile ilişkilidir; çünkü ağın genel topolojisine bakar, yol hesaplar ve yönlendirme tablolarını oluşturur. Bu nedenle ileride ağ katmanını anlatılırken forwarding ve routing’ın neden farklı başlıklar altında ele alındığı daha iyi anlaşılır.

Özetle bu slaytın ana mesajı şudur: **forwarding**, paketi bir yönlendiricinin giriş bağlantısından uygun çıkış bağlantısına aktarma işlemidir; **routing** ise paketin kaynak ile hedef arasında izleyeceği genel yolu belirler. Kısa bir ifadeyle, routing yolun planıdır, forwarding ise bu planın her yönlendiricide uygulanan yerel adımdır.

Ağ katmanı: veri düzlemi, kontrol düzlemi

Veri düzlemi:

- *local*, yönlendirici başına işlev
- Yönlendiricinin giriş portundan gelen datagramın, yönlendiricinin çıkış portuna nasıl iletileceğini belirler



Kontrol düzlemi

- *ağ genelinde geçerli mantık*
- kaynak ana bilgisayardan hedef ana bilgisayara uzanan uçtan uca yol boyunca yönlendiriciler arasında datagramın nasıl yönlendirileceğini belirler
- iki kontrol düzlemi yaklaşımı :
 - *geleneksel yönlendirme algoritmaları*: yönlendiricilerde uygulanır
 - *software-defined networking (SDN)*: implemented in (remote) servers

Ders Notu: Ağ katmanı: veri düzlemi, kontrol düzlemi

Bu slayt, ağ katmanındaki iki temel çalışma alanını birbirinden ayırmaktadır: **veri düzlemi** ve **kontrol düzlemi**. Bu ayrım çok önemlidir; çünkü ağ katmanında yapılan tüm işler aynı seviyede gerçekleşmez. Bazı kararlar her yönlendiricide, her paket için yerel olarak ve çok hızlı biçimde verilir. Bazı kararlar ise ağın genel yapısına bakılarak, uçtan uca yol mantığı çerçevesinde oluşturulur. İşte bu iki farklı işleyiş, veri düzlemi ve kontrol düzlemi kavramlarıyla ifade edilir.

Veri düzlemi, slaytta da belirtildiği gibi **local**, yani yönlendirici başına çalışan bir işlevdir. Buradaki temel soru şudur: Bir datagram yönlendiricinin giriş portuna geldiğinde, hangi çıkış portundan gönderilecektir? Veri düzlemi tam olarak bu kararı uygular. Yani paketin başlığındaki değerlere bakılır, uygun kural bulunur ve paket doğru çıkış yönüne iletilir. Bu işlem tek tek her yönlendiricide, gelen her paket için yapılır. Bu yüzden veri düzlemi, gerçek trafik üzerinde çalışan ve çok kısa zaman ölçeklerinde karar vermesi gereken katmandır.

Slayttaki küçük şekil de bunu görselleştirmektedir. Gelen paketin başlığındaki değerlere bakılarak, yönlendiricideki uygun çıkış seçilmektedir. Bu, veri düzleminin en temel doğasını gösterir: **başlığa bak, eşleştir, uygun çıkışa gönder**. Burada ağın tamamı değil, yalnızca o anda paketin bulunduğu yönlendirici önemlidir. Bu nedenle veri düzlemi, daha çok **forwarding** kavramıyla ilişkilidir.

Buna karşılık **kontrol düzlemi**, slaytta “**ağ genelinde geçerli mantık**” olarak tanımlanmıştır. Bu çok yerinde bir ifadedir. Kontrol düzlemi, bir paketin kaynak hosttan hedef hosta kadar izleyeceği yol boyunca yönlendiriciler arasında nasıl ilerlemesi gerektiğini belirler. Yani burada artık tek bir yönlendiricinin yerel çıkış kararı değil, ağın tamamını kapsayan yol seçimi mantığı vardır. Bu nedenle kontrol düzlemi, daha çok **routing** işleviyle bağlantılıdır. Veri düzlemi tek adımı uygular; kontrol düzlemi ise yolun genel mantığını kurar.

Slaytta kontrol düzlemi için iki yaklaşım verilmektedir. Birincisi **geleneksel yönlendirme algoritmalarıdır**. Bu yaklaşımda yönlendirme mantığı doğrudan yönlendiricilerin içinde çalışır. Router’lar birbirleriyle bilgi alışverişi yapar, topolojiyi öğrenir ve buna göre yönlendirme tablolarını oluşturur. Bu, klasik internet yönlendirme yaklaşımıdır.

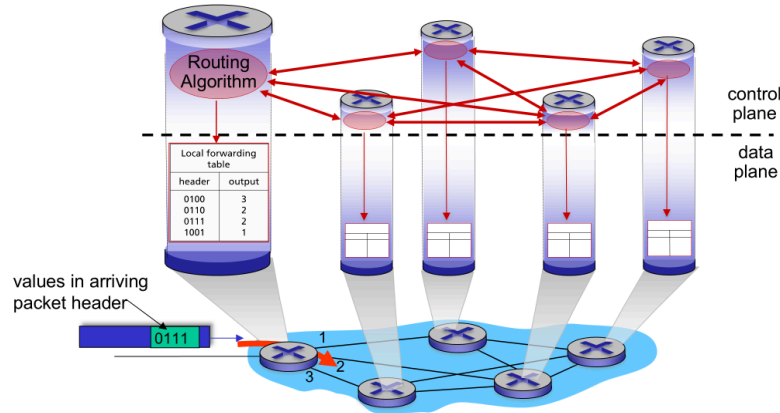
İkinci yaklaşım ise **software-defined networking (SDN)**’dir. Burada kontrol mantığı yönlendiricilerin içinde dağıtık biçimde çalışmak zorunda değildir; bunun yerine merkezi ya da uzak sunucular üzerinde uygulanabilir. Yani ağın kontrolü veri taşıyan cihazlardan ayrıştırılır. Bu nedenle SDN yaklaşımı, klasik yönlendirme modeline göre daha programlanabilir ve merkezi bir denetim anlayışı sunar. Slaytta “implemented in (remote) servers” ifadesiyle bu ayrım özellikle vurgulanmaktadır.

Bu slaytın asıl değeri, forwarding ile routing ayrımını daha sistematik hale getirmesidir. Bir önceki slaytta forwarding ve routing iki temel ağ katmanını işlevi olarak anlatılmıştı. Burada ise forwarding'in daha çok **veri düzlemi**, routing'in ise daha çok **kontrol düzlemi** içinde yer aldığı açık hale gelir. Böylece kavramsal çerçeve tamamlanır: veri düzlemi paketi taşır, kontrol düzlemi yol bilgisini oluşturur.

Özetle bu slaytın ana mesajı şudur: **veri düzlemi**, her yönlendiricide yerel olarak çalışan ve gelen datagramın hangi çıkış portuna iletileceğini belirleyen hızlı işleyiştir. **Kontrol düzlemi** ise ağ genelinde çalışan, kaynak ile hedef arasındaki yol mantığını oluşturan yapıdır. Geleneksel ağlarda bu mantık yönlendiricilerin içinde dağıtık biçimde çalışırken, SDN yaklaşımında uzak ya da merkezi sunucular üzerinde uygulanabilir.

Yönlendirici başına kontrol düzlemi

Her bir yönlendiricideki yönlendirme algoritmasının bileşenleri, kontrol düzleminde birbirleriyle etkileşime girer



Network Layer: 4-7

Ders Notu: Yönlendirici başına kontrol düzlemi

Bu slayt, **geleneksel kontrol düzlemi yaklaşımını** göstermektedir. Buradaki temel fikir şudur: Her yönlendirici, ağın tamamına ilişkin yönlendirme kararlarını merkezi bir denetleyiciden almaz; bunun yerine, kendi içinde çalışan bir **routing algorithm** bileşenine sahiptir ve bu bileşen diğer yönlendiricilerdeki benzer bileşenlerle bilgi alışverişi yapar. Yani kontrol düzlemi dağıtık yapıdadır.

Şekilde üst tarafta **control plane**, alt tarafta ise **data plane** gösterilmektedir. Bu ayrım çok önemlidir. Kontrol düzleminde yönlendirme mantığı çalışır; veri düzleminde ise gerçek paketler, oluşturulmuş kurallara göre iletilir. Başka bir deyişle, kontrol düzlemi

“hangi yolun uygun olduğunu” belirler, veri düzlemi ise “gelen paketi hangi çıkıştan göndereceğini” uygular.

Her yönlendiricinin üst kısmında yer alan **Routing Algorithm** bloğu, o cihazın kontrol düzlemi işlevini temsil eder. Kırmızı oklar, bu yönlendirme bileşenlerinin birbirleriyle etkileşim içinde olduğunu göstermektedir. Bu etkileşim sayesinde yönlendiriciler ağ topolojisi, maliyetler, erişilebilirlik ya da en iyi yollar hakkında bilgi paylaşır. Böylece her yönlendirici kendi **yerel forwarding tablosunu** oluşturabilir ya da güncelleyebilir.

Şeklin alt kısmındaki **local forwarding table**, kontrol düzleminde hesaplanan bilginin veri düzlemine nasıl aktarıldığını gösterir. Yani yönlendirme algoritmasının çıktısı, en sonunda bir tabloya dönüşür. Bu tabloda gelen paket başlığındaki bazı değerlere karşılık hangi çıkış portunun seçileceği yazılıdır. Altta gösterilen paket örneğinde başlıktaki değere bakılarak yönlendiricide **1, 2 veya 3 numaralı çıkışlardan** hangisinin seçileceği belirlenmektedir. Bu da veri düzleminin forwarding işlevine karşılık gelir.

Bu slayttaki en önemli nokta, yönlendirme kararlarının **dağıtık** biçimde alınmasıdır. Yani tek bir merkezi kontrolcü yoktur. Her router, ağın geri kalanıyla konuşur, kendi bilgisini oluşturur ve kendi tablosunu üretir. Bu yapı klasik internet yönlendirme yaklaşımının temelidir. OSPF, RIP ya da BGP gibi yönlendirme protokolleri bu mantığın farklı örnekleri olarak düşünülebilir.

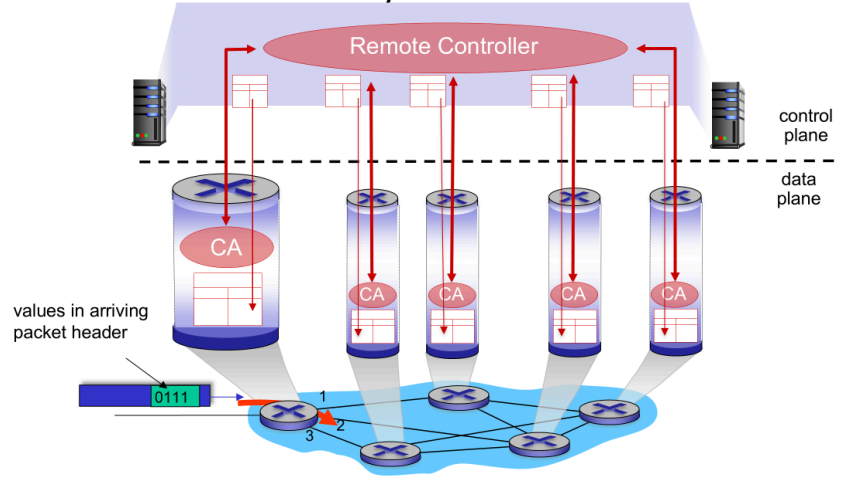
Bu yaklaşımın avantajı, ağın tek bir merkeze bağımlı olmadan çalışabilmesidir. Her cihaz kendi karar sürecine katılır. Ancak bunun karşılığında tüm yönlendiricilerin birbirleriyle haberleşmesi, topolojiyi öğrenmesi ve tutarlı biçimde güncellenmesi gerekir. Yani kontrol düzlemi dağıtık olduğu için koordinasyon yükü de ağın içine dağılmış olur.

Slayt, bir önceki “veri düzlemi-kontrol düzlemi” ayrımını daha somut hale getirmektedir. Veri düzlemi paket başlığına bakıp doğru çıkışı seçerken, kontrol düzlemi bu seçimin dayandığı tabloyu oluşturur. Bu yüzden forwarding ile routing arasındaki ilişki burada açıkça görülür: **routing algorithm** tabloları üretir, **forwarding** ise bu tabloları kullanır.

Özetle bu slaytın ana mesajı şudur: **Yönlendirici başına kontrol düzlemi** yaklaşımında her yönlendirici kendi yönlendirme algoritmasına sahiptir ve diğer yönlendiricilerle etkileşerek kendi forwarding tablosunu oluşturur. Böylece ağ genelindeki yön bilgisi dağıtık biçimde hesaplanır, veri düzleminde ise bu bilgi paketlerin doğru çıkış portuna iletilmesi için kullanılır.

Yazılım Tanımlı Ağ (SDN) kontrol düzlemi

Uzaktaki Controller, yönlendirme tablolarını hesaplar ve yönlendiricilere yükler



Ders Notu: Yazılım Tanımlı Ağ (SDN) kontrol düzlemi

Bu slayt, ağ katmanındaki **kontrol düzleminin merkezi ya da uzaktan yönetilen bir yapıya ayrıştırıldığı** SDN yaklaşımını göstermektedir. Bir önceki slaytta geleneksel yöntemde her yönlendiricinin kendi içinde çalışan bir yönlendirme algoritmasına sahip olduğu görülmüştü. Burada ise farklı bir mimari vardır: Yönlendirme kararlarını her cihaz kendi başına hesaplamaz; bunun yerine **uzaktaki bir controller**, ağın genel durumuna bakarak gerekli kuralları hesaplar ve yönlendiricilere yükler.

Slayttaki üst bölüm **control plane**, alt bölüm ise **data plane** olarak ayrılmıştır. Bu kesikli çizgi çok önemlidir; çünkü SDN'nin temel fikri tam olarak bu ayrımı görünür ve belirgin hale getirmektir. Veri düzlemindeki cihazlar, yani yönlendiriciler ya da anahtarlama cihazları, esas olarak gelen paketleri başlık bilgilerine göre iletme işini yapar. Kontrol düzlemindeki merkezi yapı ise bu cihazların hangi kurallara göre davranacağını belirler.

Şeklin üst kısmındaki **Remote Controller**, ağın genel mantığını elinde tutan merkezi ya da mantıksal olarak merkezi bileşendir. Slaytta da açıkça belirtildiği gibi, bu controller **yönlendirme tablolarını hesaplar ve yönlendiricilere yükler**. Yani klasik yapıda router'ların kendi içinde dağınık biçimde yürüttüğü yön seçimi mantığı, burada üstteki denetleyiciye taşınmıştır. Böylece ağın tamamı daha bütüncül biçimde görülebilir ve kurallar merkezi olarak dağıtılabilir.

Alt taraftaki yönlendirici benzeri cihazların içindeki **CA** blokları, bu cihazlarla controller arasındaki arayüz ya da kontrol aracısını temsil eder. Bu yapı, cihazların tamamen

“akılsız” olduđu anlamına gelmez; ancak asıl yönlendirme mantığının ve politika belirleme işinin üstteki controller’ a kaydırıldığını gösterir. Cihazlar daha çok, kendilerine yüklenen tabloları ve kuralları uygulayan veri düzlemi bileşenleri gibi davranır.

Şekildeki kırmızı oklar da bu merkezi denetim akışını göstermektedir. Controller, cihazlardan bilgi alır ve onlara kurallar gönderir. Böylece yönlendiricilerin yerel forwarding tabloları, merkezi hesaplama sonucunda oluşur. Alttaki paket örneğinde yine gelen başlığa bakılarak çıkış **1, 2 ya da 3** yönlerinden biri seçilmektedir; fakat bu seçimi mümkün kılan tablo artık cihazın kendi dağıtık yönlendirme algoritmasından değil, üstteki controller’ dan gelmektedir.

Bu yaklaşımın en önemli farkı, **kontrol mantığının veri taşıyan cihazlardan ayrılmasıdır**. Geleneksel ağlarda her router hem veri taşır hem de yönlendirme kararlarının hesaplanmasına katılır. SDN’ de ise veri taşıma işi ile karar mantığı ayrıştırılır. Bu da ağ yönetimini daha programlanabilir, daha merkezi ve çoğu zaman daha esnek hale getirir. Özellikle büyük ağlarda politikaların tek noktadan tanımlanması, trafiğin merkezi olarak optimize edilmesi ve yeni kuralların daha kolay uygulanması açısından bu önemli bir avantajdır.

Bu slayt, bir önceki “yönlendirici başına kontrol düzlemi” slaytı ile karşıtlık içinde okunmalıdır. Orada her router kendi yönlendirme bileşeniyle komşularıyla konuşuyordu. Burada ise ağ genelindeki mantık üstteki **Remote Controller** tarafından yürütülmektedir. Dolayısıyla fark, yalnızca teknik değil, mimari bir farktır: **dağıtık kontrol** yerine **mantıksal olarak merkezi kontrol**.

Özetle bu slaytın ana mesajı şudur: **SDN kontrol düzleminde**, yönlendirme ve kural üretme mantığı ağ cihazlarının içinden alınarak **uzaktaki bir controller** yapısına taşınır. Controller, ağ genelini görerek yönlendirme tablolarını hesaplar ve bunları yönlendiricilere yükler. Veri düzlemindeki cihazlar ise bu kuralları uygulayarak paketleri iletir. Böylece kontrol düzlemi ile veri düzlemi ayrımı çok daha belirgin ve yönetilebilir hale gelir.

Ağ hizmet modeli

S: Veri paketlerini göndericiden alıcıya ileten “kanal” için hangi hizmet modeli kullanılır?

Ağlar yalnızca veri taşımakla kalmaz, bunu hangi kalite düzeyinde yapacaklarını da bir hizmet modeliyle tanımlarlar.

tek tek datagramlar için örnek hizmetler :

- teslimat garantisi
- 40 milisaniyeden az gecikmeyle garantili teslimat

datagram akışı için örnek hizmetler :

- sırayla datagram iletimi
- akış için garantili minimum bant genişliği
- paketler arası mesafenin değiştirilmesine ilişkin kısıtlamalar

Network Layer: 4-9

Ders Notu: Ağ hizmet modeli

Bu slayt, ağ katmanında çok temel ama çoğu zaman gözden kaçan bir soruyu ortaya koymaktadır: Bir ağ, veriyi yalnızca bir noktadan başka bir noktaya taşımakla mı ilgilenir, yoksa bu taşımanın **hangi hizmet özellikleriyle** yapılacağını da tanımlar mı? Buradaki ana fikir şudur: Ağlar sadece paket iletmez; aynı zamanda bu iletimin ne kadar güvenilir, ne kadar hızlı, ne kadar düzenli ya da ne kadar öngörülebilir olacağını belirleyen bir **hizmet modeli** sunar.

Slaytın başındaki soru, “veri paketlerini göndericiden alıcıya ileten kanal için hangi hizmet modeli kullanılır?” biçimindedir. Bu soru önemlidir; çünkü farklı ağ mimarileri, üst katmanlara farklı garantiler sunabilir. Yani bazı ağlar teslimat güvencesi verebilir, bazıları gecikme sınırı koyabilir, bazıları ise yalnızca “elimden geleni yaparım” yaklaşımıyla çalışabilir. Dolayısıyla hizmet modeli, ağ katmanının üst katmanlara sunduğu soyut sözleşme gibi düşünülebilir.

Slaytta hizmetler iki düzeyde ele alınmaktadır. Birinci düzey, **tek tek datagramlar için** düşünülen hizmetlerdir. Burada örnek olarak **teslimat garantisi** verilmektedir. Bu tür bir hizmette ağ, gönderilen bir datagramın alıcıya mutlaka ulaştırılacağını taahhüt edebilir. Bunun bir adım ötesinde ise gecikme garantisi örneği verilmiştir: örneğin **40 milisaniyeden az gecikmeyle garantili teslimat**. Bu tür bir ifade, yalnızca paketin kaybolmamasını değil, belirli bir zaman sınırı içinde ulaştırılmasını da içerir. Yani tekil paket hizmetleri, bir paketin başına gelebilecek durumlara odaklanır.

İkinci düzey ise **datagram akışı** için tanımlanan hizmetlerdir. Burada artık tek bir paketten değil, bir paket dizisinden söz edilir. Örneğin **sırayla datagram iletimi**, akış içindeki paketlerin gönderildikleri sırayı koruyarak ulaştırılmasını ifade eder. Bu özellikle ses, video ya da sürekli veri akışı gibi uygulamalarda önemlidir. Bunun yanında **garantili minimum bant genişliği** örneği verilmektedir. Bu, ağın belirli bir akış için en az şu kadar kapasite sağlayacağını söylemesi anlamına gelir. Bir başka örnek de **paketler arası mesafenin değişimine ilişkin kısıtlamalardır**; bu da pratikte jitter, yani paketler arası zaman farklılığındaki oynama ile ilişkilidir. Özellikle gerçek zamanlı uygulamalarda bu tür özellikler kritik hale gelir.

Burada dikkat edilmesi gereken önemli nokta, hizmet modelinin yalnızca “paket gider mi gitmez mi?” sorusundan ibaret olmamasıdır. Hizmet modeli aynı zamanda **kalite boyutunu** da içerir. Teslimat, gecikme, sıra, bant genişliği ve zamanlama düzeni gibi unsurlar, ağın sunduğu hizmetin niteliğini belirler. Bu yüzden hizmet modeli, ağ katmanının üst katmanlara hangi konuda sorumluluk aldığını anlamak için temel bir çerçevedir.

Bu slayt aynı zamanda ileride internetin neden belirli türde garantiler vermediğini anlamak için de hazırlık yapmaktadır. Çünkü burada örnek olarak verilen hizmetler, teorik olarak bir ağın sunabileceği şeylerdir. Bir sonraki mantıksal adım, İnternet’in IP hizmet modelinin bunlardan hangilerini gerçekten sunduğunu sorgulamaktır. Bu nedenle bu slayt, internet mimarisini değerlendirirken “hangi garantiler var, hangileri yok?” sorusuna zemin hazırlar.

Özetle bu slaytın ana mesajı şudur: **Ağ hizmet modeli**, ağın üst katmanlara yalnızca veri taşıma değil, bu taşımanın hangi garanti ve kalite özellikleriyle yapılacağını da tanımlayan çerçevedir. Bu hizmetler tek tek datagramlar için teslimat ve gecikme garantisi biçiminde olabileceği gibi, bir akış için sıra koruma, minimum bant genişliği ve jitter kısıtları gibi daha kapsamlı özellikler de içerebilir.

Ağ katmanı hizmet modeli

Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no

Internet “best effort” hizmet modeli

Şu konularda herhangi bir garanti verilmez:

- datagramın hedefe başarılı bir şekilde iletilmesi
- iletim zamanlaması veya sırası
- uçtan uca akış için kullanılabilir bant genişliği

Network Layer: 4-10

Ders Notu: Ağ katmanı hizmet modeli

Bu slayt, bir önceki slaytta tanıtılan “hizmet modeli” kavramını doğrudan İnternet için somutlaştırmaktadır. Temel soru şudur: İnternet, ağ katmanı düzeyinde üst katmanlara ne tür garantiler verir? Verilen cevap oldukça nettir: İnternetin hizmet modeli **best effort** yaklaşımıdır. Yani ağ, paketleri taşımak için elinden geleni yapar; ancak belirli kalite ölçütleri konusunda kesin söz vermez.

Tabloda bu durum dört başlık altında özetlenmektedir: **bandwidth, loss, order** ve **timing**. İnternet için bunların karşısında garanti anlamında bir güvence verilmediği gösterilmektedir. Başka bir deyişle, ağ katmanı “paketi mutlaka şu hızda taşıyım”, “paketi kesinlikle kaybetmem”, “paketler mutlaka sırayla ulaşır” ya da “belirli bir zaman sınırı içinde teslim ederim” demez. Bu, internet mimarisinin çok temel bir özelliğidir.

Slayttaki kırmızı kutu bu noktayı daha açık hale getirir. İnternetin **best effort** hizmet modelinde şu konularda garanti verilmez. Birincisi, **datagramın hedefe başarılı biçimde ulaşması** garanti edilmez. Yani paket kaybolabilir, düşebilir ya da hiç hedefe varmayabilir. İkincisi, **iletim zamanlaması veya sırası** garanti edilmez. Bu, paketlerin farklı gecikmelerle ulaşabileceği ve gönderildikleri sıranın korunmayabileceği anlamına gelir. Üçüncüsü, **uçtan uca akış için kullanılabilir bant genişliği** garanti edilmez. Yani ağ, belirli bir veri akışına sabit ya da minimum bir kapasite ayıracağını söylemez.

Burada önemli olan nokta, “best effort” ifadesinin “ağ kötü çalışır” anlamına gelmemesidir. Anlamı, ağın teslimat için çaba göstermesi ama bunu belirli QoS

garantileriyle taahhüt etmemesidir. İnternet çoğu zaman paketleri başarıyla taşır; ancak bu başarı, protokol düzeyinde sert bir sözleşmeye bağlanmış değildir. Bu nedenle internetin sunduğu hizmet, garanti temelli bir hizmetten çok, **olasılıksal ve fırsatçı** bir hizmet gibi düşünülebilir.

Bu yaklaşımın neden tercih edildiğini anlamak da önemlidir. İnternet çok büyük, heterojen ve sürekli değişen bir ağlar ağıdır. Farklı bağlantı hızları, farklı yönlendiriciler, farklı yoğunluk seviyeleri ve farklı ağ teknolojileri birlikte çalışır. Böyle bir ortamda her akış için sıkı gecikme, sıra veya bant genişliği garantileri vermek çok zor olurdu. Bu nedenle internetin temel mimarisi daha sade bir hizmet modeli benimsemiştir: paketleri mümkün olduğunca iyi taşımak, ama katı kalite garantileri vermemek.

Bu durum aynı zamanda üst katmanların neden önemli olduğunu da açıklar. Örneğin güvenilirlik gerekiyorsa bunu ağ katmanı değil, çoğu zaman taşıma katmanındaki protokoller üstlenir. TCP'nin yeniden iletim, sıralama ve akış kontrolü gibi mekanizmaları tam da bu yüzden gereklidir. Yani internetin ağ katmanı garanti vermediği için, bazı uygulama ihtiyaçları üst katmanlarda telafi edilir.

Özetle bu slaytın ana mesajı şudur: İnternetin ağ katmanı hizmet modeli **best effort** yaklaşımıdır. Bu modelde datagram teslimatı, teslim zamanı, teslim sırası ve kullanılabilir bant genişliği konusunda kesin garantiler verilmez. Ağ elinden geleni yapar, ancak QoS türü sıkı sözler vermez. Bu nedenle internet mimarisi sade, ölçeklenebilir ve esnek kalırken; güvenilirlik ve düzen gibi bazı ihtiyaçlar üst katmanlara bırakılmış olur.

Ağ katmanı hizmet modeli

Network Architecture	Service Model	Quality of Service (QoS) Guarantees ?			
		Bandwidth	Loss	Order	Timing
Internet	best effort	none	no	no	no
ATM	Constant Bit Rate	Constant rate	yes	yes	yes
ATM	Available Bit Rate	Guaranteed min	no	yes	no
Internet	Intserv Guaranteed (RFC 1633)	yes	yes	yes	yes
Internet	Diffserv (RFC 2475)	possible	possibly	possibly	no

Ders Notu: Ağ katmanı hizmet modeli

Bu slayt, farklı ağ mimarilerinin ağ katmanı düzeyinde hangi tür **hizmet garantileri** sunabildiğini karşılaştırmaktadır. Bir önceki slaytta internetin **best effort** yaklaşımı benimsediği ve bant genişliği, kayıp, sıra ve zamanlama açısından kesin garantiler vermediği görülmüştü. Burada ise bu yaklaşım, başka hizmet modelleriyle yan yana konularak daha net hale getirilmektedir. Böylece “ağ hizmet modeli” kavramı soyut bir fikir olmaktan çıkıp, farklı mimarilerin sunduğu somut kalite düzeyleri üzerinden anlaşılır hale gelir.

Tablonun ilk satırında yine **Internet – best effort** modeli yer almaktadır. Burada bant genişliği için belirli bir garanti yoktur, kayıp olmaması garanti edilmez, paket sırasının korunacağı söylenmez ve zamanlama açısından da bir güvence verilmez. Bu, internetin temel hizmet modelinin neden esnek ama aynı zamanda garanti vermeyen bir yapı olduğunu özetler. Ağ elinden geleni yapar; ancak bunu katı QoS sözleriyle desteklemez.

İkinci satırda **ATM – Constant Bit Rate** yer almaktadır. Bu model, çok daha güçlü bir hizmet anlayışını temsil eder. Burada **sabit bir hız** sağlanır ve kayıp, sıra ve zamanlama açısından da garanti verilir. Bu tür bir hizmet modeli, özellikle sürekli ve düzenli veri akışı gerektiren uygulamalar için uygundur. Yani ağ yalnızca veriyi taşımakla kalmaz; bu taşımanın hangi hızda ve ne kadar düzenli gerçekleşeceğini de önceden tanımlar.

Üçüncü satırdaki **ATM – Available Bit Rate** modeli ise biraz daha esnek bir yapıyı temsil eder. Burada en azından **garantili minimum bant genişliği** sağlanır. Paket sırası açısından da bir güvence vardır; ancak kayıp ve zamanlama konusunda tam garanti verilmez. Bu, hizmet modelinin tek tip olmak zorunda olmadığını gösterir. Bazı ağlar her şeyi garanti etmek yerine, sadece belirli kalite boyutlarında kısmi güvence sunabilir.

Dördüncü satırda **Internet – Intserv Guaranteed** modeli görülmektedir. Bu yaklaşım, internet üzerinde de daha güçlü hizmet garantileri sağlama fikrini temsil eder. Tabloda bu model için bant genişliği, kayıp, sıra ve zamanlama alanlarının hepsinde “yes” ifadesi yer almaktadır. Yani teorik olarak internet mimarisi içinde de garanti temelli hizmet modelleri tasarlanabilir. Bu, klasik best effort yaklaşımının internet için tek olasılık olmadığını gösterir. Ancak bu tür garantili yapılar, genel internetin yaygın çalışma biçiminden daha farklı ve daha kontrollü mekanizmalar gerektirir.

Son satırda ise **Internet – Diffserv** yer almaktadır. Burada bant genişliği, kayıp ve sıra açısından “possible” ya da “possibly” ifadeleri kullanılmıştır; zamanlama için ise yine garanti verilmediği görülmektedir. Bu, Diffserv’in kesin ve katı garantiler veren bir sistemden çok, farklı trafik sınıflarına farklı hizmet öncelikleri tanıyabilen bir yaklaşım

olduğunu düşündürür. Yani bazı akışlar daha iyi hizmet alabilir, bazıları daha düşük öncelikle işlenebilir; fakat bu her zaman tam anlamıyla sert QoS garantisi anlamına gelmez.

Bu tabloyu genel olarak okuduğumuzda şunu görürüz: Ağ hizmet modelleri bir süreklilik üzerinde yer alır. Bir uçta **best effort** vardır; burada ağ basit, esnek ve ölçeklenebilirdir ama güçlü garantiler sunmaz. Diğer uçta ise **garantili hizmet modelleri** vardır; burada ağ belirli kalite özellikleri sunabilir ama bunu sağlamak için daha karmaşık kaynak yönetimi gerekir. Aradaki modeller ise bazı özellikleri garanti edip bazılarını garanti etmeyen ara çözümler sunar.

Bu slaytın önemli yönlerinden biri de, ağ tasarımı her zaman bir **denge** olduğunu göstermesidir. Daha fazla garanti vermek, daha fazla kontrol ve daha fazla kaynak planlaması gerektirir. Daha az garanti vermek ise ağı daha esnek ve büyük ölçekli hale getirebilir. İnternetin neden uzun süre best effort yaklaşımını benimsediği de bu karşılaştırmayla daha iyi anlaşılır.

Özetle bu slaytın ana mesajı şudur: farklı ağ mimarileri, ağ katmanı düzeyinde farklı **QoS garantileri** sunar. İnternetin klasik **best effort** modeli bant genişliği, kayıp, sıra ve zamanlama konusunda garanti vermezken; ATM ve bazı internet hizmet modelleri bu alanlarda daha güçlü ya da kısmi garantiler sağlayabilir. Bu karşılaştırma, ağ hizmet modelinin ağı davranışını ve uygulamalara sunduğu kaliteyi belirleyen temel bir tasarım tercihi olduğunu gösterir.

Best-effort Hizmeti Yansımaları:

- **Mekanizmanın basitliği**, İnternet'in yaygın olarak kullanılmasına olanak sağlamıştır
- **Yeterli bant genişliği sağlanması**, gerçek zamanlı uygulamaların (örneğin, etkileşimli ses, video) performansının “çoğu zaman” için “yeterince iyi” olmasını sağlar”
- **Müşterilerin ağlarına yakın konumlarda bulunan, uygulama katmanında dağıtılmış hizmetler (veri merkezleri, içerik dağıtım ağları)**, hizmetlerin birden fazla konumdan sunulmasını sağlar
- congestion control of “elastic” services helps

Best Effort hizmet modelinin başarısını yadsımak zor

Ders Notu: Best-effort Hizmeti Yansımaları

Bu slayt, internetin **best-effort** hizmet modelinin neden teorik olarak sınırlı görünmesine rağmen pratikte çok başarılı olduğunu açıklamaktadır. Bir önceki slaytta internetin bant genişliği, kayıp, sıra ve zamanlama konusunda kesin garantiler vermediği görülmüştü. Burada ise şu önemli nokta vurgulanmaktadır: Bir hizmet modelinin başarılı olması için mutlaka katı garantiler vermesi gerekmez. Bazen daha **basit**, daha **ölçeklenebilir** ve daha **esnek** bir model, gerçek dünyada daha etkili olabilir.

İlk vurgu, **mekanizmanın basitliği** üzerinedir. Best-effort yaklaşımında ağın içinde her akış için karmaşık rezervasyonlar, katı QoS sözleşmeleri ya da sıkı kaynak tahsis mekanizmaları zorunlu değildir. Bu sadelik, internetin çok geniş ölçekte yaygınlaşmasını kolaylaştırmıştır. Yani internetin büyümesinde yalnızca teknik kapasite değil, çekirdek mimarının fazla karmaşık olmaması da önemli rol oynamıştır. Basit ağ, daha kolay uygulanabilir, daha kolay genişletilebilir ve farklı teknolojilerle daha rahat birlikte çalışabilir.

İkinci önemli nokta, **yeterli bant genişliği sağlandığında**, gerçek zamanlı uygulamaların performansının çoğu zaman yeterince iyi olabilmesidir. Slaytta etkileşimli ses ve video örnekleri verilmektedir. Buradaki mantık şudur: Her akış için kesin gecikme garantisi verilmemiş olsa bile, ağda yeterli kapasite varsa kullanıcı deneyimi pratikte tatmin edici olabilir. Yani mühendislikte her zaman sıkı garanti vermek tek çözüm değildir; bazen kapasiteyi artırmak ve darboğazları azaltmak da performansı yeterli düzeye çıkarabilir.

Üçüncü vurgu, **uygulama katmanında dağıtılmış hizmetler** üzerinedir. Slaytta müşterilerin ağlarına yakın konumlarda bulunan veri merkezleri ve içerik dağıtım ağları örnek verilmektedir. Bunun anlamı şudur: Ağ katmanı kendi başına sıkı QoS garantileri vermese bile, uygulama mimarisi akıllıca tasarlanarak hizmet kalitesi iyileştirilebilir. İçeriğin kullanıcıya daha yakın noktalardan sunulması, gecikmeyi azaltır ve performansı artırır. Bu yüzden best-effort internet yalnızca ağ katmanının gücüyle değil, üst katmanlarda geliştirilen akıllı dağıtık çözümlerle birlikte başarılı hale gelmiştir.

Dördüncü nokta ise **elastic services** için tıkanıklık kontrolünün yardım etmesidir. Elastic hizmetler, örneğin dosya indirme, web erişimi ya da bazı veri aktarım uygulamaları gibi, hız değişimlerine uyum sağlayabilen hizmetlerdir. Bu tür uygulamalarda taşıma katmanındaki tıkanıklık kontrolü mekanizmaları, ağdaki yük durumuna uyum sağlayarak performansın kabul edilebilir seviyede kalmasına yardımcı olur. Yani ağ katmanı garanti vermese bile, üst katmandaki protokoller değişen koşullara uyum sağlayarak sistemi daha dengeli hale getirebilir.

Bu slaytın asıl mesajı, internetin başarısının tek başına QoS garantilerine dayanmamış olmasıdır. İnternet, basit ağ çekirdeği, artan bant genişliği, kullanıcıya yakın içerik sunumu ve üst katmanlardaki uyarlayıcı mekanizmalar sayesinde çok büyük ölçekte başarılı olmuştur. Başka bir deyişle, **best-effort modeli zayıf bir model gibi görünse de, çevresine kurulan mühendislik ekosistemi onu son derece güçlü hale getirmiştir.**

Slaytın en altındaki “Best Effort hizmet modelinin başarısını yadsımak zor” ifadesi bu yüzden önemlidir. Bu cümle, teorik olarak sınırlı görünen bir modelin pratikte ne kadar etkili olabildiğini vurgular. İnternetin küresel ölçekte yaygınlaşması, büyük ölçüde bu basit ama ölçeklenebilir hizmet modelinin üzerinde mümkün olmuştur.

Özetle bu slaytın ana mesajı şudur: **Best-effort hizmet modeli katı garantiler vermez, ancak basitliği, yüksek bant genişliği olanakları, dağıtılmış uygulama mimarileri ve üst katman tıkanıklık kontrolü sayesinde pratikte çok başarılı olmuştur.** Bu nedenle internetin başarısını anlamak için yalnızca hangi garantileri vermediğine değil, bu eksiklerin hangi tamamlayıcı mekanizmalarla telafi edildiğine de bakmak gerekir.

Ağ katmanı: “veri düzlemi” yol haritası

- Network layer: overview
 - data plane
 - control plane
- Bir yönlendiricinin içinde neler var?
 - Giriş bağlantı noktaları, anahtarlama, çıkış bağlantı noktaları
 - Tampon yönetimi, zamanlama
- IP: the Internet Protocol
 - datagram format
 - addressing
 - network address translation
 - IPv6
- Generalized Forwarding, SDN
 - Match+action
 - OpenFlow: match+action in action
- Middleboxes



Network Layer: 4-13

Ders Notu: Ağ katmanı: “veri düzlemi” yol haritası

Bu slayt, ağ katmanının veri düzlemi bölümünde odak noktasının artık **yönlendiricinin iç yapısına** kaydığını göstermektedir. Daha önce ağ katmanına genel bakış yapılmış, veri düzlemi ve kontrol düzlemi ayrımı kurulmuştu. Bu noktadan sonra soru daha somut

hale gelir: **Bir yönlendiricinin içinde neler vardır ve paket bu yapının içinde nasıl ilerler?**

Slaytta özellikle iki konu öne çıkarılmaktadır. Birincisi, yönlendiricinin temel bileşenleri olan **giriş bağlantı noktaları, anahtarlama yapısı ve çıkış bağlantı noktalarıdır**. Bir paket yönlendiriciye geldiğinde önce giriş portunda alınır, sonra iç anahtarlama yapısı üzerinden uygun yöne aktarılır ve sonunda ilgili çıkış portundan gönderilir. Yani yönlendirici, tek parça bir kutu gibi düşünülmemelidir; içinde paketin farklı aşamalardan geçtiği işlevsel bir yapı vardır.

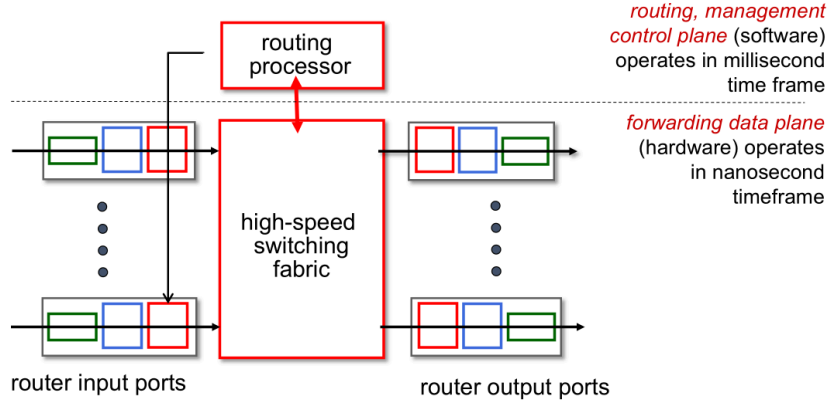
İkinci vurgu ise **tampon yönetimi ve zamanlama** konularıdır. Bunun anlamı şudur: Paketler her zaman hemen iletilemez. Bazen girişte, bazen çıkışta, bazen de iç anahtarlama kapasitesi sınırlı olduğu için beklemek zorunda kalırlar. Bu durumda tamponlar devreye girer. Birden fazla paket aynı kaynağı kullanmak istediğinde ise hangisinin önce gönderileceğine karar verilmesi gerekir; bu da zamanlama mekanizmalarını gündeme getirir. Dolayısıyla yönlendiricinin iç yapısını anlamak, yalnızca fiziksel bileşenleri tanımak değil, aynı zamanda kuyruklama, gecikme ve paket kaybı gibi performans sonuçlarını da anlamaktır.

Bu slayt, veri düzleminin pratik yüzüne geçiş yapıldığını gösterir. Artık konu yalnızca ağ katmanının ne yaptığı değil, bunu **cihaz içinde nasıl yaptığıdır**. Bu nedenle sonraki slaytlar büyük olasılıkla tek tek giriş portu işlevleri, switch fabric yapıları, çıkış portu davranışı ve kuyruklama problemleri üzerinde duracaktır.

Özetle bu slaytın ana mesajı şudur: veri düzlemi bölümünde sıradaki odak, **bir yönlendiricinin iç mimarisidir**. Giriş portları, anahtarlama yapısı, çıkış portları, tampon yönetimi ve zamanlama birlikte ele alınarak, bir paketin yönlendirici içinde nasıl işlendiği ayrıntılı biçimde incelenecektir.

Yönlendirici Mimarisine Genel Bakış

Genel yönlendirici mimarisinin genel görünümü:



Network Layer: 4-14

Ders Notu: Yönlendirici Mimarisine Genel Bakış

Bu slayt, bir yönlendiricinin iç yapısını büyük resim üzerinden göstermektedir. Burada yönlendirici tek parça bir cihaz gibi değil, farklı görevler üstlenen ana bileşenlerden oluşan bir sistem olarak ele alınmaktadır. Temel yapı üç ana bölümden oluşur: **giriş portları**, **yüksek hızlı anahtarlama yapısı (high-speed switching fabric)** ve **çıkış portları**. Bunların üstünde ise yönlendirme ve yönetim kararlarını veren **routing processor** yer alır.

Şekilde en solda **router input ports**, en sağda **router output ports** gösterilmektedir. Paketler önce giriş portlarından alınır. Burada fiziksel katman ve bağlantı katmanı işlemleri yapılır, paket başlığı incelenir ve uygun çıkış yönü belirlenir. Ardından paket, yönlendiricinin içindeki **switching fabric** üzerinden ilgili çıkış portuna taşınır. Son aşamada çıkış portu, paketi uygun bağlantı katmanı biçimine getirir ve fiziksel ortama gönderir. Yani yönlendirici içindeki temel akış, **giriş al – içeride taşı – çıkıştan gönder** mantığıyla işler.

Slayttaki merkezî yapı olan **high-speed switching fabric**, yönlendiricinin iç taşıma mekanizmasıdır. Giriş portlarından gelen paketlerin doğru çıkış portlarına hızlı biçimde aktarılmasını sağlar. Bu bölüm yönlendiricinin veri düzlemi açısından en kritik iç bağlantı yapısıdır. Eğer bu iç yapı yeterince hızlı değilse, girişte veya çıkışta kuyruklar oluşabilir ve performans düşer. Bu nedenle switching fabric, yönlendiricinin toplam aktarım kapasitesini belirleyen temel bileşenlerden biridir.

Üstteki **routing processor** ise veri taşıyan ana yolun dışında, yönlendirme ve yönetim mantığını temsil eder. Slaytta bunun **routing, management control plane (software)** olarak ve **milisaniye zaman ölçeğinde** çalıştığı belirtilmektedir. Bu işlemci, yönlendirme protokollerini çalıştırabilir, yönetim işlevlerini yerine getirebilir ve yönlendirme tablolarını hesaplayabilir. Yani hangi hedefe hangi çıkış portundan gidileceği bilgisi büyük ölçüde burada üretilir ya da güncellenir.

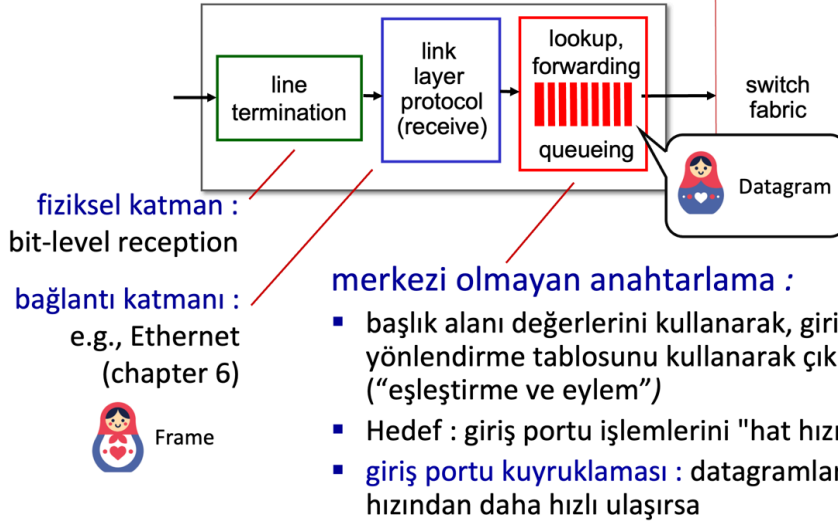
Buna karşılık alttaki asıl paket taşıma bölümü **forwarding data plane (hardware)** olarak verilmiştir ve bunun **nanosaniye zaman ölçeğinde** çalıştığı belirtilmektedir. Bu ayırım çok önemlidir. Çünkü yönlendiricinin iki farklı dünyası vardır. Bir yanda daha yavaş ama daha karmaşık mantık işlemleri yapan kontrol düzlemi bulunur. Diğer yanda ise tek tek paketleri çok yüksek hızda işlemek zorunda olan veri düzlemi bulunur. Kontrol düzlemi yol bilgisini ve kuralları oluşturur; veri düzlemi ise bunları gerçek trafik üzerinde uygular.

Bu zaman ölçeği farkı, neden yönlendiricinin tüm işlevlerinin aynı işlemci üzerinde yürütülmediğini de açıklar. Eğer her paket için karar verme işi doğrudan routing processor tarafından yapılıyorsa, sistem çok yavaş kalırdı. Bu yüzden veri düzlemi donanım temelli ve çok hızlıdır; kontrol düzlemi ise yazılım temelli ve daha üst düzey mantıkla çalışır. Bu ayırım, modern yönlendirici mimarisinin temel tasarım ilkesidir.

Şekildeki kesikli çizgi de bu iki dünyayı ayırmaktadır: üst taraf **kontrol düzlemi**, alt taraf **veri düzlemi** olarak düşünülebilir. Böylece bir yönlendiricinin yalnızca paketleri ileten donanım kutusu olmadığı; aynı zamanda ağ genelindeki yönlendirme mantığını yöneten bir kontrol sistemine sahip olduğu anlaşılır.

Özetle bu slaytın ana mesajı şudur: bir yönlendirici, **giriş portları, yüksek hızlı anahtarlama yapısı** ve **çıkış portlarından** oluşan bir veri düzlemi ile; bunun üzerinde çalışan **routing processor** tabanlı bir kontrol düzleminden meydana gelir. Kontrol düzlemi yönlendirme ve yönetim kararlarını üretirken, veri düzlemi bu kararları çok yüksek hızda paketler üzerinde uygular. Bu yapı, yönlendiricinin hem akıllı hem de hızlı çalışmasını mümkün kılar.

Giriş bağlantı noktası işlevleri



Ders Notu: Giriş bağlantı noktası işlevleri

Bu slayt, bir yönlendiricide paketin sisteme ilk girdiği yerde hangi işlemlerin yapıldığını özetlemektedir. Ağ katmanında forwarding kararı çoğu zaman "paket hangi çıkış portuna gidecek?" sorusuyla anlatılır; ancak bu kararın başlangıç noktası aslında **giriş portudur**. Paket yönlendiriciye ulaştığında, önce fiziksel olarak alınır, sonra bağlantı katmanı düzeyinde işlenir, ardından başlık bilgilerine bakılarak uygun çıkış yönü belirlenir. Bu nedenle giriş portu, yalnızca verinin içeri alındığı pasif bir kapı değil; birden fazla katmanda işlem yapan aktif bir bileşendir.

Şekilde ilk kutu **line termination** olarak verilmiştir. Bu bölüm fiziksel katman işlevini temsil eder. Burada gelen elektriksel, optik ya da radyo sinyali bit düzeyinde alınır ve anlamlı bir bit akışına dönüştürülür. Yani yönlendiriciye gelen veri önce fiziksel ortamdan soyutlanır. Bu aşama olmadan üst katmanların paketi yorumlaması mümkün değildir. Bu yüzden giriş portunun ilk görevi, fiziksel bağlantıdan gelen ham sinyali alınabilir dijital veriye çevirmektir.

İkinci kutu **link layer protocol (receive)** bölümüdür. Bu aşamada artık bitler bir **frame** olarak ele alınır. Bağlantı katmanı başlığı incelenir, çerçevenin geçerli olup olmadığı değerlendirilir ve ağ katmanındaki datagram çıkarılır. Slaytta Ethernet örneği verilmiş olması, yönlendiricinin giriş portunda yalnızca ağ katmanına değil, bağlantı katmanına da ait işlem yaptığını göstermektedir. Başka bir deyişle, giriş portu önce frame'i alır, sonra bu frame içindeki IP datagramını ayrıştırır.

Üçüncü bölüm ise slaytın asıl odak noktası olan **lookup, forwarding, queueing** kısmıdır. Burada artık veri birimi bir **datagram** olarak düşünülür. Giriş portu, datagram başlığındaki ilgili alanlara bakarak hangi çıkış portunun seçileceğini belirlemeye çalışır. Bu işlem slaytta **merkezi olmayan anahtarlama** olarak açıklanmaktadır. Bunun anlamı şudur: Her paketin kaderi merkezi bir işlemci tarafından ayrı ayrı belirlenmez; bunun yerine giriş portu, kendi belleğinde ya da çok yakınındaki yönlendirme tablosunu kullanarak uygun çıkışı bulur. Böylece işlem çok daha hızlı yapılabilir.

Slayttaki “başlık alanı değerlerini kullanarak, giriş portu belleğindeki yönlendirme tablosunu kullanarak çıkış portunu bulma” ifadesi, modern veri düzlemi mantığının temelini verir. Burada fikir yalnızca klasik hedef tabanlı iletme değildir; daha genel anlamda bir **eşleştirme ve eylem** mantığı vardır. Yani başlıktaki bazı alanlar okunur, uygun kural bulunur ve buna karşılık gelen eylem uygulanır. Bu eylem çoğu zaman uygun çıkış portuna iletmek olsa da, daha genel veri düzlemi tasarımlarında başka işlemler de olabilir.

Slayttaki önemli vurgulardan biri de hedefin giriş portu işlemlerini **hat hızında (line rate)** tamamlamak olduğudur. Bu çok önemlidir. Çünkü yönlendiriciye paketler çok yüksek hızlarda gelebilir ve her paket için uzun süre karar verilirse cihaz darboğaza girer. Bu yüzden lookup ve forwarding işleminin, paketin geldiği hızla uyumlu biçimde tamamlanması gerekir. Veri düzlemi donanımının hızlı tasarlanması temel nedeni budur.

Burada bir diğer önemli konu **giriş portu kuyruklamasıdır**. Eğer datagramlar, switch fabric’in taşıyabileceğinden daha hızlı şekilde giriş portuna ulaşırsa, paketler hemen iç yapıya aktarılamaz ve girişte beklemek zorunda kalır. Bu da kuyruklama anlamına gelir. Yani giriş portu yalnızca fiziksel alma ve yönlendirme kararı verme noktası değil, aynı zamanda yoğunluk durumunda geçici tamponlama yapan bir yerdir. Bu durum ileride gecikme, tampon taşması ve HOL blocking gibi performans sorunlarına zemin hazırlar.

Şekildeki akış sırası da bu yüzden anlamlıdır: önce **bit düzeyi**, sonra **frame düzeyi**, sonra **datagram düzeyi** işlenmektedir. Bu sıralama, katmanlı mimarinin yönlendirici içindeki gerçek çalışma karşılığıdır. Önce fiziksel katman sinyali bitlere dönüştürür, sonra bağlantı katmanını çerçeveyi işler, sonra ağ katmanını datagramı okuyup yönlendirme kararını başlatır.

Özetle bu slaytın ana mesajı şudur: giriş bağlantı noktası üç temel görevi birlikte yürütür. Önce fiziksel katmanda bitleri alır, sonra bağlantı katmanında frame’i işler, ardından ağ katmanında datagram başlığına bakarak uygun çıkış portunu belirler. Bu işlem mümkün olduğunca **merkezi olmayan** ve **hat hızında** yapılmalıdır. Eğer paketler iç anahtarlama yapısına aktarılabilirse daha hızlı gelirse, giriş portunda kuyruklama oluşur. Bu

nedenle giriş portu, yönlendiricinin performansı açısından en kritik başlangıç noktalarından biridir.

Giriş bağlantı noktası işlevleri



Ders Notu: Giriş Portu İşlevleri

Bu slayt, bir yönlendiricide paketin sisteme ilk girdiği yerde hangi işlemlerin yapıldığını göstermektedir. Ağ katmanında yönlendirmenin dışarıdan bakıldığında temel görevi, paketi uygun çıkışa ulaştırmak gibi görünür. Ancak bu karar aslında önce **giriş portunda** verilmeye başlanır. Slaytta görülen yapı, giriş portunun yalnızca fiziksel bir bağlantı noktası olmadığını; fiziksel katman, bağlantı katmanı ve iletim kararının ilk aşamasını birlikte yürüten bir işlem hattı olduğunu ortaya koymaktadır. Özellikle **lookup, forwarding, queueing** ifadesi, yönlendirme kararının merkezi bir birimde değil çoğu durumda doğrudan giriş portunda verildiğini vurgular.

İlk aşama **line termination** olarak gösterilen fiziksel katman işlevidir. Burada gelen sinyal bit düzeyinde alınır ve anlamlı bir bit akışına dönüştürülür. Yani kablo, fiber ya da başka bir fiziksel ortam üzerinden gelen elektriksel ya da optik sinyal önce bu seviyede sonlandırılır. Bundan sonra bağlantı katmanı işlemleri devreye girer. Çerçevenin yapısı incelenir, bağlantı katmanı başlıkları değerlendirilir ve ağ katmanındaki datagram üst katmana hazırlanır. Bu nedenle giriş portu, yalnızca "paketi içeri alan" bir arayüz değil; paketi yorumlayan ve bir sonraki adıma hazırlayan bir donanım bileşenidir. Kitapta da

giriş portunun fiziksel katman sonlandırma, bağlantı katmanı işlevleri ve en kritik olarak yönlendirme tablosuna başvurma görevlerini yerine getirdiği açıkça belirtilmektedir.

Slaytın asıl önemli kısmı, **decentralized switching** yani merkezi olmayan anahtarlama fikridir. Buradaki temel düşünce, her paketin nereye gideceğine karar vermek için onu merkezi bir işlemciye göndermemektir. Bunun yerine giriş portunun kendi belleğinde ya da ona çok yakın bir yapıda bulunan yönlendirme tablosu kullanılarak, paket daha iç yapıya girmeden hangi çıkış portuna yönlendirileceği belirlenir. Bu yaklaşım çok önemlidir; çünkü yönlendiriciler çok yüksek hızlarda çalışır ve her paketin karar için merkezi bir işlemciye gitmesi büyük darboğaz oluşturur. Dağıtık karar verme sayesinde veri düzlemi çok daha hızlı çalışır. Kitapta da veri düzlemi işlevlerinin donanımda, nanosaniye ölçeğinde gerçekleştiği; kontrol düzlemi işlevlerinin ise daha yavaş zaman ölçeklerinde yazılım tarafından yürütüldüğü vurgulanmaktadır.

Bu slaytta ayrıca iki farklı iletme anlayışı yan yana verilmektedir. İlki **hedef tabanlı yönlendirme**dir. Bu geleneksel IP iletme yaklaşımıdır ve paketin hangi çıkıştan gönderileceği esas olarak **hedef IP adresine** bakılarak belirlenir. Yani yönlendirici, “Bu paket hangi hedef ağa gidiyor?” sorusunu sorar ve yönlendirme tablosunda buna karşılık gelen girdiyi bulur. İnternet’in klasik yönlendirici davranışı büyük ölçüde bu mantığa dayanır.

İkincisi ise **genelleştirilmiş yönlendirme**dir. Burada karar yalnızca hedef IP adresine bağlı değildir; paketin başlığındaki farklı alanlar da kullanılabilir. Yani kaynak adres, protokol türü, port bilgisi ya da başka başlık alanları karar sürecine katılabilir. Bu yaklaşım özellikle modern veri düzlemi tasarımlarında ve SDN mantığında önem kazanır. Böylece yönlendirici sadece “hangi çıkışa gönderileceğine” değil, gerekirse paketin engellenmesine, çoğaltılmasına ya da belirli kurallara göre farklı muamele görmesine de karar verebilir. Slayttaki “match + action” mantığı tam olarak bunu ifade eder: Başlık alanlarıyla bir eşleşme yapılır, ardından buna uygun bir eylem uygulanır.

Giriş portunda **queueing** yani kuyruklama yapılmasının nedeni de pratikteki hız farklarıdır. Paket girişte çok hızlı gelebilir; fakat anahtarlama yapısı ya da hedef çıkış portu o anda aynı hızda hizmet veremeyebilir. Bu durumda paket hemen işlenemiyorsa geçici olarak bellekte bekletilir. Kuyruklama, yönlendiricinin normal çalışmasının doğal bir parçasıdır; ancak trafik yoğunluğu arttığında gecikme ve hatta taşma nedeniyle paket kaybı oluşabilir. Bu nokta, ileride tampon yönetimi ve zamanlama konularıyla birleşir. Yani giriş portu sadece karar veren değil, aynı zamanda yoğunluk altında geçici depolama yapan bir yapıdır.

Bu slaytın ağ mimarisi içindeki yeri şudur: Uygulama ve taşıma katmanlarında mesajın anlamı ve güvenilir iletimi önemliyken, ağ katmanının veri düzleminde temel soru paketin bir yönlendirici içinde nasıl hareket ettiğidir. Giriş portu, bu hareketin başlangıç

noktasıdır. Paket burada fiziksel ortamdan alınır, çerçeveden çıkarılır, başlık alanları incelenir, uygun çıkış belirlenir ve gerekiyorsa kuyrukta bekletilir. Dolayısıyla veri düzleminin hız, ölçeklenebilirlik ve performans gereksinimlerini anlamak için önce giriş portunun işlevini anlamak gerekir.

Özetle bu slayt, giriş portunun üç ana rolünü gösterir: gelen biti almak, paketi yorumlamak ve iletme kararını olabildiğince erken vermek. Böylece yönlendirici, yüksek hızlı ağlarda merkezi darboğaz oluşturmadan çalışabilir. Bu yapı, klasik hedef tabanlı iletmeden modern genelleştirilmiş iletme kadar uzanan veri düzlemi mantığının temel başlangıç noktasıdır.

Destination-based forwarding

forwarding table	
Destination Address Range	Link Interface
11001000 00010111 00010000 00000000 through 11001000 00010111 00010111 11111111	0
11001000 00010111 00010000 00000100 through 11001000 00010111 00010000 00000111	3
11001000 00010111 00011000 00000000 through 11001000 00010111 00011000 11111111	1
11001000 00010111 00011001 00000000 through 11001000 00010111 00011111 11111111	2
otherwise	3

S: Peki, aralıklar bu kadar düzgün bir şekilde bölünmezse ne olur?

Network Layer: 4-18

Ders Notu: Destination-based forwarding

Bu slayt, hedefe dayalı iletmenin temel mantığını göstermektedir. Yönlendiriciye gelen bir IP datagramı için karar, paketin **hedef IP adresine** bakılarak verilir; yani yönlendirici “bu paket hangi hedef adres aralığına ait?” sorusunu sorar ve buna karşılık gelen **çıkış arayüzünü (link interface)** seçer. Ağ katmanının veri düzlemindeki temel işlevlerinden biri de tam olarak budur: paketi giriş bağlantısından uygun çıkış bağlantısına aktarmak. Kitapta forwarding, bir paketin giriş arayüzünden uygun çıkış arayüzüne taşınması olarak tanımlanır ve bu işlemin yönlendirici üzerinde yerel, çok hızlı bir karar mekanizması olduğu vurgulanır.

Slayttaki tablo bir **forwarding table** örneğidir. Sol tarafta hedef adres aralıkları, sağ tarafta ise bu aralığa karşılık gelen çıkış arayüzü yer alır. Buradaki önemli nokta, tüm

adres uzayının tek tek yazılmaması; bunun yerine belirli bit desenleriyle ifade edilen aralıkların kullanılmasıdır. Örneğin tabloda 200.23.16.0 ile 200.23.23.255 aralığının arayüz 0'a, bunun içindeki daha dar bir aralık olan 200.23.16.4 ile 200.23.16.7 aralığının ise arayüz 3'e gönderildiği gösterilmektedir. Benzer şekilde 200.23.24.0–200.23.24.255 aralığı arayüz 1'e, 200.23.25.0–200.23.31.255 aralığı arayüz 2'ye yönlendirilmekte; bunların dışında kalan adresler ise varsayılan olarak arayüz 3'e gönderilmektedir. Bu yapı, yönlendirme tablolarının adres uzayını aralıklara bölerek ölçeklenebilir biçimde çalıştığını gösterir. Slaytta ayrıca “aralıklar bu kadar düzenli bölünmezse ne olur?” sorusuyla bu tablonun bir sonraki adımda **longest prefix matching** mantığına bağlandığı da açıkça görülmektedir.

Bu görselde özellikle dikkat edilmesi gereken nokta, bazı aralıkların diğerlerinin içine gömülü olmasıdır. Örneğin geniş bir adres bloğu için genel bir kural tanımlanırken, bu bloğun içindeki küçük bir alt blok için daha özel bir kural yazılabilir. Bu durumda bir hedef adres birden fazla tablo girdisiyle eşleşebilir. Kitapta bunun yönlendiricide önemli bir incelik olduğu belirtilir: bir hedef adres birden fazla girişle eşleştiğinde yönlendirici **en uzun ön ek eşleşmesini (longest prefix match)** kullanır; yani en özel, en fazla ortak başlangıç bitine sahip kural seçilir. Böylece 200.23.16.* gibi geniş bir blok için yazılmış genel bir kural varken, 200.23.16.4–200.23.16.7 gibi daha dar bir blok için tanımlanmış özel kural öncelik kazanır. Bu, hem doğru yönlendirme yapılmasını sağlar hem de CIDR ve alt ağ mantığının uygulanabilmesi için gereklidir.

Slayttaki renkli vurgular da tam olarak bu mantığı anlatmaktadır. Sarı alanlar, daha büyük bir adres aralığını tanımlayan ortak bit desenlerini; pembe alan ise bu büyük aralığın içindeki daha özel kısmı göstermektedir. Yani tablo yalnızca “adres hangi aralıkta?” sorusunu değil, aynı zamanda “daha özel bir eşleşme var mı?” sorusunu da gündeme getirir. Bu nedenle bu slayt, klasik destination-based forwarding'den longest prefix matching'e geçişin hazırlık slaytı olarak okunmalıdır. Nitekim ders materyalinde de, hedef adres için uygun iletme girdisi aranırken en uzun eşleşen adres ön ekinin kullanıldığı açıkça belirtilmektedir.

Özetle bu slaytın ana mesajı şudur: yönlendirici, paketin yalnızca hedefine bakarak karar verir; fakat bu karar basit bir “ilk eşleşeni seçme” işlemi değildir. Adresler aralıklara bölünür, tabloda genel ve özel kurallar birlikte bulunabilir ve doğru çıkış arayüzüne ulaşmak için gerektiğinde en özel eşleşme tercih edilir. Bu yapı, ağ katmanındaki veri düzleminin hem hızlı hem de hiyerarşik adreslemeyle uyumlu şekilde çalışmasını sağlar.

Longest prefix matching

longest prefix match

when looking for forwarding table entry for given destination address, use *longest* address prefix that matches destination address.

Destination Address Range	Link interface
11001000 00010111 00010*** *****	0
11001000 00010111 00011000 *****	1
11001000 00010111 00011*** *****	2
otherwise	3

examples:

11001000 00010111 00010110 10100001 which interface?
11001000 00010111 00011000 10101010 which interface?

Network Layer: 4-19

Ders Notu: Longest Prefix Matching

Bu slayt, hedefe dayalı iletmede (destination-based forwarding) yönlendiricinin bir paketi hangi çıkış arayüzüne göndereceğini belirlerken neden yalnızca “eşleşme var mı?” sorusunu sormadığını, bunun yerine **en uzun ön ek eşleşmesini** yani **longest prefix matching** kuralını kullandığını göstermektedir. Slaytta, yönlendirme tablosundaki bazı girişlerin aynı hedef adresle aynı anda eşleşebileceği ve bu durumda en ayrıntılı, yani en fazla bitten oluşan eşleşmenin seçildiği vurgulanmaktadır.

Buradaki temel fikir şudur: Bir yönlendirme tablosunda bazı kurallar daha genel, bazıları ise daha özeldir. Örneğin bir giriş çok geniş bir adres aralığını kapsayabilirken, başka bir giriş onun içindeki daha küçük ve daha özel bir alt kümeyi temsil edebilir. Bir paket geldiğinde hedef IP adresi birden fazla tablo girdisiyle uyushabilir. Böyle bir durumda yönlendirici rastgele seçim yapmaz ve ilk bulduğu kaydı da kullanmaz. Bunun yerine, hedef adresle **en fazla ortak başlangıç bitine** sahip olan girdiyi seçer. Bu yüzden bu yönteme “en uzun ön ek eşleşmesi” adı verilir. Slayttaki örnek tabloda da belirli bit desenlerinin farklı arayüzlere karşılık geldiği, bazı adreslerin birden fazla desenle uyushabildiği ve kararın bu kurala göre verildiği gösterilmektedir.

Bu mantığın neden önemli olduğu, ağ adreslemenin hiyerarşik yapısıyla ilgilidir. İnternette adresleme, büyük blokların daha küçük alt ağlara bölünmesiyle çalışır. Dolayısıyla daha kısa bir ön ek daha genel bir ağı, daha uzun bir ön ek ise daha özel bir alt ağı temsil eder. Eğer yönlendirici en uzun eşleşmeyi seçmeseydi, daha özel olarak tanımlanmış bir alt ağa gitmesi gereken trafik yanlışlıkla daha genel bir yola

gönderilebilirdi. Bu nedenle longest prefix matching, CIDR ve alt ağ mantığının doğal sonucudur. Kitapta da, bir hedef adresin birden fazla girişle eşleşebileceği ve yönlendiricinin bu durumda en uzun eşleşmeyi seçtiği açıkça belirtilmektedir.

Slayttaki örneklerden biri, hedef adresin ilk 24 bitinin bir tablo girdisiyle, ilk 21 bitinin ise başka bir girdiyle eşleşebildiğini ima eder. Böyle bir durumda 24 bitlik eşleşme seçilir; çünkü bu, hedefin daha dar ve daha doğru tanımlanmış bir alt ağa ait olduğunu gösterir. Aynı mantık, “otherwise” ya da varsayılan yol girişinin neden en sonda düşünüldüğünü de açıklar. Eğer hiçbir daha özel eşleşme yoksa paket varsayılan arayüze gönderilir. Yani varsayılan rota, en genel kuraldır; yalnızca başka bir uygun kural bulunmadığında devreye girer.

Bu konu, veri düzlemi açısından da önemlidir. Çünkü bu karar verme işlemi her paket için, çok yüksek hızlarda yapılır. Kitapta bu aramanın gigabit hızlarında nanosaniye ölçeğinde gerçekleşmesi gerektiği, bu yüzden basit yazılımsal arama yerine donanım destekli hızlı arama tekniklerinin kullanıldığı belirtilir. Özellikle **TCAM (Ternary Content Addressable Memory)** yapıları, yönlendirme tablolarında en uzun ön ek eşleşmesini çok hızlı yapmak için kullanılır. Slaytta da longest prefix matching işleminin çoğu zaman TCAM ile gerçekleştirildiği ve büyük ölçekli cihazlarda çok büyük yönlendirme tablolarının bu şekilde tutulabildiği ifade edilmektedir.

Özet olarak bu slaytın ana mesajı şudur: Yönlendirici, hedef IP adresine bakarak yalnızca bir eşleşme aramaz; **birden fazla eşleşme varsa en özel olanı seçer**. Bu mekanizma, internet adreslemesinin ölçeklenebilir ve doğru çalışmasını sağlar. Böylece hem geniş adres blokları için genel kurallar yazılabilir hem de onların içindeki belirli alt ağlar için daha özel yönlendirme kararları alınabilir. Bu, yönlendirme tablolarının hem esnek hem de hiyerarşik biçimde çalışmasının temelidir.

En Uzun Önek Eşleştirme

- Adresleme konusunu işlediğimizde, en uzun önek eşleştirmesinin **neden** kullanıldığını birazdan göreceğiz
- En uzun önek eşleştirme: Genellikle üçlü içerik adreslenebilir bellekler kullanılarak gerçekleştirilir (TCAMs)
 - **İçerik Üzerinden Erişim** : TCAM'de “bu bit desenine uyan kaydı bul”:
Tablo ne kadar büyük olursa olsun eşleşen kayıt çok kısa sürede bulunabilir.
 - Cisco Catalyst cihazlarında TCAM içinde yaklaşık bir milyon yönlendirme girdisinin tutulabilir.

Network Layer: 4-23

Ders Notu: Longest Prefix Matching

Bu slayt, hedefe dayalı iletmede bir paketin hangi çıkış arayüzüne gönderileceğini belirlerken neden **longest prefix matching** kuralının kullanıldığını özetlemektedir. Önceki slaytta görüldüğü gibi, bir hedef IP adresi yönlendirme tablosundaki birden fazla girişle eşleşebilir. Böyle bir durumda yönlendirici herhangi bir eşleşmeyi seçmez; **hedef adresle en fazla ortak başlangıç bitine sahip olan**, yani en özel adres bloğunu temsil eden girdiyi seçer. Bu mekanizma, özellikle CIDR ve alt ağlara bölme mantığıyla birlikte düşünüldüğünde zorunlu hale gelir. Ders materyalinde de, bir hedef adres için yönlendirme tablosunda giriş aranırken **en uzun adres ön eki eşleşmesinin** kullanıldığı açıkça belirtilmektedir.

Buradaki “prefix” kavramı, IP adresinin soldan başlayan ortak bit bölümünü ifade eder. Daha kısa bir ön ek daha genel bir ağı, daha uzun bir ön ek ise daha dar ve daha özel bir alt ağı temsil eder. Dolayısıyla bir adres hem büyük bir ağ bloğuna hem de onun içindeki daha küçük bir alt ağa ait olabilir. Yönlendiricinin doğru karar verebilmesi için daha genel kural yerine daha özel kuralı seçmesi gerekir. Longest prefix matching tam olarak bunu sağlar. Böylece ağ yöneticileri hem geniş adres öbekleri için genel yönlendirme kuralları tanımlayabilir hem de gerektiğinde belirli alt ağlar için daha özel istisnalar ekleyebilir. Bu, internet adreslemesinin hiyerarşik ve ölçeklenebilir çalışmasının temel parçalarından biridir.

Slaytta ayrıca bu işlemin pratikte nasıl hızlı yapıldığına da değinilmektedir. Longest prefix matching teorik olarak basit görünse de, gerçek yönlendiricilerde bu işlem her

paket için ve çok yüksek hızlarda yapılmak zorundadır. Kitapta veri düzlemi işlevlerinin çok kısa zaman ölçeklerinde, çoğunlukla donanım üzerinde çalıştığı vurgulanır. Çünkü yüksek hızlı bağlantılarda yönlendiricinin her pakete yalnızca çok kısa bir işlem süresi ayırması mümkündür. Bu yüzden yönlendirme tablosunda uygun girdiyi bulma işi sıradan yazılımsal arama ile değil, özel donanım teknikleriyle gerçekleştirilir.

Bu bağlamda slaytta **TCAM (ternary content addressable memory)** kullanımı öne çıkarılmaktadır. “Content addressable” ifadesi, belleğin veriye adres üzerinden değil, içerik üzerinden erişim sağlaması anlamına gelir. Yani klasik bellekte “şu adresteki veriyi getir” denirken, TCAM’de “bu bit desenine uyan kaydı bul” mantığı vardır. Bu sayede hedef adres TCAM’e verildiğinde, tablo ne kadar büyük olursa olsun eşleşen kayıt çok kısa sürede bulunabilir. Slaytta bunun tek bir saat çevriminde yapılabildiği vurgulanmaktadır. Bu, özellikle büyük omurga yönlendiricileri ve kurumsal cihazlar için kritik bir avantajdır; çünkü yönlendirme tablosu büyüse bile arama süresi aynı düzeyde tutulmaya çalışılır.

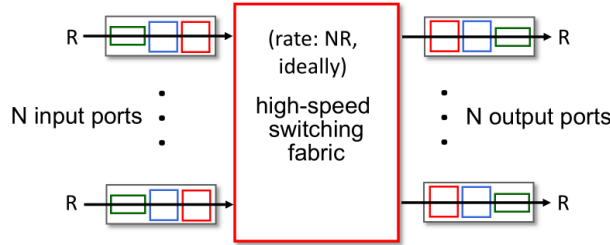
TCAM kullanımının önemli olmasının nedeni, yönlendirme tablolarının çok büyük olabilmesidir. Slaytta Cisco Catalyst cihazlarında TCAM içinde yaklaşık bir milyon yönlendirme girdisinin tutulabildiği örneği verilmiştir. Bu örnek, longest prefix matching’in sadece kavramsal bir fikir olmadığını; büyük ölçekli gerçek ağlarda uygulanabilmesi için özel donanım desteğine ihtiyaç duyduğunu göstermektedir. Yani burada konu yalnızca “hangi kural seçilecek?” sorusu değil, aynı zamanda “bu seçim yüksek hızda nasıl yapılacak?” sorusudur. Veri düzlemi başarısı, kararın doğruluğu kadar hızına da bağlıdır.

Bu slaytın ağ mimarisi içindeki yeri de önemlidir. Ağ katmanında forwarding yerel bir yönlendirici işlemidir; routing ise ağ genelinde yol belirleme sürecidir. Routing algoritmaları yönlendirme tablolarını oluşturur, forwarding ise bu tabloları kullanarak her paketi doğru çıkışa yollar. Longest prefix matching, bu forwarding aşamasındaki temel karar kuralıdır. Yani kontrol düzlemi hangi ağlara hangi yolların uygun olduğunu belirledikten sonra, veri düzlemi her gelen pakette bu bilgiyi longest prefix matching kuralıyla uygular.

Özetle bu slaytın ana mesajı şudur: Bir hedef adres birden fazla yönlendirme girdisiyle eşleşebileceğinden, yönlendirici **en uzun ön eke sahip eşleşmeyi** seçer. Bu seçim daha özel ağ bilgisini öncelikli hale getirir ve hiyerarşik IP adreslemenin doğru çalışmasını sağlar. Gerçek ağ cihazlarında ise bu işlem, çok büyük tablolar üzerinde bile yüksek hızda karar verebilmek için çoğu zaman **TCAM** gibi özel bellek yapılarıyla gerçekleştirilir.

Switching fabrics

- transfer packet from input link to appropriate output link
- **switching rate**: rate at which packets can be transfer from inputs to outputs
 - often measured as multiple of input/output line rate
 - N inputs: switching rate N times line rate desirable



Network Layer: 4-24

Ders Notu: Switching Fabrics

Bu slayt, yönlendirici içindeki **switching fabric** yapısının ne işe yaradığını açıklamaktadır. Switching fabric, bir paketi uygun **giriş portundan** uygun **çıkış portuna** taşıyan iç aktarım mekanizmasıdır. Başka bir ifadeyle, giriş portunda paketin hangi çıkışa gönderileceği belirlendikten sonra, bu kararın fiziksel olarak yönlendirici içinde uygulanmasını sağlayan bölüm switching fabric'tir. Kitapta da switching fabric, yönlendiricinin giriş portlarını çıkış portlarına bağlayan iç yapı olarak tanımlanır.

Bu noktada yönlendiricinin işleyişini üç aşamalı düşünmek yararlı olur. İlk olarak giriş portunda paket alınır, başlığı incelenir ve uygun çıkış belirlenir. İkinci olarak switching fabric devreye girer ve paketi içeride doğru çıkış tarafına taşır. Son olarak çıkış portu paketi uygun bağlantı üzerinden dışarı gönderir. Dolayısıyla switching fabric, karar verme aşaması ile fiziksel gönderim aşaması arasındaki köprüdür. Yönlendiricinin içindeki veri akışının merkezi bileşeni budur.

Slaytta öne çıkan ikinci kavram **switching rate** tir. Bu, paketlerin girişlerden çıkışlara ne kadar hızla aktarılabilirdiğini ifade eder. Yani burada ölçülen şey dış bağlantı hızı değil, yönlendiricinin iç omurgasının taşıma kapasitesidir. Slaytta bu hızın çoğu zaman giriş/çıkış hat hızının katı olarak ifade edildiği belirtilmektedir. Eğer her giriş portu saniyede R hızında paket alabiliyorsa ve yönlendiricide N adet giriş portu varsa, ideal durumda switching fabric'in toplamda yaklaşık $N \times R$ düzeyinde çalışabilmesi istenir. Böylece tüm girişler aynı anda aktif olduğunda içeride darboğaz oluşmaz.

Şekilde tam da bu fikir görselleştirilmektedir. Her giriş ve çıkış portunun hat hızı **R** olarak gösterilmiş, ortadaki yüksek hızlı switching fabric için ise ideal hızın **NR** olduğu belirtilmiştir. Bunun anlamı şudur: Eğer yönlendiricide N adet giriş portu varsa ve bu portların hepsi aynı anda veri getiriyorsa, switching fabric her birinden gelen akışı gecikmeye yol açmadan taşıyabilmelidir. Aksi halde, giriş portlarında ya da switching fabric öncesinde birikme oluşur. Bu da kuyruklama, gecikme ve trafik yoğunluğunda paket kaybı gibi sonuçlara yol açabilir. Slayt, switching fabric'in yalnızca "bağlayıcı bir iç yol" değil, performansı doğrudan belirleyen kritik bir bileşen olduğunu göstermektedir.

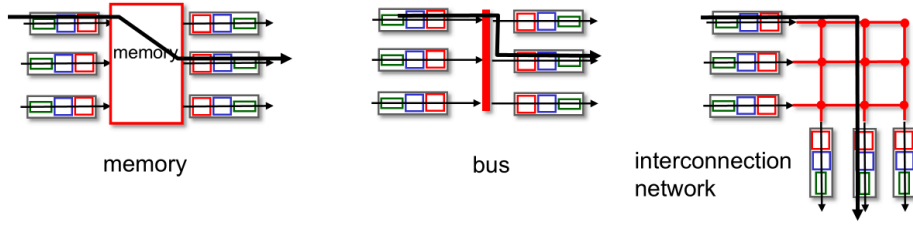
Bu kavram ağ katmanının veri düzlemi açısından oldukça önemlidir. Çünkü yönlendirici performansı sadece yönlendirme tablosuna ne kadar hızlı bakıldığıyla belirlenmez; paketin içeride ne kadar hızlı taşınabildiği de en az bunun kadar önemlidir. Giriş portunda doğru çıkış belirlenmiş olsa bile, switching fabric yeterince hızlı değilse yönlendirici toplam yük altında yavaşlar. Bu nedenle veri düzleminde hem karar verme hızı hem de iç taşıma kapasitesi birlikte düşünülmelidir. Kitapta da yönlendiricinin giriş portları, switching fabric ve çıkış portlarının çoğunlukla donanımda gerçekleştirildiği; bunun sebebinin çok kısa zaman ölçeklerinde çalışmak zorunda olmaları olduğu vurgulanır.

Bu slayt aynı zamanda ölçeklenebilirlik sorununa da işaret etmektedir. Küçük bir yönlendiricide az sayıda port olduğunda switching fabric üzerindeki baskı sınırlı olabilir. Ancak port sayısı arttıkça ve her portun hat hızı yükseldikçe içerideki toplam aktarım gereksinimi çok hızlı büyür. Bu yüzden yüksek kapasiteli yönlendiricilerde switching fabric tasarımı başlı başına kritik bir mühendislik problemidir. Amaç, girişler ile çıkışlar arasında mümkün olduğunca paralel ve yüksek hızlı bir iç veri yolu oluşturmaktır.

Özetle bu slaytın ana mesajı şudur: **switching fabric**, yönlendirici içinde paketi doğru çıkış portuna taşıyan iç anahtarlama yapısıdır. Bunun başarımı ise **switching rate** ile değerlendirilir. İdeal durumda, N giriş portlu bir yönlendiricide switching fabric'in kapasitesi toplam giriş yükünü karşılayabilecek şekilde yaklaşık **N × R** mertebesinde olmalıdır. Böylece yönlendirici, tüm portlar aynı anda aktif olduğunda bile içeride darboğaz oluşturmadan çalışabilir.

Switching Fabrics

- Switching fabric, bir paketi uygun **giriş portundan** uygun **çıkış portuna** taşıyan iç aktarım mekanizmasıdır.
- Switching Rate:** Paketlerin girişlerden çıkışlara ne kadar hızla aktarılabilirdiğini ifade eder.
 - Bu hız çoğu zaman giriş/çıkış hat hızının katı olarak ifade edilir.
 - N inputs: N giriş portlu bir yönlendiricide switching fabric'in kapasitesi toplam giriş yükünü karşılayabilecek şekilde yaklaşık $N \times R$
- Ana anahtarlama yapıları türleri:



Ders Notu: Switching Fabrics

Bu slayt, yönlendirici içindeki **switching fabric** yapısının yalnızca ne işe yaradığını değil, aynı zamanda hangi temel biçimlerde gerçekleştirilebildiğini göstermektedir. Switching fabric'in görevi, paketi giriş bağlantısından uygun çıkış bağlantısına taşımaktır. Yani giriş portunda paket için doğru çıkış belirlendikten sonra, bu paketin yönlendirici içinde gerçekten o çıkış portuna ulaştırılmasını sağlayan iç aktarım mekanizması switching fabric'tir. Slaytta ayrıca **switching rate** kavramı da verilmiştir; bu, paketlerin girişlerden çıkışlara ne hızla aktarılabilirdiğini ifade eder. İdeal durumda N giriş portu varsa ve her biri R hat hızında çalışıyorsa, switching fabric'in toplamda yaklaşık $N \cdot R$ düzeyinde bir iç taşıma kapasitesi sunması istenir.

Slaytın alt kısmında üç temel switching fabric türü gösterilmektedir: **memory**, **bus** ve **interconnection network**. Bu üç yapı aynı temel işi yapar; fark, paketin yönlendirici içinde nasıl taşındığı ve hangi ölçekte verimli çalıştıklarıdır. Bu nedenle konu sadece mimari sınıflandırma değildir; aynı zamanda yönlendiricinin performans sınırlarını, ölçeklenebilirliğini ve darboğaz noktalarını anlamakla ilgilidir.

İlk tür, **memory-based switching** yaklaşımıdır. Bu yapıda paket giriş portundan alındıktan sonra ortak belleğe yazılır, ardından uygun çıkış portu tarafından bu bellekten okunur. Mantık olarak oldukça anlaşılır bir yöntemdir; çünkü paket önce merkezi bir belleğe konur, sonra hedef çıkışa aktarılır. Ancak burada temel sınırlayıcı unsur bellek erişim hızıdır. Bir paket için hem yazma hem de okuma işlemi gerektiğinden, toplam performans belleğin ne kadar hızlı çalışabildiğine bağlı hale gelir. Port sayısı ve hızlar

arttıkça ortak bellek kolayca darboğaz oluşturabilir. Bu nedenle memory tabanlı yapı daha basit tasarımlar için uygun olsa da, çok yüksek hızlı büyük yönlendiricilerde sınırlayıcı olabilir. Kitapta da switching fabric türlerinden biri olarak bellek temelli anahtarlama anlatılır ve bunun işlemci/bellek erişim sınırlarıyla ilişkili olduğu vurgulanır.

İkinci tür, **bus-based switching** yaklaşımıdır. Burada tüm giriş ve çıkış portları ortak bir iç veri yolu, yani bir **bus**, üzerinden bağlanır. Paket, giriş portundan bu ortak yol üzerine konur ve ilgili çıkış portu paketi buradan alır. Bu yapının önemli avantajı mimari sadeliktir; merkezi ortak yol sayesinde iç bağlantı yapısı anlaşılır ve uygulanması görece kolaydır. Ancak burada da temel sınır bus kapasitesidir. Aynı anda birden fazla paketin bu ortak yolu kullanma isteği olduğunda paylaşım zorunlu hale gelir. Bu nedenle iç veri yolunun bant genişliği, toplam yönlendirici performansını doğrudan sınırlar. Yani bus yapısı küçük ve orta ölçekli cihazlarda yeterli olabilirken, yüksek paralellik gerektiren ortamlarda yetersiz kalabilir.

Üçüncü tür ise **interconnection network** yaklaşımıdır. Slayttaki ızgara benzeri yapı bunu göstermektedir. Bu mimaride girişlerle çıkışlar arasında, tek bir ortak yol yerine daha karmaşık ve daha paralel bir bağlantı ağı kurulur. Böylece farklı girişlerden gelen paketler, uygun çıkışlara aynı anda daha esnek biçimde yönlendirilebilir. Bu yaklaşımın temel avantajı, yüksek port sayılarında ve yüksek hızlarda daha iyi ölçeklenebilmesidir. Çünkü tek bir ortak bellek ya da tek bir bus yerine, daha fazla eşzamanlı iç aktarım olanağı sağlar. Bu da daha yüksek switching rate elde etmeyi mümkün kılar. Büyük kapasiteli yönlendiricilerde bu tür yapıların tercih edilmesinin nedeni de budur. Kitapta interconnection network tabanlı switching fabric'lerin, özellikle paralellik ve yüksek performans açısından önemli olduğu anlatılmaktadır.

Bu üç yaklaşım arasındaki farkı şöyle özetlemek mümkündür: memory tabanlı yapıda darboğaz ortak bellektir, bus tabanlı yapıda darboğaz ortak veri yoludur, interconnection network yapısında ise amaç bu merkezi darboğazları azaltarak daha fazla paralellik sağlamaktır. Dolayısıyla seçim yalnızca teknik bir tasarım tercihi değildir; yönlendiricinin hedeflenen hızına, port sayısına ve maliyet-performans dengesine bağlıdır.

Bu slaytın veri düzlemi açısından önemi büyüktür. Çünkü yönlendirici performansı yalnızca giriş portunda yapılan lookup işlemiyle belirlenmez. Paketin içeride ne hızla taşınabildiği de en az bunun kadar önemlidir. Giriş portu doğru çıkışı bulmuş olsa bile, switching fabric yeterince güçlü değilse içeride kuyruklama oluşur, gecikme artar ve yoğun trafik altında paket kayıpları görülebilir. Bu yüzden veri düzleminin başarımı, yönlendirme kararının doğruluğu ile iç taşıma kapasitesinin birlikte değerlendirilmesini gerektirir.

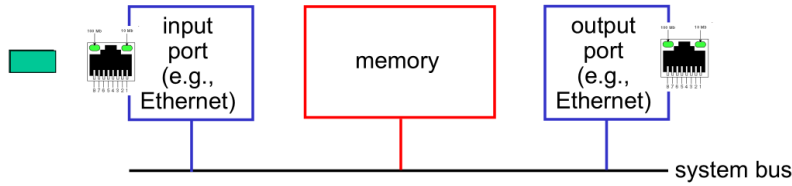
Özetle bu slaytın ana mesajı şudur: **switching fabric**, yönlendirici içinde paketi giriş portundan uygun çıkış portuna taşıyan çekirdek iç yapıdır ve bunun üç temel

gerçekleştirme biçimi vardır: **memory**, **bus** ve **interconnection network**. İlk ikisi daha merkezi kaynaklara dayandığı için ölçek büyüdükçe sınırlanabilir; üçüncüsü ise daha yüksek paralellik sunarak büyük ve hızlı yönlendiriciler için daha uygun bir çözüm sağlar.

Bellek Üzerinden Anahtarlama

Birinci Nesil Yönlendiriciler :

- traditional computers with switching under direct control of CPU
- packet copied to system's memory
- speed limited by memory bandwidth (2 bus crossings per datagram)



Network Layer: 4-26

Ders Notu: Switching via Memory

Bu slayt, **bellek üzerinden anahtarlama (switching via memory)** yaklaşımını açıklamaktadır. Bu yöntem, özellikle **ilk nesil yönlendiricilerde** kullanılan temel iç anahtarlama yapılarından biridir. Mantık oldukça doğrudandır: giriş portuna gelen paket önce sistem belleğine kopyalanır, ardından uygun çıkış portuna gönderilmek üzere bellekten okunur. Yani yönlendirici içindeki paket hareketi, giriş portu ile çıkış portu arasında doğrudan değil, ortak bir bellek üzerinden gerçekleşir. Slaytta da ilk nesil yönlendiricilerin, anahtarlama işlemi CPU'nun doğrudan kontrolü altındaki geleneksel bilgisayarlar gibi çalıştığı belirtilmektedir.

Bu yapıyı anlamak için süreci adım adım düşünmek yararlıdır. Bir paket giriş portuna gelir. Giriş portu paketi alır ve hedef çıkış portunu belirler. Ancak paket hemen çıkış portuna iletilmez; önce **system bus** üzerinden ortak belleğe taşınır. Daha sonra uygun çıkış portu aynı paketi yine sistem veri yolu üzerinden bellekten alır ve dış bağlantıya gönderir. Bu nedenle bir datagram yönlendirici içinde taşınırken aslında iki kez iç aktarım yapmış olur: bir kez **girişten belleğe**, bir kez de **bellekten çıkışa**. Slayttaki "2 bus crossings per datagram" ifadesi tam olarak bunu anlatmaktadır.

Bu yöntem mimari olarak basit ve anlaşılırdır. Çünkü yönlendiricinin içindeki temel koordinasyon noktası ortak bellektir. Giriş portları paketleri belleğe yazar, çıkış portları oradan okur. İlk dönem yönlendiriciler açısından bu yaklaşım doğal bir çözümdü; çünkü donanım karmaşıklığını azaltıyor ve genel amaçlı bilgisayar mimarisine oldukça yakın bir çalışma modeli sunuyordu. Ancak bu sadelik aynı zamanda önemli bir performans sınırlaması da getirir.

Slaytta vurgulanan temel sorun, hızın **memory bandwidth**, yani bellek bant genişliği tarafından sınırlandırılmasıdır. Çünkü bütün paketler aynı ortak belleği ve aynı iç veri yolunu kullanmak zorundadır. Paket sayısı arttığında ya da port hızları yükseldiğinde, bellek ve bus tüm trafiği taşıyamaz hale gelir. Özellikle her paket için iki ayrı bus geçişi gerektiğinden, iç mimari üzerindeki yük daha da artar. Sonuç olarak, yönlendiricinin toplam aktarım kapasitesi giriş ve çıkış portlarının teorik hızından çok, ortak belleğin ne kadar hızlı yazma-okuma yapabildiğine bağlı hale gelir. Kitapta da memory-based switching yaklaşımının performansının bellek bant genişliği ile sınırlı olduğu açıkça belirtilmektedir.

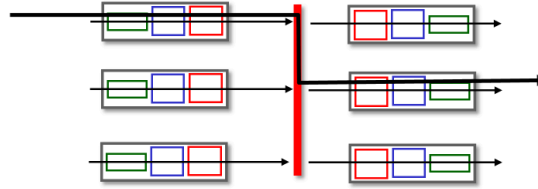
Şekilde bu yapı açık biçimde gösterilmektedir. Solda giriş portu, ortada bellek, sağda çıkış portu ve altta bunları birbirine bağlayan **system bus** yer almaktadır. Paket önce giriş portundan bus aracılığıyla belleğe gider, sonra aynı ortak iç yapı üzerinden çıkış portuna ulaşır. Bu çizim, darboğazın neden ortaya çıktığını sezgisel olarak da görünür kılar: yönlendiricinin içindeki tüm trafik aynı ortak yolu ve aynı belleği paylaşmaktadır. Birden fazla giriş aynı anda aktif olduğunda, iç kaynaklar için rekabet başlar.

Bu yapının veri düzlemi açısından anlamı şudur: yönlendirme kararını vermek tek başına yeterli değildir; paketi içeride taşıyabilmek de gerekir. Memory-based switching'de bu taşıma merkezi bir kaynak üzerinden yapıldığı için, ölçek büyüdükçe performans düşebilir. Bu nedenle daha sonra geliştirilen bus tabanlı ve özellikle interconnection network tabanlı yapılar, bu merkezi darboğazı azaltmak için ortaya çıkmıştır. Başka bir ifadeyle, switching via memory yaklaşımı tarihsel olarak önemli ve kavramsal olarak öğretici olsa da, yüksek kapasiteli modern yönlendiriciler için sınırlayıcıdır.

Özetle bu slaytın ana mesajı şudur: **switching via memory**, paketin önce ortak belleğe yazıldığı, sonra oradan uygun çıkış portuna aktarıldığı bir iç anahtarlama yöntemidir. İlk nesil yönlendiricilerde kullanılmıştır ve mimarisi sadedir; ancak her datagram için iki bus geçişi gerektiğinden, toplam hız **bellek bant genişliği** ile sınırlanır. Bu nedenle port sayısı ve trafik arttıkça bu yapı kolayca darboğaz oluşturabilir.

Ortak Veri Yolu Üzerinden Anahtarlama

- Datagram, giriş portu belleğinden çıkış portu belleğine **shared bus** üzerinden taşınır.
- **bus contention**: “Contention”, aynı kaynağı aynı anda birden fazla bileşenin kullanmak istemesi durumudur.
- 32 Gbps bus, Cisco 5600: Erişim yönlendiricileri için yeterli hız



Network Layer: 4-27

Ders Notu: Switching via a Bus

Bu slayt, yönlendirici içindeki ikinci temel anahtarlama yaklaşımını, yani **ortak veri yolu üzerinden anahtarlama (switching via a bus)** yöntemini açıklamaktadır. Bu yapıda datagram, giriş portu belleğinden çıkış portu belleğine **shared bus** üzerinden taşınır. Yani paket önce merkezi sistem belleğine yazılıp sonra okunmaz; bunun yerine giriş portu, paketi ortak iç veri yoluna koyar ve ilgili çıkış portu bu paketi oradan alır. Bu yönüyle bus tabanlı yapı, bellek temelli yapıya göre daha doğrudan bir iç aktarım modeli sunar.

Bu yapıyı süreç olarak düşünürsek, ilk adım yine aynıdır: paket giriş portuna gelir, başlığı incelenir ve hangi çıkış portuna gitmesi gerektiği belirlenir. Ancak sonra paket ortak bir **bus** üzerine yerleştirilir. Bus, yönlendirici içindeki giriş ve çıkış portlarının paylaştığı ortak iletim yoludur. Paketin hedeflendiği çıkış portu, bu veri yolundaki paketi algılar ve kendi çıkış belleğine alır. Böylece paket içeride girişten çıkışa taşınmış olur. Slayttaki çizim de bunu göstermektedir: soldaki giriş portlarından biri ortak kırmızı bus üzerinden sağdaki uygun çıkış portuna veri aktarmaktadır.

Bu yaklaşımın önemli avantajı, memory-based switching'e göre bazı durumlarda daha verimli olmasıdır. Paket, merkezi bir sistem belleğine iki aşamalı kopyalama ile taşınmak yerine, girişten çıkışa ortak iç yol üzerinden aktarılır. Bu, mimariyi sade tutarken bazı ek kopyalama maliyetlerini azaltabilir. Özellikle erişim yönlendiricileri gibi daha sınırlı ölçekli cihazlarda bu yöntem pratik ve yeterli bir çözüm olabilir.

Ancak slaytın asıl vurguladığı sınırlama **bus contention** kavramıdır. “Contention”, aynı kaynağı aynı anda birden fazla bileşenin kullanmak istemesi durumudur. Burada paylaşılan kaynak bus’tır. Eğer aynı anda birden fazla giriş portu paketini bus üzerine koymak isterse, bunların hepsi eşzamanlı olarak aktarılamaz. Çünkü bus ortak bir iletim ortamıdır ve belirli bir anda sınırlı kapasiteye sahiptir. Sonuç olarak switching speed, yani iç anahtarlama hızı, doğrudan **bus bandwidth** ile sınırlanır. Bu nedenle bus tabanlı yapının başarımı, ortak veri yolunun toplam taşıma gücünü aşamaz.

Bu sınırlamayı sezgisel olarak şöyle düşünebiliriz: Birden fazla yolcunun tek şeritli bir köprüden geçmeye çalıştığını varsayalım. Yolcular hazır olsa bile aynı anda hepsi geçemez; köprünün kapasitesi geçiş hızını belirler. Bus da yönlendirici içinde buna benzer şekilde çalışır. Giriş portları ne kadar hızlı olursa olsun, ortak veri yolu o hızları birlikte taşıyamıyorsa içeride bekleme oluşur. Bu durumda paketler giriş tarafında sıraya girebilir ve toplam verim düşer.

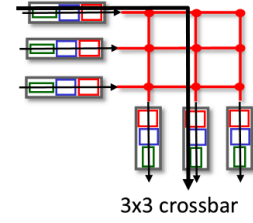
Slaytta verilen **32 Gbps bus, Cisco 5600** örneği, bu yöntemin özellikle **access routers** için yeterli olabildiğini göstermektedir. Yani bus tabanlı anahtarlama tüm yönlendiriciler için yetersiz değildir; belirli ölçeklerde ve belirli trafik yükleri altında gayet uygun olabilir. Sorun, port sayısı arttığında ve çok yüksek toplam iç kapasite gerektiğinde ortaya çıkar. Böyle durumlarda ortak bus, tüm trafiğin tek darboğazı haline gelir. Bu yüzden daha büyük ve daha yüksek performanslı yönlendiricilerde interconnection network gibi daha paralel yapılar tercih edilir.

Bu slaytın veri düzlemi açısından önemi şudur: yönlendirici içinde yalnızca doğru çıkışın bulunması yetmez, bu çıkışa **yeterli hızda** ulaşmak da gerekir. Bus tabanlı yapıda karar verme işlemi hızlı olsa bile, aynı anda çok sayıda aktarım isteği olduğunda ortak veri yolu sınırlayıcı hale gelir. Bu nedenle veri düzlemi performansını değerlendirirken yalnızca lookup hızını değil, iç taşıma mimarisinin paylaşımlı mı yoksa paralel mi olduğunu da dikkate almak gerekir.

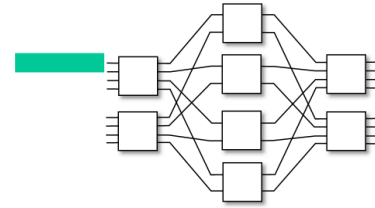
Özetle bu slaytın ana mesajı şudur: **switching via a bus**, paketin giriş portu belleğinden çıkış portu belleğine ortak bir veri yolu üzerinden taşındığı anahtarlama yaklaşımıdır. Yapı basit ve bazı yönlendiriciler için yeterince hızlıdır; ancak aynı bus’ı birden fazla paket paylaşmak zorunda olduğu için **bus contention** ortaya çıkar ve toplam anahtarlama hızı **bus bant genişliği** ile sınırlanır.

Switching via interconnection network

- Crossbar, Clos networks, other interconnection nets initially developed to connect processors in multiprocessor
- **multistage switch**: $n \times n$ switch from multiple stages of smaller switches
- **exploiting parallelism**:
 - fragment datagram into fixed length cells on entry
 - switch cells through the fabric, reassemble datagram at exit



3x3 crossbar



8x8 multistage switch
built from smaller-sized switches

Network Layer: 4-28

Ders Notu: Switching via Interconnection Network

Bu slayt, yönlendirici içindeki en gelişmiş anahtarlama yaklaşımlarından biri olan **interconnection network** tabanlı switching yapısını açıklamaktadır. Önceki slaytlarda bellek ve ortak bus temelli yöntemlerde iç anahtarlama kapasitesinin merkezi bir kaynak tarafından sınırlandırıldığı görülmüştü. Burada ise amaç, bu merkezi darboğazları azaltmak ve aynı anda birden fazla paket aktarımına izin verecek daha **paralel** bir iç yapı kurmaktır. Slaytta verilen **crossbar**, **Clos network** ve diğer bağlantı ağları bu düşüncenin örnekleridir. Bunların ilk olarak çok işlemcili sistemlerde işlemcileri birbirine bağlamak için geliştirildiği, daha sonra yüksek hızlı yönlendirici mimarilerine uyarlandığı belirtilmektedir.

Bu yapının temel fikri şudur: girişler ile çıkışlar arasında tek bir ortak bellek ya da tek bir ortak bus yerine, birden fazla eşzamanlı iç yol oluşturulur. Böylece bir giriş portundan bir çıkış portuna yapılan aktarım, başka bir giriş-çıkış çiftinin aktarımını her zaman engellemek zorunda kalmaz. Bu da yönlendiricinin iç taşıma kapasitesini artırır. Özellikle port sayısı büyüdükçe ve birden fazla akış aynı anda aktif olduğunda, interconnection network yapıları bus tabanlı sistemlere göre çok daha iyi ölçeklenir. Bu nedenle yüksek kapasiteli yönlendiricilerde tercih edilen yaklaşım genellikle budur. Kitapta da switching fabric türleri arasında interconnection network'lerin, yüksek performans için önemli bir seçenek olduğu vurgulanmaktadır.

Slayttaki ilk görsel, **3x3 crossbar** yapısını göstermektedir. Crossbar'da her giriş, her çıkışla mantıksal olarak bağlanabilir. Böylece uygun bağlantılar kurulduğunda, birden

fazla giriş farklı çıkışlara aynı anda veri gönderebilir. Buradaki avantaj, aktarımın tek bir ortak yol üzerinden değil, daha esnek bir anahtarlama matrisi üzerinden yapılmasıdır. Ancak giriş ve çıkış sayısı büyüdükçe tam bir crossbar yapısının maliyeti ve karmaşıklığı da artar. Bu nedenle daha büyük sistemlerde genellikle çok aşamalı yapılar kullanılır.

Bu noktada slaytta geçen **multistage switch** kavramı devreye girer. Buradaki fikir, büyük bir $n \times n$ anahtarlama yapısını doğrudan tek parça olarak kurmak yerine, birden fazla aşamada yer alan daha küçük anahtarlama elemanlarından oluşturmaktır. Şekilde görülen **8x8 multistage switch**, daha küçük anahtarlama bloklarının birden çok aşamada birleştirilmesiyle elde edilmiştir. Böylece hem tam çapraz bağ yapısının maliyetinden kaçınılır hem de yüksek derecede paralellik korunur. Yani büyük anahtarlama sistemi, daha küçük ve yönetilebilir parçaların birleşiminden oluşturulur. Slayttaki “ $n \times n$ switch from multiple stages of smaller switches” ifadesi tam olarak bunu anlatmaktadır.

Slaytın en önemli vurgularından biri de **exploiting parallelism**, yani paralellikten yararlanma fikridir. Burada datagram yönlendiriciye girdiğinde, tek parça halinde taşınmak yerine sabit uzunluklu küçük parçalara, yani **cells**'lere bölünebilir. Bu parçalama girişte yapılır; sonra bu sabit boyutlu hücreler switching fabric içinde paralel yollar üzerinden taşınır; en sonunda çıkışta tekrar birleştirilerek orijinal datagram elde edilir. Bu yöntem, iç anahtarlama ağının daha verimli kullanılmasını sağlar. Çünkü sabit boyutlu hücrelerle çalışmak, değişken uzunluklu paketlere göre anahtarlama planlamasını kolaylaştırır ve paralel aktarımı daha düzenli hale getirir. Slaytta da datagramın girişte sabit uzunluklu hücrelere bölündüğü, fabric içinden geçirildiği ve çıkışta yeniden birleştirildiği açıkça belirtilmektedir.

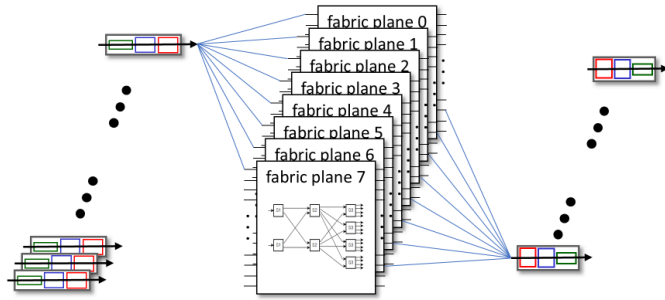
Bu yaklaşımın veri düzlemi açısından önemi büyüktür. Çünkü yönlendirici iç performansını artırmanın yolu sadece daha hızlı lookup yapmak değildir; aynı zamanda aynı anda birden fazla paketi içeride taşıyabilmektir. Interconnection network tabanlı switching fabric'ler tam da bunu sağlar. Bellek tabanlı yapılarda bellek bant genişliği, bus tabanlı yapılarda ise ortak veri yolu kapasitesi darboğaz oluştururken, burada amaç yükü daha fazla iç yola dağıtarak bu sınırları gevşetmektir. Sonuç olarak daha yüksek switching rate elde edilir ve yönlendirici daha fazla port ve daha yüksek trafik hacmi ile başa çıkabilir.

Özetle bu slaytın ana mesajı şudur: **interconnection network** tabanlı switching, girişlerle çıkışlar arasında paralel iç yollar kurarak yüksek hızlı yönlendiricilerde ölçeklenebilir anahtarlama sağlar. **Crossbar** yapılar doğrudan bağlantı matrisi sunarken, **multistage switch** yapılar daha küçük anahtarlama bloklarını birleştirerek büyük ölçekli sistemleri ekonomik ve verimli biçimde kurar. Gerekirse datagramlar sabit

uzunluklu hücrelere bölünerek paralellik daha da etkin kullanılır; hücreler fabric içinde taşındıktan sonra çıkışta yeniden birleştirilir.

Switching via interconnection network

- scaling, using multiple switching “planes” in parallel:
 - speedup, scaleup via parallelism
- Cisco CRS router:
 - basic unit: 8 switching planes
 - each plane: 3-stage interconnection network
 - up to 100's Tbps switching capacity



Network Layer: 4-29

Ders Notu: Switching via Interconnection Network

Bu slayt, interconnection network tabanlı anahtarlamanın yalnızca tek bir iç bağlantı yapısıyla sınırlı kalmadığını, gerekirse birden fazla **switching plane** kullanılarak daha da ölçeklenebildiğini göstermektedir. Önceki slaytta, crossbar ve multistage switch gibi yapıların girişler ile çıkışlar arasında paralel iç yollar oluşturduğu görülmüştü. Burada ise bu paralellik bir adım daha ileri taşınmakta ve tek bir fabric yerine birden çok fabric düzlemi aynı anda çalıştırılmaktadır. Böylece hem **speedup** hem de **scaleup**, yani hem hız artışı hem de kapasite büyütme paralellik yoluyla sağlanır.

Buradaki temel fikir şudur: Tek bir anahtarlama düzlemi belirli bir iç kapasite sağlayabilir; ancak çok büyük yönlendiricilerde bu yeterli olmayabilir. Bu durumda paket akışı birden fazla bağımsız switching plane üzerine dağıtılır. Böylece iç trafik tek bir yapıya yüklenmek yerine birden fazla paralel fabric tarafından taşınır. Bu yaklaşım, tek bir ortak bus ya da tek bir anahtarlama ağına dayanan mimarilere göre çok daha yüksek toplam aktarım kapasitesi sağlar. Slayttaki “scaling, using multiple switching planes in parallel” ifadesi tam olarak bunu anlatmaktadır.

Bu yapı sezgisel olarak çok şeritli bir otoyola benzetilebilir. Tek şeritli bir yolda trafik yoğunluğu arttığında bekleme ve yavaşlama kaçınılmaz hale gelir. Ancak aynı taşıma işi birden fazla paralel şerit üzerinden yürütülürse toplam kapasite önemli ölçüde artar.

Switching plane mantığı da buna benzer: iç anahtarlama işi birden fazla fabric düzlemine bölünerek yönlendiricinin toplam throughput'u yükseltilir. Bu, özellikle çok sayıda portun aynı anda aktif olduğu büyük omurga yönlendiricilerinde kritik bir tasarım ilkesidir.

Slaytta verilen **Cisco CRS router** örneği bu yaklaşımın gerçek bir sistemde nasıl uygulandığını göstermektedir. Burada temel birim olarak **8 switching plane** kullanıldığı belirtilmektedir. Ayrıca her bir plane'in kendi içinde **3-stage interconnection network** olduğu ifade edilmektedir. Bu, önceki slaytta anlatılan çok aşamalı anahtarlama yapısının burada bir kez değil, sekiz kez paralel biçimde kullanıldığı anlamına gelir. Yani ölçeklenebilirlik sadece tek bir çok aşamalı ağ kurarak değil, bu ağlardan birden fazlasını yan yana çalıştırarak sağlanmaktadır. Sonuç olarak sistemin toplam switching capacity değeri **yüzlerce Tbps** düzeyine kadar çıkabilmektedir.

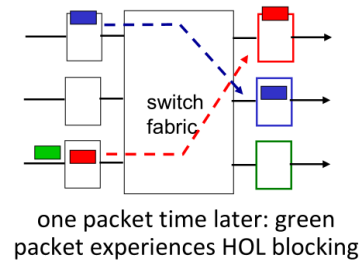
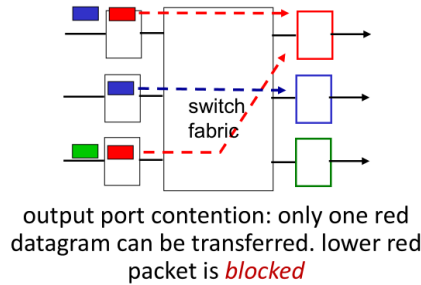
Şekilde de bu açıkça görülmektedir. Soldaki giriş akışları, tek bir fabric kutusuna değil, üst üste yığılmış birden fazla **fabric plane** üzerine yönlendirilmektedir. Sağ tarafta ise bu paralel düzlemlerden gelen veri uygun çıkışlara taşınmaktadır. Bu çizim, paketin tek bir iç yoldan değil, bir grup paralel iç ağ üzerinden işlenebildiğini göstermektedir. Böylece hem daha fazla eşzamanlılık sağlanır hem de tek bir fabric'in sınırlayıcı olması önlenir.

Bu mimarinin veri düzlemi açısından önemi büyüktür. Çünkü yönlendirici performansı yalnızca giriş portunda yapılan forwarding lookup işlemiyle ya da tek bir switching fabric'in hızına bakılarak değerlendirilemez. Çok yüksek kapasiteli cihazlarda asıl mesele, aynı anda çok büyük miktarda trafiğin içeride nasıl taşınacağıdır. Multiple switching planes yaklaşımı, bu soruna paralellik tabanlı bir çözüm getirir. Her plane kendi anahtarlama işini yürüttüğü için toplam iç kapasite plane sayısı ile birlikte artar. Bu da yönlendiricinin daha fazla port, daha yüksek hat hızı ve daha büyük trafik yüküyle başa çıkmasını sağlar.

Özetle bu slaytın ana mesajı şudur: **interconnection network** tabanlı anahtarlama, yalnızca tek bir fabric ile değil, birden fazla **switching plane** paralel çalıştırılarak çok daha büyük ölçeklere taşınabilir. Bu sayede hem hız hem de kapasite paralellik yoluyla artırılır. Cisco CRS örneğinde olduğu gibi, her biri çok aşamalı bir iç bağlantı ağı olan birden fazla plane birlikte kullanılarak yüzlerce terabit/saniye düzeyinde anahtarlama kapasitesine ulaşmak mümkün olur.

Input port queuing

- If switch fabric slower than input ports combined -> queueing may occur at input queues
 - queueing delay and loss due to input buffer overflow!
- **Head-of-the-Line (HOL) blocking**: queued datagram at front of queue prevents others in queue from moving forward



Network Layer: 4-30

Ders Notu: Input Port Queueing

Bu slayt, yönlendirici içinde kuyruklamanın neden yalnızca çıkış portlarında değil, **giriş portlarında** da oluşabildiğini göstermektedir. Temel neden şudur: Eğer **switch fabric**'in iç aktarım kapasitesi, giriş portlarının birlikte ürettiği toplam trafik hızından daha düşükse, gelen paketler hemen içeride taşınamaz ve giriş tarafında beklemek zorunda kalır. Bu durumda giriş tamponlarında birikme oluşur; bunun sonucu olarak **queueing delay** yani kuyruk gecikmesi ortaya çıkar. Trafik daha da artarsa tampon dolabilir ve **input buffer overflow** nedeniyle paket kaybı yaşanabilir. Kitapta da, yönlendirici içindeki anahtarlama yapısı yeterince hızlı değilse kuyruklama ve buna bağlı gecikme ile kaybın oluşabileceği vurgulanmaktadır.

Slaytın asıl vurgusu ise **Head-of-the-Line (HOL) blocking** problemidir. Bu problem, giriş kuyruğunun en önündeki bir datagramın ilerleyememesi nedeniyle arkasındaki diğer paketlerin de beklemek zorunda kalması durumudur. Burada önemli olan nokta şudur: Arkadaki paketlerin gitmek istediği çıkış aslında boş olabilir; yani onlar teknik olarak ilerleyebilecek durumdadır. Ancak giriş kuyruğu sırayla çalıştığı için, en öndeki paket geçemediği sürece arkadakiler de hareket edemez. Bu yüzden sorun yalnızca bir paketin gecikmesi değil, onun arkasındaki tüm akışın gereksiz yere yavaşlamasıdır. Slayttaki “queued datagram at front of queue prevents others in queue from moving forward” ifadesi tam olarak bunu anlatmaktadır.

Soldaki şekil bu durumu sezgisel biçimde göstermektedir. Birden fazla giriş portundan gelen iki kırmızı paket aynı çıkış portuna gitmek istemektedir. Ancak aynı anda yalnızca

bir kırmızı datagram ilgili çıkışa aktarılabilir. Bu nedenle alttaki kırmızı paket engellenir. Buradaki sorun, bu paketin kendi başına beklemesiyle bitmez. Çünkü o paket giriş kuyruğunun başında kaldığında, onun arkasında bulunan ve belki de başka bir çıkış portuna gitmek isteyen yeşil paket de beklemek zorunda kalır. Şeklin altındaki açıklama da tam olarak bunu söyler: çıkış portu çekişmesi vardır ve yalnızca bir kırmızı datagram aktarılabilir; diğeri bloklanır.

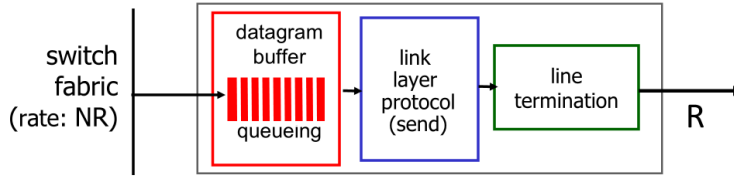
Sağdaki şekil, bir paket zamanı sonra ortaya çıkan sonucu göstermektedir. Bu sefer yeşil paket aslında ilerleyebilecek durumda olmasına rağmen, önündeki kırmızı paket yüzünden hâlâ beklemektedir. İşte bu, klasik **HOL blocking** örneğidir. Yani problem artık çıkış portunun anlık meşgul olması değil; giriş kuyruğunun başındaki paketin, arkasındaki bağımsız paketlerin ilerlemesini de durdurmasıdır. Bu nedenle HOL blocking, yönlendirici iç performansını düşüren önemli bir verimsizliktir.

Bu konunun veri düzlemi açısından önemi büyüktür. Çünkü ilk bakışta kuyruklama yalnızca trafik yoğun olduğunda ortaya çıkan sıradan bir bekleme gibi görünebilir. Oysa HOL blocking gösteriyor ki, giriş tarafındaki tek bir tıkanma tüm kuyruğun verimini etkileyebilir. Bu, switch fabric'in toplam kapasitesi yeterli görünse bile pratikte performansın daha düşük olmasına yol açabilir. Başka bir deyişle, iç anahtarlama yapısında yalnızca toplam hız değil, paketlerin giriş kuyruklarından nasıl çıkarıldığı da önemlidir.

Bu problem aynı zamanda neden daha gelişmiş anahtarlama ve zamanlama mekanizmalarına ihtiyaç duyulduğunu da açıklar. Eğer giriş kuyrukları basit bir FIFO mantığıyla çalışırsa, ön sıradaki blokaj arka sıradaki bağımsız akışları da gereksiz yere bekletir. Modern yönlendirici tasarımlarında bu tür verimsizlikleri azaltmak için daha gelişmiş kuyruk yönetimi ve eşleştirme teknikleri kullanılır. Böylece giriş tarafındaki tek bir tıkanıklığın tüm kuyruğu felç etmesi önlenmeye çalışılır.

Özetle bu slaytın ana mesajı şudur: Eğer **switch fabric**, giriş portlarının toplam trafik hızından yavaşsa, **input port queueing** ortaya çıkar; bu da gecikme ve tampon taşması nedeniyle paket kaybına yol açabilir. Buna ek olarak **HOL blocking**, giriş kuyruğunun başındaki bir paketin ilerleyememesi yüzünden arkasındaki diğer paketlerin de beklemek zorunda kalmasıdır. Bu durum, yönlendirici içindeki verimi düşüren önemli bir performans problemidir.

Output port queuing



This is a really important slide

- **Buffering** required when datagrams arrive from fabric faster than link transmission rate. **Drop policy**: which datagrams to drop if no free buffers?



Datagrams can be lost due to congestion, lack of buffers

- **Scheduling discipline** chooses among queued datagrams for transmission



Priority scheduling – who gets best performance, network neutrality

Network Layer: 4-31

Ders Notu: Output Port Queueing

Bu slayt, yönlendirici içinde en kritik darboğaz noktalarından birini, yani **çıkış portu kuyruklamasını (output port queueing)** açıklamaktadır. Paketler switching fabric'den çıktıktan sonra hemen fiziksel hatta gönderilemeyebilir. Bunun nedeni, çıkış bağlantısının iletim hızının sınırlı olmasıdır. Slaytta da görüldüğü gibi switching fabric tarafı yüksek bir iç hızla paket getirebilirken, çıkış hattı yalnızca **R** hızında veri gönderebilir. Eğer fabric'den gelen datagramlar, çıkış linkinin iletim hızından daha hızlı gelirse, paketler çıkış portundaki **datagram buffer** içinde beklemek zorunda kalır. İşte bu durum output port queueing olarak adlandırılır. Kitapta da output port'un, switching fabric'den gelen paketleri depoladığı ve ardından bağlantı katmanı ile fiziksel katman işlevleriyle dışarı gönderdiği belirtilmektedir.

Buradaki temel fikir şudur: yönlendirici içindeki karar verme ve iç aktarım işlemleri çok hızlı olabilir; ancak en sonunda paket fiziksel bir bağlantı üzerinden iletilecektir ve bu bağlantının kapasitesi sınırlıdır. Yani içerideki hız ile dışarıya çıkış hızı aynı olmak zorunda değildir. Çıkış portu bu nedenle yalnızca “paketi dışarı veren kapı” değildir; aynı zamanda trafik yoğunlaştığında geçici depolama yapan bir tampon noktasıdır. Slayttaki şekil de bunu açık biçimde göstermektedir: switch fabric'den gelen trafik önce kuyrukta birikir, ardından **link layer protocol (send)** tarafından işlenir ve son olarak **line termination** üzerinden hatta iletilir.

Bu kuyruklamanın ilk önemli sonucu **buffering** gereksinimidir. Eğer paketler anlık olarak çıkış hattının taşıyabileceğinden daha hızlı geliyorsa, tampon alanı kullanılmadan

sistem çalışmaz. Ancak tamponlar sonsuz değildir. Slaytta bu yüzden **drop policy** sorusu ortaya atılmaktadır: Eğer boş tampon kalmamışsa, hangi datagramlar atılacaktır? Bu soru oldukça önemlidir; çünkü ağlarda paket kaybı çoğu zaman yalnızca fiziksel hata nedeniyle değil, **congestion** ve **buffer yetersizliği** nedeniyle ortaya çıkar. Slaytta da açıkça “Datagrams can be lost due to congestion, lack of buffers” ifadesi yer almaktadır. Yani burada paket kaybı, yönlendiricinin içsel kaynaklarının sınırlı olmasının doğrudan sonucudur.

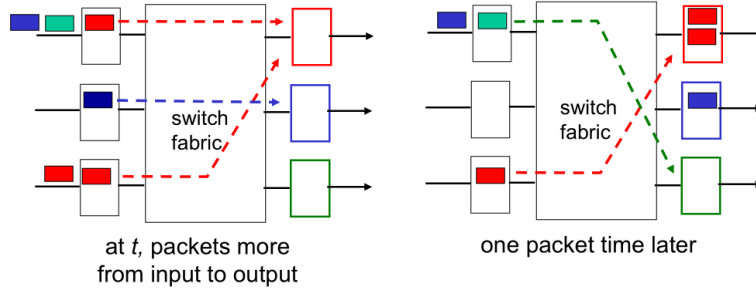
İkinci önemli konu ise **scheduling discipline** kavramıdır. Kuyrukta birden fazla datagram varken, çıkış hattı bunları aynı anda gönderemez; dolayısıyla sıradaki iletim için bir seçim yapılmalıdır. Slaytta bu seçim mekanizmasının queued datagramlar arasından hangisinin iletileceğini belirlediği söylenmektedir. İlk bakışta bu yalnızca teknik bir sıraya koyma işlemi gibi görünebilir. Oysa bu karar, ağ performansı açısından çok önemlidir. Çünkü hangi paketlerin önce gönderileceği; gecikmeyi, adaleti, farklı trafik türlerinin aldığı hizmet kalitesini ve hatta bazı durumlarda politika tartışmalarını etkiler.

Slaytın sağ alt kısmında bu nedenle **priority scheduling** ve **network neutrality** kavramlarına dikkat çekilmektedir. Eğer bazı paketler diğerlerine göre öncelikli işlenirse, bazı akışlar daha iyi performans elde eder. Örneğin gerçek zamanlı ses ya da video trafiği daha düşük gecikme alacak şekilde önceliklendirilebilir. Ancak bu tür önceliklendirme, hangi kullanıcı ya da uygulamanın “en iyi hizmeti” alacağı sorusunu da beraberinde getirir. Bu nedenle çıkış portu zamanlaması sadece teknik değil, aynı zamanda yönetim ve politika boyutu olan bir konudur. Slaytta “who gets best performance” ve “network neutrality” ifadeleriyle buna işaret edilmektedir.

Bu slaytın neden “çok önemli” olarak işaretlendiği de burada anlaşılır. Çünkü yönlendiricideki gerçek darboğazlardan biri çoğu zaman tam da çıkış portudur. Paketler doğru yönlendirilmiş, içeride taşınmış ve çıkışa ulaşmış olabilir; fakat çıkış hattı sınırlı hızda çalıştığı için asıl bekleme burada oluşabilir. Dolayısıyla veri düzleminde performans analizi yapılırken yalnızca switching fabric’in hızı ya da forwarding lookup başarımı değil, **çıkış kuyruklarının nasıl yönetildiği** de incelenmelidir. Gecikme, kayıp ve hizmet kalitesi gibi sonuçlar büyük ölçüde burada şekillenir.

Özetle bu slaytın ana mesajı şudur: **Output port queueing**, datagramlar switching fabric’den çıkış hattının iletim hızından daha hızlı geldiğinde ortaya çıkar. Bu durumda çıkış portunda tamponlama gerekir. Tamponlar dolarsa bazı paketlerin atılması gerekir; bu da **drop policy** sorununu doğurur. Kuyrukta bekleyen paketler arasından hangisinin önce gönderileceğini ise **scheduling discipline** belirler. Dolayısıyla çıkış portu kuyruklaması, hem paket kaybının hem de hizmet kalitesi ve önceliklendirme tartışmalarının merkezindeki temel konulardan biridir.

Output port queuing



- buffering when arrival rate via switch exceeds output line speed
- *queueing (delay) and loss due to output port buffer overflow!*

Network Layer: 4-32

Ders Notu: Output Port Queueing

Bu slayt, çıkış portu kuyruklamasının nasıl oluştuğunu zaman içinde iki adım halinde göstermektedir. Temel fikir şudur: **switch fabric** üzerinden belirli bir çıkış portuna gelen paketlerin geliş hızı, o çıkış bağlantısının iletim hızını aştığında, paketler hemen gönderilemez ve çıkış portunda birikmeye başlar. Bu nedenle çıkış tarafında **buffering** gerekir. Ders materyalinde de buffering'ın, switch üzerinden gelen datagramların çıkış hattı hızından daha hızlı geldiği durumda gerekli olduğu açıkça belirtilmektedir.

Soldaki şekilde, belirli bir anda birden fazla girişten gelen paketlerin aynı çıkış portlarına yönlendirildiği görülmektedir. Özellikle kırmızı çıkış portuna birden fazla paketin yönlendiği durumda, bu paketlerin hepsi aynı anda hatta çıkamaz. Çünkü çıkış portunun önündeki fiziksel bağlantı tek bir hat hızında çalışır. Yani switch fabric içeride paketleri çok hızlı taşıyabiliyor olsa bile, son aşamada çıkış hattı bu hızın altında kalıyorsa darboğaz oluşur. Bu durumda paketler çıkış portundaki tampon bellekte sıraya girer.

Sağdaki şekilde, **bir paket zamanı sonra** ne olduğuna bakılmaktadır. Kırmızı çıkışta tamponda birikme olduğu görülmektedir; çünkü o çıkışa yönelen paket sayısı, hattın bir zaman aralığında taşıyabileceğinden fazladır. Buna karşılık başka bir paket farklı bir çıkışa yönelip gönderilebilmektedir. Bu görsel, kuyruklamanın her zaman tüm yönlendiricide eşit biçimde oluşmadığını; çoğu zaman belirli çıkış portlarında, o porta olan talep yoğunlaştığında ortaya çıktığını göstermektedir. Yani sorun genel olarak “yönlendirici yavaş” olması değil, belirli bir çıkış kaynağının o anda aşırı talep görmesidir.

Slaytın altındaki iki madde bu durumu netleştirmektedir. Birincisi, **buffering when arrival rate via switch exceeds output line speed** ifadesidir. Bunun anlamı, switch fabric'den çıkış portuna gelen trafik oranı, portun dışarıya veri gönderme hızını aştığında tamponlama yapılmasının zorunlu hale gelmesidir. İkincisi ise bunun sonucudur: **queueing delay and loss due to output port buffer overflow**. Yani tampon yeterliyse paketler bekler ve gecikme oluşur; tampon dolarsa artık yeni gelen bazı paketler saklanamaz ve düşürülür. Bu nedenle çıkış portu kuyruklaması hem gecikmenin hem de paket kaybının temel nedenlerinden biridir.

Burada önemli olan nokta, kaybın fiziksel hatadan değil çoğu zaman **congestion**, yani yoğunluktan kaynaklanmasıdır. Paketler doğru şekilde yönlendirilmiş ve doğru çıkışa ulaşmış olabilir; ancak çıkış portunda yeterli tampon yoksa ya da gelen trafik çok yoğunsa bazı paketler kaçınılmaz olarak atılır. Bu nedenle ağ performansı değerlendirilirken yalnızca yönlendirme doğruluğu değil, çıkış tamponlarının ne kadar dolduğu ve nasıl yönetildiği de dikkate alınmalıdır.

Bu slayt, bir önceki çıkış portu kuyruklaması slaydındaki kavramları görselleştirerek pekiştirmektedir. Önceki slaytta buffering, drop policy ve scheduling discipline kavramları tanıtılmıştı. Bu slaytta ise özellikle ilk ikisinin mekanizması açık hale gelmektedir: Paketler aynı çıkışta birikir, bekleme süresi artar, tampon taşarsa paket kaybı oluşur. Scheduling konusu burada açıkça çizilmemiş olsa da, çıkış portunda biriken paketler arasından hangisinin önce gönderileceği sorusu yine geçerlidir.

Özetle bu slaytın ana mesajı şudur: Bir çıkış portuna switch fabric üzerinden gelen paketlerin hızı, çıkış hattının iletim hızını aştığında **output port queueing** oluşur. Önce paketler tamponda bekler, bu da **queueing delay** üretir. Trafik yoğunluğu sürer ve tampon kapasitesi aşılsa **output buffer overflow** nedeniyle paket kaybı meydana gelir.

Ne Kadar Tampon Belleği (Buffer)

- RFC 3439 Yaygın Yaklaşım: “Tipik” RTT (örneğin 250 ms) eşit ortalama arabellek süresi x bağlantı kapasitesi C 'ne eşittir.
 - e.g., $C = 10$ Gbps link: 2.5 Gbit buffer
- Modern Yaklaşım: Eğer tek bir akış değil, aynı bağlantıyı paylaşan **N adet akış** varsa, gerekli tampon miktarı

$$\frac{RTT \cdot C}{\sqrt{N}}$$

- **çok fazla buffering'in gecikmeyi artırabilir** (özellikle ev tipi yönlendiricilerde)
 - Uzun RTT'ler : Gerçek zamanlı ses, video görüşmesi veya çevrim içi oyun gibi uygulamalar için düşük gecikme çok önemlidir; büyük tamponlar nedeniyle oluşan uzun RTT değerleri bu uygulamaları ciddi biçimde olumsuz etkiler.
 - Gecikme tabanlı tıkanıklık kontrolü yaklaşımlarında amaç, darboğaz linki **tam dolu değil ama yeterince meşgul** tutmaktır.

Network Layer: 4-33

Ders Notu: How Much Buffering?

Bu slayt, bir yönlendirici ya da ağ cihazında **ne kadar tampon belleğin (buffer)** yeterli olduğu sorusunu ele almaktadır. Önceki slaytlarda, çıkış portu kuyruklamasının paketlerin beklemesine ve tampon dolduğunda paket kaybına yol açtığı görülmüştü. Ancak burada yeni ve daha incelikli soru şudur: Tamponlar az olursa erken kayıp yaşanır; çok fazla olursa bu kez gereksiz gecikme oluşur. Dolayısıyla doğru tampon boyutu, yalnızca “ne kadar çok o kadar iyi” mantığıyla belirlenemez. Slaytın ana mesajı da tam olarak budur: **buffering gerekli ama aşırısı zararlı olabilir**.

Slaytta önce klasik bir kural verilmektedir. **RFC 3439**'daki yaygın yaklaşıma göre, ortalama tampon miktarı yaklaşık olarak **tipik RTT ile bağlantı kapasitesinin çarpımına** eşit alınabilir. Yani bağlantının darboğaz link kapasitesi C , tipik gidiş-dönüş süresi **RTT** ise önerilen tampon büyüklüğü yaklaşık **$RTT \cdot C$** olur. Bunun mantığı şudur: TCP gibi protokoller ağda veri akıtırken, gönderici tarafın geri bildirim alması zaman alır. Bu geri bildirim gecikmesi boyunca bağlantının boş kalmaması için ağ içinde belli miktarda veri tutulabilmelidir. Slayttaki örnekte, **$C = 10$ Gbps** ve tipik RTT yaklaşık **250 ms** kabul edildiğinde gerekli tampon büyüklüğünün **2.5 Gbit** mertebesinde olduğu gösterilmektedir. Bu, yüksek hızlı bağlantılarda tampon gereksiniminin neden çok büyüyebildiğini ortaya koyar.

Ancak slayt burada durmamaktadır; daha yeni bir öneri de verilmektedir. Eğer tek bir akış değil, aynı bağlantıyı paylaşan **N adet akış** varsa, gerekli tampon miktarının **$RTT \cdot C / \sqrt{N}$** biçiminde azaltılabileceği belirtilmektedir. Bu fikir, çok sayıda akışın istatistiksel

olarak birbirini dengelemesi nedeniyle toplam tampon ihtiyacının tek-akış durumuna göre daha az olabileceğine dayanır. Başka bir ifadeyle, çok sayıda TCP akışı aynı bağlantıyı kullanıyorsa, her birinin pencere dalgalanması aynı anda aynı yönde gerçekleşmez; bu nedenle toplam sistemin linki dolu tutmak için tek akışlı senaryodaki kadar büyük bir tampona ihtiyacı olmayabilir. Slayttaki formül tam da bu modern yaklaşımı özetlemektedir.

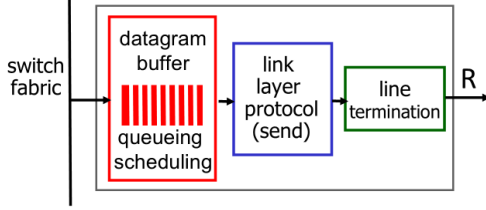
Bununla birlikte slaytın belki de en önemli vurgusu, **çok fazla buffering'in gecikmeyi artırabileceği** uyarısıdır. Özellikle ev yönlendiricilerinde bu durum ciddi bir sorun haline gelebilir. Çünkü tampon çok büyük olduğunda, paketler hemen düşmek yerine uzun süre kuyrukta bekler. Bu, ilk bakışta kaybı azaltıyor gibi görünse de, gerçekte **kuyruk gecikmesini** aşırı artırır. Sonuçta kullanıcı açısından uygulama performansı kötüleşir. Gerçek zamanlı ses, video görüşmesi veya çevrim içi oyun gibi uygulamalar için düşük gecikme çok önemlidir; büyük tamponlar nedeniyle oluşan uzun RTT değerleri bu uygulamaları ciddi biçimde olumsuz etkiler. Slaytta da “long RTTs: poor performance for real-time apps, sluggish TCP response” ifadesiyle bu durum açıkça belirtilmektedir.

Burada önemli bir kavramsal ayrım vardır: Paket kaybını azaltmak her zaman toplam kullanıcı deneyimini iyileştirmez. Eğer kaybı azaltmak uğruna çok büyük tamponlar kullanılırsa, ağ uzun kuyruklarla çalışmaya başlar. Bu durumda TCP geri bildirimleri yavaşlar, akış kontrolü ve tıkanıklık kontrolü tepkileri gecikir, etkileşimli uygulamalar hantallaşır. Bu olgu günümüzde çoğu zaman **bufferbloat** problemiyle ilişkilendirilir. Slayt bunu doğrudan bu terimle söylemese de, ev yönlendiricilerindeki aşırı tamponlamanın gecikmeyi büyütmesi tam olarak aynı probleme işaret etmektedir.

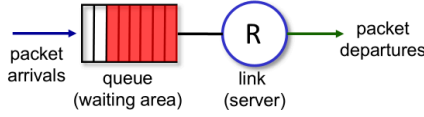
Slaytın son maddesi bu yüzden anlamlıdır: gecikme tabanlı tıkanıklık kontrolü yaklaşımlarında amaç, darboğaz linki **tam dolu değil ama yeterince meşgul** tutmaktır. Yani link boş kalmamalı, fakat kuyruklar da gereksiz yere şişmemelidir. Bu denge önemlidir. Ağ ne tamamen boş çalışmalıdır ne de aşırı tampon nedeniyle gecikme altında ezilmelidir. İdeal durumda bağlantı verimli kullanılırken kuyruklar kontrol altında tutulur.

Özetle bu slaytın ana mesajı şudur: Tampon boyutu seçimi, ağ performansında kritik bir dengedir. Klasik yaklaşım $RTT \cdot C$ kadar tampon önerirken, çok sayıda akış durumunda daha yeni öneri bunu $RTT \cdot C / \sqrt{N}$ seviyesine indirir. Ancak tampon çok büyütülürse gecikme artar, RTT uzar ve özellikle gerçek zamanlı uygulamalar ile TCP'nin tepkiselliği olumsuz etkilenir. Bu nedenle amaç, linki verimli kullanacak kadar ama gecikmeyi şişirmeyecek kadar tampon kullanmaktır.

Tampon Yönetimi



Abstraction: queue



Tampon Yönetimi :

- **drop:** Tampon dolduğunda hangi paket tutulacak ve hangisi atılacak
 - **tail drop:** Tampon doluysa, yeni gelen paket doğrudan düşürülür.
 - **priority:** Tüm paketler eşit kabul edilmez; bazı paketler daha değerli, daha öncelikli ya da daha kritik kabul edilir
- **marking:** Paketleri hemen düşürmek değil; bazı paketleri işaretleyerek ağda tıkanıklık oluşmaya başladığını uç sistemlere bildirmek (ECN, RED)

Network Layer: 4-34

Ders Notu: Buffer Management

Bu slayt, çıkış portunda biriken paketlerin yalnızca “beklediği” bir alan olmadığını, aynı zamanda aktif biçimde yönetilmesi gereken bir kaynak olduğunu göstermektedir. Önceki slaytlarda çıkış portu kuyruklamasının, switch fabric’den gelen paketlerin çıkış hattının hızını aşması durumunda ortaya çıktığı görülmüştü. Burada ise bir adım ileri gidilerek şu soru sorulmaktadır: Kuyruk dolmaya başladığında yönlendirici ne yapacaktır? Hangi paket tutulacak, hangisi düşürülecek, hangisi işaretlenecektir? İşte **buffer management** dediğimiz konu, tam olarak bu kararların nasıl verildiğiyle ilgilidir.

Şeklin sol tarafında çıkış portu yapısı yeniden gösterilmektedir. Switch fabric’den gelen datagramlar önce **datagram buffer** içinde bekler; burada hem **queueing** hem de **scheduling** yapılır. Ardından paketler bağlantı katmanı tarafından gönderilir ve fiziksel hatta çıkar. Bu yapı, tampon belleğin pasif bir depolama alanı olmadığını açıkça gösterir. Tampon, aynı anda hem bekleme alanıdır hem de hangi paketin ne zaman çıkacağına ilişkin kararların uygulandığı yerdir.

Slaytın altındaki soyutlama da bunu daha anlaşılır hale getirir. Burada sistem bir **queue** olarak modellenmiştir: soldan paket gelişleri vardır, ortada bir **bekleme alanı** bulunur, sağda ise belirli bir hızda hizmet veren **link (server)** vardır. Yani paketler rastgele bir ortamda değil, kuyruk kuramına benzeyen düzenli bir yapı içinde işlenmektedir. Bu soyutlama önemlidir; çünkü ağ tıkanıklığı, gecikme ve kayıp gibi olayları anlamak için yönlendiriciyi çoğu zaman bir kuyruk sistemi olarak düşünürüz. Paketler gelir, bekler, sırayla hizmet alır ve çıkar. Eğer geliş hızı hizmet hızını aşarsa kuyruk büyür. Tampon

sınırlıysa, bir noktadan sonra karar verilmesi gerekir: yeni gelen pakete yer açılacak mı, yoksa o paket mi düşecek?

Bu nedenle slaytta buffer management iki temel karar alanı altında özetlenmektedir: **drop** ve **marking**.

İlk alan **drop** politikasıdır. Bu, tampon dolduğunda hangi paketin tutulacağı ve hangisinin atılacağı sorusunu kapsar. Slaytta iki örnek yaklaşım verilmektedir. Birincisi **tail drop** yöntemidir. Bu yaklaşımda tampon doluysa, yeni gelen paket doğrudan düşürülür. Yani kuyrukta bekleyen mevcut paketler korunur, sonradan gelen paket kabul edilmez. Bu yöntem basit ve yaygın bir yaklaşımdır; ancak her zaman en iyi sonuçları vermez. Çünkü tampon tamamen dolana kadar hiçbir erken tepki verilmez, dolduğunda ise ardışık kayıplar oluşabilir.

İkinci yaklaşım ise **priority** temelli düşürmedir. Burada tüm paketler eşit kabul edilmez; bazı paketler daha değerli, daha öncelikli ya da daha kritik kabul edilebilir. Böyle bir durumda yönlendirici, öncelik düzeyine göre hangi paketi tutacağına ve hangisini sileceğine karar verebilir. Bu, örneğin gecikmeye duyarlı trafiğe daha iyi hizmet verilmesini sağlayabilir. Ancak aynı zamanda adalet ve kaynak paylaşımı açısından dikkatli tasarım gerektirir. Çünkü hangi paketin korunacağına dair karar, doğrudan uygulamaların ağıdan aldığı hizmet kalitesini etkiler.

İkinci büyük alan ise **marking** konusudur. Burada amaç, paketleri hemen düşürmek değil; bazı paketleri işaretleyerek ağda tıkanıklık oluşmaya başladığını uç sistemlere bildirmektir. Slaytta bu bağlamda **ECN** ve **RED** örnekleri verilmiştir. Bu mekanizmaların temel mantığı şudur: Kuyruk tamamen dolup sert kayıplar yaşanmadan önce, yönlendirici belli paketleri işaretleyerek ya da seçici olarak erken müdahale ederek göndericilere “ağ yoğunlaşıyor” sinyali gönderebilir. Böylece uç sistemler, özellikle TCP gibi protokoller, gönderim hızlarını düşürebilir ve kuyrukların aşırı büyümesi önlenir. Bu yaklaşım, yalnızca son anda paket atmak yerine daha yumuşak ve daha öngörülü bir tıkanıklık yönetimi sağlar.

Bu slaytın ağ performansı açısından önemi büyüktür. Çünkü paket kaybı ve gecikme yalnızca fiziksel hataların veya düşük kapasitenin sonucu değildir; çoğu zaman tampon yönetiminin nasıl yapıldığıyla da doğrudan ilişkilidir. İki yönlendirici aynı bağlantı hızına sahip olsa bile, buffer management politikaları farklıysa ağ davranışları da farklı olabilir. Bir cihaz kuyrukları çok geç müdahale ederek yönetebilir, diğeri ise erken işaretleme ile daha dengeli çalışabilir. Bu nedenle buffer management, yönlendiricinin yalnızca donanımsal değil, davranışsal karakterini de belirleyen temel konulardan biridir.

Özetle bu slaytın ana mesajı şudur: **Buffer management**, kuyrukta bekleyen paketler üzerindeki karar mekanizmasıdır. Tampon dolduğunda hangi paketin düşürüleceği **drop**

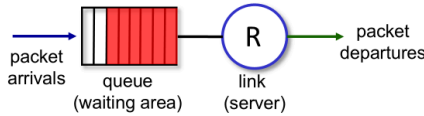
policy ile belirlenir; örneğin **tail drop** ya da **priority-based** yaklaşımlar kullanılabilir. Ayrıca bazı durumlarda paketler hemen düşürülmek yerine **marking** ile işaretlenerek tıkanıklık bilgisi göndericiye iletilir. Böylece tampon belleğin yönetimi, ağdaki gecikme, kayıp ve tıkanıklık davranışını doğrudan şekillendirir.

Paket Zamanlama: FCFS

paket zamanlama : Bağlantı üzerine **bir sonraki hangi paketin gönderileceğine karar verme** işidir.

- first come, first served
- priority
- round robin
- weighted fair queueing

Abstraction: queue



FCFS: Paketler çıkış portuna geliş sırasına göre iletilir.

- also known as: First-in-first-out (FIFO)
- real world examples?

Network Layer: 4-35

Ders Notu: Packet Scheduling: FCFS

Bu slayt, çıkış portunda kuyrukta bekleyen paketler arasından hangisinin sıradaki iletim için seçileceğini belirleyen **packet scheduling** kavramını tanıtmaktadır. Önceki slaytlarda tamponlama ve kuyruklamanın neden gerekli olduğu görülmüştü. Ancak kuyrukta birden fazla paket varsa, yönlendirici bunların hepsini aynı anda gönderemez. Bu nedenle bir seçim yapılması gerekir. Slayttaki tanıma göre packet scheduling, bağlantı üzerine **bir sonraki hangi paketin gönderileceğine karar verme** işidir. Bu karar, ağ performansını doğrudan etkiler; çünkü gecikme, adalet, öncelik ve farklı akışların deneyimi büyük ölçüde bu seçim mekanizmasına bağlıdır.

Slaytta packet scheduling için birkaç temel yaklaşım listelenmektedir: **first come, first served**, **priority**, **round robin** ve **weighted fair queueing**. Bu slaytın odak noktası bunların ilki olan **FCFS**'dir. FCFS, en basit ve en sezgisel zamanlama yaklaşımıdır. Buradaki temel fikir, paketlerin çıkış portuna geliş sırasına göre iletilmesidir. Yani önce gelen paket önce gönderilir; daha sonra gelen paket ise sırasını bekler. Bu nedenle FCFS, aynı zamanda **FIFO (First-In-First-Out)** adıyla da bilinir. Slaytta bu eş anlamlı kullanım açıkça belirtilmektedir.

Bu yaklaşımın mantığı kuyruk benzetmesiyle çok iyi anlaşılır. Slaytın alt kısmındaki soyut modelde soldan paket gelişleri vardır, ortada bekleme alanı olan kuyruk bulunur, sağda ise sabit hızda hizmet veren link yer alır. FCFS altında bu kuyruksa herhangi bir öncelik, sınıflandırma ya da ağırlıklandırma yapılmaz. Kuyruğa ilk giren paket, bağlantı boşalınca ilk çıkan paket olur. Bu açıdan FCFS hem uygulanması kolay hem de davranışı tahmin edilmesi kolay bir mekanizmadır.

FCFS'nin önemli avantajı sadeliğidir. Yönlendirici, paketleri karmaşık kurallarla sınıflandırmak zorunda kalmaz. Her paket aynı temel kurala tabidir. Bu, düşük işlem yükü ve kolay uygulanabilirlik sağlar. Ayrıca "önce gelenin önce hizmet alması" sezgisel olarak adil gibi görünür. Ancak bu adalet yalnızca geliş sırasına göredir; uygulama gereksinimlerine ya da paket türlerine göre değildir. Bu yüzden FCFS her durumda en iyi çözüm olmayabilir.

Özellikle farklı türde trafik aynı çıkış kuyruğunu paylaşıyorsa, FCFS bazı istenmeyen sonuçlar doğurabilir. Örneğin büyük veri paketleri ya da yoğun bir akış kuyruksa öne geçmişse, arkadaki gecikmeye duyarlı küçük paketler de bunların bitmesini beklemek zorunda kalır. Böylece ses, video veya etkileşimli uygulamalar gibi hızlı yanıt isteyen trafikler, sırf daha sonra geldikleri için gereksiz gecikmeye uğrayabilir. Bu nedenle FCFS basit olsa da, her trafik türünün ihtiyaçlarını ayrı ayrı gözetemeyen bir yöntem değildir.

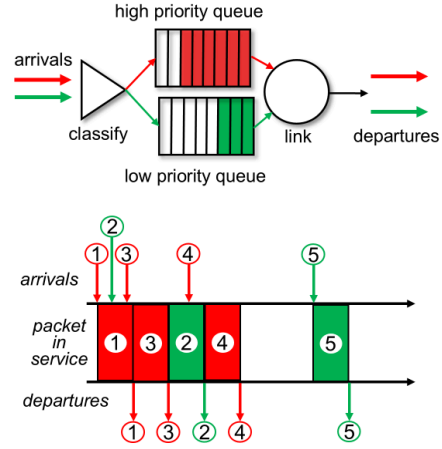
Bu noktada slaytta neden başka zamanlama disiplinlerinin de listelendiği anlaşılır. **Priority scheduling**, bazı paketleri diğerlerinden önce göndermeyi mümkün kılar. **Round robin**, farklı kuyruksa arasında dönüşümlü hizmet sunar. **Weighted fair queueing** ise akışlar arasında daha dengeli ve kontrollü bir paylaşım sağlamaya çalışır. Yani FCFS temel başlangıç noktasıdır; ama daha gelişmiş ağ hizmet kalitesi gereksinimleri ortaya çıktığında genellikle daha gelişmiş zamanlama yaklaşımları gerekir.

Özetle bu slaytın ana mesajı şudur: **packet scheduling**, çıkış portunda sıradaki hangi paketin gönderileceğini belirleyen mekanizmadır. Bunun en basit biçimi olan **FCFS**, paketleri çıkış portuna geliş sırasına göre iletir ve bu yüzden **FIFO** olarak da adlandırılır. Basit ve sezgiseldir; ancak tüm paketlere aynı şekilde davrandığı için, farklı trafik türlerinin gecikme ve hizmet kalitesi gereksinimlerini her zaman en iyi biçimde karşılamaz.

Paket Zamanlama: Öncelik (Priority)

Öncelikli planlama :

- Gelen trafik sınıflandırılıp, sınıfa göre sıralanmış
 - Sınıflandırma amacıyla herhangi bir başlık alanı kullanılabilir
- Tamponunda paket bulunan en yüksek öncelikli kuyruktan gönderim yapılır.
 - Aynı öncelik sınıfı içindeki paketler için yine geliş sırasına dayalı bir düzen uygulanır.



Network Layer: 4-36

Ders Notu: Paket Zamanlama: Öncelik (Priority)

Bu slayt, çıkış portunda kuyrukta bekleyen paketlerin yalnızca geliş sırasına göre değil, **öncelik sınıflarına göre** de zamanlanabileceğini göstermektedir. Bir önceki slaytta görülen FCFS yaklaşımında tüm paketler tek bir kuyrukta ve geliş sırasına göre ele alınıyordu. Burada ise trafik önce **sınıflandırılır**, ardından her sınıf kendi kuyruğuna yerleştirilir. Böylece yönlendirici, sıradaki paketi seçerken yalnızca “hangisi önce geldi?” sorusunu değil, aynı zamanda “hangisi daha yüksek öncelikli?” sorusunu da dikkate alır.

Slaytta bu yaklaşım **priority scheduling** olarak adlandırılmaktadır. Temel mantık şudur: Gelen trafik belirli ölçütlere göre sınıflara ayrılır ve her sınıf ayrı bir kuyruğa konur. Örneğin şekilde bir **high priority queue** ve bir **low priority queue** gösterilmektedir. Paketlerin hangi kuyruğa gideceği, bir **classify** adımıyla belirlenir. Slaytta ayrıca sınıflandırma için paketin başlığındaki **herhangi bir alanın** kullanılabileceği belirtilmektedir. Bu oldukça önemlidir; çünkü öncelik kararı yalnızca tek bir kriterle bağlı değildir. Kaynak veya hedef adres, protokol türü, port numarası, DSCP/ToS alanı ya da başka üst bilgi alanları sınıflandırmada kullanılabilir. Bu da yönlendiriciye oldukça esnek bir trafik politikası uygulama imkânı verir.

Sınıflandırma yapıldıktan sonra zamanlama kuralı basittir: **tamponunda paket bulunan en yüksek öncelikli kuyruktan gönderim yapılır**. Yani yüksek öncelik kuyruğu boş değilse, düşük öncelik kuyruğundaki paketler beklemeye devam eder. Ancak aynı öncelik sınıfı içindeki paketler için yine geliş sırasına dayalı bir düzen uygulanabilir. Slaytta bu durum “**FCFS within priority class**” ifadesiyle belirtilmektedir. Başka bir

deyişle, öncelik sınıfları arasında seçim önceliğe göre yapılır; aynı sınıfın içinde ise çoğu zaman FIFO mantığı korunur.

Sağ üstteki şekil bu yapıyı mimari olarak göstermektedir. Gelen paketler önce sınıflandırıcıdan geçer; kırmızı paketler yüksek öncelik kuyruğuna, yeşil paketler düşük öncelik kuyruğuna gider. Ardından link üzerinde iletme karar verilir. Eğer yüksek öncelik kuyruğunda paket varsa, sıradaki hizmet bu kuyruktan verilir. Bu nedenle düşük öncelikli paketler, kendi kuyruklarında hazır bekleseler bile, yüksek öncelikli trafik devam ettiği sürece hatta erişemeyebilir.

Sağ alttaki zaman diyagramı ise bu davranışın sonucunu somut biçimde göstermektedir. Şekilde 1, 3 ve 4 numaralı kırmızı paketler yüksek öncelikli; 2 ve 5 numaralı yeşil paketler düşük öncelikli olarak düşünülmektedir. Paket 2, paket 3'ten önce gelmiş olmasına rağmen, paket 3 yüksek öncelik sınıfında olduğu için 2'den önce gönderilmektedir. Aynı şekilde paket 4 de düşük öncelikli paketten önce hizmet almaktadır. Buradan çıkarılması gereken temel sonuç şudur: **priority scheduling'de çıkış sırası yalnızca varış zamanına göre belirlenmez**; öncelik sınıfı, geliş sırasının önüne geçebilir. Bu da FCFS'den en önemli farkıdır.

Bu yaklaşımın ağ performansı açısından önemli avantajları vardır. Gecikmeye duyarlı trafikler, örneğin gerçek zamanlı ses, video konferans ya da bazı kritik kontrol paketleri, daha düşük gecikmeyle iletilebilir. Böylece ağ, tüm trafiğe aynı davranmak yerine farklı hizmet ihtiyaçlarına göre daha uygun davranabilir. Bu, hizmet kalitesi (QoS) mekanizmalarının temel yapı taşlarından biridir.

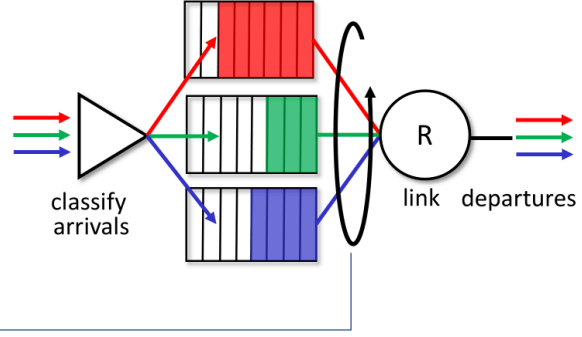
Ancak bu yaklaşımın dikkat edilmesi gereken yönü de vardır. Eğer yüksek öncelikli trafik sürekli geliyorsa, düşük öncelikli kuyruk çok uzun süre hizmet alamayabilir. Bu duruma aşırı önceliklendirme sonucu **starvation**, yani düşük öncelikli trafiğin ihmal edilmesi problemi denebilir. Dolayısıyla priority scheduling güçlü bir mekanizma olmakla birlikte, kontrolsüz kullanıldığında bazı trafik sınıflarını ciddi biçimde dezavantajlı hale getirebilir. Bu nedenle daha dengeli paylaşım gereken durumlarda round robin ya da weighted fair queueing gibi yöntemler tercih edilir.

Özetle bu slaytın ana mesajı şudur: **Priority scheduling**, gelen paketleri sınıflandırıp ayrı kuyruklara yerleştirir ve iletim için her zaman tamponunda paket bulunan **en yüksek öncelikli kuyruğu** seçer. Aynı öncelik sınıfı içinde ise çoğu zaman **FCFS** uygulanır. Böylece bazı trafik türlerine daha iyi gecikme ve hizmet kalitesi sağlanabilir; ancak yüksek öncelikli trafik baskın hale gelirse düşük öncelikli paketler uzun süre beklemek zorunda kalabilir.

Zamanlama İlkeleri: Round Robin

Round Robin (RR) zamanlama:

- Farklı trafik sınıflarına sırayla hizmet vererek bağlantı kapasitesini daha adil biçimde paylaşmaktır.
 - Sınıflandırma amacıyla herhangi bir başlık alanı kullanılabilir
- Sunucu, sınıf kuyruklarını döngüsel ve tekrar tekrar tarayarak, her sınıftan (varsa) sırayla birer tam paket gönderir



Network Layer: 4-37

Ders Notu: Scheduling Policies: Round Robin

Bu slayt, paket zamanlamasında **Round Robin (RR)** yaklaşımını açıklamaktadır. Öncelik zamanlamasında görüldüğü gibi trafik önce sınıflandırılıp ayrı kuyruklara yerleştirilebilir. Ancak priority scheduling'de en yüksek öncelikli kuyruk boş olmadığı sürece diğer kuyruklar beklemek zorunda kalır. Round Robin yaklaşımı ise bu soruna daha dengeli bir çözüm getirir. Burada amaç, farklı trafik sınıflarına sırayla hizmet vererek bağlantı kapasitesini daha adil biçimde paylaşmaktır.

Slayta göre ilk adım yine aynıdır: **arriving traffic classified, queued by class**. Yani gelen trafik belirli ölçütlere göre sınıflandırılır ve her sınıf kendi kuyruğuna yerleştirilir. Sınıflandırma için başlık alanlarındaki farklı bilgiler kullanılabilir. Bu yönüyle Round Robin, priority scheduling ile benzer bir başlangıç yapısına sahiptir. Fark, kuyruklar dolduktan sonra linkin hangi kuyruğa hangi sırayla hizmet verdiğinde ortaya çıkar.

Round Robin'in temel mantığı, sunucunun yani çıkış bağlantısının sınıf kuyruklarını **döngüsel olarak taramasıdır**. Slaytta da belirtildiği gibi sistem, kuyrukları sırayla dolaşır ve her sınıftan, eğer kuyruқта paket varsa, **bir tam paket** gönderir. Daha sonra bir sonraki kuyruğa geçer. Son kuyruğa geldikten sonra tekrar başa döner ve aynı döngü sürer. Bu yüzden bu yaklaşım "round robin", yani dönüşümlü sırayla hizmet verme olarak adlandırılır.

Şekilde bu mantık görsel olarak verilmiştir. Kırmızı, yeşil ve mavi akışlar önce sınıflandırıcıdan geçmekte ve üç ayrı kuyruğa yerleştirilmektedir. Daha sonra link, bu

kuyrukları belirli bir sabit sırayla dolaşmaktadır. Böylece örneğin kırmızı kuyruğun ardından yeşil, onun ardından mavi hizmet alır; sonra döngü tekrar kırmızıya döner. Eğer o anda bir kuyruk boşsa, sistem o kuyruğu atlayıp dolu olan bir sonraki kuyruğa geçebilir. Bu yapı, tek bir kuyruğun sürekli baskın hale gelmesini önlemeye yardımcı olur.

Bu yaklaşımın önemli avantajı, **adalet duygusunu** priority scheduling'e göre daha iyi desteklemesidir. Priority scheduling'de yüksek öncelikli trafik düşük öncelikli akışları uzun süre bekletebilir. Round Robin'de ise her sınıf, döngü içinde düzenli olarak iletim fırsatı bulur. Bu nedenle daha düşük öncelikli ya da daha az baskın akışların tamamen aç bırakılması önlenmiş olur. Bu yönüyle RR, hizmet paylaşımında daha dengeli bir yapı sunar.

Ancak burada dikkat edilmesi gereken önemli bir nokta vardır: Slaytta **bir tam paket** gönderildiği belirtilmektedir. Bu nedenle eğer kuyruklardaki paket boyutları çok farklıysa, Round Robin her kuyruk için aynı sayıda paket gönderse de, her sınıfa verilen **bit düzeyindeki bant genişliği** tam olarak eşit olmayabilir. Büyük paketler gönderen bir kuyruk, küçük paketler gönderen bir kuyruğa göre aynı turda daha fazla hat süresi kullanabilir. Bu nedenle RR, paket sayısı açısından adil görünse de, bit düzeyinde her zaman tam adil değildir. Bu durum ileride weighted fair queueing gibi daha gelişmiş yöntemlerin neden gerekli olduğunu anlamak açısından önemlidir.

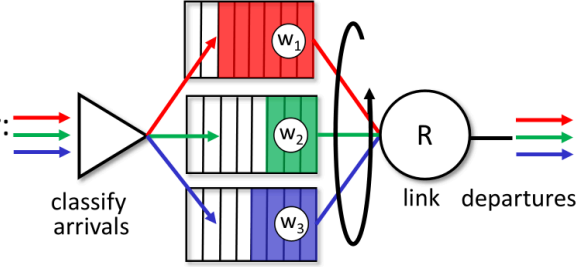
Yine de Round Robin, FCFS'ye ve katı önceliklendirmeye göre daha dengeli bir orta yol sunar. FCFS tüm paketleri tek sırada ele alırken akış sınıfları arasındaki farkı gözetmez. Priority scheduling bazı sınıflara çok büyük avantaj sağlayabilir. Round Robin ise sınıf temelli hizmeti korurken, her sınıfa düzenli erişim hakkı tanır. Bu nedenle ağ cihazlarında pratik ve öğretici bir zamanlama politikası olarak önemli bir yer tutar.

Özetle bu slaytın ana mesajı şudur: **Round Robin scheduling**, gelen paketleri sınıflara ayırıp ayrı kuyruklara yerleştirir ve çıkış bağlantısı bu kuyrukları döngüsel olarak tarayarak her sınıftan sırayla **bir tam paket** gönderir. Böylece farklı trafik sınıfları arasında daha dengeli bir hizmet paylaşımı sağlanır ve tek bir kuyruğun diğerlerini sürekli baskılaması önlenir.

Zamanlama İlkeleri: Ağırlıklı Adil Kuyruklama

Weighted Fair Queuing (WFQ):

- Temel mantık trafik sınıflarını ayrı kuyruklarda tutmaktır; ancak her sınıfa eşit değil, **ağırlığı oranında** hizmet verilir.
- Her sınıf i , bir ağırlık w_i değerine sahiptir ve her döngüde ağırlıklandırılmış hizmet miktarı alır:
$$\frac{w_i}{\sum_j w_j}$$
- Eğer bir trafik sınıfına belirli bir ağırlık verilmişse, o sınıf aktif olduğu sürece bağlantı kapasitesinden buna karşılık gelen en az bir pay alması beklenir.



Network Layer: 4-38

Ders Notu: Scheduling Policies: Weighted Fair Queueing

Bu slayt, paket zamanlamasında daha dengeli ve daha kontrollü bir yaklaşım olan **Weighted Fair Queueing (WFQ)** mekanizmasını açıklamaktadır. Önceki slaytta Round Robin yaklaşımı görülmüştü. Orada her sınıf kuyruğu sırayla hizmet alıyordu ve her turda her sınıftan bir paket gönderiliyordu. WFQ ise bu fikri genelleştirir. Slaytta da açıkça belirtildiği gibi WFQ, **generalized Round Robin** olarak düşünülebilir. Yani temel mantık yine trafik sınıflarını ayrı kuyruklarda tutmaktır; ancak her sınıfa eşit değil, **ağırlığı oranında** hizmet verilir.

İlk adım yine sınıflandırmadır. Gelen paketler başlık alanlarına göre farklı trafik sınıflarına ayrılır ve her sınıf kendi kuyruğuna yerleştirilir. Şekilde kırmızı, yeşil ve mavi trafiklerin üç ayrı kuyruğa ayrıldığı görülmektedir. Ancak bu kez kuyrukların hepsi aynı sıklıkta hizmet almaz. Her sınıf i için bir **ağırlık değeri** tanımlanır; slaytta bunlar w_1 , w_2 , w_3 olarak gösterilmektedir. Link, her çevrimde kaynaklarını bu ağırlıklara göre paylaştırır. Slayttaki formüle göre sınıf i 'nin alacağı hizmet oranı yaklaşık olarak

$$w_i / \sum w$$

şeklinde belirlenir. Bu, toplam bağlantı kapasitesinin her sınıfa kendi ağırlığı oranında tahsis edildiği anlamına gelir.

Bu yaklaşımın temel sezgisi şudur: Tüm trafik sınıfları aynı önemde değildir ama bazıları da tamamen dışlanmamalıdır. Priority scheduling'de yüksek öncelikli trafik

baskın hale gelebilir ve düşük öncelikli akışlar çok uzun süre bekleyebilir. Round Robin’de ise her sınıf dönüşümlü hizmet alır ama bit düzeyinde tam esnek bir paylaşım sağlamak zordur. WFQ bu iki uç arasında daha kontrollü bir denge sunar. Her sınıf sisteme önceden tanımlanmış bir pay ile katılır. Böylece hem hizmet farklılaştırması yapılabilir hem de hiçbir sınıf tamamen ihmal edilmez.

Slaytta ayrıca **minimum bandwidth guarantee (per-traffic-class)** ifadesi yer almaktadır. Bu, WFQ’nun en önemli özelliklerinden biridir. Eğer bir trafik sınıfına belirli bir ağırlık verilmişse, o sınıf aktif olduğu sürece bağlantı kapasitesinden buna karşılık gelen en az bir pay alması beklenir. Bu durum özellikle hizmet kalitesi gerektiren ağlarda çok önemlidir. Örneğin ses, video ya da kritik kurumsal uygulamalara daha yüksek ağırlık verilerek bu trafiğin bağlantı üzerinde asgari bir bant genişliği payı alması sağlanabilir. Aynı zamanda diğer trafikler de tamamen dışlanmaz; onlar da kendi ağırlıkları ölçüsünde hizmet alır.

Şekildeki üç kuyruğu bu gözle okumak gerekir. Eğer kırmızı kuyruğun ağırlığı en yüksekse, bağlantı zamanının ya da kapasitesinin daha büyük kısmı kırmızı akışa verilir. Yeşil ve mavi kuyruklar da kendi ağırlıklarına göre daha küçük ya da daha büyük pay alır. Yani WFQ’da amaç, yalnızca “sıradaki paket hangisi?” sorusuna cevap vermek değil; toplam bağlantı kaynağını **oranlı ve öngörülebilir** biçimde bölüştürmektir.

Bu yaklaşımın ağ mimarisi içindeki önemi büyüktür. Çünkü ağlar günümüzde tek tip trafik taşımaz. Aynı bağlantı üzerinde gerçek zamanlı ses, video akışı, web trafiği, dosya aktarımı ve arka plan senkronizasyonu aynı anda bulunabilir. Bunların hepsine FCFS uygulamak çok kaba bir yaklaşım olur; katı priority scheduling ise bazılarını aşırı avantajlı hale getirebilir. WFQ ise her sınıfa belirli bir hizmet garantisi verirken aynı zamanda genel adaleti korumaya çalışır. Bu yüzden QoS tasarımlarında çok önemli bir kavramdır.

Özetle bu slaytın ana mesajı şudur: **Weighted Fair Queueing (WFQ)**, Round Robin’in geliştirilmiş biçimidir. Her trafik sınıfına bir **ağırlık** atanır ve link kapasitesi sınıflar arasında bu ağırlıkların oranına göre paylaşılır. Böylece her sınıf yalnızca hizmet almakla kalmaz, aynı zamanda ağırlığına bağlı olarak **asgari bir bant genişliği payı** da elde eder. Bu, hem hizmet farklılaştırmasını hem de daha dengeli kaynak paylaşımını mümkün kılar.

Sidebar: Ağ Tarafsızlığı

Ağ tarafsızlığı nedir?

- **Teknik** : Bir internet servis sağlayıcısının ağ kaynaklarını farklı kullanıcılar, uygulamalar ve içerikler arasında nasıl paylaşması gerektiği sorusunu gündeme getirir.
 - Paket zamanlama ve tampon yönetimi gibi mekanizmalardır
- **sosyal ve ekonomik ilkeler**
 - ifade özgürlüğünü korumak
 - Yenilikçiliği ve rekabeti teşvik etmek
- Yürürlükteki **yasal düzenlemeler** ve politikalar

Farklı ülkeler network neutrality konusunda farklı yaklaşımlara sahiptir

Network Layer: 4-39

Ders Notu: Sidebar: Network Neutrality

Bu slayt, ağ katmanındaki zamanlama ve tampon yönetimi konularının yalnızca teknik ayrıntılar olmadığını, daha geniş bir tartışmaya bağlandığını göstermektedir. **Network neutrality** yani ağ tarafsızlığı, en genel anlamıyla bir internet servis sağlayıcısının ağ kaynaklarını farklı kullanıcılar, uygulamalar ve içerikler arasında nasıl paylaşması gerektiği sorusunu gündeme getirir. Bu nedenle konu, bir yandan yönlendirici içindeki paket zamanlama ve buffer management mekanizmalarına dayanırken, diğer yandan sosyal, ekonomik ve hukuki sonuçlar doğurur.

Slaytın ilk vurgusu, meselenin **teknik** boyutudur. Bir ISP'nin elindeki bağlantı kapasitesi sınırlıdır; dolayısıyla yoğunluk anlarında hangi trafiğin önce hizmet alacağı, hangisinin bekleyeceği ya da hangi paketlerin düşürüleceği gibi kararlar verilmek zorundadır. Önceki slaytlarda görülen **packet scheduling** ve **buffer management** mekanizmaları, bu kararların teknik araçlarıdır. Örneğin priority scheduling uygulanırsa bazı trafik sınıfları daha iyi hizmet alabilir; WFQ uygulanırsa kapasite ağırlıklara göre paylaştırılabilir. Yani ağ tarafsızlığı tartışması soyut bir fikir değil, doğrudan yönlendiricilerin ve ağ cihazlarının davranışına yansıyan bir konudur.

İkinci boyut, konunun **sosyal ve ekonomik ilkelerle** ilişkili olmasıdır. Slaytta özellikle iki nokta öne çıkarılmaktadır: **ifade özgürlüğünün korunması** ve **yenilik ile rekabetin teşvik edilmesi**. Eğer bir servis sağlayıcı belirli uygulamalara, platformlara ya da içerik türlerine sistematik olarak daha iyi hizmet verirse, bu durum yalnızca ağ performansı farkı yaratmaz; aynı zamanda hangi içeriklerin daha görünür, hangi hizmetlerin daha

erişilebilir ve hangi girişimlerin daha rekabetçi olacağını da etkileyebilir. Bu nedenle ağ tarafsızlığı, yalnızca “paket hangi sırada gönderiliyor?” sorusundan ibaret değildir; internet ekosisteminin nasıl şekilleneceğiyle de ilgilidir.

Üçüncü boyut ise **hukuki ve politik** çerçevedir. Slaytta network neutrality'nin yalnızca teknik bir tercih ya da etik bir ilke olarak değil, bazı yerlerde **yasal kurallar ve politikalarla** uygulanmaya çalışılan bir konu olduğu belirtilmektedir. Bunun anlamı şudur: ağ tarafsızlığı, ülkelerin düzenleyici kurumları, hukuk sistemleri ve telekomünikasyon politikaları tarafından farklı şekillerde ele alınabilir. Dolayısıyla aynı teknik altyapı farklı ülkelerde farklı kurallarla işletilebilir.

Slaytın son cümlesi bu yüzden önemlidir: **farklı ülkeler network neutrality konusunda farklı yaklaşımlara sahiptir**. Yani ağ tarafsızlığı evrensel olarak tek biçimde uygulanmış sabit bir kural değildir. Bazı ülkeler daha katı tarafsızlık ilkelerini benimserken, bazıları servis sağlayıcılara daha geniş trafik yönetimi esnekliği tanıyabilir. Bu da konunun hem teknik hem de politik açıdan tartışmalı ve çok katmanlı bir alan olduğunu gösterir.

Bu slaytın önceki konularla bağlantısı açıktır. Çıkış portu kuyruklaması, scheduling discipline, priority scheduling ve WFQ gibi mekanizmalar ağ kaynaklarının nasıl paylaştırılacağını belirler. Ağ tarafsızlığı tartışması ise bu teknik mekanizmaların hangi ilkelere göre kullanılmasının uygun olacağı sorusunu sorar. Başka bir deyişle, teknik araçlar mühendislik düzeyinde vardır; fakat bu araçların nasıl kullanılacağı toplumsal, ekonomik ve hukuki tercihlerle şekillenir.

Özetle bu slaytın ana mesajı şudur: **Network neutrality**, bir ISP'nin ağ kaynaklarını kullanıcılar ve uygulamalar arasında nasıl paylaşması gerektiği sorusudur. Bunun teknik tarafında paket zamanlama ve tampon yönetimi mekanizmaları bulunur; sosyal ve ekonomik tarafında ifade özgürlüğü, yenilik ve rekabet yer alır; hukuki tarafında ise düzenlemeler ve politikalar devreye girer. Bu nedenle ağ tarafsızlığı, bilgisayar ağlarında teknik tasarım ile toplumsal sonuçların kesiştiği önemli bir konudur.

Sidebar: Ağ Tarafsızlığı

2015 tarihli ABD FCC Kararı: Açık İnternetin Korunması ve Teşvik Edilmesi: üç temel “net ve kesin” kural :

Bu kurallar, bir internet servis sağlayıcısının ağ üzerindeki trafiğe nasıl müdahale edebileceğine ilişkin sınırlar çizer.

- **engelleme yok** ... “Makul ağ yönetimi ilkeleri çerçevesinde, yasal içerik, uygulamalar, hizmetler veya zararsız cihazların erişimini engellemeyecektir.”
- **hız sınırlaması yok** ... “makul ağ yönetimi ilkeleri çerçevesinde, İnternet içeriği, uygulaması veya hizmeti ya da zararsız bir cihazın kullanımı nedeniyle yasal İnternet trafiğini engellemeyecek veya kalitesini düşürmeyecektir.”
- **ücretli önceliklendirme yok.** ... “ücret karşılığında önceliklendirme yapmayacaktır”

Network Layer: 4-40

Ders Notu: Sidebar: Network Neutrality

Bu slayt, ağ tarafsızlığı tartışmasının yalnızca genel ilkeler düzeyinde kalmadığını, belirli düzenleyici kurallarla da somutlaştırıldığını göstermektedir. Burada örnek olarak **2015 tarihli ABD FCC kararı** ele alınmakta ve açık internet yaklaşımını korumaya yönelik üç temel “clear, bright line” kural özetlenmektedir. Bu kurallar, bir internet servis sağlayıcısının ağ üzerindeki trafiğe nasıl müdahale edebileceğine ilişkin sınırlar çizer. Böylece önceki slaytta değinilen teknik, sosyal, ekonomik ve hukuki boyutlar burada daha somut hale gelir.

Birinci ilke **no blocking** kuralıdır. Bunun anlamı, servis sağlayıcısının yasal içerikleri, uygulamaları, hizmetleri ya da zararsız cihazları keyfi biçimde engellememesidir. Bu ilke, internet erişiminin belirli içerik ve hizmetlere seçici biçimde kapatılmasını önlemeyi amaçlar. Ancak slaytta dikkat çekici bir ifade vardır: bu yasak, **reasonable network management** yani makul ağ yönetimi istisnasıyla birlikte düşünülmektedir. Bu çok önemlidir; çünkü ağ tarafsızlığı, ağ yönetiminin tamamen ortadan kaldırılması anlamına gelmez. Güvenlik, tıkanıklık kontrolü, teknik arızalar ya da hizmet sürekliliği gibi nedenlerle bazı meşru ağ yönetimi uygulamaları yine gerekli olabilir. Yani kural, teknik zorunlulukları değil, ayrımcı ve keyfi engellemeyi hedef alır.

İkinci ilke **no throttling** kuralıdır. Bu ilke, yasal internet trafiğinin içerik, uygulama, hizmet ya da cihaz türüne göre kasıtlı olarak yavaşlatılmamasını ifade eder. Buradaki temel mesele, servis sağlayıcısının belirli trafik türlerini açıkça engellemeden ama onları pratikte kullanılamaz hale getirecek biçimde yavaşlatmasının da tarafsızlık ihlali

sayılmasıdır. Örneğin bir uygulamanın tamamen kapatılması yerine sistematik olarak hızının düşürülmesi de kullanıcı deneyimini ciddi biçimde etkileyebilir. Bu nedenle ağ tarafsızlığı yalnızca “erişim var mı yok mu?” sorusuyla sınırlı değildir; erişimin **kalitesi** de önemlidir.

Üçüncü ilke ise **no paid prioritization** kuralıdır. Bu, bazı içerik sağlayıcıların ya da hizmetlerin para ödeyerek ağ üzerinde ayrıcalıklı, daha hızlı ya da daha düşük gecikmeli bir hizmet elde etmemesini amaçlar. Bu ilkenin önemi, teknik zamanlama mekanizmalarıyla doğrudan ilişkilidir. Önceki slaytlarda priority scheduling ve weighted fair queueing gibi yöntemlerin belirli trafiğe daha iyi hizmet sağlayabildiği görülmüştü. Paid prioritization tartışması tam da burada ortaya çıkar: Eğer bu teknik mekanizmalar, yalnızca ödeme gücü olan aktörlere sistematik avantaj sağlamak için kullanılırsa, internetin açık ve rekabetçi yapısı zarar görebilir. Dolayısıyla bu kural, teknik önceliklendirme kapasitesinin ekonomik ayrıcalığa dönüşmesini sınırlamayı hedefler.

Bu slaytın önemli mesajlarından biri, ağ tarafsızlığının teknik altyapıyla hukuk arasında doğrudan bir bağ kurduğudur. Yönlendiriciler ve ağ cihazları gerçekten de bazı paketleri önce, bazılarını sonra gönderebilir; bazı trafiği işaretleyebilir ya da düşürebilir. Ancak hangi teknik mekanizmaların hangi amaçlarla kullanılabileceği, hukuki ve düzenleyici çerçeveye belirlenebilir. Bu nedenle ağ tarafsızlığı tartışması, mühendislik düzeyinde mümkün olan her şeyin politik ve hukuki olarak kabul edilebilir olduğu anlamına gelmez.

Özetle bu slaytın ana mesajı şudur: 2015 FCC yaklaşımı, açık interneti korumak için üç temel ilke ortaya koymuştur: **engellememek (no blocking)**, **yavaşlatmamak (no throttling)** ve **ücret karşılığı ayrıcalıklı öncelik vermemek (no paid prioritization)**. Bu ilkeler, ağ tarafsızlığının somut düzenleyici karşılıkları olarak görülebilir ve teknik trafik yönetimi ile ayırmacı müdahale arasındaki sınırı çizmeye çalışır.

İSS: Telekomünikasyon mu, Bilgi Hizmeti mi?

İnternet Servis Sağlayıcı (ISP), bir “telekomünikasyon hizmeti” mi yoksa bir “bilgi hizmeti” sağlayıcısı mıdır?

1934 ve 1996 tarihli ABD Telekomünikasyon Kanunu:

- **Başlık II : Telekomünikasyon hizmetlerine** “kamu taşıyıcısı yükümlülükleri” getirir : Uygun fiyatlar, ayrımcılık yapılmaması ve düzenleme gerekliliği
- **Başlık I : Bilgi hizmetleri** için geçerlidir :
 - Genel taşıyıcı yükümlülükleri yoktur (düzenlemeye tabi değildir)
 - FCC’ye kendi görevlerini yerine getirmek için gerekli gördüğü ölçüde belirli bir yetki alanı tanınmaktadır.

Network Layer: 4-41

Ders Notu: ISP: telecommunications or information service?

Bu slayt, ağ tarafsızlığı tartışmasının teknik düzeyden çıkıp doğrudan **düzenleyici sınıflandırma** meselesine dönüştüğü noktayı göstermektedir. Temel soru şudur: Bir internet servis sağlayıcı, hukuken bir **telecommunications service** sağlayıcısı mı kabul edilmelidir, yoksa bir **information service** sağlayıcısı mı? Slaytın özellikle vurguladığı nokta, bu ayrımın yalnızca kavramsal bir tanım farkı olmadığı; düzenleme, denetim ve yükümlülükler açısından son derece önemli olduğudur. Yani bir ISP’nin hangi kategoriye yerleştirildiği, o kurumun hangi kurallara tabi olacağını doğrudan belirler.

Slaytta bu ayrımın ABD’de özellikle **1934 ve 1996 Telekomünikasyon Yasaları** bağlamında ele alındığı görülmektedir. Burada iki farklı başlık öne çıkmaktadır: **Title II** ve **Title I**. Bu iki başlık, ISP’lerin nasıl değerlendirileceği ve üzerlerinde ne ölçüde kamusal düzenleme uygulanabileceği konusunda farklı sonuçlar doğurur. Dolayısıyla ağ tarafsızlığı gibi ilkelerin uygulanabilirliği de büyük ölçüde bu sınıflandırmaya bağlı hale gelir.

İlk olarak **Title II, telecommunications services** için öngörülen çerçeveyi ifade etmektedir. Slayta göre Title II altında **common carrier duties** uygulanır. Bunun anlamı, bu tür hizmet sunucularının makul fiyatlandırma, ayrımcılık yapmama ve düzenlemeye tabi olma gibi yükümlülükler taşımasıdır. Teknik açıdan bakıldığında bu, servis sağlayıcının ağ kaynaklarını kullanıcılar ve trafik türleri arasında keyfi veya ayrımcı şekilde dağıtamayacağı yönündeki yaklaşımı güçlendirir. Başka bir ifadeyle, eğer ISP bir

telecommunications service olarak sınıflandırılırsa, ağ tarafsızlığı ilkelerine benzer yükümlülüklerin uygulanması için daha güçlü bir hukuki zemin oluşur.

Buna karşılık **Title I, information services** için kullanılan çerçevedir. Slaytta bu başlık altında **common carrier duties** bulunmadığı, yani bu hizmetlerin aynı türden sıkı düzenlemeye tabi olmadığı belirtilmektedir. Bu durum, bilgi hizmeti olarak sınıflandırılan bir yapının daha gevşek bir düzenleme rejimi altında kalması anlamına gelir. Ancak slayt önemli bir nüansı da eklemektedir: Her ne kadar Title I altında klasik common carrier yükümlülükleri olmasa da, FCC'ye kendi görevlerini yerine getirmek için gerekli gördüğü ölçüde belirli bir yetki alanı tanınmaktadır. Yani burada tamamen düzenlemesiz bir alan söz konusu değildir; fakat düzenleyici güç ve müdahale zemini, Title II'ye göre daha farklı ve daha sınırlı bir çerçevede işler.

Bu ayrımın neden bu kadar önemli olduğu, önceki network neutrality slaytlarıyla birlikte daha iyi anlaşılır. Ağ tarafsızlığı ilkeleri, örneğin **engellememe, yavaşlatmama** ya da **ücretli önceliklendirmeme** gibi kurallar, teknik olarak yönlendirici ve ağ yönetim mekanizmalarıyla ilişkilidir. Ancak bu kuralların servis sağlayıcılara hukukene ne ölçüde dayatılabileceği, ISP'nin hangi hizmet sınıfında değerlendirildiğine bağlıdır. Yani problem yalnızca "ağ bunu yapabilir mi?" sorusu değildir; aynı zamanda "hukuk bunu yasaklayabilir ya da zorunlu kılabilir mi?" sorusudur. Bu slayt tam olarak bu ikinci soruya odaklanmaktadır.

Burada temel kavramsal ayrım şu şekilde okunabilir: Eğer ISP esas olarak veriyi bir noktadan başka bir noktaya ileten tarafsız bir taşıyıcı gibi görülürse, **telecommunications service** yaklaşımı daha uygun görünür. Eğer sunduğu hizmet daha geniş bir bilgi hizmeti olarak değerlendirilirse, bu kez **information service** yaklaşımı öne çıkar. Bu iki yorum, aynı altyapıya bakıp farklı hukuki sonuçlara varabilir. Bu nedenle ISP'nin ne olduğuna dair tanım tartışması, aslında ağ tarafsızlığının uygulanıp uygulanamayacağına ilişkin temel zemini belirler.

Özetle bu slaytın ana mesajı şudur: Bir ISP'nin **telecommunications service** mi yoksa **information service** mi sayılacağı, düzenleyici açıdan kritik bir sorudur. **Title II** yaklaşımı daha güçlü yükümlülükler, ayrımcılık yapmama ilkesi ve daha açık düzenleme zemini sunarken; **Title I** yaklaşımı daha sınırlı ve daha gevşek bir düzenleme çerçevesi ifade eder. Bu yüzden ağ tarafsızlığı tartışmasında teknik mekanizmalar kadar, ISP'nin hukuki olarak nasıl sınıflandırıldığı da belirleyici hale gelir.

Ağ katmanı: “Veri Düzlemi” Yol Haritası

■ Network layer: overview

- data plane
- control plane

■ What’s inside a router

- input ports, switching, output ports
- buffer management, scheduling

■ IP: the Internet Protocol

- datagram format
- addressing
- network address translation
- IPv6

■ Generalized Forwarding, SDN

- match+action
- OpenFlow: match+action in action

■ Middleboxes



Network Layer: 4-42

Ders Notu: Network layer: “data plane” roadmap

Bu slayt, ağ katmanının veri düzlemi bölümünde şimdiye kadar işlenen konuları toparlayıp bundan sonra hangi başlıklara geçileceğini gösteren bir **yol haritası** niteliğindedir. Şu ana kadar ağ katmanına genel bakış yapılmış, veri düzlemi ile kontrol düzlemi ayrımı açıklanmış, yönlendiricinin iç yapısı ele alınmış; özellikle giriş portları, iç anahtarlama, çıkış portları, tampon yönetimi ve paket zamanlama konuları incelenmiştir. Bu slaytta ise odak noktasının artık açık biçimde **IP: the Internet Protocol** başlığına kaydığı görülmektedir.

Bu geçiş önemlidir; çünkü önceki bölümde yönlendiricinin **nasıl çalıştığı**, yani paketin bir yönlendirici içinde nasıl alındığı, işlendiği, taşındığı ve gönderildiği incelenmiştir. Şimdi ise ağ katmanının temel taşı olan **IP protokolünün kendisi** ele alınacaktır. Başka bir deyişle, önce yönlendiricinin mekanik işleyişi anlaşılmış, şimdi ise bu işleyişin üzerinde çalıştığı temel ağ-katmanı veri birimi ve adresleme mantığı incelenecektir.

Slaytta IP başlığı altında dört temel konu öne çıkarılmaktadır: **datagram formatı**, **addressing**, **network address translation** ve **IPv6**. Bunlar ağ katmanının veri düzlemini anlamak için merkezi konulardır. Çünkü yönlendirici, kararlarını doğrudan IP datagramının başlık alanlarına bakarak verir. Dolayısıyla önce datagramın biçimini bilmek gerekir. Hangi alanın ne anlama geldiği, paketin nasıl yönlendirileceği, yaşam süresinin nasıl kontrol edildiği ya da üst katman protokolünün nasıl belirtildiği gibi sorular datagram formatı ile ilgilidir.

İkinci başlık olan **addressing**, ağ katmanının belki de en temel unsurudur. Yönlendiricinin forwarding tablosunda arama yapabilmesi, longest prefix matching uygulayabilmesi ve paketi doğru çıkışa gönderebilmesi için hedefin bir ağ adresiyle ifade edilmesi gerekir. Bu nedenle adresleme konusu, önceki slaytlarda işlenen destination-based forwarding ve prefix matching mantığının doğal devamıdır. Adres yapısı anlaşılmadan yönlendirme tablolarının neden bu şekilde kurulduğu da tam olarak anlaşılamaz.

Üçüncü başlık olan **network address translation (NAT)**, ağ katmanının yalnızca teorik adresleme yapısını değil, internetin pratikte nasıl ölçeklendiğini de gösterir. NAT, özel adres alanları ile genel internet arasındaki ilişkiyi kurar ve gerçek ağlarda çok yaygın biçimde kullanılır. Bu yüzden IP adresleme yalnızca ideal uçtan uca mantıkla değil, gerçek dünyadaki adres paylaşımı ve dönüşüm mekanizmalarıyla birlikte ele alınmalıdır.

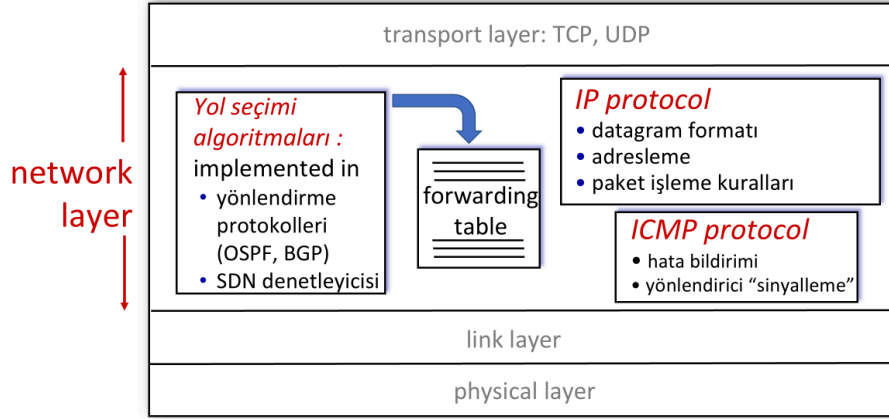
Dördüncü başlık ise **IPv6**'dır. Bu da IPv4'ün adres alanı sınırlamaları ve internetin büyümesi bağlamında ortaya çıkan yeni nesil ağ katmanı protokolünü ifade eder. Dolayısıyla bu başlık, yalnızca yeni bir adres biçimi öğrenmek anlamına gelmez; aynı zamanda internet mimarisinin evrimini anlamak anlamına gelir.

Slaytın sağ tarafında soluk biçimde görülen diğer başlıklar, yani **Generalized Forwarding**, **SDN** ve **Middleboxes**, bu bölümün ilerleyen aşamalarında tekrar ele alınacak konulara işaret etmektedir. Ancak şu anki aktif odak, açıkça IP bölümüdür. Bu da dersin akışında bir mantık olduğunu gösterir: önce yönlendiricinin iç veri düzlemi mekanizmaları, sonra IP'nin temel yapısı, ardından daha gelişmiş veri düzlemi kavramları.

Özetle bu slaytın ana mesajı şudur: ağ katmanının veri düzlemi bölümünde önce yönlendirici mimarisi ve kuyruklama-zamanlama gibi iç işleyiş konuları ele alınmıştır; şimdi ise odak **IP protokolüne**, yani datagram biçimine, adreslemeye, NAT'e ve IPv6'ya kaymaktadır. Bu geçiş, veri düzleminin donanımsal ve işlemsel boyutundan, doğrudan internetin temel ağ katmanı protokolünün mantığına geçildiğini göstermektedir.

Ağ Katmanı: İnternet

Host, yönlendirici ağ katmanı işlevleri:



Network Layer: 4-43

Ders Notu: Network Layer: İnternet

Bu slayt, İnternet'te ağ katmanının temel işlevlerini bir arada gösteren bütüncü bir çerçeve sunmaktadır. Burada ağ katmanı, yalnızca IP başlığını taşıyan bir katman olarak değil; **yol seçimi**, **iletme tablosu**, **IP protokolü** ve **ICMP protokolü** birlikte çalışan bir yapı olarak ele alınmaktadır. Slaytın ana mesajı şudur: Ağ katmanı, bir paketin ağ içinde izleyeceği yolu belirleyen mantığı da içerir, o yol bilgisini kullanarak paketi ileten veri düzlemi işlevlerini de içerir.

Şekilde üstte **transport layer: TCP, UDP**, altta ise **link layer** ve **physical layer** gösterilmektedir. Bu yerleşim, ağ katmanının katmanlı mimari içindeki konumunu hatırlatır. Taşıma katmanından gelen segmentler ağ katmanında datagrama dönüştürülür; ardından bağlantı ve fiziksel katmanlar üzerinden iletilir. Kitapta da gönderici tarafta ağ katmanının segmentleri datagrama kapsüllediği, alıcı tarafta ise datagram içinden taşıma katmanı verisini çıkarıp üst kata teslim ettiği belirtilmektedir.

Slaytın sol tarafındaki kutu **Path-selection algorithms** başlığını taşımaktadır. Bu bölüm, yol seçiminin ağ katmanının kontrol düzlemi tarafını temsil ettiğini gösterir. Burada özellikle **routing protocols (OSPF, BGP)** ve **SDN controller** örnekleri verilmiştir. Anlamı şudur: Paketlerin kaynak ile hedef arasında hangi yönlendiriciler üzerinden ilerleyeceğine ilişkin kararlar bu mekanizmalarla üretilir. Bu kararlar doğrudan her paket için sıfırdan verilmez; bunun yerine yol seçimi mantığı bir **forwarding table** üretir. Yani routing ya da kontrol düzlemi, veri düzleminin kullanacağı tabloyu hazırlar.

Kitapta da routing algoritmalarının yönlendirme tablolarının içeriğini belirlediği ve forwarding'in bu tabloyu kullanarak yerel karar verdiği açıkça anlatılmaktadır.

Ortakdaki **forwarding table** bu yüzden çok önemlidir. Bu tablo, kontrol düzleminde gelen yol bilgisinin veri düzleminde uygulanabilir hale gelmiş biçimidir. Bir paket geldiğinde yönlendirici başlıktaki ilgili alana, çoğu zaman hedef IP adresine, bakar ve bu tablo üzerinden uygun çıkış arayüzünü belirler. Önceki slaytlarda işlenen destination-based forwarding ve longest prefix matching mantığı tam olarak bu tablo üzerinde çalışır. Dolayısıyla forwarding table, kontrol düzlemi ile veri düzlemi arasındaki bağlayıcı yapıdır.

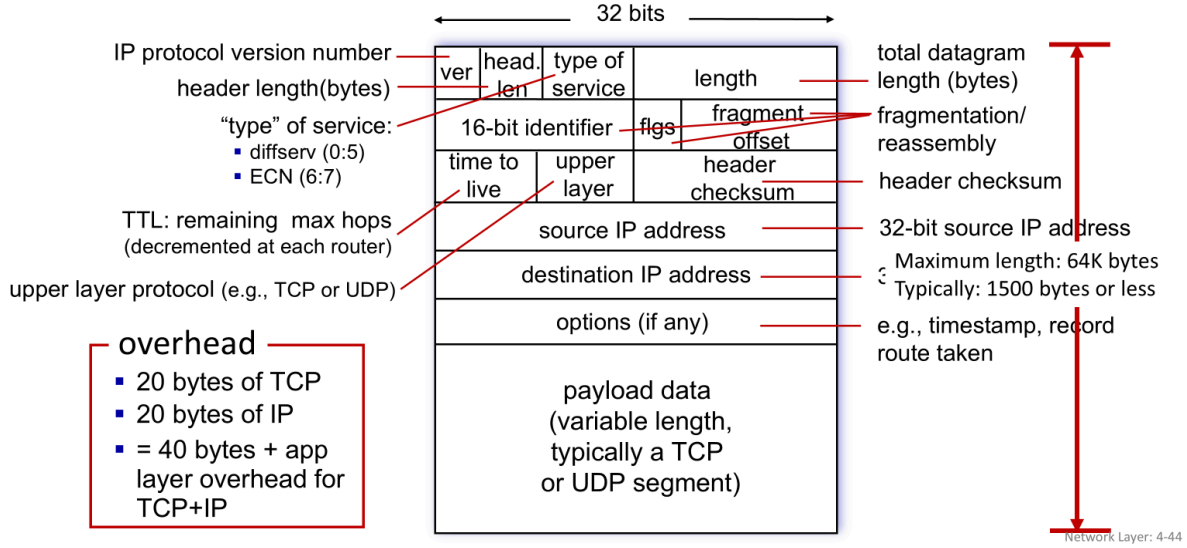
Sağ üstteki kutu **IP protocol** başlığını taşımaktadır ve üç temel unsuru vurgulamaktadır: **datagram format, addressing ve packet handling conventions**. Bu, IP'nin yalnızca bir adresleme sistemi olmadığını gösterir. IP aynı zamanda datagramın biçimini tanımlar; hangi başlık alanlarının bulunacağını belirler; adreslerin nasıl yorumlanacağını ortaya koyar; paketlerin ağda nasıl ele alınacağına ilişkin kuralları belirler. Örneğin TTL'nin nasıl kullanılacağı, parçalama ile ilgili davranışlar ya da üst katman protokolünün nasıl belirtileceği gibi konular bu genel çerçeveye dahildir. Bu nedenle IP, ağ katmanının temel veri taşıma protokolüdür.

Sağ alttaki **ICMP protocol** kutusu ise ağ katmanının yalnızca veri taşıma değil, aynı zamanda **hata bildirme ve router signaling** işlevlerine de sahip olduğunu göstermektedir. ICMP, bir paket hedefe ulaşmadığında ya da ağda belirli sorunlar oluştuğunda bu durumun bildirilmesinde kullanılır. Ayrıca yönlendiriciler ve uç sistemler arasında bazı kontrol ve tanılama amaçlı mesajların iletilmesini sağlar. Bu yönüyle ICMP, IP'nin yanına eklenmiş ikincil bir protokol değil; ağ katmanının çalışmasını gözlemlleme ve hata durumlarını yönetme açısından tamamlayıcı bir parçadır.

Bu slaytın önemli tarafı, ağ katmanını tek parçalı bir mekanizma gibi değil, birbiriyle ilişkili alt bileşenlerden oluşan bir sistem olarak göstermesidir. **Path-selection algorithms** yol bilgisini üretir, bu bilgi **forwarding table** içine yerleşir, veri düzleminde bu tablo kullanılarak paketler iletilir, **IP** datagramın yapısını ve adresleme mantığını sağlar, **ICMP** ise hata ve sinyal mesajlarıyla destekleyici işlev görür. Böylece ağ katmanının hem kontrol boyutu hem veri taşıma boyutu aynı çerçevede görünür hale gelir.

Özetle bu slaytın ana mesajı şudur: İnternet'te ağ katmanı; **routing protokolleri veya SDN denetleyicileriyle yol seçimi yapan**, bu bilgiyi **forwarding table** biçiminde kullanan, **IP** ile datagram biçimi ve adreslemeyi tanımlayan, **ICMP** ile hata raporlama ve sinyalleşmeyi sağlayan bütünleşik bir yapıdır. Bu nedenle ağ katmanı, yalnızca paketleri taşımakla kalmaz; onları doğru yola yerleştiren ve ağ içindeki durumları görünür kılan temel internet katmanıdır.

IP Datagram format



Ders Notu: IP Datagram Format

Bu slayt, IPv4 datagramının başlık yapısını ve her alanın ne işe yaradığını göstermektedir. Ağ katmanında yönlendiricilerin baktığı temel veri birimi **IP datagramı** olduğu için, bu formatı anlamak çok önemlidir. Önceki slaytlarda forwarding table, destination-based forwarding ve longest prefix matching gibi kavramlar işlendi. Bunların hepsi, sonuçta bir datagram başlığındaki alanlara bakılarak uygulanır. Bu nedenle IP datagram formatı, ağ katmanının veri düzlemindeki en temel yapı taşlarından biridir. Kitapta da IP protokolünün ağ katmanında datagram biçimini ve adresleme mantığını tanımladığı belirtilmektedir.

Şekilde datagramın üst kısmının **başlık (header)**, alt kısmının ise **payload data** olduğu görülmektedir. Başlık kısmı kontrol ve yönlendirme için gerekli bilgileri taşır; payload ise genellikle üst katmandan gelen veridir. Slaytta bu yük verisinin çoğu zaman bir **TCP ya da UDP segmenti** olduğu belirtilmiştir. Yani taşıma katmanı verisi, ağ katmanında IP datagramı içine kapsüllenmiş olur. Bu, katmanlı mimarinin doğrudan bir sonucudur. Başlık alanları sayesinde yönlendirici paketin nasıl ele alınacağını anlar; ancak taşınan asıl kullanıcı verisi aşağıdaki payload bölümündedir.

İlk alan **version**, yani IP sürüm numarasıdır. Bu alan, kullanılan IP sürümünü belirtir. Burada kastedilen yapı IPv4'tür; yani bu alan, paketin hangi tür IP başlığına sahip olduğunu bildirir. Bu küçük görünen alan oldukça önemlidir; çünkü yönlendirici paketi doğru biçimde yorumlamak için hangi protokol biçiminin kullanıldığını bilmek zorundadır.

Bunun hemen yanında **header length** alanı vardır. Bu alan, IP başlığının kaç bayt uzunluğunda olduğunu gösterir. Başlığın sabit kısmı bulunsa da, alttaki **options** alanı nedeniyle başlık uzunluğu değişebilir. Bu yüzden yönlendirici, payload'ın nerede başladığını anlamak için başlığın toplam boyunu bilmelidir. Slayt da bu alanın başlık uzunluğunu bayt cinsinden ifade ettiğini göstermektedir.

Sonraki önemli alan **type of service** olarak gösterilen bölümdür. Slaytta bunun içinde özellikle **diffserv** ve **ECN** bitlerine işaret edilmektedir. Bu alan, ağın pakete nasıl davranabileceğine dair ipuçları sağlar. Örneğin hizmet farklılaştırması veya tıkanıklık bildirimi gibi mekanizmalar burada taşınabilir. Bu yüzden bu alan, yalnızca başlık içinde duran pasif bir bilgi değil; yönlendiricilerin hizmet kalitesi ve tıkanıklık yönetimiyle ilgili kararlarında kullanılabilecek bir kontroldür.

Length alanı, tüm datagramın toplam uzunluğunu belirtir. Yani yalnızca başlığı değil, başlık artı payload toplamını ifade eder. Slaytta sağ tarafta da bu alanın **total datagram length (bytes)** olarak açıklandığı görülmektedir. Bu alanın önemi şudur: Alıcı sistem ve yönlendiriciler, paketin toplam sınırlarını bilmek zorundadır. Ayrıca slaytta maksimum uzunluğun yaklaşık **64 KB** olduğu, fakat pratikte tipik datagram boyutunun çoğu zaman **1500 bayt ya da daha az** olduğu da belirtilmiştir. Bu ikinci bilgi, gerçek ağlarda MTU sınırları nedeniyle IP paketlerinin neden çok daha küçük kaldığını anlamak açısından önemlidir.

Başlığın ortasındaki **16-bit identifier, flags ve fragment offset** alanları, **fragmentation/reassembly** işlemleriyle ilgilidir. Bu alanlar, büyük bir IP datagramı daha küçük parçalara bölündüğünde, bu parçaların hangi orijinal datagrama ait olduğunu ve alıcıda nasıl yeniden birleştirileceğini belirtir. Bu alanlar birlikte çalışır. Identifier, parçaların aynı datagramdan geldiğini gösterir; flags parçalama ile ilgili kontrol bilgisini taşır; fragment offset ise her parçanın orijinal datagram içindeki konumunu bildirir. Bu nedenle bu üç alan birlikte yorumlanmalıdır.

Time to live (TTL) alanı, slaytta da belirtildiği gibi paketin kalan azami sıçrama sayısını gösterir. Her yönlendiricide bu değer azaltılır. Amaç, yönlendirme döngülerinde bir paketin sonsuza kadar ağda dolaşmasını önlemektir. TTL sıfıra düşerse paket atılır. Bu alanın önemi büyüktür; çünkü ağda hata ya da döngü olduğunda paketin sistem kaynaklarını süresiz tüketmesini engeller. Yani TTL, ağ katmanında güvenlikten çok **kararlılık ve kaynak koruma** işlevi gören temel bir mekanizmadır.

Upper layer olarak gösterilen alan, IP datagramı içindeki yükün hangi üst katman protokolüne ait olduğunu belirtir. Slaytta örnek olarak **TCP veya UDP** verilmiştir. Bu alan, alıcı sistemin payload'ı doğru taşıma katmanı protokolüne teslim etmesini sağlar. Yani bu bilgi olmadan alıcı, datagramın içindeki verinin TCP segmenti mi, UDP segmenti mi, yoksa başka bir protokole mi ait olduğunu bilemez.

Header checksum alanı, IP başlığında hata olup olmadığını kontrol etmek için kullanılır. Burada dikkat edilmesi gereken nokta, bu alanın yalnızca başlığı korumasıdır; payload verisi bu alanla korunmaz. Bu nedenle üst katman protokolleri de kendi hata denetim mekanizmalarını taşır. Başlıktaki bozulmalar yönlendirme kararlarını doğrudan etkileyebileceği için bu koruma alanı önemlidir.

Alt tarafta yer alan **source IP address** ve **destination IP address** alanları, datagramın kaynağını ve hedefini gösterir. Özellikle hedef adres, yönlendiricilerin forwarding kararlarında kullandığı en kritik alandır. Destination-based forwarding ve longest prefix matching doğrudan bu hedef adres üzerinden yapılır. Source adres ise paketin nereden geldiğini belirtir ve geri dönüş iletişimi ya da çeşitli kontrol işlemleri açısından önemlidir.

Options (if any) alanı, başlıkta isteğe bağlı bilgilerin taşınmasına olanak tanır. Slaytta örnek olarak **timestamp** ve **record route taken** verilmiştir. Bu alanlar esneklik sağlar; ancak başlığı büyüttüğü ve işlemeyi zorlaştırdığı için pratikte sınırlı kullanılır. Yine de IP başlığının sabit değil, genişletilebilir bir yapıya sahip olduğunu göstermesi açısından önemlidir.

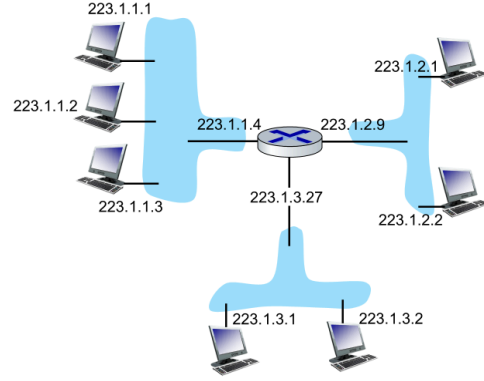
En alttaki **payload data** bölümü, değişken uzunluktadır ve tipik olarak bir TCP veya UDP segmenti taşır. Kullanıcı uygulamasının gerçek verisi en sonunda burada bulunur. Bu da bize başlığın neden “overhead” olarak adlandırıldığını gösterir: başlık doğrudan kullanıcı verisi değildir; fakat kullanıcı verisinin doğru taşınabilmesi için gereklidir.

Slayttaki kırmızı kutuda verilen **overhead** notu da bu yüzden önemlidir. Burada tipik bir TCP+IP iletişiminde yaklaşık **20 bayt TCP başlığı + 20 bayt IP başlığı = 40 bayt** protokol yükü olduğu belirtilmektedir. Buna uygulama katmanının kendi başlıkları da eklenebilir. Bu, özellikle küçük veri parçaları gönderildiğinde neden toplam iletimin önemli bir kısmının kontrol bilgisine ayrıldığını anlamak açısından önemlidir. Yani ağda taşınan her bit doğrudan kullanıcı verisi değildir; önemli bir kısmı bu veriyi düzenleyen ve yönlendiren başlıklardır.

Özetle bu slaytın ana mesajı şudur: **IP datagramı**, yönlendirme ve teslim işlemleri için gerekli başlık alanlarını taşıyan bir ağ katmanı veri birimidir. Başlıkta sürüm, başlık uzunluğu, hizmet türü, toplam uzunluk, parçalama bilgileri, TTL, üst katman protokolü, checksum ve kaynak-hedef adresleri bulunur; alttaki payload bölümünde ise genellikle TCP veya UDP verisi taşınır. Bu yapı, internet üzerindeki her paketin nasıl tanımlandığını, nasıl yönlendirildiğini ve nasıl sınırlandırıldığını belirleyen temel biçimdir.

IP Adresleme: Giriş

- **IP adresi** : Her bir host veya yönlendirici arabirimiyle ilişkili 32 bitlik **arayüz**
- **Arayüz**: Host/yönlendirici ile fiziksel bağlantı arasındaki bağlantı
 - Yönlendiriciler genellikle birden fazla arayüze sahiptir
 - Bir ana bilgisayar genellikle bir veya iki ağ arayüze sahiptir (e.g., wired Ethernet, wireless 802.11)



dotted-decimal IP address notation:

223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

Network Layer: 4-45

Ders Notu: IP addressing: introduction

Bu slayt, IP adreslemenin en temel fikrini vermektedir: **IP adresi, bir cihaza genel olarak değil, o cihazın ağa bağlı olan arayüzüne (interface) atanır.** Buradaki en önemli nokta budur. Günlük dilde çoğu zaman “bilgisayarın IP adresi” ya da “router’ın IP adresi” denir; ancak daha doğru ifade, bir **host ya da router arayüzünün IP adresi** olduğudur. Kitapta da IP adresinin, her host ya da router **interface**’i ile ilişkili 32 bitlik bir tanımlayıcı olduğu belirtilmektedir.

Slaytta IP adresinin **32 bitlik bir tanımlayıcı** olduğu açıkça yazılmıştır. Bu, IPv4 adresleme için geçerlidir. Yani her IPv4 adresi toplam 32 bitten oluşur. Ancak insanlar bit dizilerini doğrudan okumakta zorlanacağı için bu adresler genellikle **dotted-decimal notation** ile, yani noktalı ondalık gösterimle yazılır. Slaytın alt kısmındaki örnekte **223.1.1.1** adresinin aslında dört ayrı 8 bitlik bölümden oluştuğu ve ikilik karşılığının birlikte toplam 32 bit verdiği gösterilmektedir. Bu gösterim, adresin hem makine tarafından işlenebilir hem de insan tarafından okunabilir olmasını sağlar.

İkinci temel kavram **interface** kavramıdır. Slaytta interface, host ya da router ile fiziksel bağlantı arasındaki bağlantı noktası olarak tanımlanmaktadır. Bu tanım çok önemlidir; çünkü ağ katmanında adresleme, cihazların kendisini değil, cihazların ağa açılan kapılarını hedef alır. Bir cihazın birden fazla fiziksel ya da mantıksal bağlantısı varsa, bunların her biri ayrı bir interface olarak düşünülebilir ve ayrı IP adreslerine sahip olabilir. Bu nedenle IP adresleme, ağ topolojisini anlamanın da temelidir.

Bu bağlamda slaytta **router'ların tipik olarak birden fazla interface'e sahip olduğu** vurgulanmaktadır. Bunun nedeni açıktır: router, farklı ağları birbirine bağlayan cihazdır. Dolayısıyla her bağlı olduğu ağ için ayrı bir arayüze ihtiyaç duyar. Sağdaki şekilde ortadaki yönlendiricinin üç farklı ağa bağlı olduğu görülmektedir. Bu yüzden router üzerinde üç farklı IP adresi yer almaktadır: soldaki ağ için **223.1.1.4**, sağdaki ağ için **223.1.2.9**, alttaki ağ için ise **223.1.3.27**. Buradan çıkarılması gereken temel sonuç şudur: Router tek bir IP adresine sahip değildir; her arayüzü için ayrı bir IP adresi vardır.

Buna karşılık bir **host** genellikle bir veya iki arayüze sahiptir. Slaytta örnek olarak kablolu Ethernet ve kablosuz 802.11 verilmiştir. Yani bir dizüstü bilgisayar hem Wi-Fi hem Ethernet üzerinden bağlanabiliyorsa, bunlar iki ayrı interface olarak düşünülebilir. Ama birçok basit durumda host yalnızca tek bir aktif arayüzle çalışır. Bu da host'ların neden çoğu zaman tek IP adresi varmış gibi algılandığını açıklar. Aslında bu, tek aktif arayüzlü olmanın pratik sonucudur.

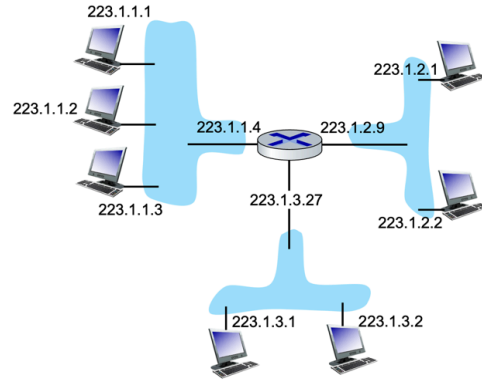
Sağdaki şekil ayrıca çok önemli bir ağ mantığını da göstermektedir: aynı fiziksel ağ parçasına bağlı cihazların adreslerinde ortak bir bölüm bulunur. Örneğin soldaki ağdaki cihazlar **223.1.1.x**, sağdaki ağdakiler **223.1.2.x**, alttaki ağdakiler ise **223.1.3.x** biçimindedir. Bu, adreslerin rastgele verilmediğini; ağ bölümlerini yansıtacak şekilde yapılandırıldığını gösterir. Daha sonraki slaytlarda bu mantık alt ağlar ve ön ekler üzerinden daha ayrıntılı biçimde açıklanacaktır. Burada ilk sezgi şudur: aynı yerel ağdaki arayüzler adreslerinin bir kısmını paylaşır, router ise her ağ segmentine o segmentle uyumlu ayrı bir adresle bağlanır.

Bu slaytın önceki konularla bağlantısı da doğrudandır. Destination-based forwarding yapılırken yönlendirici hedef IP adresine bakar. Longest prefix matching uygulanırken yine bu adresin bit yapısı kullanılır. Dolayısıyla IP adresleme yalnızca isimlendirme işi değildir; yönlendirme ve iletme kararlarının temel girdisidir. Adres yapısı anlaşılmadan forwarding tablolarının neden belirli ön eklere göre düzenlendiği de tam olarak anlaşılamaz.

Özetle bu slaytın ana mesajı şudur: **IPv4 adresi 32 bitlik bir tanımlayıcıdır ve bir cihaza değil, o cihazın ağ arayüzüne atanır.** Router'ların birden çok arayüzü olduğu için birden çok IP adresi olabilir; host'lar ise genellikle bir ya da iki arayüze sahiptir. Adresler çoğu zaman noktalı ondalık biçimde yazılır ve aynı ağ segmentindeki arayüzler ortak bir adres ön ekini paylaşır. Bu yapı, hem adreslemenin hem de yönlendirme kararlarının temelini oluşturur.

IP Adresleme: Giriş

- **IP address:** 32-bit identifier associated with each host or router *interface*
- **interface:** connection between host/router and physical link
 - router's typically have multiple interfaces
 - host typically has one or two interfaces (e.g., wired Ethernet, wireless 802.11)



dotted-decimal IP address notation:

223.1.1.1 = 11011111 00000001 00000001 00000001

223 1 1 1

Network Layer: 4-46

Ders Notu: IP addressing: introduction

Bu slayt, IP adreslemenin en temel ve en kritik ilkesini öne çıkarmaktadır: **IP adresi bir cihaza değil, o cihazın ağ arayüzüne atanır.** Önceki slaytta bu tanım verilmişti; burada ise görsel taraf öne çıkarılarak bu mantık daha somut hale getirilmektedir. Bu yüzden bu slaytı, tanımın kendisinden çok, tanımın ağ topolojisi üzerinde nasıl görüldüğünü açıklayan bir pekiştirme slaytı olarak okumak gerekir.

Sağdaki şekilde ortada bir yönlendirici ve ona bağlı üç ayrı ağ görülmektedir. Her ağdaki uç sistemlerin adresleri birbirine benzer bir ön ek taşımaktadır. Soldaki ağdaki cihazlar **223.1.1.x**, sağdaki ağdaki cihazlar **223.1.2.x**, alttaki ağdaki cihazlar ise **223.1.3.x** adres yapısına sahiptir. Bu, IP adreslerinin rastgele verilmediğini; aynı fiziksel ya da mantıksal ağ segmentinde bulunan arayüzlerin ortak bir adres bölümünü paylaştığını gösterir. Buradaki ortak bölüm, ileride daha ayrıntılı işlenecek olan ağ ön eki mantığının ilk sezgisel örneğidir.

Şekilde yönlendiricinin de tek bir IP adresiyle değil, bağlı olduğu her ağ için ayrı bir arayüz adresiyle gösterildiğine dikkat edilmelidir. Sol ağa bakan arayüz **223.1.1.4**, sağ ağa bakan arayüz **223.1.2.9**, alttaki ağa bakan arayüz ise **223.1.3.27** adresine sahiptir. Bu çok önemli bir noktadır; çünkü yönlendirici, farklı ağları birbirine bağlayan cihaz olduğu için her ağ segmentine o segmentin adresleme düzeniyle uyumlu ayrı bir arayüz üzerinden bağlanır. Dolayısıyla router'ın "tek bir IP adresi" yoktur; her bağlantı noktası için ayrı bir IP adresi vardır.

Bu görsel aynı zamanda host ile router arasındaki farkı da netleştirir. Uç sistemler çoğu zaman yalnızca bir yerel ağa bağlıdır; bu yüzden genellikle tek bir aktif IP adresi varmış gibi görünürler. Router ise birden fazla ağı birbirine bağladığından, aynı anda birden fazla IP adres taşır. Bu da forwarding işleminin nasıl mümkün olduğunu açıklar: Router, bir ağdan aldığı paketi başka bir ağdaki uygun arayüzü üzerinden iletir.

Slayttaki sol alt metinlerin soluklaştırılmış olması da anlamlıdır. Burada artık odak tanımın kendisinden çok örneğin kendisine kaymıştır. Yani “IP address 32-bit identifier’dır” ve “interface, host/router ile fiziksel link arasındaki bağlantıdır” bilgileri daha önce verilmiştir; bu slaytta ise bu tanımların görsel karşılığı gösterilmektedir. Başka bir deyişle, teori geri planda bırakılmış, uygulamalı ağ resmi ön plana alınmıştır.

Bu slaytın ileride işlenecek subnetting ve longest prefix matching konularına doğrudan hazırlık yaptığı da görülmelidir. Çünkü aynı ağdaki makinelerin neden ortak bir ön ek taşıdığı, yönlendiricinin neden her ağa farklı bir adresle bağlandığı ve paketlerin neden hedef adrese göre doğru ağ segmentine yönlendirildiği gibi sorular, birazdan ağ adresi ve host adresi ayrımı üzerinden daha sistematik biçimde açıklanacaktır.

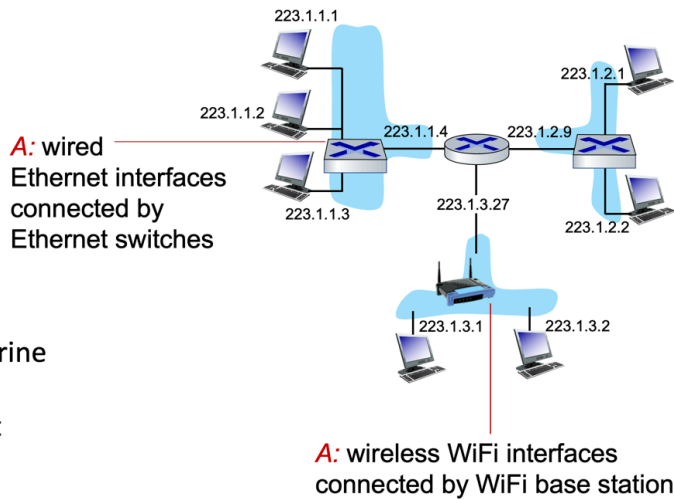
Özetle bu slaytın ana mesajı şudur: **IP adresleme, arayüz temellidir** ve bir ağ topolojisinde aynı segmentteki arayüzler ortak bir adres ön ekini paylaşır. Router ise her bağlı olduğu ağ için ayrı bir arayüze ve dolayısıyla ayrı bir IP adresine sahiptir. Bu yapı, ağların birbirinden ayrılmasını ve paketlerin doğru ağ segmentine yönlendirilmesini mümkün kılar.

IP Adresleme: Giriş

S: Arayüzler aslında nasıl birbirine bağlanır?

C: Bunu 6. ve 7. bölümlerde göreceğiz

Şimdilik: Bir arayüzün diğerine nasıl bağlandığını (arada yönlendirici olmadan) dert etmenize gerek yok



Ders Notu: IP Adresleme: Giriş

Bu slayt, önceki IP adresleme örneğinde görülen ağ yapısına yeni bir açıklık getirmektedir. Aynı ağ içindeki arayüzlerin birbirine “mantıksal olarak” bağlı olduğunu söylemek yeterli değildir; pratikte bu bağlantının hangi cihazlar üzerinden kurulduğunu da düşünmek gerekir. Slaytın sorduğu temel soru budur: **Arayüzler gerçekte nasıl birbirine bağlanır?**

Burada verilmek istenen ana fikir, IP adreslemenin ağ katmanına ait olduğudur; ancak arayüzlerin fiziksel ya da yerel ağ düzeyinde nasıl bağlandığı sorusu, daha çok alt katmanların konusudur. Bu yüzden slayt açıkça “bunu 6. ve 7. bölümlerde öğreneceğiz” demektedir. Yani şu aşamada amaç, bağlantının ayrıntılı mekanizmasını değil, adresleme yapısının üstten görünümünü anlamaktır.

Şekilde üç farklı yerel ağ bölgesi yine aynı adres bloklarıyla gösterilmektedir: **223.1.1.x**, **223.1.2.x** ve **223.1.3.x**. Ancak bu kez bu ağların içinde arayüzlerin doğrudan birbirine bağlı olmadığı, arada yerel ağ cihazlarının bulunduğu gösterilmektedir. Soldaki ve sağdaki ağlarda cihazlar **Ethernet switch** üzerinden bağlanmaktadır. Bu, aynı IP ağındaki kablolu arayüzlerin çoğu zaman fiziksel olarak bir anahtar cihaz üzerinden birbirine ulaştığını gösterir. Ortadaki yönlendiricinin ilgili arayüzü de bu yerel Ethernet yapısına bağlanır. Dolayısıyla aynı IP ön ekini paylaşan arayüzler, pratikte çoğu zaman bir anahtarlama cihazı üzerinden aynı yerel ağda bir araya gelir.

Alt taraftaki ağda ise farklı bir örnek verilmiştir. Burada cihazlar bir **WiFi base station**, yani kablosuz erişim noktası üzerinden bağlanmaktadır. Bu da aynı IP ağının her zaman kablolu bir Ethernet yapısı olmak zorunda olmadığını gösterir. Yani ağ katmanı açısından bakıldığında alttaki cihazlar yine aynı **223.1.3.x** ağındadır; fakat bağlantı biçimi kablolu değil, kablosuzdur. Bu ayrım önemlidir çünkü IP adresleme mantığı değişmezken, alt katmandaki erişim teknolojisi değişebilir.

Bu slaytın özellikle vurguladığı nokta şudur: **Şimdilik bir arayüzün başka bir arayüze nasıl bağlandığı ayrıntısına takılmak gerekmiyor.** Yani IP adresleme konusunu öğrenirken öncelikle “hangi arayüz hangi ağda?” sorusuna odaklanmak yeterlidir. O arayüzlerin Ethernet anahtarıyla mı, WiFi erişim noktasıyla mı, yoksa başka bir bağlantı teknolojisiyle mi birleştirildiği daha sonra ele alınacaktır. Bu, top-down yaklaşımın doğal bir sonucudur: önce ağ katmanının mantığını kurmak, sonra alt katman ayrıntılarına inmek.

Bu yüzden bu slayt, iki önemli kavramı ayırmaya yardımcı olur. Birincisi **mantıksal ağ üyeliği**dir: aynı IP ön ekine sahip arayüzler aynı ağdadır. İkincisi ise **fiziksel/yerel bağlantı biçimi**dir: bu arayüzler switch, access point ya da başka bir yerel ağ

mekanizmasıyla bağlanabilir. Ağ katmanında asıl önemli olan birincisidir; ikincisi ise bağlantı katmanı ve fiziksel katman konularında ayrıntılandırılır.

Özetle bu slaytın ana mesajı şudur: Aynı IP ağındaki arayüzler gerçek hayatta doğrudan değil, çoğu zaman **Ethernet switch** ya da **WiFi erişim noktası** gibi yerel ağ cihazları üzerinden bağlanır. Ancak IP adresleme açısından şu aşamada önemli olan, bu arayüzlerin aynı ağ ön ekini paylaşmasıdır; bağlantının tam olarak hangi alt katman teknolojisiyle kurulduğu ise daha sonra incelenecektir.

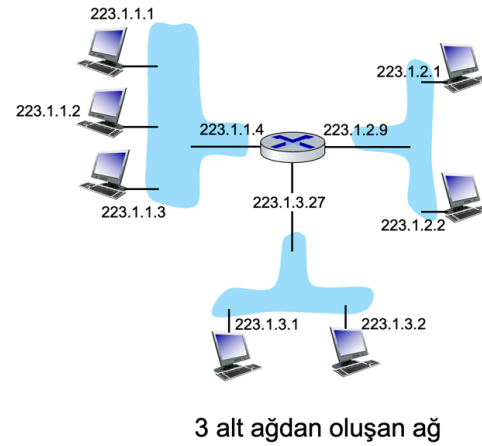
Alt Ağlar

■ Alt ağ nedir?

- Aralarında bir yönlendiriciye gerek kalmadan fiziksel olarak birbirlerine ulaşabilen cihaz arayüzleri

■ IP adreslerinin bir yapısı vardır :

- **Alt ağ kısmı** : Aynı alt ağdaki cihazların üst bitleri ortaktır
- **Host kısmı**: Kalan sıralı bitler



Network Layer: 4-48

Ders Notu: Subnets

Bu slayt, IP adreslemede çok önemli bir kavram olan **subnet** kavramını tanıtmaktadır. Buradaki temel fikir şudur: Aynı ağın içinde yer alan bazı arayüzler birbirine ulaşmak için bir yönlendiriciden geçmek zorunda değildir. Eğer iki cihaz ya da arayüz, arada bir yönlendirici olmadan fiziksel olarak birbirine erişebiliyorsa, bunlar aynı **subnet** içinde kabul edilir. Slaytta da subnet, **bir intervening router'dan geçmeden birbirine fiziksel olarak ulaşabilen arayüzler kümesi** olarak tanımlanmaktadır. Bu tanım çok önemlidir; çünkü subnet kavramı yalnızca adres benzerliğine değil, aynı zamanda yönlendirme gerekir gerekmediğine de dayanır.

Sağdaki şekilde bu mantık açık biçimde görülmektedir. Ortadaki yönlendirici üç ayrı yerel ağ segmentine bağlıdır. Soldaki grup **223.1.1.x**, sağdaki grup **223.1.2.x**, alttaki

grup ise **223.1.3.x** adreslerini kullanmaktadır. Bu üç grup kendi içlerinde birbirine doğrudan erişebilir; ancak bir gruptan diğerine geçmek için ortadaki router üzerinden yönlendirme gerekir. Bu nedenle şeklin altında da belirtildiği gibi burada **3 subnetten oluşan bir ağ** vardır. Yani tüm topoloji tek bir büyük ağ gibi görünse de, ağ katmanı açısından üç ayrı alt ağ söz konusudur.

Bu noktada IP adreslerinin neden yapılandırılmış biçimde verildiği anlaşılır. Slaytta özellikle **IP addresses have structure** ifadesi yer almaktadır. Yani bir IP adresi yalnızca benzersiz bir sayı değildir; kendi içinde iki temel parçaya ayrılır: **subnet part** ve **host part**.

Subnet part, aynı subnet içindeki cihazların ortak olarak paylaştığı yüksek anlamlı bitleri ifade eder. Başka bir deyişle, aynı alt ağdaki tüm arayüzlerin adreslerinde ortak bir üst bölüm vardır. Örneğin şekilde soldaki subnet içindeki tüm cihazlar **223.1.1** ön ekini paylaşmaktadır. Sağdaki subnet için bu ortak bölüm **223.1.2**, alttaki subnet için ise **223.1.3** biçimindedir. Bu ortak kısım, adresin hangi alt ağa ait olduğunu gösterir.

Host part ise geri kalan düşük anlamlı bitlerdir. Aynı subnet içindeki cihazları birbirinden ayıran bölüm budur. Örneğin **223.1.1.1**, **223.1.1.2**, **223.1.1.3** ve router'ın soldaki arayüzü olan **223.1.1.4** adreslerinde ilk üç bölüm ortak subnet kısmını, son bölüm ise host kısmını temsil etmektedir. Böylece aynı alt ağ içinde hangi cihazın hangi arayüz olduğuna karar verilebilir.

Bu ayrımın ağ katmanı açısından önemi çok büyüktür. Çünkü bir host, hedef adresin kendi subnetinde mi yoksa başka bir subnette mi olduğunu bu yapıya bakarak anlar. Eğer hedef aynı subnet içindeyse, paket doğrudan yerel ağ üzerinden iletilir. Eğer hedef başka bir subnetteyse, paket doğrudan gönderilemez; bir yönlendiriciye iletilmesi gerekir. Dolayısıyla subnet kavramı, yalnızca adresleme düzeni değil, aynı zamanda **paketin yerel olarak mı yoksa yönlendirme yoluyla mı iletileceğini belirleyen temel mantıktır**.

Bu slayt aynı zamanda önceki IP adresleme örneklerini de netleştirir. Daha önce aynı ön eki paylaşan cihazların aynı ağ segmentinde olduğu görülmüştü. Burada bunun adı konmaktadır: Bu ortak ön eki paylaşan ve arada router olmadan erişebilen cihazlar aynı **subnet** içindedir. Böylece adres ön ekleri ile fiziksel/topolojik ağ yapısı arasındaki ilişki görünür hale gelir.

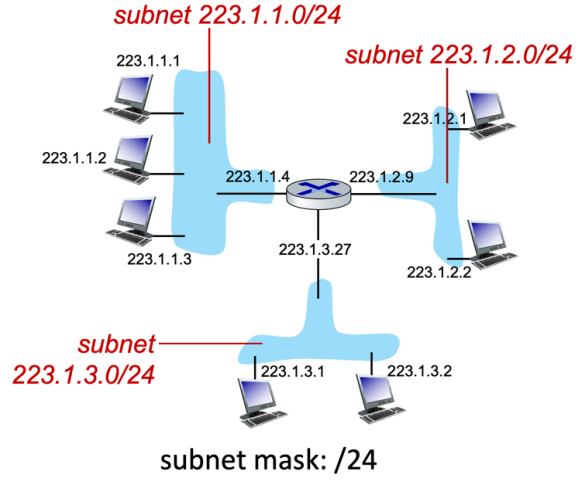
Özetle bu slaytın ana mesajı şudur: **Subnet**, aralarında bir yönlendirici olmadan birbirine fiziksel olarak erişebilen arayüzler kümesidir. IP adresleri de bu yapıyı yansıtacak şekilde iki parçalı düşünülür: aynı subnet içindeki cihazların paylaştığı **subnet part** ve onları birbirinden ayıran **host part**. Bu yapı, hem adreslerin düzenli

verilmesini sağlar hem de bir paketin doğrudan mı yoksa router üzerinden mi gönderileceğini belirler.

Alt Ağlar

Alt ağları tanımlama yöntemi :

- Her bir arayüzü host veya yönlendiricisinden ayırarak, birbirinden izole edilmiş ağlardan oluşan “**adacıklar**” oluşturmak
- Her bir izole ağa **alt ağ (subnet)** denir.



(Üst sıralı bitler 24 bits: Bu üç subnet içinde yer alan cihazlar, adreslerinin ilk 24 bitini ortak paylaşır; geri kalan bitler ise host kısmını oluşturur.)

Network Layer: 4-49

Ders Notu: Subnets

Bu slayt, subnet kavramını tanımlamaktan bir adım ileri gidip **bir ağ topolojisi**nde **subnetlerin nasıl belirleneceğini** göstermektedir. Önceki slaytta subnet, aralarında bir yönlendirici olmadan birbirine fiziksel olarak ulaşabilen arayüzler kümesi olarak açıklanmıştı. Bu slaytta ise bunu sistematik hale getiren bir yöntem verilmektedir: Her arayüzü bağlı olduğu host ya da router'dan kavramsal olarak ayır ve geriye kalan izole ağ parçalarına bak. Her bir izole parça bir **subnet** olarak kabul edilir.

Slaytta buna “**recipe for defining subnets**” denmektedir. Buradaki yöntem oldukça öğreticidir. Ağdaki tüm host ve router arayüzlerini, bağlı oldukları cihazlardan ayırdığını düşün. O zaman yönlendiriciler artık ağ parçalarını birbirine bağlayan merkezler olarak değil, sadece uçta kalan arayüzler olarak görünür. Geriye kalan her bağımsız bağlantı bölgesi bir “ada” gibi ortaya çıkar. İşte her böyle ada, bir **subnet**'tir. Bu yöntem subnet kavramını yalnızca adres benzerliği üzerinden değil, topolojik ayrışma üzerinden de anlamayı sağlar.

Şekilde ortadaki router üç farklı bağlantı bölgesine bağlıdır. Soldaki bölge, **223.1.1.x** adresli cihazları ve router'ın bu ağa bakan arayüzünü içerir. Sağdaki bölge, **223.1.2.x** adresli cihazları ve router'ın sağ arayüzünü içerir. Altındaki bölge ise **223.1.3.x** adresli

cihazları ve router'ın alt arayüzünü içerir. Router'ın kendisi bu üç bölgeyi birbirine bağlayan cihazdır; fakat subnet tanımlarken her bağlantı bölgesi ayrı düşünülür. Bu yüzden bu topolojide üç ayrı subnet vardır.

Slaytta bu subnetler açık biçimde adlandırılmıştır:

223.1.1.0/24

223.1.2.0/24

223.1.3.0/24

Bu gösterim, CIDR biçimindeki subnet gösterimidir. Buradaki **/24**, adresin ilk 24 bitinin subnet kısmını oluşturduğunu belirtir. Slaytın alt kısmında da açıkça “**subnet mask: /24**” ve “**high-order 24 bits: subnet part of IP address**” ifadesi yer almaktadır. Bunun anlamı şudur: Bu üç subnet içinde yer alan cihazlar, adreslerinin ilk 24 bitini ortak paylaşıyor; geri kalan bitler ise host kısmını oluşturur.

Örneğin **223.1.1.0/24** subnetinde bulunan adresler için ilk üç oktet, yani **223.1.1**, ortak subnet bölümüdür. Son oktet ise bu subnet içindeki hostu belirtir. Bu yüzden **223.1.1.1**, **223.1.1.2**, **223.1.1.3** ve router'ın sol arayüzü olan **223.1.1.4**, aynı subnet içinde yer alır. Benzer şekilde **223.1.2.0/24** ve **223.1.3.0/24** için de aynı mantık geçerlidir.

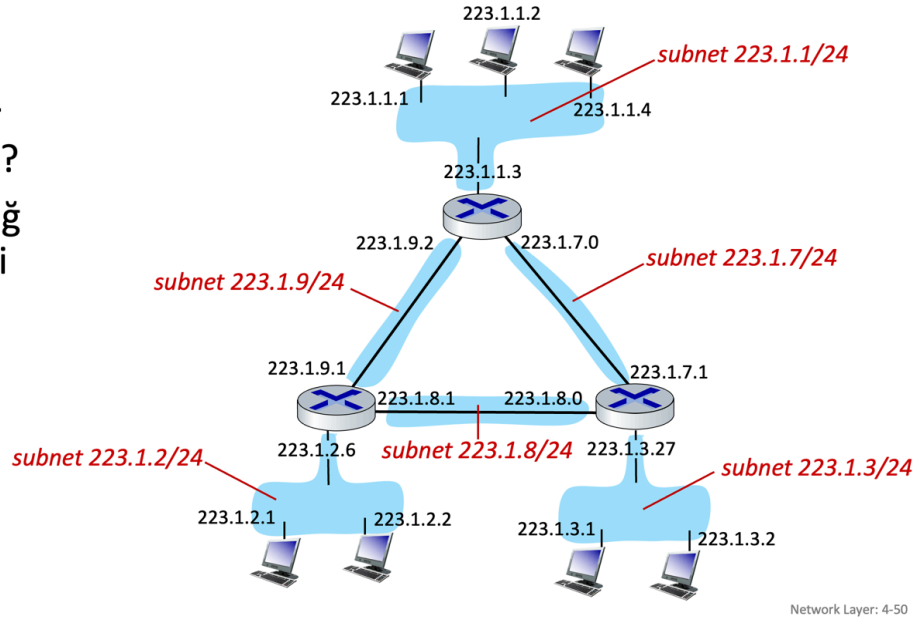
Burada önemli olan nokta, subnet adresi ile host adresinin aynı şey olmamasıdır. Örneğin **223.1.1.0/24** ifadesi, tek bir cihaza verilmiş bir adres değil; tüm alt ağı temsil eden ağ tanıdır. İçindeki tek tek cihazlar ise bu subnetin host adresleriyle ifade edilir. Yani subnet, adreslerin ait olduğu ortak ağ bağlamını; host kısmı ise o ağ içindeki bireysel arayüzü gösterir.

Bu slayt aynı zamanda forwarding mantığıyla da doğrudan ilişkilidir. Yönlendiriciler çoğu zaman tek tek her host için değil, **subnetler** için yönlendirme bilgisi tutar. Çünkü hedef adresin hangi subnet içinde olduğunu anlamak, paketin hangi çıkış arayüzüne gönderileceğini belirlemek için yeterlidir. Bu da neden longest prefix matching ve ağ ön eki kavramlarının bu kadar önemli olduğunu açıklar. Yönlendirici çoğu zaman “bu paket 223.1.2.0/24 ağına mı gidiyor?” gibi düşünür; “tek tek 223.1.2.1 veya 223.1.2.2'ye özel ayrı fiziksel yol var mı?” diye düşünmez.

Özetle bu slaytın ana mesajı şudur: **Subnet tanımlamak için, ağdaki arayüzleri bağlı oldukları cihazlardan ayırarak ortaya çıkan izole ağ bölgelerine bakılır; her izole bölge bir subnettir.** Bu örnekte üç subnet vardır: **223.1.1.0/24**, **223.1.2.0/24** ve **223.1.3.0/24**. Buradaki **/24**, adresin ilk 24 bitinin subnet kısmı olduğunu, geri kalan bitlerin ise host kısmını oluşturduğunu gösterir. Bu yapı, hem adreslemeyi düzenli hale getirir hem de yönlendirmeyi ölçeklenebilir kılar.

Alt Ağlar

- Alt ağlar nerede??
- /24 alt ağ adresleri nedir?



Ders Notu: Alt Ağlar

Bu slayt, subnet kavramını biraz daha karmaşık bir topoloji üzerinde uygulamayı göstermektedir. Temel soru şudur: Bir ağ diyagramında **alt ağlar nerede başlar ve nerede biter?** Bunu anlamamanın anahtarı, yine aynı kuraldır: **Aralarında bir yönlendirici olmadan birbirine ulaşabilen arayüzler aynı subnet içindedir.** Yani her router, subnet sınırı oluşturan bir ayırım noktasıdır.

Şekilde üç yönlendirici ve bunlara bağlı hem uç sistem ağları hem de router-router bağlantıları görülmektedir. Burada önemli bir nokta, yalnızca uç bilgisayarların bulunduğu yerel ağların değil, **iki router arasındaki bağlantıların da ayrı birer subnet** olmasıdır. Çünkü bu bağlantılar da kendi içinde yönlendirici araya girmeden erişilebilen bağımsız ağ parçalarıdır. Bu yüzden diyagram tek parça bir ağ değildir; birden fazla /24 subnetten oluşan daha büyük bir yapıdır.

Şekilde görülen subnetler şunlardır:

223.1.1.0/24

Üstteki yerel ağdır. Bu ağda 223.1.1.1, 223.1.1.2, 223.1.1.4 gibi adresler ve bu ağa bakan router arayüzü bulunur. Hepsi aynı yüksek anlamlı 24 bitli paylaşırlar.

223.1.2.0/24

Sol alttaki yerel ağdır. 223.1.2.1, 223.1.2.2 ve router'ın ilgili arayüzü bu subnet içindedir.

223.1.3.0/24

Sağ alttaki yerel ağıdır. 223.1.3.1, 223.1.3.2 ve router'ın bu ağa bağlı arayüzü burada yer alır.

223.1.7.0/24

Üst router ile sağ alttaki router arasındaki bağlantıyı oluşturan subnettir. Bu, bir uç kullanıcı ağı değil, iki router arasında kullanılan ayrı bir alt ağıdır.

223.1.8.0/24

Sol alttaki router ile sağ alttaki router arasındaki bağlantının subnetidir.

223.1.9.0/24

Üst router ile sol alttaki router arasındaki bağlantının subnetidir.

Dolayısıyla bu topolojide toplam **6 ayrı subnet** vardır. Üçü uç sistemlerin bulunduğu yerel ağlardır, üçü ise routerlar arası bağlantı subnetleridir.

Burada **/24** gösterimi çok önemlidir. Bu, adresin ilk 24 bitinin subnet kısmı olduğunu söyler. Yani aynı subnet içindeki tüm arayüzler ilk 24 biti ortak paylaşır; son 8 bit ise host kısmıdır. Bu yüzden 223.1.2.1 ile 223.1.2.2 aynı subnettedir, ama 223.1.2.1 ile 223.1.3.1 aynı subnette değildir. Aynı mantık router-router bağlantıları için de geçerlidir; örneğin 223.1.9.x adresleri kendi başına ayrı bir subnet oluşturur.

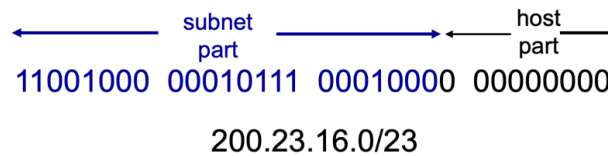
Bu slaytın önemli katkısı, subnet kavramının yalnızca “bilgisayarların olduğu LAN” anlamına gelmediğini göstermesidir. **Her bağımsız bağlantı bölgesi**, ister kullanıcı ağı olsun ister iki router arasındaki point-to-point bağlantı olsun, ayrı bir subnet olarak düşünülmelidir. Bu bakış açısı, daha sonra yönlendirme tablolarının neden ağ ön ekleri üzerinden çalıştığını anlamayı kolaylaştırır.

Özetle bu slaytın ana mesajı şudur: Bir topolojide subnetleri bulmak için routerları sınır kabul etmek gerekir. Router araya girmeden birbirine erişebilen her bağlantı bölgesi ayrı bir subnettir. Bu örnekte **223.1.1.0/24, 223.1.2.0/24, 223.1.3.0/24, 223.1.7.0/24, 223.1.8.0/24 ve 223.1.9.0/24** olmak üzere toplam altı subnet vardır.

IP Adresleme: CIDR

CIDR: Classless InterDomain Routing (pronounced “cider”)

- İsteğe bağlı uzunluktaki adresin alt ağ kısmı
- Adres formatı: **a.b.c.d/x**, burada x, adresin alt ağ kısmındaki bit sayısıdır



Network Layer: 4-51

Ders Notu: IP addressing: CIDR

Bu slayt, IP adreslemede klasik sınıflı yaklaşımın yerine geçen **CIDR (Classless InterDomain Routing)** kavramını tanıtmaktadır. CIDR'in temel önemi, adresin ağ kısmının sabit uzunlukta olmak zorunda olmadığını göstermesidir. Yani artık “A sınıfı, B sınıfı, C sınıfı” gibi katı kalıplar yerine, adresin **subnet portion** denilen ağ kısmı istenen uzunlukta seçilebilir. Slaytta da bu durum açıkça “subnet portion of address of arbitrary length” ifadesiyle belirtilmektedir. Bu esneklik, hem adres alanının daha verimli kullanılmasını hem de yönlendirme tablolarının daha ölçeklenebilir hale gelmesini sağlar.

CIDR gösteriminde adresler **a.b.c.d/x** biçiminde yazılır. Buradaki **x**, adresin kaç bitinin ağ ya da subnet kısmını oluşturduğunu belirtir. Geri kalan bitler ise host kısmıdır. Bu yüzden CIDR gösterimi, yalnızca bir IP adresi yazma biçimi değil; aynı zamanda o adresin hangi ağ bloğuna ait olduğunu da aynı anda söyleyen bir gösterimdir. Slayttaki tanım da tam olarak bunu vurgular: **x**, adresin subnet kısmındaki bit sayısını ifade eder.

Altındaki örnek bu mantığı somutlaştırmaktadır: **200.23.16.0/23**. Buradaki **/23**, ilk 23 bitin subnet kısmı olduğunu söyler. Geri kalan bitler host kısmıdır. Şekilde de ilk 23 bit maviyle subnet part olarak, son bitler ise host part olarak ayrılmıştır. Bu örnekten çıkarılması gereken temel sonuç şudur: ağ sınırı artık mutlaka 8, 16 veya 24 bit gibi oktet sınırlarında olmak zorunda değildir. **/23** gibi bir değer, ağ kısmının üçüncü oktetin ortasında bitebileceğini gösterir. İşte CIDR'in esnekliği tam burada ortaya çıkar.

Bu esnek yapı, önceki slaytlarda işlenen subnet ve prefix mantığının doğal devamıdır. Subnet içindeki cihazlar ortak yüksek anlamlı bitleri paylaşır; CIDR ise bu ortak bölümün uzunluğunu serbestçe belirlemeyi mümkün kılar. Böylece bir ağ yöneticisi ihtiyaca göre daha küçük ya da daha büyük adres blokları tanımlayabilir. Bu, gereğinden fazla büyük ağ blokları ayırma zorunluluğunu azaltır ve IPv4 adres alanının daha dikkatli kullanılmasını sağlar. Ayrıca yönlendiriciler de hedef adresleri değerlendirirken bu ön ek uzunluklarını kullanır; bu nedenle CIDR, **longest prefix matching** mantığıyla doğrudan ilişkilidir. Daha uzun ön ek daha özel bir ağı, daha kısa ön ek daha genel bir ağı temsil eder.

Özetle bu slaytın ana mesajı şudur: **CIDR**, IP adresinin ağ kısmını sabit sınıflara bağlı olmadan, istenen uzunlukta tanımlamaya yarayan esnek adresleme yaklaşımıdır. **a.b.c.d/x** gösteriminde **x**, subnet kısmındaki bit sayısını belirtir. Örneğin **200.23.16.0/23** ifadesinde ilk 23 bit ağ kısmını, geri kalan bitler host kısmını oluşturur. Bu yapı, hem adres kullanımını verimli hale getirir hem de modern yönlendirme mantığının temelini oluşturur.

IP Adresleri: Bir IP Adresi Nasıl Alınır?

Aslında bu **iki** soru :

1. S: Bir host, kendi ağı içindeki IP adresini (adresin host kısmını) nasıl alır?
2. S: Bir ağ, kendisi için IP adresini nasıl alır (adresin ağ kısmı) ?

Sunucu IP adresini nasıl alır?

- Elle yapılandırma sysadmin tarafından yapılandırma dosyasında (e.g., /etc/rc.config in UNIX)
- **DHCP**: Dynamic Host Configuration Protocol: Sunucudan ip adresini dinamik olarak almak
 - “plug-and-play”

Network Layer: 4-52

Ders Notu: IP addresses: how to get one?

Bu slayt, bir IP adresinin nasıl elde edildiği sorusunun aslında tek bir soru olmadığını vurgulamaktadır. İlk bakışta “Bir cihaz IP adresini nasıl alır?” diye düşünülse de, burada iki farklı düzey vardır. Birincisi, **bir host’un bulunduğu ağ içinde kendisine düşen host kısmını nasıl aldığıdır**. İkincisi ise, **bir ağın kendi network kısmını nasıl elde**

ettiğidir. Bu ayrım önemlidir; çünkü IP adresleme hem ağ ön ekinin atanmasını hem de o ağ içindeki tek tek cihazların adreslenmesini içerir.

Slaytın ilk sorusu, bir host'un kendi yerel ağı içinde IP adresini nasıl aldığıdır. Burada özellikle adresin **host part** kısmı vurgulanmaktadır. Yani ağın hangi adres bloğunu kullandığı zaten bellidir; asıl soru, o blok içinden bu cihaza hangi adresin verileceğidir. Bu noktada iki temel yaklaşım gösterilmektedir.

Birinci yaklaşım **elle yapılandırma**, yani statik atamadır. Slaytta bunun sistem yöneticisi tarafından bir yapılandırma dosyasında **hard-coded** biçimde tanımlanabileceği belirtilmektedir. Bu yöntemde cihazın IP adresi önceden belirlenir ve işletim sistemi ayarlarına doğrudan yazılır. Böyle bir yapılandırma, özellikle sunucular, yönlendiriciler, yazıcılar ya da adresinin değişmemesi istenen ağ cihazları için uygundur. Çünkü bu tür cihazların sabit bir adresle çalışması yönetim açısından kolaylık sağlar. Ancak bu yöntem büyük ağlarda zahmetlidir; her cihaz için ayrı elle ayar yapmak gerekir ve hata yapma olasılığı da yüksektir.

İkinci yaklaşım ise **DHCP (Dynamic Host Configuration Protocol)** kullanmaktır. Slaytta da belirtildiği gibi DHCP ile cihaz, bir sunucudan adresini **dinamik olarak** alır. Bu, günümüzde en yaygın kullanılan yöntemdir. Mantık şudur: Cihaz ağa bağlandığında, adresini önceden bilmek zorunda değildir. Ağ üzerindeki DHCP sunucusu, uygun bir IP adresini cihaz için seçer ve ona tahsis eder. Bu nedenle slaytta bu yöntem için **“plug-and-play”** ifadesi kullanılmaktadır. Yani kullanıcı cihazı ağa bağlar ve adresleme işlemi büyük ölçüde otomatik gerçekleşir. Özellikle ev ağlarında, kurumsal kablolu ağlarda ve çok sayıda istemcinin olduğu ortamlarda bu yaklaşım büyük kolaylık sağlar.

Bu slaytın ikinci sorusu ise farklı bir düzeye işaret eder: **Bir ağ, kendi network part'ını nasıl alır?** Yani örneğin bir yerel ağın neden 223.1.2.0/24 ya da başka bir ön eki kullandığı sorulmaktadır. Bu, tek tek host adreslerinden daha üst düzey bir tahsis problemidir. Burada artık cihaz içi ayarlardan değil, ağ bloğunun organizasyonlara, servis sağlayıcılara ya da ağ yöneticilerine nasıl verildiğinden söz edilir. Slayt bu konuyu burada sadece ayırmakta, ayrıntısını sonraya bırakmaktadır. Bu da doğru bir ayrımdır; çünkü bir host'un DHCP ile adres alması ile bir kurumun kendine ait IP bloklarını edinmesi aynı süreç değildir.

Bu nedenle slaytın en önemli öğretici yönü, **IP adres edinimini iki farklı problem olarak ayırmasıdır.** Birincisi yerel ağ içi atamadır; ikincisi küresel ya da kurumsal ölçekte ağ ön eki tahsisidir. Öğrenciler çoğu zaman bu ikisini tek bir şey sanabilir. Oysa DHCP, yalnızca host'a yerel ağ içinde bir adres verir; o ağın hangi adres bloğunu kullanacağı ise başka mekanizmalarla belirlenir.

Özetle bu slaytın ana mesajı şudur: “IP adresi nasıl alınır?” sorusu aslında iki parçalıdır. **Host**, bulunduğu ağ içinde adresini ya elle yapılandırma ile ya da daha yaygın olarak **DHCP** ile alır. Buna karşılık **ağın kendi adres ön ekini** nasıl elde ettiği daha üst düzey, ayrı bir adres tahsis problemidir. Bu ayrım, IP adreslemenin hem yerel hem de küresel ölçekte nasıl çalıştığını anlamak için temeldir.

DHCP: Dynamic Host Configuration Protocol

Hedef : Ana bilgisayar (host), ağa “katıldığında” ağ sunucusundan dinamik olarak IP adresi alır

- Kullanımdaki adresin kiralama süresini yenileyebilir;
- Adreslerin yeniden kullanılmasına izin verir (adres yalnızca ağa bağlıken veya cihaz açıkken tutulur);
- Ağa katılan veya ağdan ayrılan mobil kullanıcılar için destek

DHCP genel bakış :

- host broadcasts **DHCP discover** msg [optional]
- DHCP server responds with **DHCP offer** msg [optional]
- host requests IP address: **DHCP request** msg
- DHCP server sends address: **DHCP ack** msg

Network Layer: 4-53

Ders Notu: DHCP: Dynamic Host Configuration Protocol

Bu slayt, bir host’un ağa bağlandığında IP adresini nasıl otomatik olarak alabildiğini açıklayan **DHCP (Dynamic Host Configuration Protocol)** mekanizmasını tanıtmaktadır. Önceki slaytta, bir host’un IP adresini ya elle yapılandırma ile ya da DHCP ile alabileceği belirtilmişti. Burada ise DHCP’nin temel amacı, neden gerekli olduğu ve hangi mesajlaşma adımlarıyla çalıştığı özetlenmektedir. Slaytta açıkça belirtildiği gibi DHCP’nin hedefi, host’un ağa “katıldığı” anda bir ağ sunucusundan **dinamik olarak** IP adresi almasını sağlamaktır.

DHCP’nin temel avantajı, adres atama işlemini otomatik hale getirmesidir. Bu, özellikle çok sayıda istemcinin bulunduğu ağlarda büyük kolaylık sağlar. Kullanıcı cihazı ağa bağlandığında adresi önceden bilmek ya da elle girmek zorunda kalmaz. Bu yüzden DHCP çoğu zaman **plug-and-play** mantığıyla ilişkilendirilir. Yani cihaz ağa katılır, gerekli mesajları gönderir ve uygun bir IP adresini kendiliğinden edinir. Bu yaklaşım, hem kullanıcı açısından rahatlık sağlar hem de ağ yönetimini büyük ölçüde kolaylaştırır.

Slaytta DHCP'nin yalnızca adres vermekle kalmayıp adres kullanımını daha verimli hale getirdiği de vurgulanmaktadır. Bir cihaz adresi kalıcı olarak sonsuza kadar tutmaz; çoğu zaman belirli bir **lease**, yani kiralama süresi boyunca kullanır. Gerekirse bu kiralamayı yenileyebilir. Bu çok önemlidir; çünkü ağdaki adresler sınırlıdır. Eğer bir kullanıcı ağı terk ettiğinde ya da cihazını kapattığında adres serbest bırakılabiliyorsa, aynı adres başka bir cihaza yeniden tahsis edilebilir. Slayttaki “allows reuse of addresses” ifadesi tam olarak bunu anlatır. Bu nedenle DHCP, özellikle ev ağları, öğrenci laboratuvarları, misafir ağları ve kurumsal Wi-Fi ortamlarında çok verimli bir çözümdür.

DHCP'nin bir başka önemli avantajı da **hareketli kullanıcıları** desteklemesidir. Günümüzde dizüstü bilgisayarlar, telefonlar ve tabletler farklı ağlara bağlanıp ayrılmaktadır. Böyle bir ortamda her ağ için ayrı statik adres tanımlamak pratik değildir. DHCP ise kullanıcının bulunduğu ağa göre ona o ağdan geçerli bir adres verebilir. Bu yüzden slaytta özellikle “support for mobile users who join/leave network” ifadesine yer verilmiştir. Yani DHCP, modern ağların dinamik ve geçici bağlantı yapısına uyumlu bir protokoldür.

Slaytın alt kısmında DHCP'nin temel çalışma akışı özetlenmektedir. Bu süreç çoğu zaman dört adımlı bir mesajlaşma olarak anlatılır. İlk adımda host, ağda bir DHCP sunucusu olup olmadığını bulmak ve kendisine uygun bir adres istemek için **DHCP discover** mesajı gönderir. Bu mesaj genellikle yayın (broadcast) mantığıyla gönderilir; çünkü host bu aşamada henüz kendi adresini bilmediği gibi sunucunun adresini de bilmeyebilir.

İkinci adımda DHCP sunucusu, istemciye bir **DHCP offer** mesajı ile cevap verir. Bu mesaj, sunucunun istemciye verebileceği bir IP adresini ve bazı ilgili yapılandırma bilgilerini içerir. Buradaki fikir, sunucunun “Sana şu adresi verebilirim” şeklinde bir teklif sunmasıdır. Böylece istemci, kullanabileceği olası adresi öğrenir.

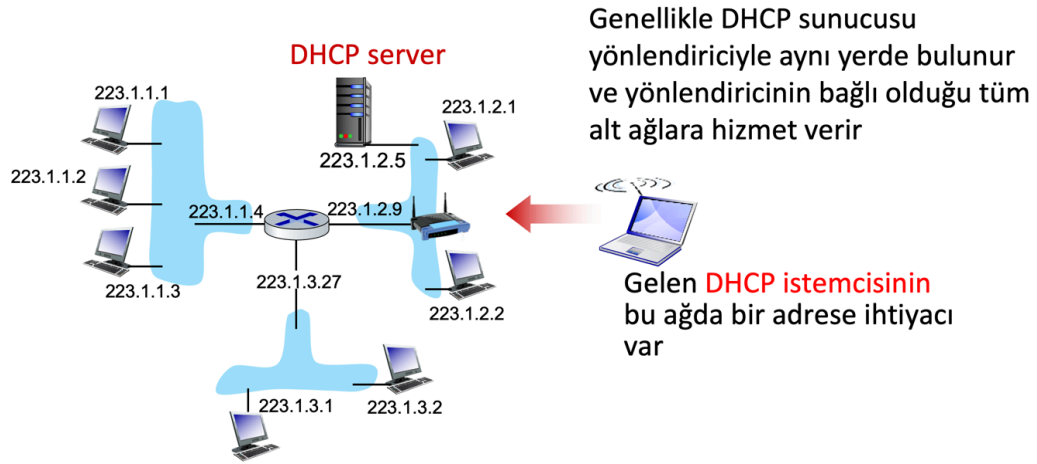
Üçüncü adımda host, kendisine sunulan bu adresi almak istediğini belirten **DHCP request** mesajını yollar. Bu, istemcinin teklifi kabul ettiğini ve o adresi resmi olarak talep ettiğini gösterir. Son adımda ise sunucu **DHCP ACK** mesajı göndererek adres atamasını onaylar. Böylece istemci ilgili IP adresini kullanmaya başlayabilir. Slaytta bu dört adım, discover, offer, request ve ack olarak özetlenmiştir. Bu süreç çoğu zaman DORA kısaltmasıyla da anılır; ancak slaytın dili içinde en önemli olan, her mesajın işlevini mantığıyla anlamaktır.

Burada dikkat edilmesi gereken bir nokta, DHCP'nin yalnızca “bir sayı vermek”ten ibaret olmadığıdır. Bu protokol sayesinde ağ yönetimi merkezi hale gelir. Hangi adreslerin kimlere verileceği, ne kadar süreyle kiralanacağı ve hangi adreslerin yeniden kullanılacağı ağ sunucusu tarafından kontrol edilebilir. Bu, özellikle büyük ağlarda çakışmaları azaltır ve adres kullanımını düzenli hale getirir. Ayrıca DHCP genellikle

sadece IP adresi değil, başka yapılandırma bilgileri de sağlayabilir; ancak bu slaytta ağırlıklı odak adres edinim sürecidir.

Özetle bu slaytın ana mesajı şudur: **DHCP**, bir host'un ağı bağlandığında IP adresini bir sunucudan **dinamik olarak** almasını sağlayan protokoldür. Adresler kiralama mantığıyla verilebilir, yeniden kullanılabilir ve böylece ağ kaynakları daha verimli yönetilir. Temel işlem akışı da sırasıyla **DHCP discover**, **DHCP offer**, **DHCP request** ve **DHCP ACK** mesajlarından oluşur. Bu yapı, özellikle çok sayıda istemcinin bulunduğu ve kullanıcıların sık sık ağı girip çıktığı ortamlarda IP adreslemesini pratik ve otomatik hale getirir.

DHCP İstemci-Sunucu Senaryosu



Network Layer: 4-54

Ders Notu: DHCP client-server scenario

Bu slayt, DHCP'nin yalnızca soyut bir protokol akışı olmadığını, gerçek bir ağ topolojisi içinde nasıl konumlandığını göstermektedir. Önceki slaytta DHCP'nin discover, offer, request ve ack mesajlarıyla çalışan bir istemci-sunucu protokolü olduğu görülmüştü. Burada ise bu mesajlaşmanın **hangi cihazlar arasında, hangi ağda ve hangi adresleme bağlamı içinde** gerçekleştiği görselleştirilmektedir.

Şekilde ortada üç farklı alt ağı bağlı bir yönlendirici vardır. Soldaki ağ **223.1.1.x**, sağdaki ağ **223.1.2.x**, alttaki ağ ise **223.1.3.x** adres bloğunu kullanmaktadır. DHCP sunucusu sağdaki subnet içinde, **223.1.2.5** adresiyle gösterilmiştir. Bu, önemli bir noktayı vurgular: DHCP sunucusu ağın herhangi bir yerinde rastgele duran bağımsız bir sistem değil, çoğu zaman yerel ağ altyapısının bir parçası olarak düşünülür. Slayttaki

notta da belirtildiği gibi, DHCP sunucusu genellikle yönlendiriciyle aynı ortamda bulunur ve yönlendiricinin bağlı olduğu alt ağlara hizmet verebilir.

Sağ tarafta kablolu bir istemci, yani ağa yeni katılan bir cihaz gösterilmektedir. Bu istemcinin henüz bir IP adresi yoktur ve bulunduğu ağda iletişim kurabilmesi için önce bir adres edinmesi gerekir. Slayttaki “Gelen DHCP istemcisinin bu ağda bir adrese ihtiyacı var” ifadesi bunu anlatır. Yani DHCP’nin devreye girdiği an, bir cihazın ağa bağlandığı ve henüz yapılandırılmamış olduğu andır. Bu yüzden DHCP özellikle taşınabilir bilgisayarlar, telefonlar ve geçici istemciler için çok uygundur.

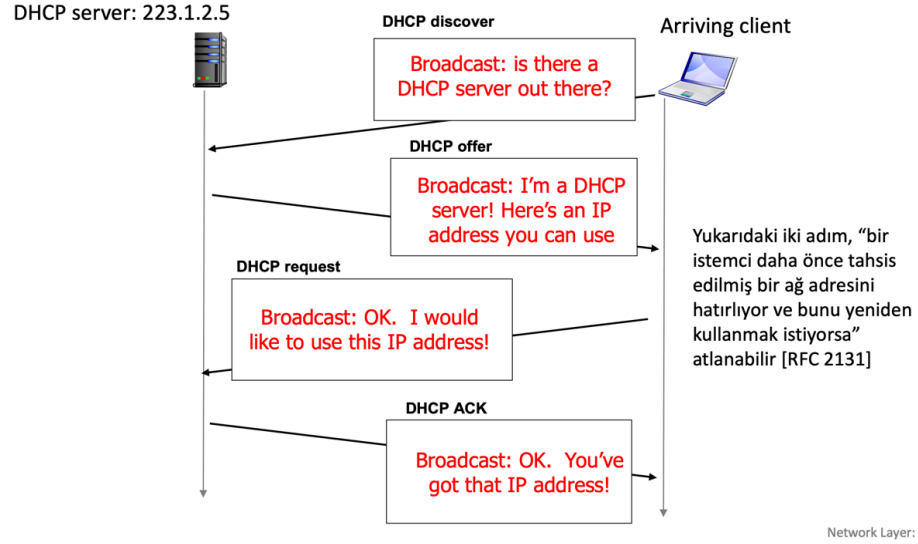
Bu slayttaki temel mesajlardan biri, DHCP’nin bir **istemci-sunucu mimarisi** ile çalıştığıdır. İstemci adres ister, sunucu uygun adresi tahsis eder. Ancak bu ilişki sadece iki cihaz arasında soyut bir mantık değil; ağ topolojisine oturmuş gerçek bir hizmettir. DHCP sunucusu, belirli subnetler için hangi adreslerin kullanılabilir olduğunu bilir ve istemciye o subnet içinde geçerli bir adres verir. Yani sunucu yalnızca rastgele bir IP göndermez; istemcinin bağlandığı ağ segmentine uygun bir adres atar.

Burada bir başka önemli nokta da, DHCP sunucusunun birden fazla alt ağa hizmet verebilmesidir. Şekilde sunucu 223.1.2.x ağında yer alsa da, yönlendiricinin bağlı olduğu diğer subnetlere de hizmet verebileceği ima edilmektedir. Bu, büyük ağlarda yönetim kolaylığı sağlar. Her subnet için ayrı fiziksel DHCP sunucusu kurmak yerine, merkezi ya da yarı merkezi bir sunucu kullanılarak adres tahsisi yapılabilir. Böylece ağ yönetimi daha düzenli hale gelir.

Bu slayt, DHCP’nin subnet kavramıyla da doğrudan ilişkili olduğunu göstermektedir. Çünkü istemcinin aldığı adres, bağlandığı subnetin adres bloğuna uygun olmalıdır. Örneğin 223.1.2.x ağında bulunan bir istemciye 223.1.1.x bloğundan bir adres verilmesi doğru olmaz. Bu nedenle DHCP, yalnızca adres dağıtan bir mekanizma değil, aynı zamanda ağ topolojisini ve subnet sınırlarını dikkate alan bir yapılandırma hizmetidir.

Özetle bu slaytın ana mesajı şudur: **DHCP, gerçek ağ ortamında istemci ile sunucu arasında çalışan bir adres atama hizmetidir.** Ağa yeni katılan istemci, bulunduğu subnet için uygun bir IP adresine ihtiyaç duyar; DHCP sunucusu da bu adresi sağlar. Sunucu çoğu zaman yönlendiriciye yakın bir konumda bulunur ve yönlendiricinin bağlı olduğu bir veya birden fazla alt ağa hizmet verebilir. Bu da DHCP’yi modern ağlarda merkezi, pratik ve ölçeklenebilir bir adresleme çözümü haline getirir.

DHCP client-server scenario



Ders Notu: DHCP client-server scenario

Bu slayt, DHCP'nin çalışma mantığını adım adım gösteren klasik **istemci-sunucu zaman diyagramını** vermektedir. Önceki slaytta DHCP'nin temel amacının, bir host'un ağa katıldığında IP adresini dinamik olarak almasını sağlamak olduğu belirtilmişti. Burada ise bu işlemin hangi mesajlarla ve hangi sırayla gerçekleştiği açık biçimde gösterilmektedir. Slayttaki akış, DHCP'nin neden "plug-and-play" hissi verdiğini de açıklamaktadır: istemci henüz hiçbir şey bilmeden ağa gelir, ağdaki sunucuyu bulur, uygun adresi ister ve onay alarak kullanmaya başlar.

İlk adım **DHCP Discover** mesajıdır. Ağa yeni katılan istemci henüz kendi IP adresini bilmediği gibi, DHCP sunucusunun adresini de bilmeyebilir. Bu yüzden ilk mesaj genellikle **broadcast** olarak gönderilir. Slayttaki ifade bunu çok sezgisel biçimde anlatmaktadır: "Ağda bir DHCP sunucusu var mı?" Bu adımın amacı, ağda kendisine adres verebilecek bir DHCP hizmeti olup olmadığını bulmaktır. Bu noktada istemci, yapılandırılmamış bir cihazdır; yani adres edinim sürecini başlatan taraf odur.

İkinci adım **DHCP Offer** mesajıdır. DHCP sunucusu, istemcinin yayın mesajını aldıktan sonra kullanılabilir bir adres önerir. Slaytta bu, "Ben bir DHCP sunucusuyum, kullanabileceğin bir IP adresi var" mantığıyla gösterilmektedir. Bu aşamada sunucu henüz adresi kesin olarak teslim etmiş olmaz; daha çok istemciye uygun bir adres ve ilgili yapılandırma bilgisini teklif eder. Buradaki önemli fikir, adresin istemci tarafından kendi kendine seçilmemesi; ağ tarafındaki merkezi bir hizmet tarafından tahsis edilmesidir.

Üçüncü adım **DHCP Request** mesajıdır. Bu kez istemci, kendisine sunulan adresi kabul ettiğini ve onu kullanmak istediğini bildirir. Slaytta bu durum “Tamam, bu IP adresini kullanmak istiyorum” biçiminde özetlenmiştir. Yani istemci, yapılan tekliflerden birini seçip bunu resmileştiren isteği geri yollar. Bu adım, adres tahsisini tek taraflı bir karar olmaktan çıkarır; istemcinin de seçimi onayladığı iki yönlü bir sürece dönüştürür.

Dördüncü ve son adım ise **DHCP ACK** mesajıdır. Sunucu bu mesajla birlikte adres tahsisini kesinleştirir ve istemciye ilgili IP adresini kullanabileceğini bildirir. Slayttaki ifade de bunu açıkça özetlemektedir: “Tamam, bu IP adresini aldın.” Bu andan sonra istemci artık ağa uygun biçimde yapılandırılmış olur ve normal IP iletişimine başlayabilir. Böylece dört adımlı süreç tamamlanmış olur. Bu akış çoğu zaman **Discover–Offer–Request–ACK** sırasıyla hatırlanır.

Slaytın sağ tarafındaki not da önemli bir ayrıntı eklemektedir. Eğer istemci daha önce kendisine tahsis edilmiş bir adresi hatırlıyor ve yeniden kullanmak istiyorsa, ilk iki adımın bazı durumlarda atlanabileceği belirtilmektedir. Bu, DHCP’nin tamamen sıfırdan başlayan katı bir süreç olmadığını, daha önceki kira bilgisini ve yeniden kullanım ihtimalini de dikkate alabildiğini gösterir. Bu ayrıntı, DHCP’nin pratikte neden verimli çalıştığını açıklayan önemli bir noktadır: her bağlantıda tüm süreci en baştan tekrar etmek zorunda kalmayabilir.

Bu slaytın temel öğretici değeri, DHCP’nin yalnızca “sunucu bir adres verir” kadar basit olmadığını göstermesidir. Burada istemci ile sunucu arasında karşılıklı bir pazarlık ve onaylaşma vardır. Ayrıca broadcast kullanılması, istemcinin başlangıçta ağ hakkında yeterli bilgiye sahip olmamasıyla doğrudan ilişkilidir. Yani protokol yapısı, istemcinin başlangıç koşullarına göre tasarlanmıştır.

Özetle bu slaytın ana mesajı şudur: DHCP adres edinimi dört temel adımda gerçekleşir. İstemci önce **DHCP Discover** ile sunucuyu arar, sunucu **DHCP Offer** ile bir adres önerir, istemci **DHCP Request** ile bu adresi talep eder ve sunucu **DHCP ACK** ile tahsisi kesinleştirir. Böylece ağa yeni katılan istemci, yerel ağ içinde geçerli bir IP adresini otomatik ve dinamik biçimde edinmiş olur.

DHCP: IP Adreslerinden Daha Fazlası

DHCP, alt ağda tahsis edilen IP adresinden başka bilgiler de döndürebilir :

- İstemci için ilk atlama yönlendiricisinin adresi
- DNS sunucusunun adı ve IP adresi
- Ağ maskesi (adresin ağ kısmını ve ana bilgisayar kısmını ayıran)

Network Layer: 4-56

Ders Notu: DHCP: more than IP addresses

Bu slayt, DHCP'nin yalnızca bir cihaza IP adresi vermekle sınırlı olmadığını göstermektedir. Önceki slaytlarda DHCP'nin temel görevinin, ağa yeni katılan bir host'a dinamik olarak IP adresi tahsis etmek olduğu anlatılmıştı. Burada ise bu sürecin aslında daha kapsamlı bir **başlangıç yapılandırması (initial configuration)** sağladığı vurgulanmaktadır. Yani DHCP, istemciye sadece "hangi IP adresini kullanacağını" değil, ağ içinde düzgün çalışabilmesi için gerekli başka bilgileri de verir.

İlk önemli bilgi, istemcinin kullanacağı **first-hop router**, yani ilk sıçrama yönlendiricisidir. Bu bilgi çoğu zaman **varsayılan ağ geçidi (default gateway)** olarak düşünülür. Bir host kendi subneti dışındaki bir hedefe paket göndermek istediğinde, paketi doğrudan o uzak hedefe iletemez; bunun yerine önce yerel ağdaki yönlendiriciye gönderir. Bu yüzden cihazın yalnızca kendi IP adresini bilmesi yetmez, aynı zamanda "yerel ağ dışına çıkarken hangi router'a başvuracağını" da bilmesi gerekir. DHCP bu bilgiyi otomatik olarak sağlayabildiği için, kullanıcı ayrıca ağ geçidi ayarı yapmak zorunda kalmaz.

İkinci önemli bilgi **DNS sunucusunun adı ve IP adresidir**. Bir cihaz internette çoğu zaman sayısal IP adresleriyle değil, alan adlarıyla çalışır. Örneğin bir web sitesine erişmek için önce onun adının IP adresine çevrilmesi gerekir. Bu iş DNS tarafından yapılır. Dolayısıyla istemcinin ağa bağlandıktan sonra sadece kendi adresine değil, hangi DNS sunucusuna soru soracağını bilmeye de ihtiyacı vardır. DHCP'nin bu bilgiyi de birlikte vermesi, istemcinin ağa bağlanır bağlanmaz isim çözümleme yapabilmesini

sağlar. Yani DHCP olmasaydı, IP adresi verilse bile cihaz pratikte internet hizmetlerini tam kullanamayabilirdi.

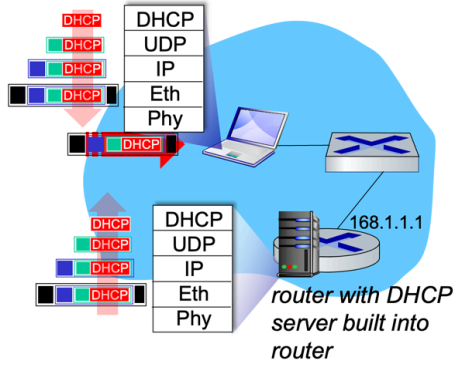
Üçüncü bilgi ise **network mask**, yani ağ maskesidir. Slaytta bunun, adresin hangi bölümünün ağ kısmı, hangi bölümünün host kısmı olduğunu gösterdiği belirtilmektedir. Bu bilgi kritik öneme sahiptir; çünkü host, bir hedef adresin kendi subnetinde mi yoksa başka bir subnet'te mi olduğunu bu maskeye bakarak belirler. Eğer hedef aynı subnet içindeyse yerel ağ üzerinden doğrudan erişim denir; değilse paket varsayılan yönlendiriciye gönderilir. Bu yüzden yalnızca IP adresini bilmek yeterli değildir; o adresin hangi ön ekle yorumlanacağını da bilmek gerekir. DHCP'nin ağ maskesini de iletmesi, istemcinin kendi yerel ağ sınırlarını doğru anlamasını sağlar.

Bu slaytın verdiği temel mesaj şudur: DHCP, bir **adres dağıtım protokolü** olmanın ötesinde, bir **host yapılandırma protokolüdür**. Zaten açılımındaki "Configuration" kelimesi bunu açıkça gösterir. Amaç sadece boş bir adresi tahsis etmek değil; istemcinin ağa katıldığı anda çalışabilir bir ağ yapılandırmasına sahip olmasını sağlamaktır. Bu nedenle DHCP, IP adresi, varsayılan ağ geçidi, DNS bilgisi ve ağ maskesi gibi öğeleri birlikte sunarak cihazı iletişime hazır hale getirir.

Bu durum önceki subnet ve yönlendirme konularıyla da doğrudan bağlantılıdır. Bir host'un hedefin kendi subnetinde olup olmadığını anlayabilmesi için maskeye ihtiyacı vardır. Uzak hedeflere ulaşabilmesi için first-hop router bilgisini bilmesi gerekir. Alan adlarıyla çalışabilmesi için DNS sunucusunu tanıması gerekir. Yani DHCP'nin sağladığı ek bilgiler, ağ katmanındaki adresleme ve yönlendirme mantığını, uygulama katmanındaki isim çözümleme ile birleştiren pratik yapılandırma parçalarıdır.

Özetle bu slaytın ana mesajı şudur: **DHCP, istemciye sadece IP adresi vermez**. Aynı zamanda **varsayılan yönlendirici**, **DNS sunucusu bilgisi** ve **network mask** gibi ağda çalışmak için gerekli diğer temel yapılandırma bilgilerini de sağlar. Bu yüzden DHCP, modern ağlarda cihazların ağa bağlanır bağlanmaz kullanılabilir hale gelmesini sağlayan kapsamlı bir otomatik yapılandırma mekanizmasıdır.

DHCP: Örnek



- Bağlanacak dizüstü bilgisayar, IP adresini, ilk atlama yönlendiricisinin adresini ve DNS sunucusunun adresini almak için DHCP'yi kullanır.
- UDP içinde kapsüllenmiş, IP içinde kapsüllenmiş, Ethernet içinde kapsüllenmiş DHCP İSTEK mesajı
- LAN üzerinde Ethernet çerçeve yayını (hedef: FFFFFFFF), DHCP sunucusu çalıştıran yönlendiricide alındı
- Ethernet'ten IP'ye ayrıştırma, UDP'den DHCP'ye ayrıştırma

Network Layer: 4-57

Ders Notu: DHCP: Örnek

Bu slayt, DHCP'nin gerçek bir yerel ağ içinde nasıl çalıştığını ve aynı zamanda **katmanlı kapsülleme (encapsulation)** mantığıyla nasıl taşındığını göstermektedir. Önceki slaytlarda DHCP'nin istemci ile sunucu arasında discover, offer, request ve ack mesajlarıyla çalışan bir protokol olduğu açıklanmıştı. Burada ise bu süreç, ağa yeni bağlanan bir dizüstü bilgisayar örneği üzerinden somutlaştırılmaktadır.

Slayttaki temel senaryo şudur: Ağa bağlanan laptop, yalnızca bir IP adresine değil, aynı zamanda **ilk sıçrama yönlendiricisinin adresine** ve **DNS sunucusunun adresine** de ihtiyaç duyar. Bu nedenle DHCP, sadece “boş bir IP numarası veren” bir mekanizma değildir; istemcinin ağa katılır katılmaz çalışabilir hale gelmesini sağlayan başlangıç yapılandırmasını sunar. Slaytta da bu açıkça belirtilmektedir: bağlanan laptop, DHCP kullanarak IP adresini, first-hop router bilgisini ve DNS sunucusu bilgisini alır.

Şekilde ayrıca DHCP hizmetinin çoğu zaman doğrudan yönlendirici içine gömülü olabildiği gösterilmektedir. Yani ayrı bir fiziksel sunucu olmak zorunda değildir; ev ve küçük ofis ağlarında router aynı zamanda DHCP sunucusu gibi davranabilir. Bu, pratikte çok yaygın bir durumdur. Kullanıcı kablosuz ağa bağlandığında ya da Ethernet kablosunu taktığında, çoğu zaman adresi doğrudan bu yerel ağ yönlendiricisinden alır.

Slaytın en öğretici taraflarından biri, bir **DHCP REQUEST** mesajının ağ içinde nasıl taşındığını göstermesidir. Bu mesaj doğrudan çıplak halde gitmez; katmanlı mimariye uygun olarak önce **UDP** içine, sonra **IP** içine, en sonunda da **Ethernet çerçevesi** içine

kapsülendirir. Şekilde soldaki katman yığını bunu açıkça göstermektedir: en üstte DHCP, onun altında UDP, sonra IP, ardından Ethernet ve fiziksel katman yer almaktadır. Bu, uygulama katmanındaki bir protokolün, alt katmanlar tarafından taşınmasının doğrudan bir örneğidir.

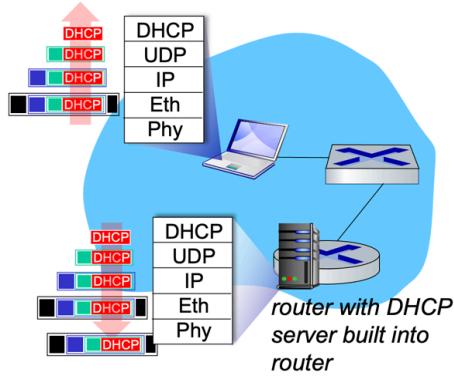
Burada önemli bir nokta, istemcinin başlangıçta henüz yapılandırılmış bir IP adresine sahip olmamasıdır. Bu nedenle ilk DHCP mesajları çoğu zaman **yayın (broadcast)** mantığıyla gönderilir. Slaytta Ethernet çerçevesinin hedef adresinin **FFFFFFFFFFFF** olduğu belirtilmektedir; bu, yerel ağdaki tüm cihazlara gönderilen Ethernet broadcast adresidir. Yani istemci, “DHCP sunucusu kim bilmiyorum; ama bu yerel ağda varsa beni duysun” mantığıyla mesaj yollar. Bu yayın çerçevesi LAN içindeki tüm cihazlara ulaşır; DHCP sunucusu çalıştıran router da bu çerçeveyi alır.

Daha sonra gelen çerçeve yönlendirici tarafında katman katman açılır. Slaytın son maddesi bunu **de-multiplexing** açısından özetlemektedir. Önce Ethernet başlığına bakılarak çerçevenin ilgili üst katmana iletilmesi sağlanır. Ardından IP paketi çıkarılır, sonra UDP segmenti ayrıştırılır ve sonunda veri doğru uygulama protokolüne, yani DHCP’ye teslim edilir. Bu, daha önce taşıma katmanında işlenen çoklama ve çoklamadan ayırma mantığının burada gerçek bir ağ örneği üzerinde uygulanmasıdır.

Bu slayt, DHCP’yi yalnızca bir adresleme protokolü olarak değil, aynı zamanda katmanlı ağ mimarisinin çalışan bir örneği olarak da görmeyi sağlar. Uygulama katmanındaki DHCP mesajı, taşıma katmanında UDP ile, ağ katmanında IP ile, bağlantı katmanında Ethernet ile taşınır. Alıcı tarafta ise bu işlem tersine çevrilir. Böylece hem adres edinimi sağlanır hem de katmanlı mimarinin pratikte nasıl işlediği görünür hale gelir.

Özetle bu slaytın ana mesajı şudur: Ağa bağlanan istemci, DHCP kullanarak yalnızca IP adresini değil, ağ geçidi ve DNS gibi temel yapılandırma bilgilerini de edinir. DHCP mesajları **UDP içinde, IP içinde, Ethernet çerçevesi içinde** taşınır; ilk aşamada çoğu zaman **broadcast** olarak gönderilir ve DHCP sunucusu çalıştıran router tarafından alınıp katman katman ayrıştırılarak işlenir. Bu nedenle DHCP, hem otomatik adresleme hem de katmanlı kapsülleme mantığını aynı anda gösteren çok iyi bir örnektir.

DHCP: Örnek



- DHCP sunucusu, istemcinin IP adresini, istemci için ilk atlama yönlendiricisinin IP adresini, DNS sunucusunun adını ve IP adresini içeren bir DHCP ACK mesajı oluşturur
- Kapsüllenmiş DHCP sunucu yanıtı istemciye iletilir, istemcide DHCP'ye kadar çoklayıcıdan ayırma
- İstemci artık kendi IP adresini, DNS sunucusunun adını ve IP adresini ile ilk atlama yönlendiricisinin IP adresini biliyor

Network Layer: 4-58

Ders Notu: DHCP: Örnek

Bu slayt, DHCP sürecinin istemci tarafında nasıl tamamlandığını göstermektedir. Önceki örnekte istemci DHCP sunucusunu bulmuş, uygun adresi istemişti. Bu aşamada artık DHCP sunucusu, istemci için son yapılandırma bilgisini içeren **DHCP ACK** mesajını hazırlar ve istemciye gönderir. Yani süreç burada yalnızca “adres önerme” aşamasından çıkıp, yapılandırmanın kesinleştiği noktaya gelir.

Slaytta özellikle vurgulanan nokta, DHCP ACK mesajının sadece istemcinin IP adresini taşımadığıdır. Bu mesaj içinde genellikle üç temel bilgi bulunur: istemcinin kullanacağı **IP adresi**, istemcinin subnet dışına çıkarken başvuracağı **first-hop router** yani varsayılan ağ geçidi adresi ve isim çözümleme için gerekli **DNS sunucusunun adı ile IP adresi**. Böylece istemci sadece yerel ağda kim olduğunu öğrenmez; aynı zamanda ağ dışına nasıl çıkacağını ve alan adlarını hangi sunucuya soracağını da öğrenmiş olur.

Şeklin sol tarafındaki katmanlı yapı burada yine önemlidir. Sunucunun cevabı doğrudan gönderilmez; önce **DHCP** verisi olarak oluşturulur, sonra **UDP** içine kapsülendir, ardından **IP** paketi içine yerleştirilir ve son olarak **Ethernet çerçevesi** içinde istemciye iletilir. Bu, DHCP'nin yine uygulama katmanında çalışan bir protokol olduğunu; fakat ağ içinde alt katmanların desteğiyle taşındığını gösterir. Başka bir deyişle, sunucu cevabı da istemci isteği gibi katmanlı mimari içinde iletilir.

Slaytta ayrıca gelen sunucu yanıtının istemci tarafında **de-multiplexing** ile yukarı taşındığı belirtilmektedir. Yani istemci önce Ethernet çerçevesini alır, sonra IP katmanına

çıkart, ardından UDP segmenti ayrıştırılır ve en sonunda veri DHCP istemcisine teslim edilir. Böylece DHCP istemcisi, kendisine tahsis edilen yapılandırma bilgisini okuyabilir. Bu süreç, daha önce işlenen çoklama ve çoklamadan ayırma mantığının çok somut bir örneğidir.

Bu aşamadan sonra istemci artık ağ üzerinde çalışabilir durumdadır. Slaytta da açıkça söylendiği gibi, istemci artık kendi **IP adresini**, **DNS sunucusunun adını ve IP adresini** ve **ilk sıçrama yönlendiricisinin adresini** bilmektedir. Bu bilgilerle cihaz, hem yerel ağ içindeki iletişimi başlatabilir hem de uzak ağlara paket gönderebilir. Ayrıca alan adlarını çözümleyerek web, e-posta ve benzeri internet hizmetlerini kullanabilir.

Bu slaytın önemli mesajı şudur: DHCP süreci, yalnızca bir adres tahsisi değil, istemcinin ağ üzerinde kullanılabilir hale gelmesini sağlayan tamamlanmış bir başlangıç yapılandırmasıdır. ACK mesajı geldikten sonra istemci artık “ağa bağlı ama ne yapacağını bilmeyen” bir cihaz olmaktan çıkar; tam anlamıyla yapılandırılmış bir ağ düğümüne dönüşür.

Özetle bu slaytın ana mesajı şudur: **DHCP ACK**, istemciye son ve geçerli yapılandırmayı veren mesajdır. Bu mesajla birlikte istemci IP adresini, varsayılan yönlendiricisini ve DNS bilgilerini öğrenir. Sunucunun cevabı katmanlı biçimde kapsüllenerek istemciye taşınır ve istemci tarafında katman katman ayrıştırılarak DHCP’ye ulaştırılır. Bu aşamadan sonra istemci ağda normal iletişime başlayabilir.

IP adresleri: Nasıl edinilir?

Q: Ağ, IP adresinin alt ağ kısmını nasıl elde eder??

A: Sağlayıcısı olan İSS’nin adres alanından bir bölüm tahsis edilir

ISP's block 11001000 00010111 00010000 00000000 200.23.16.0/20

ISP, adres alanını 8 bloğa ayırabilir:

Organization 0	<u>11001000 00010111 00010000</u>	00000000	200.23.16.0/23
Organization 1	<u>11001000 00010111 00010010</u>	00000000	200.23.18.0/23
Organization 2	<u>11001000 00010111 00010100</u>	00000000	200.23.20.0/23
...			
Organization 7	<u>11001000 00010111 00011110</u>	00000000	200.23.30.0/23

Ders Notu: IP addresses: how to get one?

Bu slayt, önceki sorunun ikinci kısmını yanıtlamaktadır: Bir **host** kendi ağında IP adresini DHCP ile alabilir; peki bir **ağ** kendi **subnet kısmını** nasıl elde eder? Buradaki temel cevap şudur: Bir kurum ya da yerel ağ, adres alanını genellikle doğrudan “kendisi üretmez”; bunun yerine, bağlı olduğu **ISP’nin adres bloğundan bir parça** tahsis edilir. Slaytta da bu açıkça “network gets allocated portion of its provider ISP’s address space” şeklinde ifade edilmektedir.

Şekilde örnek olarak bir ISP’ye ait daha büyük bir blok verilmiştir: **200.23.16.0/20**. Buradaki **/20**, ilk 20 bitin ağ ön eki olduğunu gösterir. Yani ISP, elindeki daha büyük adres uzayını tek tek son kullanıcılara değil, daha küçük ama yine düzenli bloklara bölerek dağıtabilir. Bu, CIDR mantığının pratikte nasıl kullanıldığını gösterir. Önce büyük bir adres bloğu vardır, sonra bu blok daha küçük ön ekli parçalara ayrılır ve farklı kuruluşlara verilir.

Slaytta ISP’nin bu **/20** bloğunu **8 ayrı /23 blok** halinde kuruluşlara paylaştırdığı gösterilmektedir. Örneğin:

- **Organization 0** → **200.23.16.0/23**
- **Organization 1** → **200.23.18.0/23**
- **Organization 2** → **200.23.20.0/23**
- ...
- **Organization 7** → **200.23.30.0/23**

Buradaki mantık şudur: ISP’nin elindeki daha büyük blok, daha küçük kuruluş ağlarına tahsis edilebilecek alt bloklara bölünmektedir. Böylece her kuruluş, kendisine verilen ön eki kendi içinde kullanabilir; sonra o kuruluş da gerekirse bu bloğu daha küçük subnetlere ayırabilir. Yani adres tahsisi katmanlı biçimde ilerler: üstte daha büyük bir sağlayıcı bloğu, altta ise ondan türetilmiş daha küçük kurumsal bloklar vardır.

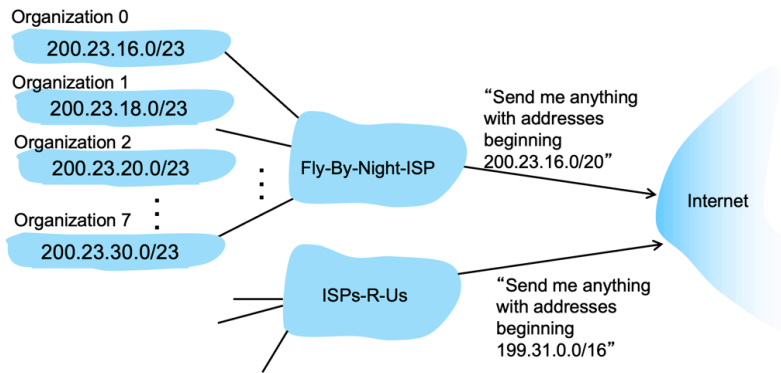
Bu slaytın önemli yönü, **host adres edinimi** ile **network prefix tahsisi** arasındaki farkı netleştirmesidir. DHCP, bir host’a yerel ağ içinde bir adres verir; ama DHCP, ağın hangi ön eki kullanacağını belirlemez. Ağın ön eki, daha üst düzeyde, çoğu zaman ISP tarafından tahsis edilir. Başka bir deyişle, bir cihazın **host part**’ını edinmesi ile bir kurumun **subnet part**’ını edinmesi farklı süreçlerdir. Bu slayt özellikle bu ayrımı görünür hale getirmektedir.

Ayrıca bu örnek, CIDR'ın neden önemli olduğunu da gösterir. Eğer adres alanı yalnızca katı sınıflarla bölünseydi, ISP'nin elindeki adresleri ihtiyaçlara göre esnek biçimde paylaşması zor olurdu. CIDR sayesinde ISP, elindeki büyük adres bloğunu uygun büyüklükte parçalara ayırabilir ve farklı kuruluşlara daha verimli biçimde dağıtabilir. Bu, hem adres alanının daha ekonomik kullanılmasını sağlar hem de yönlendirme örnekleri mantığını destekler.

Özetle bu slaytın ana mesajı şudur: Bir ağ kendi **subnet ön ekini**, genellikle bağlı olduğu **ISP'nin adres alanından tahsis edilmiş bir blok** olarak alır. Örneğin ISP'nin **200.23.16.0/20** bloğu, daha küçük **/23** bloklara bölünerek farklı organizasyonlara verilebilir. Böylece adresleme, üst düzeyden alt düzeye doğru hiyerarşik ve ölçeklenebilir biçimde dağıtılmış olur.

Hiyerarşik Adresleme: Yol Birleştirme

Hiyerarşik adresleme, yönlendirme bilgilerinin verimli bir şekilde duyurulmasını sağlar:



Network Layer: 4-60

Ders Notu: Hierarchical Addressing: Route Aggregation

Bu slayt, hiyerarşik adreslemenin neden sadece adresleri düzenli vermek için değil, aynı zamanda **yönlendirme bilgisini ölçeklenebilir hale getirmek** için de gerekli olduğunu göstermektedir. Buradaki temel kavram **route aggregation**, yani rota özetlemesidir. Ana fikir şudur: Eğer bir ISP'nin müşterilerine verdiği adres blokları belirli bir ortak ön eki paylaşıyorsa, internetin geri kalanına bu müşterilerin her biri için ayrı ayrı rota duyurmak yerine, hepsini kapsayan **tek bir daha genel rota** duyurulabilir. Slaytta da bu durum, "hierarchical addressing allows efficient advertisement of routing information" ifadesiyle açıkça vurgulanmaktadır.

Şeklin sol tarafında bir ISP'ye bağlı birden fazla organizasyon görülmektedir. Bu organizasyonlara verilen adres blokları şunlardır: **200.23.16.0/23**, **200.23.18.0/23**, **200.23.20.0/23** ... **200.23.30.0/23**. Bunların hepsi daha önceki slaytta da görüldüğü gibi, ISP'nin elindeki daha büyük **200.23.16.0/20** bloğundan türetilmiş alt bloklardır. Yani organizasyonların kullandığı adresler birbirinden bağımsız ve rastgele seçilmiş değildir; aynı üst adres alanının alt parçalarıdır. Bu ortaklık, rota özetlemesini mümkün kılan temel koşuldur.

Şekilde ortadaki **Fly-By-Night-ISP**, internete her organizasyon için ayrı ayrı "200.23.16.0/23 bana gelsin", "200.23.18.0/23 bana gelsin", "200.23.20.0/23 bana gelsin" diye duyuru yapmak zorunda kalmaz. Bunun yerine, çok daha genel bir ifade kullanabilir:

"200.23.16.0/20 ile başlayan tüm adresleri bana gönder."

Bu, slaytta doğrudan metin halinde de verilmiştir. Böylece internetin geri kalan yönlendiricileri, bu ISP'nin tüm müşteri ağlarını temsil eden tek bir ön ek öğrenmiş olur. Bu tam olarak **route aggregation**'dır. Çok sayıda küçük rota, daha kısa ve daha genel tek bir ön ek altında özetlenir.

Bu yaklaşımın en önemli sonucu, yönlendirme tablolarının daha küçük ve daha yönetilebilir olmasıdır. Eğer her müşteri ağı dünya üzerindeki tüm yönlendiricilere tek tek duyurulmak zorunda kalsaydı, internet omurgasındaki yönlendirme tabloları çok daha hızlı büyürdü. Oysa hiyerarşik adresleme sayesinde bir üst düzey servis sağlayıcı, kendi müşterilerine ait birçok ağı tek bir önekle temsil edebilir. Böylece hem yönlendirme güncellemeleri azalır hem de yönlendirme tabloları daha ölçeklenebilir hale gelir. İnternetin bugünkü boyutlarda çalışabilmesi için bu tür özetleme mekanizmaları kritik önemdedir.

Şeklin alt tarafında ikinci bir ISP örneği de verilmiştir: **ISPs-R-U's**, internete **199.31.0.0/16** ile başlayan adresleri kendisine göndermesini söylemektedir. Bu ikinci örnek, aynı mantığın başka bir adres bloğu için de geçerli olduğunu göstermektedir. Yani route aggregation belirli tek bir senaryoya değil, hiyerarşik adres tahsis yapılan tüm yapılaraya uygulanabilen genel bir ilkedir. Her servis sağlayıcı, kendisine tahsis edilmiş üst bloğu kullanarak alt müşterilerini tek bir daha genel rota altında toplayabilir.

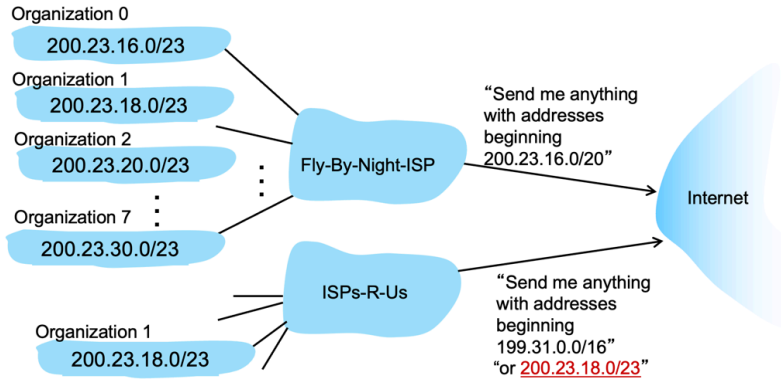
Bu slayt, önceki CIDR slaytıyla da doğrudan bağlantılıdır. CIDR, ağ ön ekinin esnek uzunlukta tanımlanmasını mümkün kılmıştı. Route aggregation ise bu esnek ön eklerin yönlendirme açısından nasıl büyük fayda sağladığını göstermektedir. Çünkü özetleme yapabilmek için, birçok küçük ağın ortak bir daha kısa ön eki paylaşması gerekir. Başka bir ifadeyle, CIDR adresleme esnekliği sağlar; route aggregation ise bu esnekliğin yönlendirme sisteminde ölçeklenebilirliğe dönüşmüş halidir.

Aynı zamanda bu konu, **longest prefix matching** ile de ilişkilidir. İnternetin geri kalanı, ISP'ye doğru gönderim yaparken genel örnek olan **200.23.16.0/20**'yi kullanabilir. Ancak ISP'nin iç yönlendirme yapısı, kendi müşterileri arasında ayırım yaparken daha özel örnekleri, örneğin **200.23.18.0/23** gibi daha uzun ön ekleri kullanabilir. Yani dışarıya genel rota duyurulurken, içeride daha ayrıntılı rota bilgisi tutulabilir. Bu da hiyerarşik yönlendirmenin doğal sonucudur.

Özetle bu slaytın ana mesajı şudur: **Hiyerarşik adresleme, rota özetlemeyi mümkün kılar.** Bir ISP, müşterilerine tahsis ettiği çok sayıdaki küçük adres bloğunu, internete tek tek duyurmak yerine, hepsini kapsayan daha genel bir önekle duyurabilir. Bu da yönlendirme bilgisinin daha verimli yayılmasını, yönlendirme tablolarının küçülmesini ve internetin daha ölçeklenebilir çalışmasını sağlar.

Hiyerarşik Adresleme: Daha Spesifik Rotalar

- Organization 1 moves from Fly-By-Night-ISP to ISPs-R-Us
- ISPs-R-Us now advertises a more specific route to Organization 1



Network Layer: 4-61

Ders Notu: Hierarchical addressing: more specific routes

Bu slayt, hiyerarşik adresleme ve rota özetlemesinin çok güçlü bir yaklaşım olduğunu, ancak gerçek ağlarda bazen tek başına yeterli olmadığını göstermektedir. Önceki slaytta bir ISP'nin kendisine ait daha büyük bir adres bloğunu tek bir önekle internete duyurabildiği görülmüştü. Burada ise istisnai bir durum ele alınmaktadır: **Organization 1**, önce bağlı olduğu **Fly-By-Night-ISP**'den ayrılıp **ISPs-R-Us** üzerinden internete bağlanmaktadır. Buna rağmen adres bloğunu değiştirmemekte ve **200.23.18.0/23**

önekini kullanmaya devam etmektedir. Slayt tam olarak bu durumu “Organization 1 moves from Fly-By-Night-ISP to ISPs-R-Us” ifadesiyle vermektedir.

Normalde Fly-By-Night-ISP, internete “**200.23.16.0/20 ile başlayan her şeyi bana gönder**” diye daha genel bir rota duyurmaktadır. Bu genel duyuru, kendi müşteri bloklarını tek bir önek altında topladığı için verimlidir. Ancak Organization 1 artık fiziksel olarak bu ISP üzerinden değil, başka bir ISP üzerinden bağlandığında bir sorun ortaya çıkar: İnternetin geri kalanı hâlâ **200.23.18.0/23** adreslerini, daha genel önek olan **200.23.16.0/20** kapsamında Fly-By-Night-ISP’ye göndermeye devam edebilir. Bu durumda trafik yanlış yere gider. Slaytta ve kitapta bu problemin çözümü olarak, yeni ISP’nin Organization 1 için **daha özel bir rota** duyurması gerektiği anlatılmaktadır.

İşte bu yüzden **ISPs-R-Us**, internete sadece kendi büyük bloğu olan **199.31.0.0/16**’yı değil, ayrıca Organization 1 için özel olarak **200.23.18.0/23** önekini de duyurur. Slayttaki alt metinde de bu açıkça görülmektedir:

“Send me anything with addresses beginning 199.31.0.0/16 or 200.23.18.0/23”.

Bu, özetlemenin üstüne eklenen istisna duyurusudur. Yani daha genel rota hâlâ vardır, ancak belirli bir alt blok için daha ayrıntılı, daha özel yeni bir rota ilan edilmiştir.

Buradaki temel ağ mantığı, daha önce işlenen **longest prefix matching** kuralıyla doğrudan ilişkilidir. İnternetteki bir yönlendirici, hedef adres olarak **200.23.18.x** gördüğünde iki farklı duyuru ile karşılaşabilir:

biri **200.23.16.0/20**, diğeri **200.23.18.0/23**.

Bu durumda yönlendirici daha kısa olan genel öneki değil, **daha uzun ve daha özel olan /23 önekini** seçer. Çünkü longest prefix matching, eşleşen rotalar arasında en fazla ortak başlangıç bitine sahip olanı tercih eder. Böylece Organization 1’e gidecek trafik artık Fly-By-Night-ISP yerine ISPs-R-Us yönüne gönderilir. Kitapta da açıkça, 200.23.16.0/20 ve 200.23.18.0/23 birlikte görüldüğünde, yönlendiricilerin en uzun, yani en özel öneki seçerek Organization 1 trafiğini ISPs-R-Us’a yönlendireceği belirtilmektedir.

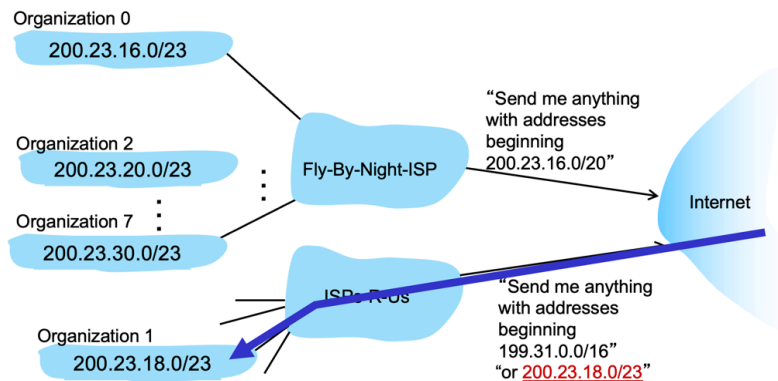
Bu örnek çok öğreticidir; çünkü route aggregation’ın pratikte her zaman tek başına kusursuz çalışmadığını gösterir. Hiyerarşik adresleme, adresler düzgün ve üstten alta doğru tahsis edildiğinde çok verimlidir. Ancak bir kuruluş servis sağlayıcısını değiştirip eski adres bloğunu korursa, özetlenmiş genel rota ile gerçek fiziksel bağlantı yapısı arasında uyumsuzluk doğabilir. Böyle durumlarda çözüm, genel rotayı iptal etmek değil; o kuruluş için **daha özel bir istisna rota** duyurmaktır. Yani internet yönlendirmesi, hem özetleme hem de gerektiğinde özel istisnalarla birlikte çalışır.

Bu durum aynı zamanda neden yönlendirme tablolarında bazen hem çok genel hem de çok özel örneklerin birlikte bulunduğunu da açıklar. Genel örnekler tabloyu küçültür ve ölçeklenebilirlik sağlar. Daha özel örnekler ise topolojideki istisnai durumları doğru biçimde yakalar. Ağın hem verimli hem de doğru çalışması için bu iki yaklaşım birlikte kullanılır.

Özetle bu slaytın ana mesajı şudur: **Bir kuruluş adres bloğunu koruyarak başka bir ISP'ye geçtiğinde, yeni ISP o kuruluş için daha özel bir rota duyurur.** Bu örnekte Fly-By-Night-ISP genel **200.23.16.0/20** önekini duyurmaya devam ederken, ISPs-R-Us ayrıca **200.23.18.0/23** önekini ilan eder. İnternetteki yönlendiriciler de **longest prefix matching** sayesinde daha özel olan /23 önekini seçerek trafiği doğru ISP'ye yollar.

Hiyerarşik Adresleme: Daha Spesifik Rotalar

- Organization 1 moves from Fly-By-Night-ISP to ISPs-R-Us
- ISPs-R-Us now advertises a more specific route to Organization 1



Network Layer: 4-62

Ders Notu: Hiyerarşik Adresleme: Daha Spesifik Rotalar

Bu slayt, bir önceki slayttaki "more specific routes" mantığını aynı örnek üzerinden daha görsel ve sezgisel biçimde pekiştirmektedir. Temel durum şudur: **Organization 1**, önce bağlı olduğu **Fly-By-Night-ISP**'den ayrılıp **ISPs-R-Us** üzerinden internete bağlanmaktadır. Ancak kullandığı adres bloğu **200.23.18.0/23** olarak aynı kalmaktadır. Bu durumda internetin geri kalanının bu adres bloğuna giden trafiği hangi ISP'ye göndereceği kritik hale gelir.

Sorunun kaynağı, Fly-By-Night-ISP'nin internete hâlâ daha genel bir önek duyuruyor olmasıdır:

200.23.16.0/20.

Bu genel önek, içinde **200.23.18.0/23** bloğunu da kapsar. Yani internet yalnızca bu genel duyuruyu görseydi, Organization 1'e gitmesi gereken paketler eski ISP'ye yönlenebilirdi. Oysa Organization 1 artık fiziksel olarak yeni ISP üzerinden bağlıdır. Bu nedenle eski genel önek tek başına artık doğru yönlendirme için yeterli değildir.

Slaytın ana mesajı burada ortaya çıkar: **ISPs-R-Us**, Organization 1 için daha özel bir rota duyurur. Yani internete sadece kendi büyük bloğunu değil, ayrıca şu öneki de ilan eder:

200.23.18.0/23.

Böylece internetteki yönlendiriciler aynı hedef için iki farklı eşleşme görebilir: biri daha genel olan **200.23.16.0/20**, diğeri daha özel olan **200.23.18.0/23**. Bu durumda seçim kuralı bellidir: **en uzun ön ek eşleşmesi**, yani **longest prefix matching** uygulanır. Daha uzun olan /23 öneki, daha kısa olan /20 öneğine göre öncelik kazanır. Sonuç olarak Organization 1'e giden trafik artık doğru biçimde **ISPs-R-Us** yönüne gönderilir.

Şekildeki mavi ok bu durumu özellikle görünür hale getirmektedir. Ok, internetten çıkan trafiğin artık eski toplulaştırılmış rota üzerinden değil, yeni ISP'nin duyurduğu **daha spesifik rota** üzerinden Organization 1'e ulaştığını göstermektedir. Yani bu slayt, soyut bir yönlendirme kuralını değil, onun topoloji üzerindeki gerçek etkisini göstermektedir: trafik fiziksel olarak yer değiştirmiş kuruluşa, en özel önek sayesinde doğru yoldan ulaşır.

Bu örnek, **route aggregation** ile **istisna rotaların** birlikte nasıl çalıştığını anlamak açısından çok önemlidir. Hiyerarşik adresleme normalde çok sayıda küçük ağı tek bir büyük önek altında özetlemeye yarar. Bu, yönlendirme tablolarını küçültür ve ölçeklenebilirlik sağlar. Ancak bazı durumlarda bu özetleme bozulmadan kalırken, içindeki belirli bir alt blok için özel bir yönlendirme ihtiyacı doğabilir. İşte burada daha spesifik rota devreye girer. Yani internet yönlendirmesi yalnızca genel özetlerden değil, gerektiğinde o özetleri delen daha özel istisnalardan da oluşur.

Bu nedenle bu slaytın asıl öğretici noktası şudur: **Hiyerarşik adresleme çok verimlidir, fakat gerçek dünyada hareket eden organizasyonlar ve değişen bağlantılar nedeniyle bazen daha spesifik rota duyuruları gerekir**. Bu istisnalar sayesinde hem genel özetleme korunur hem de özel durumlar doğru biçimde yönlendirilir.

Özetle bu slaytın ana mesajı şudur: Organization 1 başka bir ISP'ye geçtiğinde, eski ISP'nin genel **200.23.16.0/20** duyurusu hâlâ var olsa bile, yeni ISP'nin duyurduğu **200.23.18.0/23** öneki daha spesifik olduğu için tercih edilir. Böylece **longest prefix**

matching sayesinde trafik doğru ISP'ye yönlendirilir ve hiyerarşik adresleme, istisna rotalarla birlikte esnek biçimde çalışır.

IP Adresleme: Son Sözler...

S: Bir Internet Servis Sağlayıcısı (ISS) adres bloklarını nasıl elde eder?

C: **ICANN:** Internet Corporation for Assigned Names and Numbers
<http://www.icann.org/>

- IP adreslerini 5 bölgesel kayıt kuruluşu (RR) aracılığıyla tahsis eder (bu kuruluşlar daha sonra yerel kayıt kuruluşlarına tahsis edebilir)
- DNS kök bölgesini yönetir; buna, tek tek üst düzey alan adlarının (.com, .edu, ...) yönetiminin devri de dahildir

S: Yeterli sayıda 32 bit IP adresi var mı?

- ICANN, 2011 yılında IPv4 adreslerinin son bloğunu Kayıt Yöneticilerine tahsis etti
- NAT (Next), IPv4 adres alanının tükenmesine yardımcı olur
- IPv6, 128 bitlik bir adres alanına sahiptir

"Adres alanı ihtiyacımızın ne kadar olacağını kim bilebilirdi ki?" Vint Cerf (IPv4 adreslerinin 32 bit uzunluğunda olması kararını değerlendirirken)

Network Layer: 4-63

Ders Notu: IP Adresleme: Son Sözler

Bu slayt, IP adresleme konusunu teknik ayrıntıların ötesine taşıyarak iki büyük resmi bir araya getiriyor: Birincisi, internet üzerindeki adres bloklarının **kurumsal olarak nasıl dağıtıldığı**; ikincisi ise **IPv4 adres alanının neden yetersiz hale geldiği** ve bunun nasıl yönetildiğidir. Bu nedenle slayt, adreslemenin yalnızca subnet, CIDR ve DHCP gibi yerel mekanizmalarla sınırlı olmadığını; küresel ölçekte yönetilen bir kaynak olduğunu vurgulamaktadır.

Slaytın sol tarafındaki ilk soru şudur: **Bir internet servis sağlayıcısı adres bloklarını nasıl elde eder?** Buradaki cevap **ICANN** üzerinden verilmektedir. ICANN, yani *Internet Corporation for Assigned Names and Numbers*, internetin küresel isim ve numara kaynaklarının yönetiminde merkezi rol oynayan yapıdır. Slayta göre IP adresleri, ICANN tarafından doğrudan son kullanıcılara değil, önce **bölgesel kayıt kuruluşları** aracılığıyla tahsis edilir. Bu yapı önemlidir; çünkü internet adreslemesi tek merkezden tek tek cihazlara dağıtılan bir sistem değildir. Bunun yerine hiyerarşik bir tahsis mekanizması vardır: üstte küresel düzey, onun altında bölgesel kayıt kuruluşları, sonra servis sağlayıcılar ve en altta organizasyonlar ile yerel ağlar bulunur. Bu yapı, daha önce işlenen hiyerarşik adresleme mantığının kurumsal karşılığıdır.

Slaytta ayrıca ICANN'in yalnızca IP adresleriyle değil, **DNS kök bölgesinin yönetimiyle** de ilişkili olduğu belirtilmektedir. Yani bu kurum internetin hem sayısal adresleme kaynaklarıyla hem de üst düzey alan adlarının yönetimiyle bağlantılıdır. Bu nokta önemlidir; çünkü internet altyapısında isimlendirme ve adresleme birbirinden farklı işlevler olsa da, her ikisi de küresel ölçekte dikkatli yönetilmesi gereken kaynaklardır.

Slaytın sağ tarafındaki ikinci soru ise çok daha çarpıcıdır: **Yeterli sayıda 32 bit IP adresi var mı?** Bu soru doğrudan IPv4'ün sınırlarına işaret etmektedir. IPv4 adresleri 32 bit uzunluğundadır ve bu teorik olarak büyük bir adres uzayı sunsa da, internetin büyümesi, adreslerin geçmişte verimsiz kullanılması ve çok sayıda cihazın ağa bağlanması nedeniyle bu alan zamanla yetersiz hale gelmiştir. Slaytta, ICANN'in **2011 yılında IPv4 adreslerinin son bloklarını bölgesel kayıt yöneticilerine tahsis ettiği** belirtilmektedir. Bu ifade, pratikte IPv4 serbest adres havuzunun tükenme noktasına geldiğini anlatır.

Bu noktada slayt iki temel çözüm yönüne işaret etmektedir. Birincisi **NAT**'tır. NAT, tek bir genel IP adresinin arkasında birden çok özel adresli cihazın internete çıkabilmesini sağlayarak IPv4 adres alanı üzerindeki baskıyı azaltır. Bu nedenle NAT, adres kıtlığı döneminde çok önemli bir ara çözüm olmuştur. Ancak NAT, adres alanı sorununu kökten çözmekten çok, mevcut adreslerin daha verimli kullanılmasını sağlar.

İkinci ve daha kalıcı yön ise **IPv6**'dır. Slaytta IPv6'nın **128 bitlik bir adres alanına** sahip olduğu belirtilmektedir. Bu, IPv4'e göre çok büyük bir genişleme anlamına gelir. Buradaki temel fikir, artık adresleri dikkatle idare edilen kıt bir kaynak gibi değil, çok daha geniş ve uzun vadeli bir alan içinde ele alabilmektir. Bu nedenle IPv6, internetin gelecekteki büyümesini desteklemek üzere tasarlanmış yeni nesil adresleme yaklaşımıdır.

Slaytın altındaki Vint Cerf alıntısı da bu tartışmaya tarihsel bir boyut kazandırmaktadır. Alıntının ima ettiği şey şudur: İnternet ilk tasarlanırken bugünkü ölçekte milyarlarca cihazın çevrimiçi olacağı öngörülmemiştir. Bu nedenle 32 bitlik bir adres alanı başlangıçta yeterli görünmüş olabilir. Ancak internetin büyüme hızı, mobil cihazlar, IoT sistemleri ve her alana yayılan dijital bağlantı ihtiyacı, bu alanın sınırlı olduğunu zamanla açık biçimde göstermiştir.

Bu slayt, önceki konularla da doğrudan bağlantılıdır. DHCP, subnetting ve CIDR, mevcut adres alanının ağ içinde verimli kullanılmasını sağlar. Hiyerarşik adresleme ve route aggregation, bu adreslerin küresel yönlendirme açısından ölçeklenebilir biçimde duyurulmasını sağlar. Ancak tüm bu mekanizmaların üzerinde daha büyük bir gerçek vardır: IPv4 adres uzayı sonludur. Bu yüzden ağ mühendisliği yalnızca adresleri düzenlemekle kalmamış, aynı zamanda kıt bir kaynağı yönetmeye de dönüşmüştür.

Özetle bu slaytın ana mesajı şudur: IP adresleme hem **küresel ölçekte yönetilen** hem de **sınırlı bir kaynak** olan bir yapıdır. Adres blokları ICANN ve bölgesel kayıt kuruluşları üzerinden servis sağlayıcılara tahsis edilir. Ancak IPv4'ün 32 bitlik adres alanı zamanla yetersiz hale gelmiştir. Bu sorunu hafifletmek için NAT kullanılmış, daha kalıcı çözüm olarak ise çok daha geniş adres alanı sunan **IPv6** geliştirilmiştir.

Network layer: “data plane” roadmap

- Network layer: overview
 - data plane
 - control plane
- What's inside a router
 - input ports, switching, output ports
 - buffer management, scheduling
- IP: the Internet Protocol
 - datagram format
 - addressing
 - network address translation
 - IPv6
- Generalized Forwarding, SDN
 - match+action
 - OpenFlow: match+action in action
- Middleboxes



Network Layer: 4-64

Ders Notu: Network layer: “data plane” roadmap

Bu slayt, ağ katmanının veri düzlemi bölümünde yeni bir alt başlığa geçildiğini göstermektedir. Daha önce **IP datagram formatı** ve **addressing** konuları ele alınmıştı; şimdi odak **network address translation (NAT)** başlığına kaymaktadır. Yani ders akışında artık şu soruya geçilmektedir: Bir yerel ağ içindeki özel adresli cihazlar, küresel internetle nasıl haberleşir? Bu geçiş, IPv4 adresleme konusunun doğal devamıdır. Çünkü IP adresleme anlatıldıktan sonra, gerçek ağlarda bu adreslerin birebir ve doğrudan kullanılmadığı; çoğu zaman dönüştürüldüğü görülür.

Bu slaytta koyu renkle vurgulanan başlıklar, veri düzlemi içinde şu ana kadar gelinecek noktayı özetler. Ağ katmanının genel yapısı, veri ve kontrol düzlemi ayrımı, yönlendirici iç yapısı, giriş-çıkış portları, tampon yönetimi ve zamanlama konuları daha önce işlendi. Ardından IP bölümünde **datagram formatı** ve **adresleme** ele alındı. Şimdi aynı IP başlığı altında **NAT** ve daha sonra **IPv6** konuları işlenecektir. Bu yapı, dersin mantıksal sırasını gösterir: önce adresin ne olduğu anlaşılır, sonra bu adresin gerçek ağlarda nasıl kullanıldığı ve neden dönüştürülmeye ihtiyaç duyulduğu açıklanır.

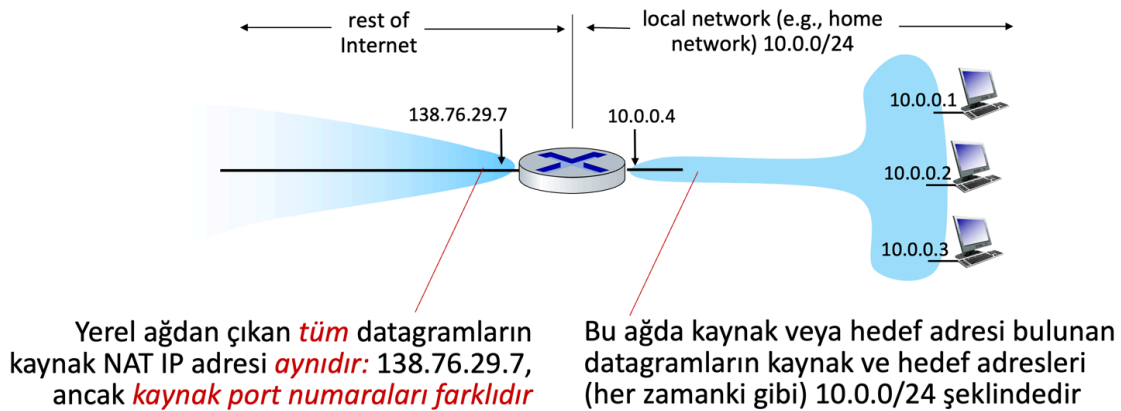
Burada NAT konusuna geçilmesinin nedeni, daha önce tartışılan çok önemli bir problemle doğrudan bağlantılı olmasıdır: **IPv4 adres alanının sınırlı olması**. Önceki slaytlarda 32 bit IPv4 adres alanının uzun vadede yetersiz hale geldiği, bu nedenle adreslerin dikkatli ve verimli kullanılmasının zorunlu olduğu görülmüştü. NAT tam da bu noktada devreye girer. Yerel ağ içindeki çok sayıda cihazın özel IP adresleri kullanmasına, dış dünya ile konuşurken ise daha az sayıda genel IP adresi üzerinden internete çıkmasına imkân verir. Yani NAT, IP adresleme konusunun soyut bir ayrıntısı değil; IPv4 kıtlığına verilmiş çok pratik bir mühendislik cevabıdır.

Bu yüzden bu slayt bir özet değil, aynı zamanda bir **köprü slayt** olarak okunmalıdır. Datagrama bakıp yönlendirme yapmayı öğrendikten, subnet, CIDR ve hiyerarşik adreslemeyi gördükten sonra artık “gerçek ev ve kurum ağlarında cihazlar neden özel adres kullanıyor?” sorusuna gelinmektedir. NAT bu sorunun cevabıdır. Hemen ardından gelen **IPv6** başlığı da daha geniş çerçeveyi tamamlar: NAT kısa ve orta vadede IPv4 adres yetersizliğini hafifletir; IPv6 ise uzun vadeli adresleme çözümünü temsil eder.

Özetle bu slaytın ana mesajı şudur: ağ katmanının veri düzlemi bölümünde IP başlığı altında sıradaki konu **network address translation (NAT)**'tir. Bu geçiş önemlidir; çünkü IP adresleme mantığını öğrendikten sonra, gerçek ağlarda adreslerin neden ve nasıl dönüştürüldüğünü anlamak gerekir. NAT konusu, hem IPv4 adres kıtlığı hem de yerel ağların internetle pratik bağlantı kurma biçimi açısından merkezi bir başlıktır.

NAT: Network Address Translation

NAT: Dış dünya açısından bakıldığında, yerel ağdaki tüm cihazlar **tek** bir IPv4 adresini paylaşır



Ders Notu: NAT: Network Address Translation

Bu slayt, **NAT (Network Address Translation)** mekanizmasının temel mantığını göstermektedir. NAT'in ana fikri şudur: Yerel ağdaki çok sayıda cihaz, dış dünyaya görünürken **tek bir ortak IPv4 adresini** paylaşabilir. Slaytta da açıkça belirtildiği gibi, dış internet açısından bakıldığında yerel ağdaki tüm cihazlar sanki tek bir IP adresi kullanıyormuş gibi görünür. Bu, özellikle IPv4 adres kıtlığı bağlamında son derece önemli bir çözümdür.

Şekilde sağ tarafta bir **yerel ağ** görülmektedir: **10.0.0.0/24**. Bu ağ içindeki cihazlar **10.0.0.1, 10.0.0.2, 10.0.0.3** gibi adresler kullanmaktadır. Bunlar yerel ağ içinde normal biçimde çalışan adreslerdir. Yani bu ağ içindeki cihazlar birbirleriyle haberleşirken kaynak ve hedef adres olarak bu **10.0.0.x** adreslerini kullanır. Slayttaki “datagrams with source or destination in this network have 10.0.0/24 address” ifadesi bunu anlatmaktadır.

Ortadaki yönlendirici ise iki farklı dünyayı birbirine bağlamaktadır. İç tarafta yerel ağ arayüzü **10.0.0.4** adresine sahiptir. Dış tarafta ise internete bakan arayüz **138.76.29.7** adresini kullanmaktadır. NAT işlemi tam da bu sınırdadır yapılır. Yani yerel ağdan dışarı çıkan paketler, yönlendirici üzerinde çevrilir ve dış dünyaya artık içteki 10.0.0.x kaynak adresleriyle değil, **138.76.29.7** kaynak adresiyle görünür.

Buradaki kritik nokta, dışarı çıkan tüm paketlerin aynı dış IP adresini kullanmasına rağmen yine de birbirinden ayırt edilebilmesidir. Slaytta bu durum özellikle vurgulanmaktadır: Yerel ağdan ayrılan tüm datagramlar **aynı kaynak NAT IP adresine**, yani **138.76.29.7**'ye sahiptir; ancak **farklı kaynak port numaraları** kullanırlar. Yani NAT yalnızca IP adresini çevirmekle kalmaz, çoğu zaman taşıma katmanındaki port numaralarını da kullanarak farklı iç akışları birbirinden ayırır. Böylece aynı anda birden fazla iç istemci internete çıkabilir ve yanıt paketleri doğru iç cihaza geri yönlendirilebilir.

Bu yapıyı sezgisel olarak şöyle düşünebiliriz: Evdeki tüm cihazlar dış dünyaya tek bir apartman kapısı üzerinden çıkıyor gibidir. Dışarıdan bakıldığında tek bir adres vardır; fakat içeride hangi daireye gidileceğini belirlemek için ek bilgi gerekir. NAT'te bu ek bilgi çoğu zaman **port numarasıdır**. Böylece dış dünya yalnızca tek bir genel IP görürken, yönlendirici içeride hangi paketin hangi cihaza ait olduğunu takip edebilir.

NAT'in neden bu kadar yaygın kullanıldığı da buradan anlaşılır. Eğer yerel ağdaki her cihaz için ayrı bir küresel IPv4 adresi gerekseydi, evlerde, küçük ofislerde ve birçok kurumda adres ihtiyacı çok daha hızlı büyüdü. NAT sayesinde çok sayıda özel adresli cihaz, tek bir genel IPv4 adresi üzerinden internete erişebilir. Bu da IPv4 adres alanı üzerindeki baskıyı ciddi ölçüde azaltır.

Ancak burada önemli bir mimari sonuç da vardır: Yerel ağdaki cihazların adresleri dış internet tarafından doğrudan görülmez. Dış dünya yalnızca NAT yapan yönlendiricinin

dış IP adresini görür. Bu nedenle NAT, adres tasarrufu sağlarken aynı zamanda uçtan uca adres görünürlüğünü de değiştirir. Bu konu, NAT'in avantajları ve sınırlamaları tartışılırken daha da önem kazanacaktır.

Özetle bu slaytın ana mesajı şudur: **NAT**, yerel ağdaki çok sayıda cihazın dış internet açısından **tek bir ortak IPv4 adresini** paylaşmasını sağlar. Yerel ağ içinde cihazlar özel adreslerini normal biçimde kullanır; fakat dışarı çıkan paketlerin kaynak adresi NAT yapan yönlendiricinin genel IP adresine çevrilir. Farklı iç bağlantılar ise çoğu zaman **port numaraları** kullanılarak birbirinden ayırt edilir. Bu yapı, IPv4 adres alanının daha verimli kullanılmasını sağlayan en önemli pratik mekanizmalardan biridir.

NAT: Network Address Translation

- Yerel ağdaki tüm cihazlar, yalnızca yerel ağda kullanılabilen “özel” IP adres alanındaki (10/8, 172.16/12, 192.168/16 örnekleri) 32 bitlik adreslere sahiptir
- Avantajları :
 - Tüm cihazlar için internet servis sağlayıcısından sadece **bir** IP adresi yeterlidir
 - Yerel ağdaki ana bilgisayarların adreslerini dış dünyaya haber vermeden değiştirebilir
 - Yerel ağdaki cihazların adreslerini değiştirmeden İSS'yi değiştirebilir
 - Güvenlik : Yerel ağ içindeki cihazlara doğrudan erişilemiyor, ancak dış dünyadan görülebiliyor

Network Layer: 4-66

Ders Notu: NAT: Network Address Translation

Bu slayt, NAT'in yalnızca “birçok cihazın tek bir genel IP adresini paylaşması” olmadığını, aynı zamanda bunun **neden tercih edildiğini** açıklamaktadır. Temel yapı şudur: Yerel ağ içindeki cihazlar, internet üzerinde doğrudan yönlendirilmeyen **özel (private) IP adresleri** kullanır. Slaytta bu özel adres alanları **10/8, 172.16/12 ve 192.168/16** ön ekleriyle verilmiştir. Bunlar, yalnızca yerel ağ içinde kullanılmak üzere ayrılmış adres bloklarıdır; küresel internet üzerinde doğrudan hedeflenmezler. Bu yüzden yerel ağdaki cihazlar dış dünyaya çıkarken NAT yapan yönlendiricinin genel adresi üzerinden görünür hale gelir.

Buradaki ilk büyük avantaj, **adres tasarrufudur**. Slaytta da açıkça belirtildiği gibi, yerel ağdaki tüm cihazlar için servis sağlayıcıdan ayrı ayrı genel IP adresi almak gerekmez;

çoğu durumda **tek bir genel IP adresi** yeterlidir. Ev ağlarında, küçük ofislerde ve birçok kurumsal ortamda NAT'ın yaygın olmasının en önemli nedeni budur. Yerel ağ içinde onlarca cihaz olabilir; fakat dış internet açısından bunlar NAT yapan yönlendiricinin tek genel adresi arkasında çalışabilir. Bu, IPv4 adres kıtlığını hafifleten çok güçlü bir pratik çözümdür.

İkinci avantaj, **yerel ağ içindeki adreslerin dış dünyadan bağımsız olarak değiştirilebilmesidir**. Eğer içerideki cihazların adresleri değiştirilirse, internetin geri kalanına bunu duyurmak gerekmez. Çünkü dış dünya zaten doğrudan içteki adresleri görmez; yalnızca NAT cihazının dış arayüzünü görür. Bu, ağ yöneticilerine esneklik sağlar. İç ağ yeniden numaralandırılabilir, cihazlar farklı özel adres bloklarına taşınabilir; fakat bu değişiklik küresel yönlendirme sistemini etkilemez.

Üçüncü avantaj, **ISP değiştirmenin daha kolay hale gelmesidir**. Slaytta da belirtildiği gibi, bir kuruluş servis sağlayıcısını değiştirse bile yerel ağdaki cihazların iç adreslerini değiştirmek zorunda kalmayabilir. Çünkü yerel ağ zaten özel adres alanı kullanmaktadır; dış dünyaya açılan kısım yalnızca NAT cihazının genel adresidir. Bu da iç ağ yapısını dış bağlantı sağlayıcısından kısmen bağımsız hale getirir. Başka bir deyişle, iç adresleme planı ile dış internet bağlantısı birbirinden ayrılmış olur.

Dördüncü avantaj ise **güvenlik ve görünürlük** açısından ortaya çıkar. Slaytta, yerel ağ içindeki cihazların dış dünya tarafından doğrudan adreslenebilir ve görünür olmadığı belirtilmektedir. Bu çok önemlidir. NAT tek başına tam bir güvenlik çözümü değildir; ancak içteki cihazların küresel internet üzerinde doğrudan hedef haline gelmesini zorlaştırır. Dışarıdan bakıldığında görülen ana nokta NAT cihazının genel adresidir. Bu nedenle iç ağdaki makineler, doğrudan kamuya açık uç sistemler gibi davranmaz. Bu da pratikte belirli bir koruma katmanı hissi oluşturur.

Ancak bu avantajları doğru okumak gerekir. NAT'ın asıl gücü, özel adres alanı ile genel internet arasında bir **çeviri sınırı** koymasıdır. Yerel ağ içinde cihazlar özel adreslerle çalışır; dışarı çıkarken NAT cihazı kaynak adresi kendi genel adresiyle değiştirir. İçeri dönen trafikte de doğru iç cihaza yönlendirme yapılır. Böylece hem adres tasarrufu, hem yönetim kolaylığı, hem de belirli ölçüde görünürlük kontrolü sağlanmış olur.

Bu slayt, önceki NAT slaytının mantıksal devamıdır. Önceki slaytta tüm yerel cihazların dış dünyaya tek bir IP ile görünmesi anlatılmıştı. Burada ise bunun neden yararlı olduğu sıralanmaktadır: daha az genel IP ihtiyacı, iç adreslerin serbestçe değiştirilebilmesi, ISP değişiminin kolaylaşması ve iç cihazların doğrudan dış dünyaya açık olmaması.

Özetle bu slaytın ana mesajı şudur: **NAT, özel IP adres alanı kullanan yerel ağların tek ya da az sayıda genel IP adresi üzerinden internete çıkmasını sağlar**. Bunun sonucunda IPv4 adres tasarrufu yapılır, iç ağ dış dünyadan bağımsız biçimde yeniden

düzenlenebilir, servis sağlayıcı değişikliği kolaylaşır ve iç cihazlar dış internet tarafından doğrudan görünür hale gelmez.

NAT: Network Address Translation

Uygulama : NAT yönlendirici (şeffaf bir şekilde):

- **Giden datagramlar:** Giden her datagramın (kaynak IP adresi, bağlantı noktası numarası) değerini (NAT IP adresi, yeni bağlantı noktası numarası) ile değiştir
 - Uzaktaki istemciler/sunucular, hedef adres olarak (NAT IP adresi, yeni bağlantı noktası numarası) kullanarak yanıt verecektir
- (NAT çeviri tablosunda) her (kaynak IP adresi, port numarası) – (NAT IP adresi, yeni port numarası) çeviri çiftini hatırla
- **Gelen datagramlar :** Her gelen datagramın hedef alanlarındaki (NAT IP adresi, yeni port numarası) değerlerini, NAT tablosunda depolanan karşılık gelen (kaynak IP adresi, port numarası) değerleriyle değiştir.

Network Layer: 4-67

Ders Notu: NAT: Network Address Translation

Bu slayt, NAT'in mantığını değil, **nasıl çalıştırıldığını** açıklamaktadır. Önceki slaytlarda yerel ağdaki çok sayıda cihazın dış dünyaya tek bir genel IP adresi üzerinden görünebildiği anlatılmıştı. Burada ise bu sonucun yönlendirici tarafından hangi işlemler yapılarak sağlandığı gösterilmektedir. Ana fikir şudur: NAT yapan yönlendirici, geçen paketlerin adres ve port bilgilerini **şeffaf biçimde değiştirir**, bu değişiklikleri bir tabloda saklar ve dönüş trafiğinde ters çevirme işlemini uygular.

İlk işlem **giden paketler** üzerinde yapılır. Yerel ağdaki bir cihaz internete paket gönderdiğinde, paket üzerinde o cihaza ait **kaynak IP adresi** ve çoğu zaman taşıma katmanına ait **kaynak port numarası** bulunur. NAT yönlendirici bu bilgileri olduğu gibi dış dünyaya bırakmaz. Bunun yerine paketin kaynak alanını, kendi genel **NAT IP adresi** ve yeni bir **port numarası** ile değiştirir. Böylece dış internet açısından bakıldığında paket, artık içerideki gerçek cihazdan değil, NAT yönlendiriciden geliyormuş gibi görünür.

Bu noktada port numarasının neden gerekli olduğu çok önemlidir. Eğer yerel ağdaki birden fazla cihaz aynı anda dışarıya bağlantı kuruyorsa, hepsi aynı genel IP adresini paylaşacaktır. Bu durumda bu akışların birbirinden ayırt edilmesi gerekir. NAT bunu yeni

port numaraları vererek çözer. Yani ayırt edici bilgi yalnızca IP adresi değil, **IP adresi + port numarası çifti** haline gelir. Böylece aynı genel IP adresi üzerinden çok sayıda eşzamanlı bağlantı yönetilebilir.

Slayttaki ikinci önemli nokta, NAT yönlendiricinin bu dönüşümleri **hatırlamak zorunda** olmasıdır. Bunun için bir **NAT translation table**, yani NAT çeviri tablosu tutulur. Bu tabloda, iç ağdaki gerçek kaynak bilgisi ile dış dünyaya sunulan çevrilmiş bilgi eşleştirilir. Başka bir ifadeyle tablo, şu tür bir ilişkiyi saklar:

yerel kaynak IP ve port ↔ NAT'ın dış IP'si ve atanmış yeni port.

Bu tablo olmadan NAT yalnızca giden paketi çevirebilir, fakat dönen yanıtı hangi iç cihaza göndermesi gerektiğini bilemez.

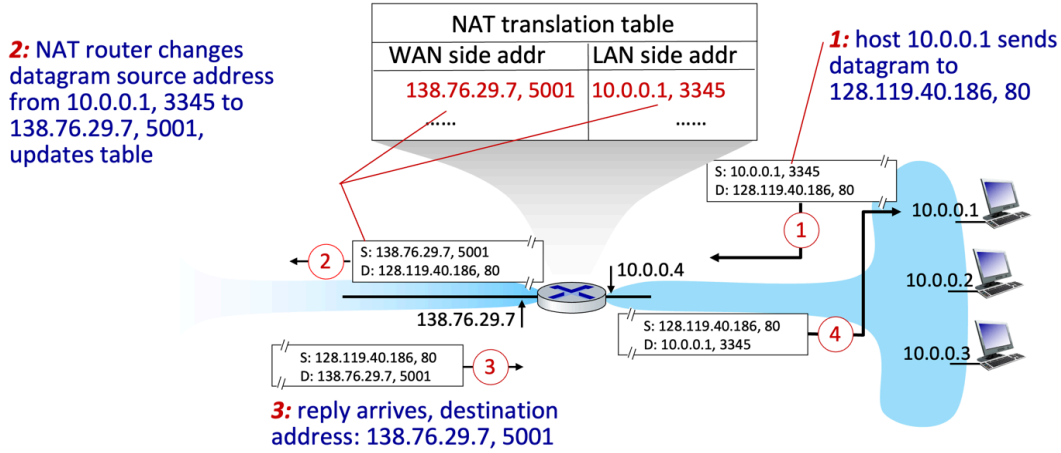
Üçüncü adım **gelen paketler** üzerinde gerçekleşir. Dış dünyadaki bir sunucu ya da uzak istemci, yanıt paketini NAT yönlendiricinin genel IP adresine ve ilgili port numarasına gönderir. Çünkü onun gördüğü kaynak adres budur. NAT yönlendirici gelen paketin **hedef alanına** bakar, kendi çeviri tablosunda bu IP ve port eşleşmesini arar, sonra paketin hedef bilgisini uygun iç adres ve iç port ile değiştirir. Böylece paket doğru yerel cihaza ulaştırılır. Yani gidiş yönünde yapılan çevirme, dönüş yönünde ters çevrilmiş olur.

Burada önemli olan şey, NAT'ın yalnızca IP adresini değil, çoğu durumda **port numarasını da çevirmesidir**. Bu nedenle modern kullanımda NAT çoğu zaman port adres çevirme ile birlikte düşünülür. Çünkü tek genel IP ile çok sayıda iç cihazı desteklemenin pratik yolu budur. Dış dünya açısından tek kaynak varmış gibi görünür; ama NAT tablosu sayesinde içeride hangi akışın hangi cihaza ait olduğu kaybolmaz.

Bu slayt aynı zamanda NAT'ın neden durum bilgisi tutan bir mekanizma olduğunu da gösterir. Klasik yönlendirme çoğu zaman hedef öneke bakıp karar verirken, NAT ek olarak **bağlantı benzeri eşleşmeleri** takip eder. Çünkü dönüş paketinin nereye gideceğini anlayabilmek için geçmişte yapılmış çeviriyi bilmek gerekir. Bu yüzden NAT, basit bir adres değiştirme işleminden daha fazlasıdır; akışları takip eden ve eşleştiren bir ara işlev görür.

Özetle bu slaytın ana mesajı şudur: NAT yönlendirici, **giden paketlerde** iç kaynak IP ve port bilgisini kendi genel IP'si ve yeni bir port ile değiştirir, bu eşleşmeyi **NAT çeviri tablosunda** saklar ve **gelen paketlerde** bu işlemi tersine çevirerek paketi doğru iç cihaza teslim eder. Böylece çok sayıda yerel cihaz, dış dünyaya tek bir genel IP adresi üzerinden erişebilir.

NAT: network address translation



Network Layer: 4-68

Ders Notu: NAT: Network Address Translation

Bu slayt, NAT'in yalnızca kavramsal olarak ne yaptığını değil, **tek bir bağlantı akışı üzerinde adım adım nasıl çalıştığını** göstermektedir. Şekilde yerel ağdaki **10.0.0.1** adresli bir istemcinin dışarıdaki **128.119.40.186** adresli ve **80 numaralı porta** sahip bir sunucuya paket gönderdiği örnek ele alınmaktadır. Bu örnek, NAT'in neden hem adres hem de port bilgisi üzerinde işlem yaptığını çok net biçimde ortaya koyar.

İlk adımda, yerel ağdaki host normal davranır. **10.0.0.1** adresli cihaz, dışarıdaki sunucuya bir datagram gönderir. Paketin kaynak kısmında kendi yerel adresi ve kendi kaynak portu yer alır; şekilde bu bilgi **S: 10.0.0.1, 3345** olarak gösterilmiştir. Hedef kısmında ise dış sunucunun adresi ve portu bulunur: **D: 128.119.40.186, 80**. Yani host, NAT varmış gibi özel bir işlem yapmaz; kendi açısından sıradan bir IP iletişimi başlatmaktadır.

İkinci adımda NAT yönlendirici devreye girer. Slaytta özellikle belirtildiği gibi yönlendirici, datagramın kaynak bilgisini **10.0.0.1, 3345** değerinden **138.76.29.7, 5001** değerine çevirir. Burada **138.76.29.7**, NAT cihazının dış dünyaya bakan genel IP adresidir; **5001** ise NAT tarafından bu akış için atanmış yeni port numarasıdır. Yani paket artık internete çıkarken yerel host'tan geliyormuş gibi değil, NAT yönlendiriciden geliyormuş gibi görünür. Aynı anda NAT çeviri tablosuna da şu eşleşme yazılır:

WAN side: 138.76.29.7, 5001 ↔ LAN side: 10.0.0.1, 3345.

Bu tablo, dönüş trafiğinin doğru iç cihaza gönderilebilmesi için zorunludur.

Üçüncü adımda dışarıdaki sunucudan yanıt gelir. Sunucu, kendisine görünen kaynak kimliği ne ise ona cevap verir. Yani artık hedef olarak **10.0.0.1, 3345**'i değil, **138.76.29.7, 5001**'i kullanır. Şekilde bu açıkça **S: 128.119.40.186, 80** ve **D: 138.76.29.7, 5001** olarak gösterilmiştir. Bu çok önemli bir noktadır: dış dünya, yerel ağdaki gerçek istemciyi hiç görmez; yalnızca NAT cihazının genel adresini ve ona atanmış portu görür.

Dördüncü adımda NAT yönlendirici gelen yanıtı alır ve hedef alanına bakarak kendi NAT çeviri tablosunda eşleşme arar. Tabloda **138.76.29.7, 5001** için kayıtlı iç eşleşmenin **10.0.0.1, 3345** olduğunu bulur. Bunun üzerine paketin hedef bilgisini bu iç adres ve port ile değiştirir ve paketi yerel ağa iletir. Böylece istemciye ulaşan paket artık şu biçimdedir:

S: 128.119.40.186, 80

D: 10.0.0.1, 3345

Yani gidiş yönünde yapılan çeviri, dönüş yönünde tersine çevrilmiş olur.

Bu örnekten çıkarılması gereken en önemli sonuç, NAT'in yalnızca IP adresini değiştirmedir. Eğer sadece IP adresi değişseydi, aynı anda birden fazla iç istemci dışarıya bağlantı kurduğunda bunları ayırt etmek mümkün olmazdı. Burada ayırt edici unsur, **IP adresi + port numarası** çiftidir. Bu nedenle NAT pratikte çoğu zaman taşıma katmanındaki portları da kullanır ve bu sayede tek bir genel IP üzerinden çok sayıda eşzamanlı akışı destekler.

Şekildeki NAT çeviri tablosu da bu yüzden merkezî rol oynar. NAT yönlendirici bağlantı benzeri bir durum bilgisi tutar. Hangi iç istemcinin hangi dış port ile temsil edildiğini bilmeden, gelen yanıtları doğru iç cihaza iletmesi mümkün değildir. Bu nedenle NAT, sıradan yönlendirmeden farklı olarak akışlara dair geçici eşleşmeler saklayan bir ara mekanizma gibi çalışır.

Bu slayt aynı zamanda önceki NAT slaytlarının mantığını pekiştirir. Yerel ağdaki cihazlar özel adresler kullanır; dışarıya çıkarken hepsi NAT'ın genel adresi arkasında görünür; fakat her akış farklı port numaraları ile ayrılır. Böylece hem adres tasarrufu sağlanır hem de birden fazla cihazın internet erişimi aynı genel IPv4 adresi üzerinden yürütülebilir.

Özetle bu slaytın ana mesajı şudur: NAT yönlendirici, yerel istemciden çıkan paketin **kaynak IP ve port** bilgisini kendi **genel IP'si ve yeni bir port** ile değiştirir, bu eşleşmeyi tabloda saklar, dışarıdan gelen yanıt paketlerinde de bu işlemi tersine çevirerek paketi doğru iç host'a teslim eder. Bu mekanizma sayesinde **10.0.0.1** gibi özel adresli bir cihaz, dış dünyaya **138.76.29.7:5001** kimliğiyle görünür ve yanıtlar güvenli biçimde tekrar kendisine ulaştırılır.

NAT: network address translation

- NAT tartışmalı bir konu olmuştur :
 - Yönlendiriciler “sadece” 3. katmana kadar işlem yapmalıdır
 - IPv6 ile “kısıtlılık” sorunu çözülmelidir
 - Uçtan uca ilkesini ihlal eder (ağ katmanı cihazı tarafından bağlantı noktası numarasının değiştirilmesi)
 - NAT geçişi: Ya istemci NAT arkasında bulunan sunucuya bağlanmak isterse?
- Pratikte NAT kalıcıdır :
 - Ev ve kurumsal ağlarda, 4G/5G hücresel ağlarda yaygın olarak kullanılmaktadır

Network Layer: 4-69

Ders Notu: NAT: Network Address Translation

Bu slayt, NAT'in neden çok yaygın kullanıldığını anlattıktan sonra, neden aynı zamanda **tartışmalı** bir çözüm olduğunu açıklamaktadır. Yani burada NAT'in teknik olarak işe yaradığı kabul edilir; fakat bunun internet mimarisi açısından bazı bedelleri olduğu vurgulanır. Slaytın ana mesajı iki parçalıdır: **NAT bazı temel ağ ilkeleriyle gerilim içindedir, ama buna rağmen pratikte kalıcı hale gelmiştir.**

İlk eleştiri, yönlendiricilerin normalde yalnızca **3. katmana kadar** işlem yapması gerektiği düşüncesidir. Klasik internet mimarisinde router, temel olarak IP başlığına bakar ve paketi uygun çıkışa yollar. Oysa NAT yapan cihaz yalnızca ağ katmanı bilgisini değil, çoğu durumda **taşıma katmanındaki port numaralarını** da değiştirmektedir. Bu yüzden NAT, klasik “router sadece layer 3'e kadar bakmalı” anlayışını aşan bir davranış sergiler. Başka bir ifadeyle, NAT cihazı saf bir yönlendirici olmaktan çıkıp daha müdahaleci bir ara cihaza dönüşür.

İkinci eleştiri, adres kıtlığının asıl çözümünün NAT değil, **IPv6** olması gerektiği fikridir. NAT, IPv4 adres alanı yetersiz kaldığı için geliştirilmiş çok pratik bir ara çözümdür. Ancak mimari açıdan bakıldığında, problem adres uzayının küçüklüğünden kaynaklanıyorsa bunu kalıcı olarak çözmenin daha doğru yolu daha büyük adres alanına geçmektir. Bu nedenle bazı bakış açıları, NAT'i yaratıcı ama geçici bir “yama”,

IPv6'yı ise daha temiz ve uzun vadeli çözüm olarak görür. Slaytta da açıkça, adres “shortage” probleminin IPv6 ile çözülmesi gerektiği belirtilmektedir.

Üçüncü ve belki de en temel eleştiri, NAT'in **end-to-end argument** denilen ilkeyi zedelemesidir. İnternetin klasik tasarım anlayışında ağın içi olabildiğince basit kalmalı, asıl karmaşık işlevler uç sistemlerde olmalıdır. NAT ise ağın ortasındaki bir cihazın, uçtan uca iletişimin bir parçası olan adres ve port bilgilerini değiştirmesi anlamına gelir. Bu yüzden uçtan uca şeffaflık bozulur. Özellikle port numaralarının bir ağ katmanı cihazı tarafından değiştirilmesi, katmanlar arası sınırların bulanıklaşmasına neden olur. Bu, NAT'in en çok tartışılan mimari etkilerinden biridir.

Dördüncü sorun **NAT traversal** problemidir. Slayttaki soru çok önemlidir: “İstemci, NAT arkasındaki bir sunucuya bağlanmak isterse ne olur?” NAT dışarıdan bakıldığında iç ağdaki cihazları doğrudan görünmez hale getirir. Bu, bir avantaj gibi görünse de, dışarıdan içeri doğru başlatılan bağlantılarda sorun yaratır. Çünkü dış dünya genellikle yalnızca NAT cihazının genel adresini görür; içeride hangi cihaza gidilmesi gerektiğini bilmez. Bu nedenle eşler arası iletişim, VoIP, görüntülü görüşme, oyunlar ve uzaktan erişim gibi senaryolarda NAT geçişi ek teknikler gerektirir. Yani NAT, istemcinin dışarı çıkmasını kolaylaştırırken, dışarıdan içeri erişimi zorlaştırabilir.

Buna rağmen slaytın ikinci yarısı çok nettir: **NAT is here to stay**. Yani tüm bu mimari eleştirilere karşın NAT, pratikte vazgeçilmez hale gelmiştir. Bunun nedeni çok açıktır: adres tasarrufu sağlar, yerel ağ yönetimini kolaylaştırır ve mevcut IPv4 interneti üzerinde çalışır. Bu yüzden ev ağlarında, kurumsal ağlarda ve hücresel ağlarda yaygın biçimde kullanılmaktadır. Slaytta da özellikle ev ve kurumsal ağlar ile **4G/5G cellular nets** örnekleri verilmiştir. Bu ifade, NAT'in yalnızca küçük ev yönlendiricilerinde değil, çok büyük ölçekli operatör ağlarında da kullanıldığını göstermektedir.

Bu nedenle NAT'i değerlendirirken iki düzeyi birlikte düşünmek gerekir. **Mimari düzeyde** NAT, internetin ilk tasarım ilkeleriyle tam uyumlu değildir. **Mühendislik düzeyinde** ise son derece başarılı ve yaygın bir çözümdür. Gerçek internet çoğu zaman bu iki yaklaşımın arasında yaşar: teoride daha temiz çözümler vardır, ama pratikte en çalışabilir olanlar baskın hale gelir. NAT bunun en iyi örneklerinden biridir.

Özetle bu slaytın ana mesajı şudur: **NAT mimari olarak tartışmalıdır**, çünkü router'ın yalnızca ağ katmanında kalması ilkesini aşar, uçtan uca şeffaflığı bozar ve NAT traversal gibi sorunlar doğurur. Buna rağmen **IPv4 internetinin pratik gerçekliği içinde çok yaygın ve kalıcı bir çözüm** haline gelmiştir; ev ağlarında, kurumsal ağlarda ve mobil operatör ağlarında yoğun biçimde kullanılmaya devam etmektedir.

IPv6: motivation

- **initial motivation:** 32-bit IPv4 address space would be completely allocated
- additional motivation:
 - speed processing/forwarding: 40-byte fixed length header
 - enable different network-layer treatment of “flows”

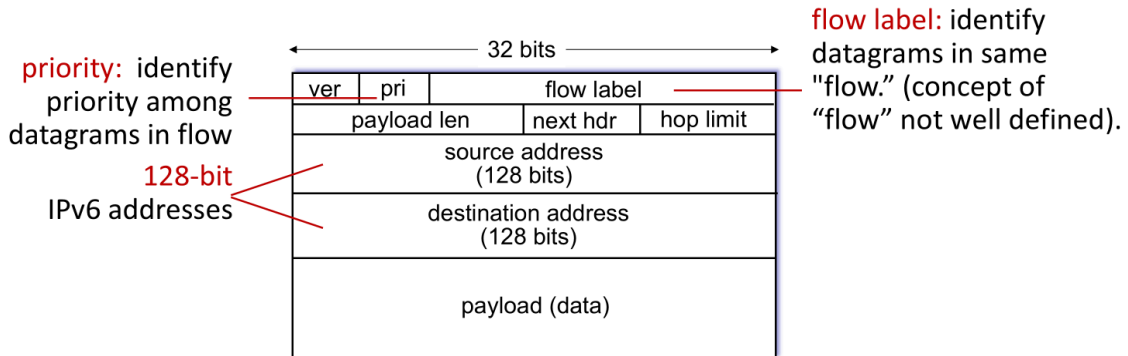
IPv6'e geçişin ilk ve en temel nedeni, IPv4'ün 32 bitlik adres alanının zamanla yetersiz hale gelmesidir. İnternete bağlanan cihaz sayısı çok hızlı arttıkça mevcut IPv4 adreslerinin tamamen tahsis edileceği öngörülmüştür. Bu nedenle IPv6, çok daha geniş bir adres alanı sağlayarak uzun vadeli çözüm olarak geliştirilmiştir.

Slaytta belirtilen ek motivasyonlardan biri, IPv6 başlığının sabit uzunlukta 40 bayt olacak şekilde tasarlanmasıdır. Bu yapı, paket işleme ve yönlendirme işlemlerini daha düzenli hale getirir. Başlık yapısının sadeleştirilmesi, yönlendiricilerin paketi daha hızlı işlemesine yardımcı olur.

Bir diğer önemli nokta, IPv6'in “flow” kavramını desteklemesidir. Buradaki amaç, ağ içinde bazı veri akışlarının farklı şekilde ele alınabilmesini sağlamaktır. Özellikle gerçek zamanlı uygulamalar, ses, video ya da belirli hizmet kalitesi beklentileri olan trafik türleri açısından bu yaklaşım önemlidir.

Özetle IPv6 yalnızca daha fazla adres üretmek için ortaya çıkmış bir çözüm değildir. Aynı zamanda daha ölçeklenebilir, daha düzenli ve gelecekteki ağ gereksinimlerine daha uygun bir internet mimarisi oluşturma amacı taşır.

IPv6 datagram format



What's missing (compared with IPv4):

- no checksum (to speed processing at routers)
- no fragmentation/reassembly
- no options (available as upper-layer, next-header protocol at router)

Network Layer: 4-71

Ders Notu: IPv6 datagram format

Bu slayt, IPv6 başlığının temel alanlarını ve IPv4'e göre getirilen yapısal değişiklikleri göstermektedir. Buradaki ana fikir, IPv6'in yalnızca adres boyutunu büyütmediği, aynı zamanda başlık yapısını da **daha sade ve daha hızlı işlenebilir** hale getirdiğidir. Bu nedenle IPv6 datagram biçimi, hem ölçeklenebilirlik hem de yönlendiricilerde hızlı forwarding açısından önemlidir.

Başlığın en üst kısmında yine **version (ver)** alanı bulunur. Bu alan, paketin IPv6 olduğunu belirtir. Bunun yanında slaytta **priority (pri)** olarak gösterilen alan, aynı akış içindeki datagramların öncelik bilgisini ifade etmek için düşünülmüştür. Amaç, bazı paketlerin diğerlerine göre daha öncelikli ele alınabilmesini desteklemektir. Bu, hizmet farklılaştırması ya da belirli trafik türlerine özel davranış sağlama fikriyle ilişkilidir.

Bir diğer önemli alan **flow label**'dir. Slaytta da belirtildiği gibi bu alan, aynı **flow** içinde yer alan datagramları tanımlamak için kullanılır. Buradaki "flow" kavramı, kabaca aynı iletişim akışına ait paketler olarak düşünülebilir. Amaç, ağın belirli bir akışa özel işlem uygulayabilmesini kolaylaştırmaktır. Örneğin bazı gerçek zamanlı trafik türlerinin birlikte tanınması ve benzer şekilde ele alınması hedeflenmiştir. Ancak slaytın da vurguladığı gibi, flow kavramının nasıl tanımlanacağı tarihsel olarak her zaman çok net ve tek biçimli olmamıştır.

Başlıktaki **payload length** alanı, başlıktan sonra gelen veri bölümünün uzunluğunu belirtir. **Next header** alanı ise bir sonraki üst katman protokolünü ya da bir uzantı

başlığını işaret eder. IPv4'teki protocol alanına benzer bir işlev görür; ancak IPv6'te bu alan daha esnek bir zincirleme yapıya da imkân verir. **Hop limit** alanı ise IPv4'teki TTL alanının karşılığıdır. Paket her yönlendiriciden geçtiğinde bu değer azaltılır; değer sıfıra düşerse paket dolaşımda sonsuza kadar kalmasın diye atılır.

IPv6 başlığındaki en dikkat çekici farklardan biri, **kaynak ve hedef adreslerinin 128 bit** olmasıdır. Slaytta bu açıkça gösterilmektedir. IPv4'te 32 bit olan adres alanının IPv6'te 128 bite çıkarılması, adres uzayını çok büyük ölçüde genişletir. Bu, IPv6'in ortaya çıkışındaki en temel motivasyonlardan biridir. Böylece çok daha fazla cihaz, çok daha esnek adresleme yapılarıyla internete bağlanabilir.

Slaytın alt kısmı özellikle önemlidir; çünkü burada IPv4'e göre **hangi alanların artık bulunmadığı** belirtilmektedir. Birincisi, IPv6 başlığında **checksum yoktur**. Bunun temel nedeni, yönlendiricilerde işleme hızını artırmaktır. IPv4'te her yönlendirici TTL değiştiği için başlık sağlama toplamını yeniden hesaplamak zorundaydı. IPv6'te bu yük kaldırılmıştır. Hata denetimi zaten alt katmanlarda ve gerektiğinde üst katmanlarda yapılabildiği için, ağ katmanı başlığından checksum alanı çıkarılmıştır.

İkinci önemli fark, IPv6 temel başlığında **fragmentation/reassembly alanlarının bulunmamasıdır**. IPv4'te yönlendiriciler gerekirse paketleri parçalayabiliyordu. IPv6 tasarımında bu iş yönlendiricilerden alınmıştır. Böylece ağ içindeki cihazların görevi sadeleşmiştir. Parçalama gerekiyorsa bu daha çok uç sistemlerin ya da özel uzantı başlıklarının konusu haline gelir. Bu da veri düzlemini hızlandıran önemli tasarım kararlarından biridir.

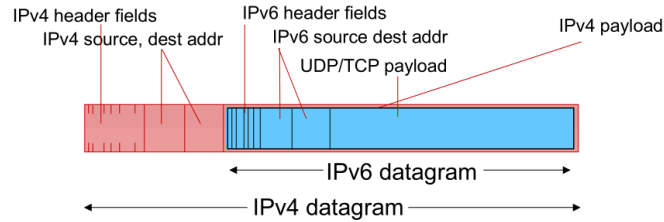
Üçüncü fark, temel başlıkta **options alanının bulunmamasıdır**. IPv4'te seçenekler başlığı karmaşıktırabiliyordu. IPv6'te bu tür ek bilgiler gerektiğinde **extension headers** mantığıyla, yani next header zinciri üzerinden taşınır. Böylece her paket için temel başlık sabit ve öngörülebilir kalır; yalnızca gerekli olduğunda ek başlıklar kullanılır.

Bu noktada IPv6'in bir başka önemli özelliği de ortaya çıkar: **başlık sabit uzunluktadır ve 40 bayttır**. Slayt bunu doğrudan yazmasa da, gösterilen alan yapısı bu sabitliği desteklemektedir. Sabit başlık uzunluğu, yönlendiricilerin paketi daha hızlı ayırıştırmasına ve donanım düzeyinde daha verimli işlemesine yardımcı olur. Bu, IPv6 motivasyon slaytındaki "speed processing/forwarding" hedefiyle doğrudan bağlantılıdır.

Özetle bu slaytın ana mesajı şudur: IPv6 datagram biçimi, **128 bit adresler, flow label** ve daha sadeleştirilmiş bir temel başlık yapısı ile IPv4'ten ayrılır. IPv6 başlığında checksum, yönlendirici tabanlı parçalama ve klasik options alanı kaldırılmıştır. Bu tasarım, hem adresleme kapasitesini çok büyütür hem de yönlendiricilerde daha hızlı ve daha verimli paket işleme sağlamayı amaçlar.

Transition from IPv4 to IPv6

- not all routers can be upgraded simultaneously
 - no “flag days”
 - how will network operate with mixed IPv4 and IPv6 routers?
- **tunneling**: IPv6 datagram carried as *payload* in IPv4 datagram among IPv4 routers (“packet within a packet”)
 - tunneling used extensively in other contexts (4G/5G)



Network Layer: 4-72

Ders Notu: Transition from IPv4 to IPv6

Bu slayt, IPv4'ten IPv6'ya geçişin neden teknik olarak zor bir süreç olduğunu ve bu zorluğu aşmak için kullanılan temel yöntemlerden birini açıklamaktadır. Temel problem şudur: İnternet gibi çok büyük ve dağınık bir yapıda, tüm yönlendiricileri ve tüm ağ cihazlarını aynı anda IPv6'ya geçirmek mümkün değildir. Yani bir gün belirlenip herkesin aynı anda yeni protokole geçtiği bir “**flag day**” yaklaşımı gerçekçi değildir. Bu nedenle geçiş süreci, uzun süre boyunca **IPv4 ve IPv6'nın birlikte bulunduğu karma bir ortamda** yürümek zorundadır.

Slayttaki ilk önemli vurgu, “not all routers can be upgraded simultaneously” ifadesidir. Bunun anlamı, bazı ağların ve yönlendiricilerin IPv6 desteklerken bazılarının yalnızca IPv4 ile çalışmaya devam edeceğidir. Bu durumda temel soru ortaya çıkar: **IPv6 kullanan uçlar arasında iletişim, arada yalnızca IPv4 destekleyen yönlendiriciler varsa nasıl sürdürülecektir?** İşte bu soru geçiş mekanizmalarının doğmasına yol açmıştır.

Bu slaytta verilen temel çözüm **tunneling** yöntemidir. Tunneling'in mantığı şudur: Bir **IPv6 datagramı**, geçici olarak bir **IPv4 datagramının yükü (payload)** içine yerleştirilir. Böylece IPv4 ağı, içeriğini anlamasa bile bu paketi sıradan bir IPv4 paketi gibi taşıyabilir. Bu nedenle slaytta “**packet within a packet**” ifadesi kullanılmaktadır. Yani burada dışta IPv4 başlığı, içte ise gerçek IPv6 paketi vardır.

Şekil bu yapıyı açık biçimde göstermektedir. En dışta bir **IPv4 datagramı** vardır. Bunun başlığında IPv4 kaynak ve hedef adresleri bulunur. Bu dış başlık, paketin IPv4 ağı içinde bir uçtan diğer uca taşınmasını sağlar. Ancak bu IPv4 paketinin yük kısmında aslında bir **IPv6 datagramı** taşınmaktadır. İçteki bu IPv6 datagramı kendi IPv6 başlığını ve onun taşıdığı TCP/UDP verisini içerir. Yani IPv4 ağı, içte taşınan asıl IPv6 paketiyle ilgilenmez; sadece dıştaki IPv4 paketini iletir.

Bu işlemi bir tünel benzetmesiyle düşünmek faydalı olur. İki IPv6 adası arasında, ortada yalnızca IPv4 konuşabilen bir ağ varsa, IPv6 paketi bu IPv4 bölgesinden kendi kimliğini koruyarak doğrudan geçemez. Bunun yerine bir kapsül içine alınır, IPv4 ağı boyunca taşınır ve tünelin diğer ucunda tekrar açılır. Böylece iki IPv6 bölgesi, aradaki IPv4 altyapısına rağmen birbirine bağılıymış gibi çalışabilir.

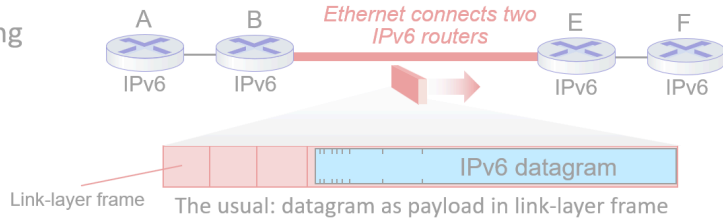
Bu yöntemin önemli yönlerinden biri, geçişi **aşamalı** hale getirmesidir. Yani tüm interneti bir anda dönüştürmek yerine, bazı bölgeler IPv6'ya geçerken diğer bölgeler bir süre IPv4 kalabilir. Tunneling bu karma dönemde birlikte çalışabilirliği sağlar. Bu yüzden IPv4'ten IPv6'ya geçiş, tek seferlik bir olay değil; uzun süreli, ara çözümler kullanan evrimsel bir süreç olarak düşünülmelidir.

Slayt ayrıca tunneling'in yalnızca IPv4/IPv6 geçişine özgü olmadığını da belirtmektedir. Özellikle **4G/5G** gibi modern mobil ağlarda da tünelleme mantığı yaygın biçimde kullanılmaktadır. Bu da şunu gösterir: Tunneling, ağ mühendisliğinde çok genel ve güçlü bir tekniktir. Bir protokol birimini başka bir protokolün içine yerleştirerek, normalde doğrudan geçemeyeceği bir ağ altyapısından geçirilmesini sağlar.

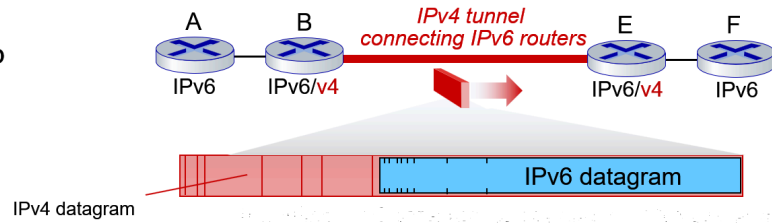
Özetle bu slaytın ana mesajı şudur: **IPv4'ten IPv6'ya geçiş bir anda yapılamaz**, çünkü tüm yönlendiriciler aynı anda yükseltilemez. Bu nedenle karma IPv4-IPv6 ortamlarında iletişim için **tunneling** kullanılır. Bu yöntemde bir IPv6 datagramı, bir IPv4 datagramının yükü içine kapsüllenerak IPv4 ağı üzerinden taşınır; tünelin diğer ucunda tekrar çıkarılır. Böylece kademeli geçiş mümkün hale gelir.

Tunneling and encapsulation

Ethernet connecting two IPv6 routers:



IPv4 tunnel connecting two IPv6 routers



Network Layer: 4-74

Bu slayt, **tünelleme (tunneling)** ile **kapsülleme (encapsulation)** arasındaki ilişkiyi çok net gösterir. Temel fikir şudur: Bir ağ teknolojisi ya da protokol, kendi anlayamadığı başka bir protokolün paketini, onu kendi veri alanı içinde taşıyarak iletebilir. Yani burada asıl veri, başka bir paketin içine yerleştirilmiş olur.

Slaydın üst kısmında normal durum gösterilmektedir. İki IPv6 yönlendirici arasında doğrudan bir Ethernet bağlantısı varsa, süreç alışılmış biçimde çalışır. IPv6 datagramı, bağlantı katmanı çerçevesinin yükü olarak taşınır. Burada ekstra bir ağ katmanı kapsüllemesi yoktur. Yani Ethernet çerçevesi, IPv6 paketini doğrudan taşır. Bu, katmanlı mimaride daha önce gördüğümüz standart kapsülleme mantığıdır: ağ katmanı verisi, alt katman tarafından taşınır.

Alt kısımda ise daha ilginç bir durum vardır. Burada iki IPv6 yönlendirici arasında doğrudan IPv6 taşıyan bir altyapı yoktur; aradaki bölüm IPv4 ağıdır. Bu durumda IPv6 yönlendiricileri birbirine bağlamak için bir **IPv4 tüneli** oluşturulur. Tünelin girişindeki yönlendirici, IPv6 datagramını alır ve onu bir **IPv4 datagramının yükü** haline getirir. Yani IPv6 paketi kaybolmaz, içeriği değişmez; sadece dışına yeni bir IPv4 başlığı eklenir. Böylece aradaki IPv4 ağ, bu paketi sıradan bir IPv4 paketi gibi taşır.

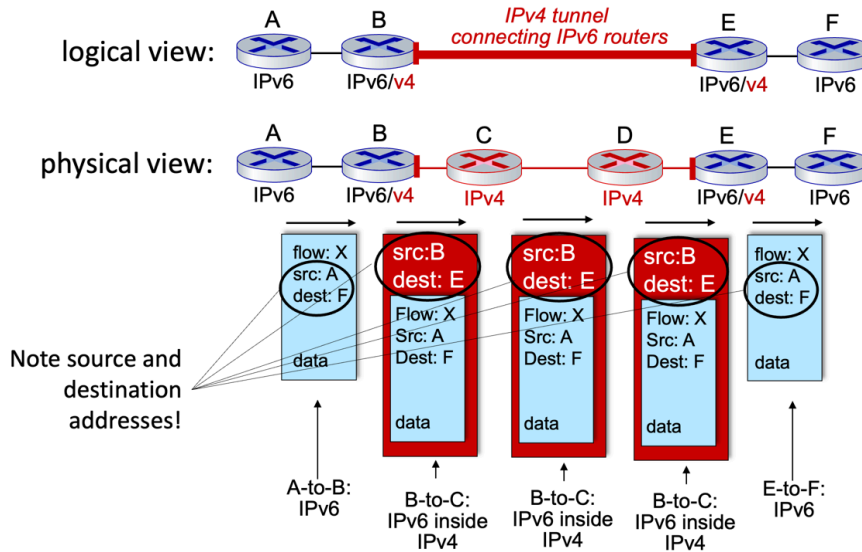
Tünelin çıkışındaki yönlendiriciye ulaşıldığında dıştaki IPv4 başlığı çıkarılır. Geriye özgün IPv6 datagramı kalır ve bu paket IPv6 ortamında normal şekilde yoluna devam eder. Bu nedenle tünelleme, mantıksal olarak bir protokolün başka bir protokolün içinden geçirilmesidir. Burada taşıyıcı protokol IPv4, taşınan asıl protokol ise IPv6'dır.

Bu yapının en önemli amacı **geçiş sürecini mümkün kılmaktır**. İnternette tüm yönlendiriciler ve tüm ağlar aynı anda IPv6'ya geçirilemeyeceği için, bazı bölgeler IPv6 desteklerken bazı bölgeler yalnızca IPv4 çalıştırabilir. Tünelleme bu iki dünya arasında köprü kurar. Böylece IPv6 destekleyen iki uç, arada yalnızca IPv4 çalışan bir bölüm olsa bile haberleşmeye devam edebilir.

Burada dikkat edilmesi gereken nokta, tünellemenin sadece “adres taşımak” olmadığıdır. Aslında bu, ağ katmanının kendi içinde yaptığı özel bir kapsüllemedir. Önceki konularda taşıma katmanı segmentinin IP datagramı içinde, IP datagramının da bağlantı katmanı çerçevesi içinde taşındığını görmüştük. Bu slaytta ise bir **IPv6 datagramının, başka bir ağ katmanı paketi olan IPv4 datagramı içinde taşındığı** gösterilmektedir. Yani kapsülleme mantığı burada katmanlar arasında değil, aynı katmanın farklı protokol sürümleri arasında uygulanmaktadır.

Bu slayttan çıkarılması gereken ana fikir şudur: **Tünelleme, uyumsuz ya da farklı protokol ortamlarını birbirine bağlamak için kullanılan bir kapsülleme tekniğidir**. IPv6 geçişinde bunun önemi çok büyüktür; çünkü gerçek dünyada ağ dönüşümleri ani değil, aşamalı gerçekleşir. Bu yüzden tünelleme, IPv4'ten IPv6'ya geçişin temel yardımcı mekanizmalarından biri olmuştur.

Tunneling



Bu slayt, tünellemenin yalnızca “paket içinde paket” fikrinden ibaret olmadığını; aynı zamanda **mantıksal görünüm** ile **fiziksel görünüm** arasındaki farkı anlamamız gerektiğini gösterir.

Üstteki **logical view** kısmında, B ile E arasında sanki doğrudan bir bağlantı varmış gibi düşünülür. Yani IPv6 açısından bakıldığında, B ve E iki uç IPv6/v4 yönlendiricisidir ve aradaki IPv4 bölüm tek bir “tünel” gibi görünür. Bu mantıksal bakış açısı, ağ tasarımını sadeleştirir. IPv6 yönlendiricileri, aradaki tüm IPv4 ayrıntılarıyla tek tek uğraşmaz; sadece B’den E’ye uzanan bir tünel varmış gibi davranır.

Altteki **physical view** kısmında ise gerçekte ne olduğu gösterilir. Fiziksel olarak B ile E arasında C ve D gibi yalnızca IPv4 çalışan yönlendiriciler vardır. Yani mantıksal olarak tek bir tünel gibi görünen yapı, fiziksel olarak birden fazla IPv4 yönlendirici ve bağlantıdan oluşmaktadır. Bu ayrım çok önemlidir: Mantıksal görünüm ağ mühendisine soyutlama sağlar, fiziksel görünüm ise paketin gerçekten hangi cihazlardan geçtiğini gösterir.

Slaytın en kritik noktası, ortadaki paket çizimlerinde gösterilen **kaynak ve hedef adreslerinin değişmesidir**. A’dan çıkan özgün IPv6 paketinde kaynak adres A, hedef adres F’tir. Bu, gerçek uçtan uca iletişimin adres bilgisidir. Yani asıl iletişim A ile F arasındadır. Ancak paket B yönlendiricisine geldiğinde ve tünele sokulacağı zaman, bu IPv6 paketinin dışına yeni bir **IPv4 başlığı** eklenir. Bu dış başlıkta kaynak artık B, hedef ise E olur. Çünkü IPv4 ağı paketi A’dan F’ye değil, tünelin giriş noktası olan B’den çıkış noktası olan E’ye taşıyacaktır.

Burada çok önemli bir kavramsal ayrım vardır: **İç başlık** gerçek uçtan uca iletişimi temsil eder, **dış başlık** ise geçici taşıma işini temsil eder. İçerideki IPv6 datagramında kaynak A ve hedef F bilgisi korunur. Dışarıdan eklenen IPv4 başlığında ise kaynak B ve hedef E yazılır. Yani aradaki IPv4 ağ, paketin içindeki gerçek IPv6 hedefini anlamak zorunda değildir; sadece kendi açısından B’den E’ye giden bir IPv4 paketi görür.

C ve D gibi ara IPv4 yönlendiricileri de bu yüzden yalnızca dış IPv4 başlığına bakar. Onlar için paket, B kaynaklı ve E hedefli sıradan bir IPv4 datagramıdır. İçeride taşınan IPv6 verisiyle ilgilenmezler. Bu durum tünellemenin temel gücünü ortaya koyar: Aradaki ağ, taşıdığı üstteki protokolü desteklemek zorunda kalmadan sadece dış kapsülü işleyerek iletim yapabilir.

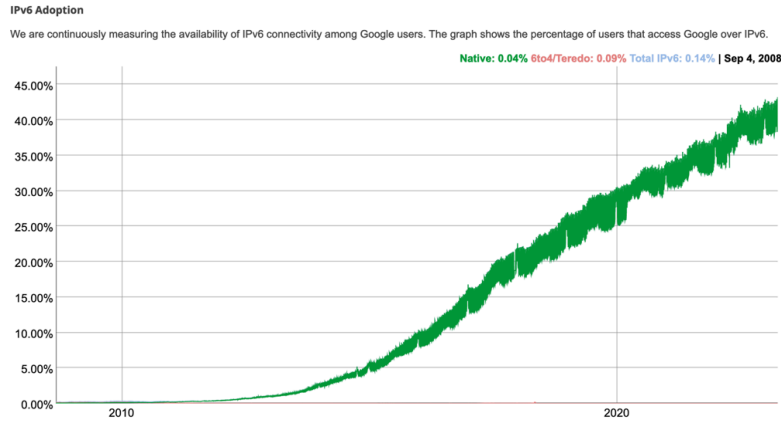
Paket E’ye ulaştığında tünel sona erer. E yönlendiricisi dıştaki IPv4 başlığını kaldırır. Böylece elde yine özgün IPv6 datagramı kalır; kaynak A, hedef F olarak yoluna devam eder. Sonrasında E’den F’ye iletim tekrar normal IPv6 mantığıyla gerçekleşir. Slaytın altındaki “A-to-B: IPv6”, “B-to-C: IPv6 inside IPv4”, “E-to-F: IPv6” notları da tam olarak bunu anlatır. Yani tünel yalnızca belirli bir ara bölümde devrededir; tünel öncesi ve sonrası paket yine kendi doğal protokolüyle taşınır.

Bu slayttan çıkarılması gereken ana fikir şudur: **Tünelleme sırasında paket iki farklı adresleme bağlamına sahip olur**. İçteki başlık gerçek uç noktaları gösterir; dıştaki

başlık ise tünelin iki ucunu gösterir. Bu sayede IPv6 paketleri, IPv4 altyapısı içinden bozulmadan geçirilebilir. IPv6 geçiş sürecinde tünellemenin neden işe yaradığını anlamamanın en doğru yolu da budur: ağın bir bölümü geçici olarak yalnızca “taşıyıcı” rolü üstlenir, asıl iletişim bilgisi ise iç pakette korunur.

IPv6: adoption

- Google¹: ~ 40% of clients access services via IPv6 (2023)
- NIST: 1/3 of all US government domains are IPv6 capable



Bu slayt, IPv6'nın teknik olarak neden geliştirildiğini anlattıktan sonra, **gerçek dünyada ne kadar benimsendiği** sorusuna cevap verir. Yani artık konu yalnızca “IPv6 mümkün mü?” değil, “IPv6 gerçekten kullanılıyor mu?” sorusudur. Slayttaki iki madde, IPv6'nın uzun ve yavaş bir geçiş sürecinden geçmesine rağmen artık ciddi ölçüde yaygınlaştığını göstermektedir. Google kullanıcılarının yaklaşık %40'ının hizmetlere IPv6 üzerinden erişmesi ve ABD hükümetine ait alan adlarının yaklaşık üçte birinin IPv6 uyumlu olması, IPv6'nın artık sadece teorik ya da deneysel bir teknoloji olmadığını ortaya koymaktadır.

Burada dikkat edilmesi gereken ilk nokta, IPv6'nın yaygınlaşmasının **bir anda değil, kademeli olarak** gerçekleşmiş olmasıdır. Slayttaki grafikte 2008 civarında neredeyse yok denecek kadar az olan IPv6 kullanım oranının yıllar içinde sürekli yükseldiği görülmektedir. Bu artış, ağ teknolojilerinde geçişlerin neden uzun sürdüğünü de açıklar. Çünkü yalnızca istemcilerin değil; servis sağlayıcıların, yönlendiricilerin, işletim sistemlerinin, veri merkezlerinin, güvenlik cihazlarının ve uygulama altyapılarının da IPv6'yı desteklemesi gerekir. Bu nedenle IPv6'ya geçiş, tek bir yazılım güncellemesiyle tamamlanabilecek kadar basit değildir; tüm ekosistemin dönüşmesini gerektirir.

Google verisi özellikle önemlidir çünkü son kullanıcı tarafındaki gerçek erişim davranışını gösterir. Yani burada ölçülen şey yalnızca bir sunucunun IPv6 desteklemesi

değildir; kullanıcıların gerçekten Google hizmetlerine IPv6 üzerinden ulaşım ulaşımadığıdır. Bu da IPv6'nın yalnızca ağ omurgasında değil, istemci erişim ağlarında ve servis sağlayıcı altyapılarında da belirli bir olgunluğa ulaştığını gösterir. Başka bir ifadeyle, IPv6 artık “hazır olunması gereken gelecek” değil, güncel internet trafiğinin anlamlı bir parçasıdır.

NIST tarafından verilen bilgi ise kurumsal ve kamusal ölçekte geçişin durumunu göstermektedir. ABD hükümetine ait alan adlarının üçte birinin IPv6 uyumlu olması, büyük ölçekli kurumların da bu dönüşüme dahil olduğunu gösterir. Bu tür büyük kurumsal yapılarda geçiş genellikle daha yavaş ilerler; çünkü eski sistemlerle uyumluluk, güvenlik testleri, politika yönetimi ve operasyonel süreklilik gibi konular çok kritik hale gelir. Buna rağmen böyle bir oranın yakalanmış olması, IPv6'nın kurumsal dünyada da giderek daha fazla benimsenmeye başladığını gösterir.

Bu slaydın verilmek istenen ana mesajı şudur: **IPv6'ya geçiş yavaş olmuştur, fakat durmuş değildir.** Hatta uzun yıllara yayılan istikrarlı bir yükseliş görülmektedir. Bu da önceki slaytlarda anlatılan tünelleme, çift yığınlı çalışma ve geçiş mekanizmalarının neden önemli olduğunu doğrular. Eğer internet bir gecede IPv6'ya geçemiyorsa, bu tür ara çözümler sayesinde geçiş sürdürülebilir hale gelir. Bugün görülen yaygınlaşma oranları da bu stratejilerin pratikte işe yaradığını göstermektedir.

Ayrıca bu slayt, ağ mühendisliği açısından önemli bir ders daha verir: Bir protokolün teknik olarak üstün olması, onun hemen yaygınlaşacağı anlamına gelmez. IPv6'nın adres uzayı çok daha geniştir, başlığı daha sade bir yapıya sahiptir ve geleceğe dönük önemli avantajlar sunar. Buna rağmen dağıtımın çok uzun sürmesi, internetin ne kadar büyük, dağınık ve geriye dönük uyumluluğa bağımlı bir sistem olduğunu gösterir. Yani ağ teknolojilerinde başarı sadece iyi tasarım değil, aynı zamanda **yaygın uygulanabilirlik** meselesidir.

Bu nedenle bu slaytı şöyle özetlemek doğru olur: **IPv6 artık teorik bir ihtiyaç değil, gerçek kullanım alanı bulunan bir standarttır; ancak benimsenmesi uzun yıllara yayılan aşamalı bir süreç olarak ilerlemektedir.** Bu da internet mimarisinde değişimin ne kadar dikkatli, kademeli ve uyumluluk odaklı yürütüldüğünü gösterir.

IPv6: Uygulama

- Google¹: ~ Müşterilerin %40'ı hizmetlere IPv6 üzerinden erişiyor (2023)
- NIST: ABD hükümetine ait tüm alan adlarının 1/3'ü IPv6 uyumludur
- Kurulum ve kullanım süresi oldukça (gerçekten!) uzun
 - 25 yıl ve devam ediyor!
 - Son 25 yılda uygulama düzeyinde yaşanan değişimleri düşünün: WWW, sosyal medya, akıllı medya, oyun, telepresence, ...
 - *Neden?*

¹ <https://www.google.com/intl/en/ipv6/statistics.html>

Bu slayt, IPv6'nın teknik olarak gerekli olmasına rağmen neden bu kadar **yavaş yayıldığını** sorguluyor. Buradaki temel soru gerçekten çok önemlidir: İnternet son 25 yılda uygulama düzeyinde inanılmaz hızla değiştiyse, neden ağ katmanındaki bu kadar temel bir dönüşüm aynı hızda gerçekleşmedi?

İlk neden, IPv6'ya geçişin yalnızca yeni bir protokol eklemekten ibaret olmamasıdır. IPv4'ten IPv6'ya geçiş, internetin adresleme yapısını doğrudan etkiler. Bu da yönlendiricilerden sunuculara, istemci işletim sistemlerinden güvenlik duvarlarına, DNS altyapısından uygulama sunucularına kadar çok geniş bir ekosistemin birlikte uyumlu hale gelmesini gerektirir. Bir web uygulamasını güncellemek çoğu zaman tek bir kurumun kontrolündedir; ama IP adresleme yapısını değiştirmek, birbirinden bağımsız sayısız ağ işletmecisinin, servis sağlayıcının ve cihaz üreticisinin ortak hareket etmesini gerektirir. Bu yüzden ağ katmanındaki dönüşümler, uygulama katmanındaki yeniliklere göre doğal olarak çok daha yavaş ilerler.

İkinci önemli neden, IPv4'ün tüm eksiklerine rağmen uzun süre “yeterince çalışıyor” olmasıdır. Aslında adres kıtlığı ciddi bir problemdi; ancak NAT gibi çözümler bu baskıyı bir süre hafifletti. Yani IPv6 teknik olarak daha doğru ve uzun vadeli çözüm olsa da, IPv4 dünyası tamamen işlemez hale gelmedi. Ağ işletmecileri açısından bakıldığında, mevcut sistem çalışmaya devam ederken büyük bir geçiş maliyetine girmek her zaman ertelenebilir bir karar haline geldi. Bu da IPv6'nın benimsenmesini ciddi biçimde yavaşlattı. Chapter 4'te NAT ve middlebox'ların da bu bağlamda ele alınması tesadüf değildir; çünkü bunlar IPv4'ün ömrünü uzatan mekanizmalardır.

Üçüncü neden, geçiş sürecinin **çift işletim yükü** doğurmasıdır. Uzun bir süre boyunca ağların hem IPv4'ü hem IPv6'yı birlikte desteklemesi gerekir. Bu da çift yığınlı yapı, tünelleme, geçiş mekanizmaları, ek konfigürasyon, ek test ve ek operasyonel karmaşıklık anlamına gelir. Kurumlar açısından mesele sadece "IPv6 açalım" değildir; güvenlik politikalarının, izleme sistemlerinin, erişim listelerinin, yük dengeleyicilerin ve uygulama bağımlılıklarının da yeni yapıya göre yeniden düşünülmesi gerekir. Dolayısıyla teknik geçiş, operasyonel ve yönetsel maliyet de üretir.

Bir başka neden de faydanın her paydaş için aynı anda görünür olmamasıdır. Örneğin bir içerik sağlayıcı IPv6 desteği verdiğinde bunun gerçek faydasını ancak erişim sağlayıcılar ve istemci tarafı da IPv6 kullanıyorsa hisseder. Aynı şekilde bir servis sağlayıcı IPv6 desteği verse bile, kullanıcının eriştiği servisler IPv6 sunmuyorsa avantaj sınırlı kalır. Yani burada klasik bir ağ etkisi problemi vardır: geçişin değeri, ekosistemdeki diğer aktörlerin de geçiş yapmasına bağlıdır. Bu karşılıklı bağımlılık da benimsenme hızını düşürür.

Slayttaki "Son 25 yılda WWW, sosyal medya, akıllı medya, oyun, telepresence..." ifadesi de tam bu farkı vurgular. Uygulama katmanında yenilik yapmak daha çeviktir; çünkü çoğu zaman mevcut IP altyapısının üstünde yeni servisler geliştirirsiniz. Oysa IPv6, bu uygulamaların altında duran temel taşı değiştirmeye çalışır. Temel katmanlar daha yavaş değişir; çünkü üzerlerine kurulu çok büyük bir sistem vardır. Ağ katmanındaki her değişiklik, üst katmanlardaki çok sayıda sistemle uyumlu olmak zorundadır.

Bu nedenle slayttaki "Neden?" sorusunun en güçlü cevabı şudur: **IPv6'nın yayılması yavaştır, çünkü bu bir uygulama güncellemesi değil, internetin temel altyapısını geriye dönük uyumluluğu koruyarak dönüştürme problemidir.** IPv4'ün bir süre daha çalışmasını sağlayan NAT gibi ara çözümler, geçiş baskısını azaltmıştır. Ayrıca maliyet, operasyonel karmaşıklık ve ekosistem bağımlılığı da bu süreci uzatmıştır. Buna rağmen Google ve NIST verileri, yavaş da olsa geçişin gerçekten ilerlediğini göstermektedir.

Bu slaytın ana mesajı şu şekilde özetlenebilir: **IPv6 teknik olarak gerekli ve giderek daha yaygın hale gelen bir standarttır; ancak internetin temelini değiştirmek, uygulama katmanında yenilik yapmaktan çok daha zor olduğu için benimsenme süreci onlarca yıla yayılmıştır.**

Network layer: “data plane” roadmap

■ Network layer: overview

- data plane
- control plane

■ What’s inside a router

- input ports, switching, output ports
- buffer management, scheduling

■ IP: the Internet Protocol

- datagram format
- addressing
- network address translation
- IPv6

■ Generalized Forwarding, SDN

- Match+action
- OpenFlow: match+action in action

■ Middleboxes



Network Layer: 4-78

Bu slayt, Chapter 4’ün veri düzlemi kısmında şimdiye kadar işlenen konuları tamamladıktan sonra, bundan sonraki yönün **genelleştirilmiş iletme (generalized forwarding)** ve **SDN** olacağını gösteren bir geçiş slaytıdır. Yani burada yeni bir kavram ayrıntılı biçimde anlatılmaktan çok, veri düzlemi tartışmasının klasik IP yönlendirme sınırlarının ötesine taşınacağı ilan edilmektedir. Slaytta önceki başlıklar sol tarafta soluklaştırılmış, yeni odak alanı ise sağ tarafta vurgulanmıştır. Bu da ders akışında artık klasik yönlendirici veri düzleminden daha esnek ve programlanabilir ağ davranışlarına geçildiğini gösterir.

Bu geçişin mantığı şudur: Şimdiye kadar veri düzlemini büyük ölçüde **hedefe dayalı iletme** çerçevesinde ele aldık. Yani yönlendirici, paketin başlığındaki hedef IP adresine bakar, en uzun önek eşleşmesini bulur ve paketi uygun çıkış arayüzüne yollar. Bu yaklaşım internetin temelinde yer alır ve klasik IP yönlendirme için son derece önemlidir. Ancak modern ağlarda veri düzlemi yalnızca “hangi çıkış portuna gönderilecek?” sorusuna cevap veren basit bir mekanizma değildir. Ağ cihazları artık çok daha zengin kararlar verebilmekte, paketleri yalnızca iletmekle kalmayıp farklı işlemler de uygulayabilmektedir. Slayttaki yeni başlığın önemi tam burada ortaya çıkar.

“Generalized Forwarding” ifadesi, iletme kararının sadece hedef IP adresine göre değil, paketin farklı başlık alanlarına göre de verilebileceğini anlatır. Yani ağ cihazı kaynak adres, hedef adres, protokol türü, TCP/UDP port numaraları veya başka alanlara bakarak karar verebilir. Böylece veri düzlemi daha esnek hale gelir. Artık cihaz yalnızca “bu paketi nereye göndereyim?” değil, aynı zamanda “bu paket üzerinde ne yapayım?”

sorusunu da cevaplayabilir. Bu bakış açısı, geleneksel yönlendiriciden daha genel bir paket işleme mantığına geçiş anlamına gelir.

Burada öne çıkan temel kavram **match + action** yaklaşımıdır. Bu mantıkta ağ cihazı önce gelen paketin belirli alanlarını bir kuralla eşleştirir, sonra da bu eşleşmeye uygun bir eylem uygular. Eylem yalnızca iletme olmayabilir; paketi düşürme, kopyalama, işaretleme, değiştirme veya denetleyiciye gönderme gibi seçenekler de olabilir. Bu yaklaşım, modern veri düzleminin neden daha programlanabilir olarak görüldüğünü açıklar. Çünkü artık cihaz davranışı sabit ve tek boyutlu değildir; kurallarla tanımlanabilir hale gelmiştir.

Slaytta SDN'nin de bu başlık altında verilmesi tesadüf değildir. **Software-Defined Networking**, kontrol mantığının daha merkezi ve yazılım tabanlı biçimde ele alınmasını sağlar. Bu durumda veri düzlemindeki cihazlar, çoğu zaman kendilerine yüklenen kurallara göre çalışır. Yani hangi paket alanlarının eşleştirileceği ve hangi eylemlerin uygulanacağı, daha üst düzey bir denetim mantığı tarafından belirlenebilir. Bu da klasik dağıtık yönlendirme anlayışından daha esnek bir yönetim modeli sunar.

“OpenFlow: match+action in action” ifadesi ise bu fikrin somut bir örneği olarak OpenFlow'un ele alınacağını gösterir. Başka bir deyişle, biraz önce soyut olarak anlatılan match+action mantığı, OpenFlow üzerinden gerçek kurallar ve gerçek paket işlemleriyle gösterilecektir. Bu, konunun yalnızca teorik düzeyde kalmayacağını; uygulamada paketlerin nasıl sınıflandırıldığı ve hangi işlemlerin uygulandığının da inceleneceğini gösterir.

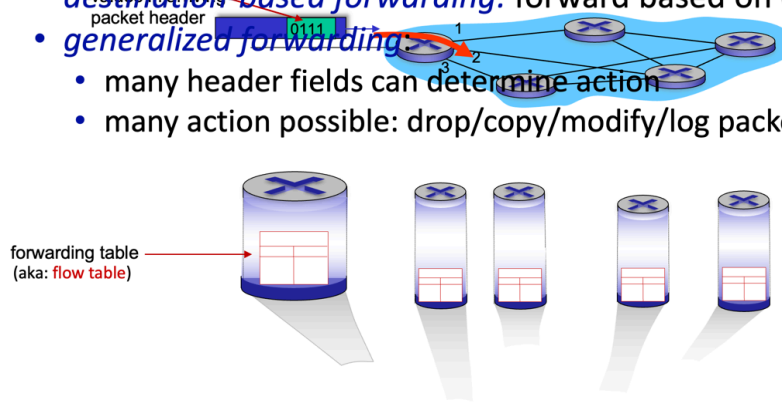
Slaydın alt kısmında soluk biçimde görülen “Middleboxes” başlığı da önemli bir işarettir. Bu, genelleştirilmiş iletme ve SDN tartışmasından sonra ağ içindeki güvenlik duvarı, NAT, yük dengeleyici gibi ara cihazların da gündeme geleceğini düşündürür. Çünkü bu cihazlar da klasik yönlendiriciden farklı olarak yalnızca iletme yapmaz; paketleri inceler, değiştirir, engeller veya yeniden yönlendirir. Dolayısıyla generalized forwarding yaklaşımı, middlebox kavramına geçiş için de güçlü bir hazırlık oluşturur.

Bu slaydın ana mesajı şu şekilde özetlenebilir: **Veri düzlemi konusu artık klasik IP iletme mantığından modern, esnek ve programlanabilir ağ davranışlarına doğru genişlemektedir.** Bundan sonraki odak, paketin yalnızca hedefe göre yönlendirilmesi değil; farklı başlık alanlarına göre eşleştirilmesi ve uygun eylemlerin uygulanması olacaktır. Bu da SDN ve OpenFlow gibi yapıları anlamak için gerekli kavramsal zemini oluşturur.

Generalized forwarding: match plus action

Review: each router contains a **forwarding table** (aka: **flow table**)

- “**match plus action**” abstraction: match bits in arriving packet, take action
 - **destination-based forwarding:** forward based on dest. IP address
 - **generalized forwarding:**
 - many header fields can determine action
 - many action possible: drop/copy/modify/log packet



Network Layer: 4-79

Bu slayt, veri düzlemindeki modern yaklaşımın özünü veren en önemli kavramlardan birini tanıtıyor: **match plus action**, yani **eşleştir ve eylem uygula** yaklaşımı. Buradaki temel fikir şudur: Her yönlendirici ya da ağ cihazı, gelen paketin belirli bitlerine veya başlık alanlarına bakar, bunları bir tabloyla eşleştirir ve ardından önceden tanımlı bir işlemi uygular. Slaytta da her yönlendiricinin bir **forwarding table**, yani daha genel adıyla bir **flow table** içerdiği açıkça belirtilmektedir.

Klasik internet yönlendirmesinde bu mantığın daha dar bir örneğini zaten görmüştük. Paket gelir, yönlendirici paketin **hedef IP adresine** bakar, yönlendirme tablosunda en uygun eşleşmeyi bulur ve paketi ilgili çıkış arayüzüne yollar. Bu, slaytta “destination-based forwarding” olarak adlandırılmaktadır. Yani karar tek bir temel alana, hedef IP adresine göre verilir. Bu yapı internetin çalışması için son derece güçlü ve ölçeklenebilir bir model sunmuştur; ancak modern ağ ihtiyaçları açısından her zaman yeterli değildir.

Genelleştirilmiş iletme yaklaşımı tam bu noktada devreye girer. Burada artık karar yalnızca hedef IP adresine bağlı değildir. Slaytta da vurgulandığı gibi, **birçok başlık alanı eylemi belirleyebilir**. Yani kaynak adres, hedef adres, taşıma katmanı port numaraları, protokol türü, hatta bazı durumlarda daha başka alanlar da karar sürecine dahil edilebilir. Böylece veri düzlemi, yalnızca “paketi nereye yollayayım?” sorusuna cevap veren basit bir mekanizma olmaktan çıkar; “bu pakete nasıl davranayım?” sorusunu da cevaplayan daha zengin bir yapıya dönüşür.

Bu yaklaşımın ikinci önemli boyutu, uygulanabilecek eylemlerin çeşitlenmesidir. Slaytta açıkça belirtildiği gibi, olası eylemler yalnızca iletme değildir; **drop, copy, modify, log** gibi işlemler de mümkündür. Yani eşleşme sonucunda paket düşürülebilir, başka bir yere kopyalanabilir, bazı alanları değiştirilebilir ya da kayıt altına alınabilir. Bu, ağ cihazlarının artık yalnızca pasif taşıyıcılar değil; aynı zamanda politika uygulayan, güvenlik denetimi yapan, trafik yöneten ve gerektiğinde paketin içeriğine göre farklı davranan aktif bileşenler haline geldiğini gösterir.

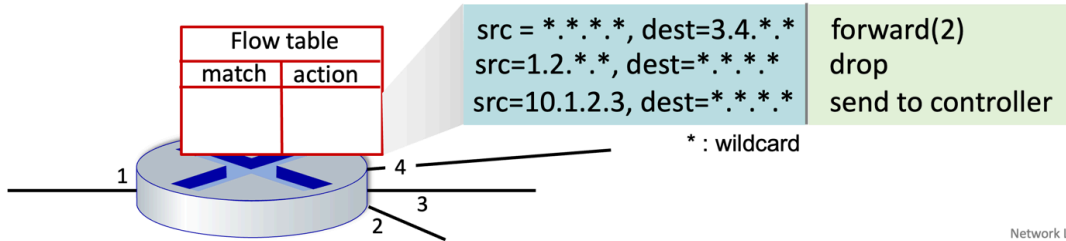
Bu nedenle flow table kavramı, klasik yönlendirme tablosundan daha genel düşünülmelidir. Klasik tabloda “şu hedef önek için şu arayüz” gibi kurallar bulunurken, flow table içinde “şu koşullar sağlanıyorsa şu işlemi yap” biçiminde kurallar yer alabilir. Başka bir ifadeyle, artık tablo sadece yol seçen bir yapı değil; paket işleme mantığını tanımlayan bir karar mekanizmasıdır. Bu da SDN yaklaşımı için temel oluşturur, çünkü merkezi ya da yazılım tabanlı bir denetleyici bu akış tablolarına kurallar yükleyerek ağın davranışını programlayabilir.

Buradaki en önemli kavramsal ayrım şudur: **destination-based forwarding**, match plus action yaklaşımının özel ve daha sınırlı bir örneğidir. Yani klasik IP yönlendirme tamamen ortadan kalkmış değildir; aksine daha genel bir soyutlamanın içinde yerini almıştır. Önceden yalnızca hedef adrese göre karar verilirken, şimdi birçok alan eşleştirilebilir ve birçok farklı eylem uygulanabilir. Bu da modern ağların neden daha esnek ve programlanabilir hale geldiğini açıklar.

Bu slaytın ana mesajı şu şekilde özetlenebilir: **Genelleştirilmiş iletme, ağ cihazlarının gelen paketi farklı başlık alanlarına göre sınıflandırıp sadece iletme değil, farklı işlemler de uygulayabildiği daha genel bir veri düzlemi modelidir.** Klasik hedefe dayalı yönlendirme bu modelin yalnızca bir alt durumudur. Bu bakış açısı, bir sonraki adımda OpenFlow’un ve daha sonra middlebox mantığının anlaşılması için temel kavramsal zemini hazırlar.

Flow table abstraction

- **flow**: defined by header fields
- **generalized forwarding: simple** packet-handling rules
 - **match**: pattern values in packet header fields
 - **actions**: for matched packet: drop, forward, modify, matched packet or send matched packet to controller
 - **priority**: disambiguate overlapping patterns
 - **counters**: #bytes and #packets



Network Layer: 4-81

Bu slayt, bir önceki slaytta tanıtilen **match + action** yaklaşımını daha somut hale getirerek **flow table abstraction** kavramını açıklar. Buradaki temel fikir, ağ cihazının gelen paketleri yalnızca hedefe göre yönlendiren bir kutu değil; belirli başlık alanlarına bakan, bu alanları kurallarla eşleştiren ve eşleşmeye göre işlem yapan bir sistem olduğudur. Yani burada veri düzlemi davranışı, bir **akış tablosu** üzerinden tanımlanmaktadır.

Slaytın ilk önemli kavramı **flow**, yani **akış** kavramıdır. Akış, başlık alanlarıyla tanımlanır. Başka bir ifadeyle, belirli ortak özelliklere sahip paketler aynı akışın parçası olarak değerlendirilebilir. Bu ortak özellikler kaynak adres, hedef adres, port numarası, protokol tipi gibi başlık alanları olabilir. Böylece ağ cihazı, tek tek her paketi tamamen bağımsız düşünmek yerine, belirli bir kurala uyan paket gruplarına aynı davranışı uygulayabilir. Bu soyutlama, veri düzlemini hem daha düzenli hem de daha programlanabilir hale getirir.

Slaytta “generalized forwarding: simple packet-handling rules” ifadesiyle anlatılan şey, bu akış tablolarının aslında basit paket işleme kurallarından oluştuğudur. Her kuralın birkaç temel bileşeni vardır. İlk bileşen **match**, yani eşleşme bölümüdür. Burada paketin başlık alanlarında hangi örüntünün aranacağı tanımlanır. Örneğin belirli bir hedef adres öneki, belirli bir kaynak adres aralığı ya da daha özel bir adres kombinasyonu eşleşme ölçütü olabilir. Eşleşme alanında kullanılan * işareti, yani **wildcard**, ilgili alanın “herhangi bir değer olabilir” anlamına geldiğini gösterir. Bu sayede kurallar çok katı değil, gerektiğinde daha genel yazılabilir.

İkinci bileşen **action**, yani eylemdir. Eşleşen pakete ne yapılacağı burada belirtilir. Slaytta örnek olarak paketin belirli bir çıkış portuna iletilmesi, düşürülmesi veya denetleyiciye gönderilmesi verilmektedir. Bu çok önemlidir; çünkü artık tablo sadece “hangi arayüze gönder?” sorusuna cevap vermez. Paket tamamen engellenebilir, kontrol düzlemine yükseltilebilir ya da başka şekillerde işlenebilir. Bu da veri düzlemi ile kontrol düzlemi arasındaki ilişkiyi daha esnek hale getirir. Özellikle “send to controller” seçeneği, SDN mantığının temel taşlarından biridir; çünkü bazı paketler yerel cihaz tarafından değil, merkezi ya da üst düzey bir denetim mantığı tarafından ele alınmak istenir.

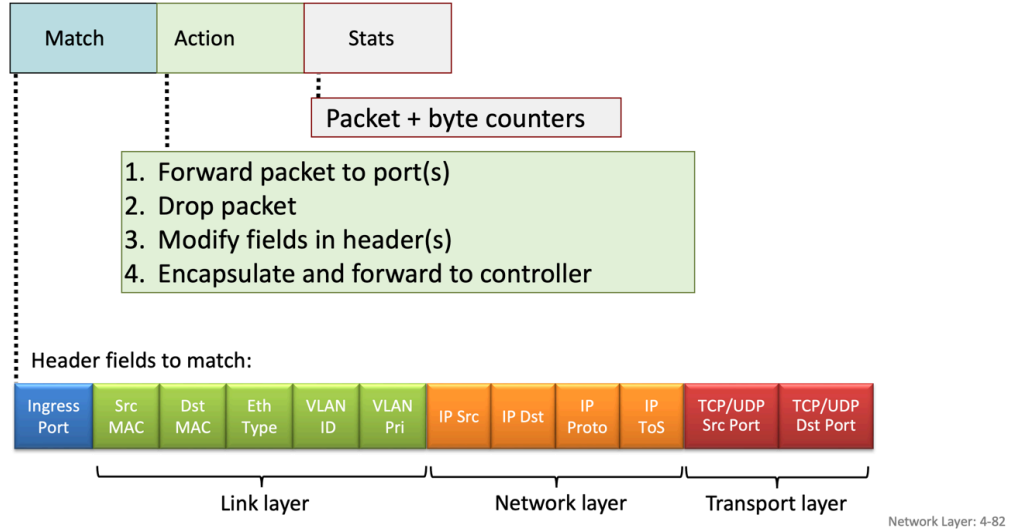
Üçüncü bileşen **priority**, yani önceliklidir. Bunun gerekli olmasının nedeni, bazı kuralların birbiriyle çakışabilmesidir. Örneğin bir paket hem genel bir kurala hem de daha özel bir kurala uyabilir. Böyle bir durumda hangi kuralın uygulanacağını öncelik alanı belirler. Bu, klasik yönlendirmedeki en uzun önek eşleşmesine benzer bir ihtiyacı daha genel biçimde çözer. Yani sistem, yalnızca “eşleşti mi, eşleşmedi mi?” diye bakmaz; birden çok eşleşme varsa hangisinin daha baskın olduğuna da karar verir.

Dördüncü bileşen ise **counters**, yani sayaçlardır. Sayaçlar kaç paket ve kaç bayt işlendiğini tutar. İlk bakışta bu ayrıntı küçük görünebilir; ancak ağ yönetimi açısından çok değerlidir. Çünkü bir kuralın ne kadar kullanıldığını, hangi tür trafiğin yoğun olduğunu, hangi akışların baskın hale geldiğini ya da belirli bir güvenlik kuralının ne kadar devreye girdiğini anlamak için bu sayaçlar gerekir. Yani akış tabloları yalnızca karar vermez; aynı zamanda gözlem ve izleme için de veri üretir.

Slayttaki örnek kurallar bu mantığı oldukça iyi somutlaştırır. Örneğin hedef adresi 3.4.*.* ile eşleşen paketlerin forward(2) ile 2 numaralı porta yönlendirilmesi, klasik iletme davranışının bir örneğidir. Kaynağı 1.2.*.* olan paketlerin drop edilmesi, veri düzleminde politika uygulamanın ve erişim denetiminin bir örneğidir. Kaynağı tam olarak 10.1.2.3 olan paketlerin send to controller yapılması ise belirli özel trafiğin merkezi denetime devredilmesini gösterir. Bu örnekler, veri düzleminin artık yalnızca yönlendirme değil, aynı zamanda filtreleme, istisna yönetimi ve kontrol düzlemine iş devretme işlevlerini de üstlenebildiğini açıkça göstermektedir.

Bu slayttan çıkarılması gereken ana fikir şudur: **Akış tablosu soyutlaması, ağ cihazı davranışını kurallar halinde tanımlayan genel bir veri düzlemi modelidir.** Her kural; hangi paketlerin eşleşeceğini, bu paketlere ne yapılacağını, çakışan durumlarda hangi kuralın baskın olacağını ve bu kuralın ne kadar kullanıldığını belirler. Böylece veri düzlemi, klasik hedefe dayalı yönlendirmeden çok daha esnek ve çok daha programlanabilir bir yapıya dönüşür. Bu da OpenFlow gibi mekanizmaların ve modern SDN mimarisinin neden bu kadar güçlü olduğunu anlamak için temel oluşturur.

OpenFlow: Akış Tablosu Girdileri



Bu slayt, bir önceki slayttaki genel **flow table abstraction** fikrini şimdi **OpenFlow** üzerinden daha somut biçimde gösteriyor. Yani burada artık sadece “eşleştir ve eylem uygula” mantığından söz edilmiyor; bir OpenFlow akış tablosu girdisinin hangi parçalardan oluştuğu açıkça gösteriliyor. Slaytta her akış girdisinin üç temel bölümden oluştuğu görülmektedir: **Match**, **Action** ve **Stats**. Ayrıca eşleşmenin hangi başlık alanları üzerinden yapılabildiği de bağlantı, ağ ve taşıma katmanları boyunca örneklenmektedir.

İlk bölüm olan **Match**, gelen paketin hangi özelliklerine bakılacağını tanımlar. Buradaki önemli nokta, OpenFlow’un yalnızca hedef IP adresine göre çalışan klasik yönlendirme mantığıyla sınırlı olmamasıdır. Slaytta eşleşme için kullanılabilecek alanlar arasında giriş portu, kaynak MAC adresi, hedef MAC adresi, Ethernet tipi, VLAN kimliği ve VLAN önceliği gibi **bağlantı katmanı** alanları; IP kaynak ve hedef adresi, IP protokol türü ve ToS gibi **ağ katmanı** alanları; TCP/UDP kaynak ve hedef portları gibi **taşıma katmanı** alanları yer almaktadır. Bu, OpenFlow’un çok katmanlı bir başlık görünümüne bakarak karar verebildiğini gösterir. Başka bir ifadeyle, veri düzlemi artık yalnızca “hedef adres nereye işaret ediyor?” sorusunu sormaz; paketin hangi porttan geldiği, hangi VLAN’a ait olduğu, hangi uygulama trafiğini taşıdığı gibi bilgileri de karar sürecine dahil edebilir.

İkinci bölüm olan **Action**, eşleşme sağlandıktan sonra pakete ne yapılacağını belirtir. Slaytta dört temel eylem özellikle vurgulanmaktadır. Birincisi, paketin bir veya birden fazla çıkış portuna iletilmesidir. Bu, klasik yönlendirme ve anahtarlama davranışına karşılık gelir. İkincisi, paketin tamamen düşürülmesidir; bu da veri düzleminde filtreleme

ve erişim kontrolü yapılabildiğini gösterir. Üçüncüsü, paketin başlık alanlarının değiştirilmesidir. Bu çok önemlidir; çünkü ağ cihazı yalnızca ileten değil, gerektiğinde paketi dönüştüren bir unsur haline gelir. Dördüncüsü ise paketin kapsüllenerek denetleyiciye gönderilmesidir. Bu, OpenFlow'un SDN ile bağlandığı en kritik noktadır; çünkü bazı kararlar yerel cihaz içinde değil, daha üst düzey bir denetleyici mantığı tarafından verilmek istenir.

Üçüncü bölüm olan **Stats**, yani istatistikler, paket ve bayt sayaçlarını içerir. Bu alan ilk bakışta sadece izleme amacı taşıyormuş gibi görünebilir; ancak aslında veri düzlemi yönetimi açısından çok değerlidir. Çünkü hangi akışın ne kadar trafik ürettiği, hangi kuralın yoğun kullanıldığı, hangi politikaların devreye girdiği ve ağdaki davranışın nasıl değiştiği bu sayaçlar sayesinde anlaşılır. Bu da OpenFlow tablolarının yalnızca karar veren değil, aynı zamanda ölçülebilir ve gözlemlenebilir bir yapı sunduğunu gösterir.

Bu slaytta özellikle dikkat edilmesi gereken şey, eşleşme alanlarının **katmanlar arası** yayılmasıdır. Klasik bir yönlendirici çoğunlukla ağ katmanındaki hedef adrese bakarken, OpenFlow cihazı bağlantı katmanındaki MAC alanlarından taşıma katmanındaki TCP/UDP portlarına kadar birçok bilgiyi aynı anda kullanabilir. Bu nedenle OpenFlow, yönlendirici, anahtar, güvenlik duvarı ve bazı middlebox işlevlerini ortak bir soyutlama altında birleştirebilir. Yani cihazın türü ne olursa olsun, davranış “hangi alana bak, ne yap” kuralı ile tanımlanır. Bu da önceki slayttaki “match+action abstraction unifies different kinds of devices” düşüncesinin doğal devamıdır.

Buradan şu önemli sonuç çıkar: OpenFlow'daki akış tablosu girdileri, klasik yönlendirme tablosundan çok daha güçlüdür. Klasik tabloda çoğunlukla yalnızca hedef önek ve çıkış arayüzü bulunurken, burada birden çok katmandan alanlar, çeşitli eylemler ve sayaç bilgileri birlikte yer alır. Bu da veri düzlemini sabit davranan bir donanım mantığından çıkarıp, programlanabilir bir kurallar sistemi haline getirir. Özellikle “modify fields in header(s)” ve “encapsulate and forward to controller” seçenekleri, veri düzleminin yalnızca ileten değil, gerektiğinde karar sürecini yukarı taşıyan ve paketi dönüştüren aktif bir bileşen olduğunu gösterir.

Bu slaytın ana mesajı şu şekilde özetlenebilir: **OpenFlow akış tablosu girdileri, paketi çoklu katman başlık alanlarına göre eşleştirip iletme, düşürme, değiştirme ya da denetleyiciye gönderme gibi eylemler uygulayan programlanabilir veri düzlemi kurallarıdır.** İstatistik sayaçları da bu kuralların ağ içindeki etkisini görünür hale getirir. Böylece OpenFlow, modern SDN veri düzleminin nasıl çalıştığını somut biçimde gösteren temel örnek haline gelir.

OpenFlow: examples

Destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	51.6.0.8	*	*	*	*	port6

IP datagrams destined to IP address 51.6.0.8 should be forwarded to router output port 6

Firewall:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	*	*	*	*	22	drop

Block (do not forward) all datagrams destined to TCP port 22 (ssh port #)

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	128.119.1.1	*	*	*	*	*	drop

Block (do not forward) all datagrams sent by host 128.119.1.1

Network Layer: 4-83

Bu slayt, OpenFlow'un aynı **flow table** yapısıyla çok farklı ağ işlevlerini nasıl gerçekleştirebildiğini gösteriyor. Yani burada verilmek istenen ana fikir, aynı “eşleştir ve eylem uygula” mantığının hem klasik yönlendirme için hem de güvenlik politikaları için kullanılabilmesidir. Böylece OpenFlow, farklı ağ cihazı türlerini ortak bir soyutlama altında birleştirir.

İlk örnek **destination-based forwarding**, yani hedefe dayalı iletmedir. Burada tabloda yalnızca **IP hedef adresi** alanı anlamlıdır; diğer alanlar * ile gösterildiği için “fark etmez” anlamına gelir. Kural şunu söyler: Hedef IP adresi **51.6.0.8** olan tüm IP datagramları **port 6** üzerinden iletilsin. Bu aslında klasik yönlendiricinin yaptığı işe çok benzer. Yani OpenFlow yalnızca yeni ve karmaşık politikalar için değil, bildiğimiz geleneksel yönlendirme davranışını tanımlamak için de kullanılabilir. Buradaki önemli nokta, klasik yönlendirme mantığının artık daha genel bir akış tablosu modelinin özel bir durumu olarak görülmesidir.

İkinci örnek bir **firewall** davranışıdır. Bu kez tabloda anlamlı olan alan **TCP hedef portudur**. Değer olarak **22** verilmiştir; bu da SSH servisi için kullanılan porttur. Eylem kısmında ise **drop** yazmaktadır. Yani hedefi TCP port 22 olan tüm paketler ileilmeyecek, doğrudan düşürülecektir. Bu örnek, veri düzleminin yalnızca “nereye gönderelim?” sorusuna cevap vermediğini; aynı zamanda “buna izin verelim mi, engelleyelim mi?” sorusuna da cevap verebildiğini gösterir. Böylece ağ cihazı, klasik yönlendirici olmanın ötesinde bir güvenlik politikası da uygulayabilmektedir.

Üçüncü örnekte ise yine bir güvenlik ya da erişim kontrolü politikası vardır; ancak bu kez karar **kaynak IP adresine** göre verilmektedir. Kaynağı **128.119.1.1** olan tüm datagramlar için eylem yine **drop** olarak belirlenmiştir. Yani bu hosttan gelen bütün trafik engellenmektedir. Bu örnek çok önemlidir; çünkü ağın yalnızca hedefe göre değil, gönderen kim olduğuna göre de davranış değiştirebildiğini gösterir. Başka bir ifadeyle, veri düzlemindeki kurallar hem trafik yönlendirme hem de trafik kısıtlama amacıyla kullanılabilir.

Bu üç örnek birlikte düşünüldüğünde çok güçlü bir sonuç ortaya çıkar: **Aynı akış tablosu yapısı ile hem yönlendirici davranışı hem güvenlik duvarı davranışı tanımlanabilmektedir.** Birinci örnek “ilet”, ikinci ve üçüncü örnekler ise “engelle” mantığını göstermektedir. Demek ki cihazın fiziksel adı yönlendirici, anahtar ya da güvenlik duvarı olsa bile, veri düzlemi açısından hepsi aslında belirli başlık alanlarına göre karar verip uygun eylemi uygulayan sistemlerdir.

Burada * ile gösterilen joker karakterler de önemli bir ayrıntıdır. Çünkü her alanı tek tek belirtmek zorunda değiliz. Sadece ilgilendiğimiz alanı seçip diğerlerini “önemsiz” bırakabiliyoruz. Bu da kuralları hem daha esnek hem de daha güçlü hale getirir. Örneğin yalnızca hedef IP’ye göre iletme yapmak istiyorsak diğer tüm alanları yok sayabiliriz; yalnızca TCP portuna göre engelleme yapmak istiyorsak diğer alanları dikkate almayabiliriz. Bu yaklaşım, akış tablosu mantığının neden bu kadar genel ve programlanabilir olduğunu açıkça gösterir.

Bu slaytın ana mesajı şudur: **OpenFlow, aynı match+action yapısıyla hem klasik hedefe dayalı yönlendirmeyi hem de güvenlik duvarı benzeri filtreleme politikalarını gerçekleştirebilir.** Böylece ağ işlevleri ayrı ayrı ve tamamen farklı mekanizmalar olarak değil, ortak bir akış tablosu mantığı içinde düşünülebilir. Bu da modern ağların neden daha esnek, merkezi olarak yönetilebilir ve yazılımla tanımlanabilir hale geldiğini anlamak için çok önemli bir adımdır.

OpenFlow: Örnekler

Layer 2 destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	22:A7:23:11:E1:02	*	*	*	*	*	*	*	*	*	port3

Hedef MAC adresi 22:A7:23:11:E1:02 olan Katman 2 çerçeveleri, çıkış bağlantı noktası 3'e iletilmelidir.

Network Layer: 4-84

Bu slayt, OpenFlow'un yalnızca ağ katmanında değil, **bağlantı katmanında da** iletme kararı verebildiğini gösteriyor. Yani bir önceki örneklerde gördüğümüz hedef IP adresine göre yönlendirme mantığına ek olarak, burada kararın **hedef MAC adresine** göre verildiği bir durum anlatılmaktadır. Bu da OpenFlow'un yalnızca yönlendirici mantığını değil, aynı zamanda **anahtar (switch)** mantığını da aynı akış tablosu soyutlaması içinde ifade edebildiğini gösterir.

Buradaki örnekte anlamlı olan alan **MAC dst**, yani **hedef MAC adresi** alanıdır. Tabloda bu alan için 22:A7:23:11:E1:02 değeri verilmiştir. Diğer alanların hepsi * ile gösterildiği için bunların karar açısından önemli olmadığı anlaşılır. Yani paket hangi giriş portundan gelirse gelsin, kaynak MAC adresi ne olursa olsun, IP katmanında hangi adresleri taşırsa taşırsın ya da TCP/UDP portları ne olursa olsun; eğer **hedef MAC adresi bu değere eşitse**, paket için uygulanacak eylem **port3** üzerinden iletmek olacaktır.

Bu davranış, klasik bir Katman 2 anahtarın çalışma mantığına çok benzer. Ethernet anahtarları, gelen çerçevenin hedef MAC adresine bakar ve bu adresin hangi çıkış portuna karşılık geldiğini tablo üzerinden bulup çerçeveyi o porta iletir. Bu slaytta anlatılan da tam olarak budur; ancak önemli fark şudur: Bu davranış ayrı ve özel bir donanım mantığı olarak değil, yine aynı **match + action** tablosu ile ifade edilmektedir. Böylece aynı soyutlama ile hem Katman 2 anahtarlama hem Katman 3 yönlendirme yapılabilir.

Bu slaytın asıl önemi burada ortaya çıkar. OpenFlow ve genelleştirilmiş iletme yaklaşımı sayesinde ağ cihazlarını "bu bir switch", "bu bir router" diye tamamen ayrı mantıklarla

düşünmek yerine, “hangi alanlara bakıyor ve hangi eylemi uyguluyor?” sorusuyla düşünebiliriz. Eğer cihaz hedef MAC adresine göre karar veriyorsa Katman 2 anahtarlama davranışı sergiler. Eğer hedef IP adresine göre karar veriyorsa Katman 3 yönlendirme davranışı sergiler. Eğer TCP portuna göre engelleme yapıyorsa güvenlik duvarı gibi davranır. Yani cihaz işlevleri ortak bir kurallar sistemi altında birleşmiş olur.

Burada Switch Port alanının da tabloda bulunması önemlidir. Bu alan, paketin hangi porttan geldiğine göre farklı davranış tanımlanabilmesine imkan verir. Bu örnekte giriş portu önemli olmadığı için * bırakılmıştır. Ancak istenirse “şu MAC adresine sahip ve şu giriş portundan gelen çerçeveler için farklı işlem yap” gibi daha ayrıntılı kurallar da yazılabilir. Bu da veri düzleminin ne kadar esnek hale geldiğini gösterir.

Bu slayttan çıkarılması gereken ana fikir şudur: **OpenFlow yalnızca IP yönlendirme için değil, Ethernet anahtarlama için de kullanılabilir.** Hedef MAC adresine göre eşleşme yapıp belirli bir çıkış portuna iletme kararı vererek klasik Katman 2 anahtar davranışını akış tablosu üzerinden tanımlayabilir. Böylece veri düzlemindeki farklı cihaz işlevleri, tek bir genel “eşleştir ve eylem uygula” mantığı içinde ifade edilebilir.

OpenFlow abstraction

- **match+action**: abstraction unifies different kinds of devices

Router

- **match**: longest destination IP prefix
- **action**: forward out a link

Firewall

- **match**: IP addresses and TCP/UDP port numbers
- **action**: permit or deny

Switch

- **match**: destination MAC address
- **action**: forward or flood

NAT

- **match**: IP address and port
- **action**: rewrite address and port

Network Layer: 4-85

Bu slayt, OpenFlow ve genel olarak **match + action** yaklaşımının neden bu kadar güçlü olduğunu tek cümlede özetliyor: **Bu soyutlama, farklı türde ağ cihazlarını ortak bir mantık altında birleştirir.** Yani yönlendirici, anahtar, güvenlik duvarı ya da NAT gibi cihazlar ilk bakışta çok farklı görevler yapıyor gibi görünse de, aslında hepsi “hangi alana bakacağım?” ve “eşleşirse ne yapacağım?” sorularıyla tanımlanabilir.

Slaytta ilk olarak **router**, yani yönlendirici ele alınmaktadır. Yönlendiricide eşleşme ölçütü, en uzun hedef IP öneğidir. Yani gelen paketin hedef IP adresi alınır, tabloyla karşılaştırılır ve en uygun önek bulunur. Eylem ise paketi uygun bağlantıdan dışarı göndermektir. Bu, klasik ağ katmanı yönlendirme davranışdır. Burada önemli olan, yönlendiricinin artık “özel ve tamamen farklı bir cihaz” olarak değil, belirli bir eşleşme ve belirli bir eylem uygulayan bir sistem olarak düşünülmesidir.

İkinci olarak **switch**, yani anahtar verilmiştir. Burada eşleşme ölçütü hedef MAC adresidir. Eylem ise çerçeveyi ilgili porta iletmek ya da gerekirse flood etmektir. Bu da klasik Katman 2 anahtarlama davranışdır. Böylece bir switch'in yaptığı iş de aynı soyutlama içine oturur: hedef MAC adresine bak, sonra uygun işlemi uygula.

Üçüncü örnek **firewall**, yani güvenlik duvarıdır. Güvenlik duvarı paketin IP adreslerine ve TCP/UDP port numaralarına bakar. Eylem ise izin vermek ya da engellemektir. Burada artık amaç iletmekten çok politika uygulamaktır. Buna rağmen mantık yine aynıdır: başlık alanlarını eşleştir, sonra bir karar uygula. Bu da güvenlik duvarının bile aynı veri düzlemi soyutlamasıyla ifade edilebildiğini gösterir.

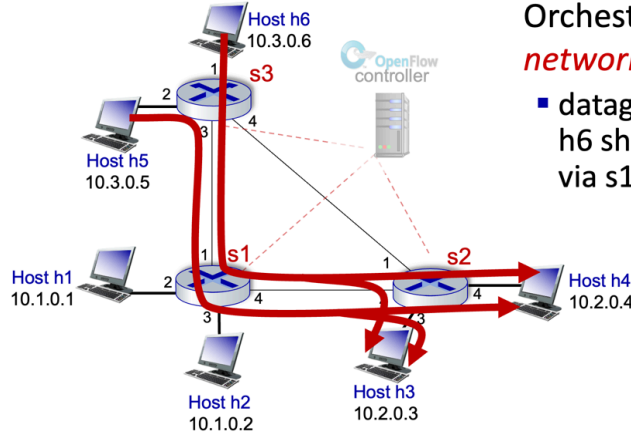
Dördüncü örnek ise **NAT**'tır. NAT'ta eşleşme ölçütü IP adresi ve porttur; eylem ise adresi ve portu yeniden yazmaktır. Bu örnek özellikle önemlidir çünkü burada cihaz sadece “ilet” ya da “engelle” yapmaz; paketin başlığını değiştirir. Yani match + action soyutlaması yalnızca yönlendirme ve filtreleme değil, aynı zamanda **başlık dönüştürme** işlevlerini de kapsar.

Bu dört örnek birlikte düşünüldüğünde çok önemli bir kavramsal sonuç ortaya çıkar. Ağ cihazlarını ayrı ayrı, tamamen farklı mantıklara sahip kutular olarak düşünmek yerine, hepsini ortak bir modelle açıklayabiliriz. Fark yaratan şey cihazın adı değil; **hangi alanlara baktığı ve hangi eylemi uyguladığıdır**. Yani bir cihaz hedef IP öneğine bakıp iletirse router gibi davranır, hedef MAC adresine bakıp iletirse switch gibi davranır, portlara bakıp engellerse firewall gibi davranır, adres ve port değiştirirse NAT gibi davranır.

Bu bakış açısı SDN açısından çok değerlidir. Çünkü kontrol mantığı, farklı cihaz türleri için tamamen ayrı ayrı değil, ortak bir kurallar sistemi üzerinden tanımlanabilir. Böylece ağ daha programlanabilir, daha esnek ve daha merkezi biçimde yönetilebilir hale gelir.

Bu slaytın ana mesajı şu şekilde özetlenebilir: **match + action yaklaşımı, farklı ağ cihazlarının görünürde farklı olan işlevlerini tek bir ortak veri düzlemi soyutlaması altında birleştirir**. Router, switch, firewall ve NAT gibi yapılar aslında aynı temel mantığın farklı uygulamalarıdır.

OpenFlow Örneği



Orchestrated tables can create **network-wide** behavior, e.g.,:

- datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2

Network Layer: 4-86

Bu slayt, OpenFlow'un en önemli gücünü gösteriyor: **tek tek cihazlarda tanımlanan akış tabloları, birlikte çalıştırıldığında tüm ağın davranışını belirleyebilir.** Yani burada artık sadece bir anahtarın ya da bir yönlendiricinin ne yaptığını bakmıyoruz; birden fazla cihaz üzerindeki kuralların koordineli biçimde uygulanmasıyla **network-wide behavior**, yani **ağ genelinde davranış** oluşturulduğunu görüyoruz.

Şekilde üç OpenFlow anahtarı vardır: **s1, s2 ve s3**. Ayrıca bunlara bağlı altı host bulunmaktadır. Kırmızı kalın çizgiler, belirli trafiğin ağ içinde hangi yoldan geçirilmek istendiğini göstermektedir. Sağ taraftaki açıklama da bunu açıkça ifade eder: **h5 ve h6 hostlarından gelen datagramlar, h3 veya h4'e giderken önce s1 üzerinden, sonra da s2 üzerinden ilerlemelidir.** Bu, çok önemli bir noktadır; çünkü burada ağ trafiği yalnızca "en kısa yol neyse oradan gitsin" mantığıyla bırakılmamaktadır. Bunun yerine, kontrolör tarafından yerleştirilen kurallarla trafik için özel bir yol dayatılmaktadır.

Bu örnek, SDN'nin klasik dağıtık yönlendirme yaklaşımından nasıl ayrıldığını anlamak için çok değerlidir. Geleneksel ağlarda her cihaz çoğunlukla kendi yerel tablosuna bakarak karar verir ve ağ genelindeki davranış, bu yerel kararların birleşiminden doğar. OpenFlow ve SDN yaklaşımında ise merkezi denetleyici, farklı cihazların akış tablolarını **orkestre ederek** önceden istenen bir uçtan uca yol oluşturabilir. Yani s3, s1 ve s2 üzerindeki kurallar ayrı ayrı bakıldığında yerel görünür; fakat birlikte düşünüldüğünde tek bir küresel politika uygulanır.

Şekildeki trafik örneğini adım adım düşünersek mantık daha net hale gelir. h5 veya h6'dan çıkan bir paket önce **s3** anahtarına gelir. s3 üzerindeki akış kuralı, bu trafiği

doğrudan başka bir yöne değil, özellikle **s1**'e gönderecek şekilde ayarlanmıştır. Daha sonra s1 üzerindeki kural, paketi **s2**'ye iletir. Son olarak s2, hedefin h3 mü yoksa h4 mü olduğuna göre paketi uygun hosta ulaştırır. Böylece her cihaz yalnızca kendi küçük görevini yapar; ama bu küçük görevler toplandığında, tüm ağ için belirlenmiş özel bir yol elde edilir.

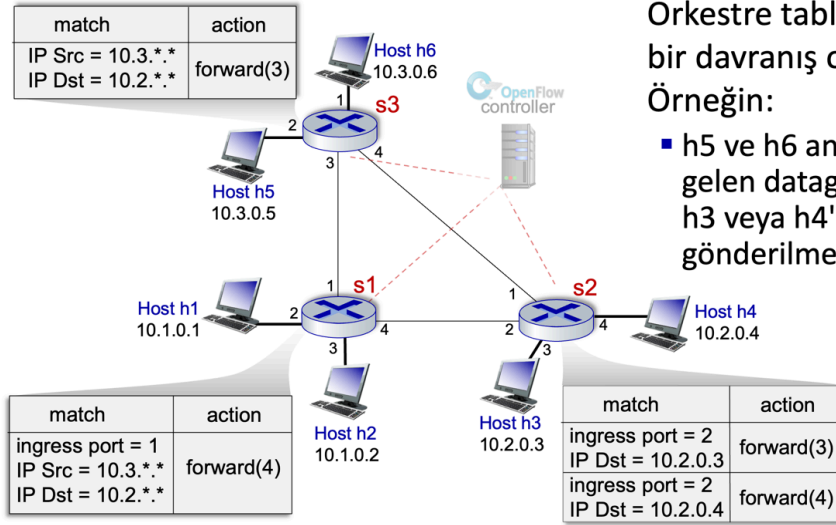
Burada verilmek istenen ana mesaj şudur: **OpenFlow tabloları yalnızca yerel iletme kararları üretmez; koordineli kullanıldığında trafik mühendisliği yapılmasını sağlar.** Başka bir ifadeyle, ağ yöneticisi belirli türde trafiğin hangi cihazlardan geçeceğini önceden tasarlayabilir. Bu sayede yük dengeleme, güvenlik denetimi, belirli bir middlebox üzerinden geçirme, gecikme optimizasyonu veya politikaya dayalı yönlendirme gibi hedefler gerçekleştirilebilir.

Bu örnek aynı zamanda “match + action” mantığının neden merkezi denetimle birleştğinde çok güçlü olduğunu da gösterir. Çünkü her anahtarın akış tablosuna uygun kurallar yüklendiğinde, paketler ağ boyunca istenen biçimde yönlendirilebilir. Yani tek başına bir akış tablosu yerel bir araçtır; fakat bir denetleyici tarafından koordine edilen çok sayıda akış tablosu, küresel ağ davranışı üretir.

Slayttaki “orchestrated tables” ifadesi tam da bunu anlatır. Orkestrasyon, sadece ayrı ayrı kural yazmak değil; bu kuralların bir bütün olarak tutarlı çalışmasını sağlamaktır. Eğer s3 paketi s1'e gönderiyor ama s1 üzerinde buna uygun devam kuralı yoksa istenen yol oluşmaz. Bu nedenle SDN yaklaşımında asıl güç, kuralların merkezi ve koordineli biçimde dağıtılabilesidir.

Bu slayttan çıkarılması gereken ana fikir şudur: **OpenFlow ile ağ içindeki farklı anahtarların akış tabloları birlikte düzenlenerek, belirli hostlardan gelen trafiğin belirli bir yoldan ilerlemesi sağlanabilir.** Böylece veri düzlemi yalnızca yerel iletme yapan bir yapı olmaktan çıkar; merkezi denetleyici tarafından yönlendirilen, politika tabanlı ve ağ-geneli davranış üreten programlanabilir bir sisteme dönüşür.

OpenFlow Örneği



Orkestre tablolar **ağ genelinde** bir davranış ortaya çıkarabilir, Örneğin:

- h5 ve h6 ana bilgisayarlarından gelen datagramlar, s1 üzerinden h3 veya h4'e, oradan da s2'ye gönderilmelidir.

Network Layer: 4-87

Bu slayt, bir önceki şeklin devamı olarak, **ağ-geneli davranışın aslında tek tek anahtarlara yerleştirilen somut akış kurallarıyla nasıl üretildiğini** göstermektedir. Önceki slaytta kırmızı yollarla gösterilen trafik politikası burada artık tablo girdileri şeklinde açıkça yazılmıştır. Yani “h5 ve h6’dan çıkan, h3 veya h4’e giden trafik önce s1’e, sonra s2’ye gitsin” ifadesi artık yalnızca sözlü bir politika değil; **s3, s1 ve s2 üzerindeki belirli match-action kuralları** ile uygulanmaktadır.

İlk olarak **s3** üzerindeki kurala bakılır. Burada eşleşme koşulu, **kaynak IP adresinin 10.3.. olması ve hedef IP adresinin 10.2.. olmasıdır**. Bu, h5 ve h6’dan çıkan ve 10.2 ağındaki hostlara giden trafiği seçmek anlamına gelir. Eylem kısmında ise **forward(3)** yazmaktadır. Yani bu koşulu sağlayan paketler, s3 üzerindeki 3 numaralı porta gönderilecektir. Şekilde bu portun s1’e giden bağlantıya karşılık geldiği görülmektedir. Böylece ilk adımda trafik doğrudan s2’ye değil, özellikle s1’e yönlendirilmiş olur.

Daha sonra paket **s1** anahtarına gelir. s1 üzerindeki kural biraz daha özeldir. Burada yalnızca kaynak ve hedef IP aralıklarına bakılmamış, ayrıca **ingress port = 1** koşulu eklenmiştir. Bu çok önemlidir; çünkü s1 yalnızca belirli bir porttan, yani s3 tarafından gönderilen trafik için bu kuralı uygulamaktadır. Eşleşme yine **IP Src = 10.3.. ve IP Dst = 10.2..** biçimindedir. Eylem ise **forward(4)** olarak verilmiştir. Bu da s1’in bu trafiği 4 numaralı portundan çıkarıp **s2’ye doğru** iletmesi anlamına gelir. Burada ingress port alanının kullanılması, yalnızca doğru kaynaktan gelen trafik için bu yolun uygulanmasını sağlar; yani akış tablosunun ne kadar hassas kontrol sunabildiğini gösterir.

Son olarak paket **s2** anahtarına ulaştığında, artık hedefin h3 mü yoksa h4 mü olduğuna göre ayırım yapılır. s2 üzerinde iki ayrı kural bulunmaktadır. Her iki kuralda da **ingress port = 2** şartı vardır; yani bu trafik s1'den gelmiş olmalıdır. İlk kuralda **IP Dst = 10.2.0.3** için eylem **forward(3)** olarak tanımlanmıştır; bu da paketin h3'e gönderileceğini gösterir. İkinci kuralda ise **IP Dst = 10.2.0.4** için eylem **forward(4)** verilmiştir; bu durumda paket h4'e gönderilir. Böylece s2, son adımda trafiği doğru nihai hosta ayıran cihaz haline gelir.

Bu üç tablo birlikte düşünüldüğünde çok önemli bir sonuç ortaya çıkar: **Ağ-geneli politika, her cihazın yalnızca kendi yerel kuralını uygulamasıyla gerçekleştirilir.** s3 sadece “bu trafiği s1'e gönder” der. s1 sadece “bu trafik benden s2'ye gitsin” der. s2 ise “hedef hangi hostsa ona göre son porta ilet” der. Her anahtar kendi yerel kararını verir; fakat bu kararlar merkezi denetleyici tarafından uyumlu biçimde tasarlandığı için, sonuçta tüm ağ boyunca tek bir tutarlı uçtan uca davranış elde edilir.

Bu örnek, **SDN'nin temel gücünü** çok açık biçimde gösterir. Klasik ağlarda her yönlendirici çoğunlukla kendi yönlendirme mantığına göre hareket eder ve ağın genel davranışı dolaylı biçimde ortaya çıkar. Burada ise denetleyici, s3, s1 ve s2 üzerine koyduğu kuralları bilinçli biçimde koordine ederek trafiğin izleyeceği yolu önceden belirlemektedir. Yani ağ davranışı kendiliğinden oluşmaz; **tasarlanır ve uygulanır.** Bu da trafik mühendisliği, güvenlik politikaları, yük dengeleme ve belirli trafiği belirli cihazlardan geçirme gibi hedefler için büyük esneklik sağlar.

Slayttaki kurallar ayrıca bir başka önemli noktayı daha gösterir: **aynı politika farklı anahtarlarda farklı eşleşme alanlarıyla ifade edilebilir.** s3'te yalnızca kaynak ve hedef ağlara bakmak yeterliyken, s1 ve s2'de giriş portu bilgisi de kullanılmıştır. Bu, match-action modelinin sabit ve tek biçimli olmadığını; her cihazın bulunduğu konuma ve üstlendiği göreve göre farklı ayrıntı seviyesinde kural yazılabildiğini gösterir.

Bu slaytın ana mesajı şu şekilde özetlenebilir: **OpenFlow'da ağ-geneli bir trafik politikası, farklı anahtarlara yerleştirilen uyumlu match-action kurallarıyla gerçekleştirilir.** Her anahtar yalnızca kendi yerel kuralını uygular; ancak denetleyici bu kuralları orkestre ettiği için, h5 ve h6'dan çıkan trafiğin önce s1'e, sonra s2'ye gitmesi ve sonunda doğru hedef hosta ulaşması gibi uçtan uca davranışlar elde edilir. Bu da SDN'nin veri düzlemine programlanabilirlik ve merkezi kontrol kazandırmasının en somut örneklerinden biridir.

Generalized forwarding: summary

- “**match plus action**” abstraction: match bits in arriving packet header(s) in any layers, take action
 - matching over many fields (link-, network-, transport-layer)
 - local actions: drop, forward, modify, or send matched packet to controller
 - “program” *network-wide* behaviors
- simple form of “network programmability”
 - programmable, per-packet “processing”
 - *historical roots*: active networking
 - *today*: more generalized programming: P4 (see p4.org).

Network Layer: 4-88

Bu slayt, genelleştirilmiş iletme konusunun kısa ama çok güçlü bir özetini veriyor. Bölüm boyunca anlatılan temel fikir şudur: Modern veri düzleminde ağ cihazı, gelen paketin yalnızca hedef adresine bakıp onu bir çıkış portuna gönderen basit bir yapı değildir. Bunun yerine cihaz, paketin başlığındaki farklı alanları inceleyebilir, bu alanları kurallarla eşleştirebilir ve eşleşmeye göre farklı işlemler uygulayabilir. İşte bu genel yaklaşım “**match plus action**” soyutlaması ile ifade edilir.

Buradaki “match” kısmı, paketin hangi bitlerine ya da hangi başlık alanlarına bakılacağını anlatır. Slaytta da belirtildiği gibi bu alanlar yalnızca tek bir katmanla sınırlı değildir; **bağlantı katmanı, ağ katmanı ve taşıma katmanı** alanları birlikte kullanılabilir. Yani cihaz hedef MAC adresine, IP kaynak-hedef adreslerine, protokol alanına, TCP/UDP port numaralarına ya da başka başlık bilgilerine göre karar verebilir. Bu, klasik hedefe dayalı IP yönlendirmeye göre çok daha genel bir eşleştirme modelidir.

“Action” kısmı ise eşleşen pakete ne yapılacağını belirler. Burada uygulanabilecek işlemler yalnızca iletme değildir. Paket düşürülebilir, belirli bir porta gönderilebilir, başlığı değiştirilebilir ya da doğrudan denetleyiciye gönderilebilir. Bu nedenle veri düzlemi, sadece taşıyan değil; aynı zamanda seçen, filtreleyen, dönüştüren ve gerektiğinde kontrol düzlemine başvuran aktif bir işleme katmanı haline gelir.

Slayttaki en önemli ifadelerden biri, bu yaklaşım sayesinde **ağ-geneli davranışların programlanabilmesidir**. Bu çok kritik bir noktadır. Tek bir cihaz üzerindeki kural yerel bir işlem tanımlar; fakat çok sayıda cihaz üzerindeki kurallar uyumlu biçimde yerleştirildiğinde, ağ boyunca belirli bir politika uygulanabilir. Böylece belirli trafik akışları

belli yollardan geçirilebilir, bazı trafik türleri engellenebilir, bazı paketler güvenlik ya da izleme amacıyla denetleyiciye yönlendirilebilir. Yani artık ağ davranışı, büyük ölçüde kurallarla tanımlanabilen bir yapıya dönüşür.

Slaytın ikinci kısmı bunu **network programmability**, yani **ağ programlanabilirliği** olarak adlandırır. Burada kastedilen şey, paketlerin işleme biçiminin yazılım benzeri bir mantıkla tanımlanabilmesidir. Her paket için hangi alanlara bakılacağı ve hangi işlemlerin uygulanacağı önceden belirlenebilir. Bu, veri düzlemine belirli bir ölçüde esneklik kazandırır. Ağ yöneticisi ya da kontrol mantığı, cihazların davranışını yalnızca sabit protokol kurallarına bırakmaz; kurallar seti üzerinden şekillendirir.

Slaytta ayrıca bu düşüncenin tarihsel köklerinin **active networking** yaklaşımına dayandığı belirtilmektedir. Yani ağın daha programlanabilir hale getirilmesi fikri tamamen yeni değildir. Ancak günümüzde bu düşünce daha uygun, daha sistematik ve daha uygulanabilir biçimlerde karşımıza çıkmaktadır. Slaytın sonunda verilen **P4** örneği de bunu gösterir. Buradaki ana mesaj, OpenFlow'un ve match-action yaklaşımının önemli bir adım olduğu; fakat veri düzlemi programlanabilirliğinin bununla sınırlı kalmadığıdır. Daha genel ve daha güçlü programlama modelleri de ortaya çıkmıştır.

Bu slayttan çıkarılması gereken ana sonuç şudur: **Genelleştirilmiş iletme, paketin farklı katmanlardaki başlık alanlarını eşleştirip uygun eylemi uygulayan programlanabilir bir veri düzlemi modelidir.** Bu model sayesinde yerel paket işleme kuralları tanımlanabilir, bu kurallar bir araya getirilerek ağ-geneli davranış oluşturulabilir ve böylece modern ağlar daha esnek, daha denetlenebilir ve daha yazılımla yönetilebilir hale gelir. Bu nedenle bu konu, klasik IP yönlendirmeden modern SDN ve veri düzlemi programlanabilirliğine geçişin temel kavramsal köprüsünü oluşturur.

Network layer: “data plane” roadmap

- Network layer: overview
- What’s inside a router
- IP: the Internet Protocol
- Generalized Forwarding
- Middleboxes
 - middlebox functions
 - evolution, architectural principles of the Internet



Network Layer: 4-89

Şimdiye kadar ağ katmanında klasik yönlendirici işlevleri, IP, IPv6 ve genelleştirilmiş iletme ele alındıktan sonra, artık ağ içinde yalnızca “paketi ileten” cihazların ötesine geçiliyor. Bu geçiş önemlidir; çünkü modern internet yalnızca uç sistemler ve yönlendiricilerden oluşan sade bir yapı değildir. Ağın içinde, trafiği inceleyen, değiştiren, filtreleyen veya yeniden yönlendiren çok sayıda ara cihaz bulunmaktadır. Slaytta da bu bölüm için iki vurgu yapılmaktadır: **middlebox işlevleri** ve bunların **internet mimarisinin evrimi ile mimari ilkeler üzerindeki etkisi**.

“Middlebox” kavramı genel olarak, ağ yolunun ortasında bulunan ve yalnızca klasik IP iletme yapmayan cihazları ifade eder. Yani bu cihazlar paketi alıp sadece bir sonraki çıkışa göndermekle yetinmez; paketin başlık alanlarını inceleyebilir, bazı trafiği engelleyebilir, adres veya port değiştirebilir, yük dengelemesi yapabilir, oturum izleyebilir ya da uygulama düzeyine kadar bakarak özel işlemler uygulayabilir. Bu nedenle middlebox’lar, genelleştirilmiş iletme bölümünde gördüğümüz **match + action** mantığının gerçek dünyadaki yaygın yansımalarından biri olarak düşünülebilir.

Bu başlığa gelmesinin nedeni de tam olarak budur. Bir önceki bölümde, aynı akış tablosu mantığıyla router, switch, firewall ve NAT gibi farklı cihaz davranışlarının ortak bir soyutlama içinde ifade edilebildiğini gördük. Şimdi ise bu soyutlamanın gerçek internet mimarisinde nasıl somutlaştığına bakılıyor. Başka bir ifadeyle, middlebox konusu veri düzlemindeki esnek paket işleme mantığının, modern ağlarda nasıl yaygın biçimde kullanıldığını gösteren doğal bir devam niteliğindedir.

Slayttaki ikinci vurgu, yani **internetin evrimi ve mimari ilkeleri**, oldukça önemlidir. İnternetin ilk tasarımında temel fikirlerden biri, ağın çekirdeğinin görece sade tutulması ve karmaşıklığın uç sistemlerde yer almasıydı. Bu anlayış çoğu zaman “uçtan uca ilke” ile ilişkilendirilir. Ancak zamanla güvenlik, adres kıtlığı, trafik yönetimi, içerik dağıtımı, kurumsal politika uygulamaları ve performans gereksinimleri nedeniyle ağın içine çok sayıda middlebox yerleşti. Bu da internetin pratikte, ilk baştaki saf ve sade mimariden daha karmaşık bir yapıya evrildiğini gösterir.

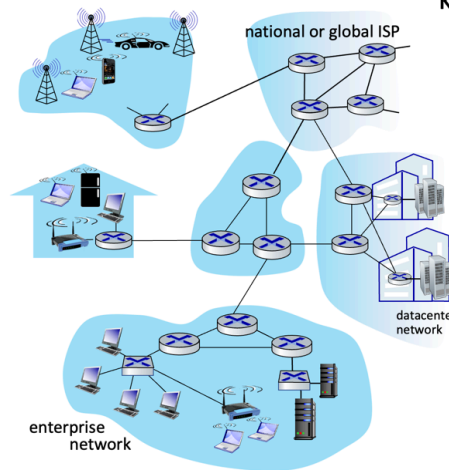
Dolayısıyla bu yeni bölüm yalnızca teknik cihaz türlerini tanıtmak için değil, aynı zamanda şu daha büyük soruyu tartışmak için açılmaktadır: **İnternet bugün neden ilk tasarlandığı kadar sade değil?** Güvenlik duvarları, NAT’lar, yük dengeleyiciler, uygulama vekilleri ve benzeri yapılar neden bu kadar yaygınlaştı? Bu soruların cevabı, ağ mühendisliğinde teorik mimari ilkeler ile gerçek operasyonel ihtiyaçlar arasındaki gerilimi anlamak açısından çok değerlidir.

Bu slaytın ana mesajı şudur: **Veri düzlemi konusu artık sadece yönlendirme ve OpenFlow ile sınırlı kalmayacak; trafiği iletmenin ötesinde işleyen ara ağ cihazları, yani middlebox’lar da incelenecektir.** Bu cihazlar modern internetin vazgeçilmez parçalarıdır ve hem teknik işlevleri hem de internet mimarisinin zaman içindeki değişimini anlamak için kritik öneme sahiptir.

Middleboxlar Heryerde!

NAT: ev, cep telefonu, kurumsal ağlar

Uygulamaya Özel : Hizmet sağlayıcıları, Kurumlar, CDN



Firewalls, IDS: Şirketler, kurumlar, hizmet sağlayıcılar, İnternet Servis Sağlayıcıları (ISS)

Load balancers: Şirketler, hizmet sağlayıcılar, veri merkezleri, mobil ağlar

Caches: Hizmet sağlayıcı, mobil, CDN’ler

Bu slaytın ana mesajı çok açıktır: **middlebox’lar istisna değil, modern internetin sıradan ve yaygın bileşenleridir.** Yani ağ içinde yalnızca yönlendiriciler ve uç sistemler

yoktur; trafięi inceleyen, dönüştüren, filtreleyen, hızlandıran veya yeniden dağıtan çok sayıda ara cihaz bulunmaktadır. Slayttaki “Her yerde” vurgusu tam olarak bunu anlatır. Middlebox’lar ev ağından mobil şebekelere, kurumsal ağlardan veri merkezlerine, hatta servis sağlayıcı omurgalarına kadar internetin hemen her bölümünde karşımıza çıkar.

İlk dikkat çeken örnek **NAT**’tır. Slaytta NAT’ın ev, cep telefonu ve kurumsal ağlarda bulunduğu belirtilmektedir. Bu çok anlamlıdır; çünkü NAT artık çoęu kullanıcı için fark edilmeyen ama günlük internet erişiminin doğal parçası haline gelmiş bir ara işlevdir. Evdeki modemden mobil operatör altyapısına kadar birçok noktada NAT kullanılır. Bunun nedeni çoęu zaman adres tasarrufu, özel adres alanlarının kullanımı ve iç ağların dış dünyadan soyutlanmasıdır. Yani middlebox mantığı yalnızca büyük kurumsal ortamlara özgü değildir; sıradan bir ev ağına bile middlebox işlevi vardır.

İkinci grup **firewall** ve **IDS**’lerdir. Slaytta bunların şirketlerde, kurumlarda, hizmet sağlayıcılarda ve ISS düzeyinde bulunduğu gösterilmektedir. Bu cihazlar veya işlevler, ağ trafiğini yalnızca iletmek yerine güvenlik açısından değerlendirir. Hangi trafięe izin verileceęi, hangi akışların şüpheli olduęu, hangi bağlantıların engelleneceęi gibi kararlar bu tür middlebox’lar tarafından uygulanır. Bu da bize modern internetin yalnızca bağlantı sağlama değil, aynı zamanda güvenlik politikaları uygulama amacı taşıdığını gösterir.

Üçüncü örnek **load balancer**, yani yük dengeleyicidir. Slaytta bunların şirketlerde, hizmet sağlayıcılarda, veri merkezlerinde ve mobil ağlarda kullanıldığını belirtilmektedir. Yük dengeleyiciler, gelen trafięi birden fazla sunucu ya da servis örneęi arasında dağıtarak performans ve ölçeklenebilirlik sağlar. Bu tür cihazlar klasik yönlendirici gibi sadece “hedefe ulaştırma” yapmaz; hangi sunucunun isteęi işleyeceęine karar vererek hizmetin çalışma şeklini doğrudan etkiler. Bu nedenle middlebox kavramı yalnızca güvenlikle sınırlı değildir; performans ve hizmet süreklilięi açısından da kritik rol oynar.

Dördüncü örnek **cache**’lerdir. Slaytta bunların hizmet sağlayıcılarda, mobil ağlarda ve CDN yapılarında yer aldığı belirtiliyor. Cache sistemleri, sık erişilen içerięi kullanıcıya daha yakın noktalarda tutarak gecikmeyi azaltır ve omurga üzerindeki yükü düşürür. Bu da middlebox’ların yalnızca paketleri filtreleyen ya da dönüştüren yapılar olmadığını; aynı zamanda içerik dağıtımını ve kullanıcı deneyimini iyileştiren bileşenler olduğunu gösterir.

Slaytta ayrıca **uygulamaya özel** middlebox’lardan da söz edilmektedir. Bu çok önemlidir; çünkü bazı middlebox’lar genel ağ işlevleri yerine belirli bir uygulama veya hizmet için tasarlanır. Hizmet sağlayıcılar, kurumlar ve CDN’ler içinde yer alan bu tür yapılar, belirli uygulama trafiğini optimize etmek, denetlemek ya da yönlendirmek için kullanılır. Bu da internetin giderek daha fazla “uygulama farkında” hale geldiğini gösterir. Başka bir ifadeyle, ağ çekirdeęi tamamen kör ve genel amaçlı bir taşıma ortamı olmaktan çıkıp, zamanla uygulama ihtiyaçlarına daha duyarlı hale gelmiştir.

Bu slaytın mimari açıdan en önemli sonucu şudur: **Middlebox'lar internetin kenarında görülen geçici eklemeler değil, internetin bugünkü işleyişinin kalıcı parçalarıdır.** Ev ağında NAT, kurumsal ağda firewall, veri merkezinde load balancer, CDN'de cache bulunuyorsa, bu artık ağ mimarisinin pratik gerçekliğidir. Yani internetin teorik sade modeli ile gerçek dünyadaki dağıtık hizmet altyapısı arasında önemli bir fark oluşmuştur.

Bu yüzden bu slaytı şöyle özetlemek doğru olur: **Modern internet, yalnızca yönlendiricilerden oluşan sade bir ağ değildir; NAT, firewall, IDS, load balancer, cache ve uygulamaya özel cihazlar gibi çok sayıda middlebox ile çevrilidir.** Bu yapılar performans, güvenlik, adresleme, ölçeklenebilirlik ve içerik dağıtımı gibi nedenlerle ağın her katmanında ve her ortamında yaygın biçimde kullanılmaktadır.

Middleboxes

- Başlangıçta : Özel (kapalı) donanım çözümleri
- Open API'yi uygulayan "whitebox" donanıma geçiş
 - Özel (kapalı) donanım çözümlerinden uzaklaşmak
 - **match+action aracılığıyla programlanabilir** yerel eylemler
 - Yazılım alanında yenilikçilik ve farklılaşmaya yönelmek
 - **SDN:** mantıksal olarak merkezi kontrol ve yapılandırma yönetimi yaklaşımını temsil eder.
- **network functions virtualization (NFV):** ğ işlevlerinin özel fiziksel cihazlar yerine sanallaştırılmış servisler olarak çalıştırılmasıdır.

Bu slayt, middlebox kavramının yalnızca “ağ içindeki özel cihazlar” olarak kalmadığını; zamanla **nasıl evrildiğini** anlatıyor. Buradaki ana fikir şudur: Middlebox işlevleri başlangıçta çoğunlukla kapalı, üreticiye bağımlı donanımlar üzerinde sunuluyordu; ancak zamanla bu işlevler daha açık, daha programlanabilir ve daha yazılım ağırlıklı yapılara kaymaya başladı.

Slaytın ilk maddesi bunu açıkça söylüyor: başlangıçta middlebox çözümleri büyük ölçüde **proprietary**, yani kapalı ve üreticiye özgü donanımlardı. Bu tür cihazlarda hem donanım hem yazılım çoğu zaman tek bir üreticinin kontrolündeydi. Güvenlik duvarı, yük dengeleyici, WAN optimizasyon cihazı ya da başka bir middlebox satın alındığında, kullanıcı aynı zamanda o üreticinin kapalı ekosistemine de girmiş oluyordu. Bu yaklaşım

çalışıyordu; ancak esneklik düşüktü, maliyet yüksekti ve yenilik hızı çoğu zaman üreticinin sunduğu özelliklerle sınırlı kalıyordu.

İkinci önemli nokta, sektörün zamanla **whitebox hardware** yaklaşımına yönelmesidir. Whitebox burada, belirli bir üreticinin kapalı çözümüne sıkı sıkıya bağlı olmayan, daha genel amaçlı ve daha açık arayüzlerle çalışan donanımı ifade eder. Bu geçişin arkasındaki mantık şudur: Ağ işlevleri donanımdan ayrışsın, donanım daha standart hale gelsin ve asıl farklılaşma yazılım tarafında gerçekleşsin. Böylece kurumlar her işlev için özel bir kapalı kutu satın almak zorunda kalmadan, daha esnek altyapılar kurabilir.

Slaytta ayrıca **programmable local actions via match+action** ifadesi yer alıyor. Bu, önceki bölümle doğrudan bağlantılıdır. Yani middlebox işlevleri artık sabit ve üretici tarafından önceden belirlenmiş davranışlarla değil; daha çok programlanabilir yerel işlemlerle gerçekleştirilmeye başlanmaktadır. Paket başlık alanlarına bak, eşleşme kur, sonra uygun işlemi uygula mantığı burada da devreye girer. Böylece middlebox davranışları daha modüler ve daha esnek biçimde tanımlanabilir hale gelir.

Buradaki çok önemli bir başka vurgu, **yenilik ve farklılaşmanın donanımdan yazılıma kaymasıdır**. Yani artık asıl değer, kutunun fiziksel biçiminde değil; üstünde çalışan yazılım mantığında, politika motorlarında, trafik işleme kurallarında ve yönetim araçlarında ortaya çıkmaktadır. Bu dönüşüm, ağ mühendisliğinde donanım merkezli düşünceden yazılım merkezli düşünceye geçişin önemli bir parçasıdır.

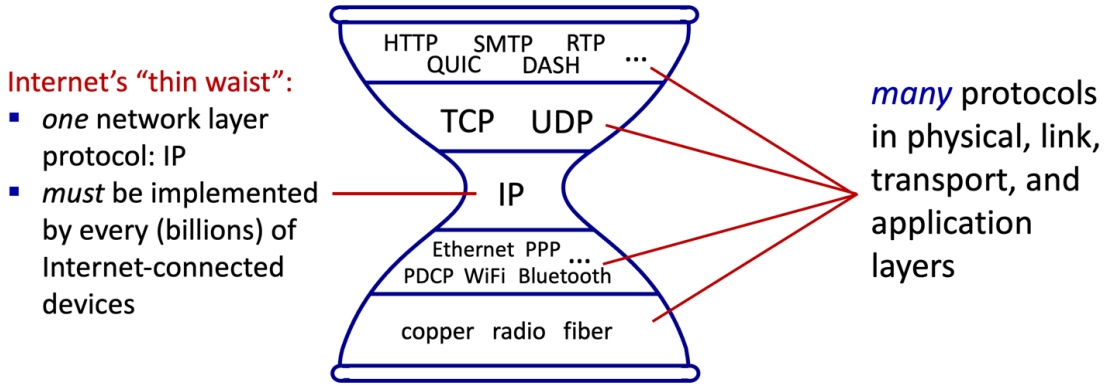
Slaytın bir sonraki maddesi **SDN**'i bu dönüşümle ilişkilendiriyor. SDN burada, mantıksal olarak merkezi kontrol ve yapılandırma yönetimi yaklaşımını temsil eder. Başka bir ifadeyle, cihazların davranışları tek tek yerel kutular üzerinde manuel olarak ayarlanmak yerine, daha merkezi bir kontrol mantığıyla yönetilebilir hale gelir. Bu kontrolün özel ya da genel bulut içinde bulunabilmesi, ağ yönetiminin fiziksel cihazlardan giderek soyutlandığını gösterir. Yani middlebox'lar da artık tek başına duran bağımsız kutular değil; daha büyük bir yazılım kontrollü altyapının parçaları haline gelmektedir.

Son maddede geçen **NFV**, yani **network functions virtualization**, bu dönüşümün daha da ileri aşamasını gösterir. Buradaki düşünce, ağ işlevlerinin özel fiziksel cihazlar yerine sanallaştırılmış servisler olarak çalıştırılmasıdır. Yani güvenlik duvarı, yük dengeleyici ya da başka bir ağ işlevi, artık illa özel bir kutu olarak değil; standart sunucu, ağ ve depolama altyapısı üzerinde çalışan yazılım bileşeni olarak sunulabilir. Bu da dağıtım hızını artırır, ölçeklenebilirliği kolaylaştırır ve maliyetleri azaltabilir.

Bu slaytın en önemli mesajı şudur: **Middlebox'lar kapalı ve özel donanım çözümlerinden, açık arayüzlü, programlanabilir, yazılım kontrollü ve sanallaştırılabilir yapılara doğru evrilmektedir**. Bu evrimde üç temel eğilim öne çıkar: donanımın standartlaşması, kontrolün merkezileşmesi ve ağ işlevlerinin

yazılımla/sanallaştırmayla sunulması. Başka bir ifadeyle, modern ağlarda middlebox kavramı artık yalnızca fiziksel bir ara cihaz değil; yazılım, otomasyon, SDN ve NFV ile birlikte düşünülen daha geniş bir hizmet mimarisine dönüşmüştür.

The IP hourglass



Bu slayt, internet mimarisinin en temel tasarım fikrini gösteriyor: **IP hourglass**, yani **IP kum saati** modeli. Buradaki ana düşünce, internetin ortasında çok dar ve ortak bir katman bulunması; bunun üstünde ve altında ise çok sayıda farklı protokol ve teknoloji yer alabilmesidir. Slaytta bu dar bölüm **IP** ile gösterilmektedir ve "Internet's thin waist" ifadesiyle adlandırılmaktadır.

"Kum saati" benzetmesi çok yerindedir. Üst tarafta çok sayıda uygulama ve taşıma katmanı protokolü vardır. Örneğin HTTP, SMTP, RTP, QUIC, DASH gibi uygulama düzeyi protokoller ile TCP ve UDP gibi taşıma katmanı protokolleri birlikte gösterilmektedir. Alt tarafta da yine çok sayıda bağlantı ve fiziksel katman teknolojisi vardır: Ethernet, PPP, WiFi, Bluetooth gibi bağlantı katmanı yapıları ile bakır, radyo ve fiber gibi fiziksel iletim ortamları. Buna karşılık tam ortada, bu büyük çeşitliliği birbirine bağlayan **tek ortak ağ katmanı protokolü IP** yer alır.

Bu yapının en önemli sonucu şudur: İnternete bağlanan her cihazın, her uygulamayı ya da her alt katman teknolojisini desteklemesi gerekmez; ama **IP'yi desteklemesi gerekir**. Slaytta da özellikle vurgulandığı gibi, internetin bu "ince beli" milyarlarca internet bağlantılı cihaz tarafından uygulanmak zorundadır. Çünkü üstteki çeşitliliğin alttaki çeşitlilikle buluşabilmesi ancak ortak bir ara katman sayesinde mümkündür.

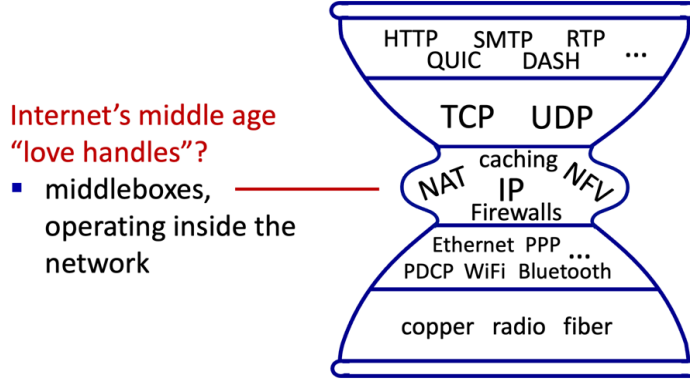
Bu tasarım internetin başarısının temel nedenlerinden biridir. Üst katmanda yeni uygulamalar geliştirilebilir; örneğin zaman içinde web, e-posta, video akışı, gerçek zamanlı medya ve başka birçok servis ortaya çıkmıştır. Alt katmanda da yeni erişim ve iletim teknolojileri geliştirilebilir; örneğin Ethernet'ten WiFi'ye, mobil erişimden fiber altyapılara kadar çok farklı teknolojiler kullanılabilir. Bütün bu yenilikler yapılırken, eğer ortak IP katmanı korunursa sistemin bütünü birlikte çalışmaya devam eder. Yani internet, **ortadaki ortak protokol sayesinde üstte ve altta yeniliğe açık** hale gelir.

Bu nedenle IP'nin “thin waist” olması hem avantaj hem de kısıttır. Avantajdır; çünkü ortak bir standart etrafında büyük ölçekli birlikte çalışabilirlik sağlar. Farklı üreticiler, farklı cihazlar, farklı ağ ortamları ve farklı uygulamalar aynı internet içinde çalışabilir. Kısıttır; çünkü bu kadar merkezi bir rol oynayan katmanı değiştirmek çok zordur. Önceki IPv6 tartışmalarında gördüğümüz geçiş zorluğu tam da buradan gelir. Eğer IP internetin dar beli ise, bu beli değiştirmek tüm yapıyı etkiler.

Slayttaki oklarla verilen mesaj da budur: fiziksel, bağlantı, taşıma ve uygulama katmanlarında çok sayıda protokol olabilir; ama bunların hepsi ortadaki ortak IP katmanı üzerinden birbirine bağlanır. Başka bir ifadeyle, internetin mimari sadeliği tam merkezdeki bu ortak ağ katmanından gelir.

Bu slayttan çıkarılması gereken ana fikir şudur: **İnternetin mimarisi, ortada tek ve ortak bir ağ katmanı protokolü olan IP etrafında kurulmuştur.** Üstte çok sayıda uygulama ve taşıma protokolü, altta çok sayıda bağlantı ve fiziksel teknoloji bulunabilir; fakat hepsini birlikte çalıştıran ortak zemin IP'dir. Bu da internetin hem ölçeklenebilirliğini hem de yeniliğe açıklığını mümkün kılan temel mimari ilkedir.

The IP hourglass, at middle age



Bu slayt, önceki **IP kum saati** modelinin artık eskisi kadar “ince bel” olmadığını anlatıyor. Başlıktaki “**at middle age**” ifadesi ve soldaki “**love handles**” benzetmesi bilinçli olarak seçilmiş. Verilmek istenen mesaj şudur: İnternetin ortasındaki sade ve dar IP katmanı zamanla çevresine ek işlevler almış, yani ağın içi giderek daha karmaşık hale gelmiştir.

Klasik kum saati modelinde ortada yalnızca **IP** vardı. Üstte çok sayıda uygulama ve taşıma protokolü, altta da çok sayıda bağlantı ve fiziksel teknoloji bulunuyordu. Bu yapı internetin sadeliğini ve birlikte çalışabilirliğini temsil ediyordu. Ancak bu slaytta IP’nin etrafına **NAT, caching, firewalls, NFV** gibi ek işlevlerin yerleştirildiğini görüyoruz. Bu da ağ içinde artık yalnızca saf IP iletiminin yapılmadığını, çok sayıda **middlebox işlevinin** de devreye girdiğini gösteriyor.

Buradaki en önemli fikir, internetin başlangıçtaki mimari sadeliğinin zaman içinde pratik ihtiyaçlar nedeniyle değişmiş olmasıdır. Adres kıtlığı NAT’ı, güvenlik ihtiyacı firewall’ları, performans ve içerik dağıtımı caching yapılarını, esneklik ve hizmet sunumu ise NFV benzeri yaklaşımları öne çıkarmıştır. Yani internet hâlâ IP etrafında kuruludur; fakat artık bu dar belin çevresinde çok sayıda yardımcı ve müdahaleci işlev bulunmaktadır.

“Love handles” benzetmesi aslında hafif mizahi ama çok güçlü bir eleştiridir. Kum saatinin ortası eskiden inceydi; şimdi ise yanlarında çıkıntılar oluşmuş gibi gösteriliyor. Bunun anlamı şudur: Ağın çekirdeği artık tamamen minimal değildir. İçeride çalışan middlebox’lar, internetin başlangıçta öngörülen sade uçtan uca modeline ek karmaşıklık katmıştır.

Bu durumun iki yönü vardır. Bir yandan bu ek işlevler çok faydalıdır; güvenlik, ölçeklenebilirlik, adresleme çözümü, yük azaltma ve hizmet optimizasyonu sağlarlar. Öte yandan internetin mimari saflığını azaltırlar. Çünkü artık ağ, sadece paket taşıyan tarafsız bir ortam değil; pakete müdahale eden, onu filtreleyen, değiştiren, önbelleğe alan ve bazen yeniden yazan bir yapıya dönüşmüştür.

Bu slayttan çıkarılması gereken ana mesaj şudur: **Modern internet, klasik IP kum saati modelini tamamen terk etmemiştir; ancak IP'nin etrafında biriken middlebox işlevleri nedeniyle bu model daha kalın, daha karmaşık ve daha müdahaleci bir hale gelmiştir.** Başka bir ifadeyle, internetin “ince beli” bugün hâlâ vardır, ama artık etrafında ciddi bir middlebox ekosistemi oluşmuştur.

Architectural Principles of the Internet

RFC 1958

“Many members of the Internet community would argue that there is no architecture, but only a tradition, which was not written down for the first 25 years (or at least not by the IAB). However, in very general terms, the community believes that **the goal is connectivity, the tool is the Internet Protocol, and the intelligence is end to end rather than hidden in the network.**”

Three cornerstone beliefs:

- simple connectivity
- IP protocol: that narrow waist
- intelligence, complexity at network edge

Bu slayt, internetin yalnızca bir teknoloji yığını değil, aynı zamanda belirli **mimari ilkeler** üzerine kurulmuş bir sistem olduğunu vurguluyor. Burada verilen RFC 1958 alıntısı, internet topluluğunun uzun yıllar boyunca benimsediği temel yaklaşımı özetler: **Amaç bağlantı kurmaktır, araç IP'dir ve asıl zeka ağın içinde değil uçlarda yer almalıdır.** Bu ifade, internetin neden bu kadar yaygınlaştığını ve neden uzun süre ölçeklenebilir kalabildiğini anlamak için çok önemlidir.

İlk temel ilke **simple connectivity**, yani **basit bağlantısallık** anlayışıdır. İnternetin birincil hedefi, farklı ağları ve cihazları birbirine bağlayabilmektir. Başka bir deyişle, internetin çekirdeği mümkün olduğunca temel bir işi yapmalıdır: paketleri bir uçtan diğer uca taşıyabilmek. Bu yaklaşım, ağın çok fazla özel amaçla yüklenmemesini ve her yeni

gereksinim için çekirdeğin baştan tasarlanmamasını sağlar. Böylece internet, farklı teknolojileri ve farklı uygulamaları bir arada taşıyabilen genel amaçlı bir altyapı haline gelir.

İkinci temel ilke, **IP protokolünün dar bel (narrow waist)** oluşturmastır. Önceki kum saati slaytlarında gördüğümüz gibi, üstte çok sayıda uygulama ve taşıma protokolü, altta da çok sayıda bağlantı ve fiziksel katman teknolojisi olabilir. Fakat bunların hepsini birbirine bağlayan ortak zemin IP'dir. Bu dar bel yaklaşımı, internetin hem birlikte çalışabilirliğini hem de yeniliğe açıklığını mümkün kılar. Çünkü üst katmanda yeni uygulamalar geliştirilebilir, alt katmanda yeni erişim teknolojileri üretilebilir; yeter ki ortadaki ortak IP katmanı korunsun.

Üçüncü temel ilke ise en kritik olanlardan biridir: **zeka ve karmaşıklık ağın kenarında, yani uçlarda olmalıdır**. Slaytta "the intelligence is end to end rather than hidden in the network" ifadesiyle bu açıkça belirtilmektedir. Bunun anlamı şudur: Ağın çekirdeği mümkün olduğunca sade tutulmalı, karmaşık kontrol ve uygulama mantıkları ise uç sistemlerde yer almalıdır. Örneğin güvenilirlik, uygulama mantığı, oturum yönetimi ya da içerik işleme gibi daha karmaşık görevlerin büyük bölümü uçlarda çözülmelidir. Bu anlayış, internetin erken dönem mimarisinin temel felsefesidir ve çoğu zaman **uçtan uca ilke** ile ilişkilendirilir.

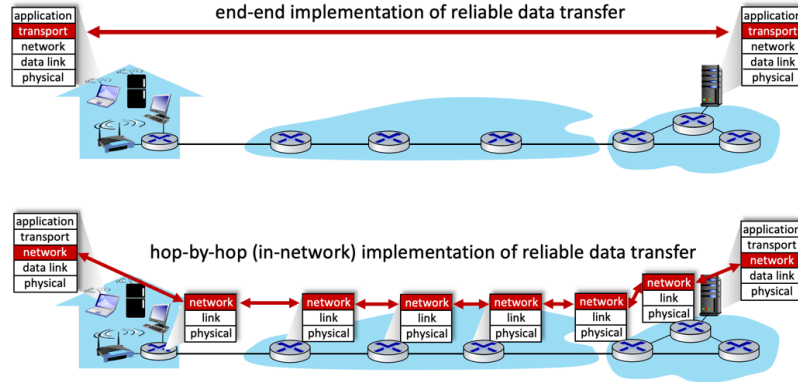
Bu üç ilke birlikte düşünüldüğünde internetin neden başarılı olduğu daha net anlaşılır. Çekirdek sade tutulduğu için sistem ölçeklenebilir olmuştur. Ortak IP katmanı sayesinde çok farklı teknolojiler birlikte çalışabilmiştir. Zekanın uçlarda bulunması da yeniliği hızlandırmıştır; çünkü yeni servisler geliştirmek için ağın içeriğini değiştirmek gerekmemiştir. Web, e-posta, video akışı ve daha birçok uygulamanın hızla yayılabilmesi biraz da bu mimari tercihlerin sonucudur.

Ancak bu slayt, önceki middlebox tartışmalarıyla birlikte düşünüldüğünde daha da anlam kazanır. Çünkü modern internet artık bu ilkelerin tamamen saf biçimde uygulandığı bir yapı değildir. NAT, firewall, cache ve benzeri middlebox'lar ağın içine ek zeka ve işlev yerleştirmiştir. Bu nedenle bu slayt bir bakıma "internetin ideal mimarisi neydi?" sorusuna cevap verirken, middlebox bölümü de "gerçek dünyada bu ideal ne kadar değişti?" sorusunu gündeme getirir. Yani burada anlatılan ilkeler, hem internetin tasarım felsefesini hem de bu felsefeden zamanla nasıl uzaklaştığını anlamak için temel oluşturur.

Bu slaytın ana mesajı şu şekilde özetlenebilir: **İnternetin mimarisi üç temel inanca dayanır: temel amaç bağlantısallıktır, ortak zemin IP'dir ve asıl zeka ağın çekirdeğinde değil uçlarda yer almalıdır**. Bu ilkeler internetin sade, esnek ve ölçeklenebilir bir sistem olarak büyümesini sağlamıştır.

The end-end argument

- some network functionality (e.g., reliable data transfer, congestion) can be implemented **in network**, or at **network edge**



Bu slayt, internet mimarisinin en temel düşüncelerinden birini anlatır: **uçtan uca argüman**. Buradaki temel soru şudur: Ağda ihtiyaç duyulan bazı işlevler, örneğin **güvenilir veri aktarımı** veya **tıkanıklıkla başa çıkma**, ağın içinde mi yapılmalıdır, yoksa uç sistemlerde mi gerçekleştirilmelidir?

Slaytın üst kısmında, **güvenilir veri aktarımının uçtan uca uygulanması** gösterilmektedir. Burada güvenilirlik işlevi doğrudan iletişimin iki ucu arasında, yani gönderici ve alıcı uç sistemlerde gerçekleştirilir. Başka bir deyişle, ağın içindeki yönlendiriciler özel bir güvenilirlik mantığı yürütmez; sadece paketleri taşır. Asıl kontrol, hata denetimi, yeniden gönderim ve doğru teslim güvencesi uçlardaki protokoller tarafından sağlanır. İnternette TCP'nin oynadığı rol büyük ölçüde bu mantığa dayanır.

Slaytın alt kısmında ise alternatif yaklaşım gösterilir: **hop-by-hop**, yani **atlama-atlama ağ içi uygulama**. Bu yaklaşımda her ara düğüm ya da ağ içindeki her bölüm, güvenilirliği kendi yerel bağlantısı için sağlamaya çalışır. Yani paket bir sonraki düğüme güvenilir biçimde aktarılır, sonra bir sonraki atlamada aynı işlem tekrar edilir. Böyle bir modelde ağın içi daha akıllı hale gelir; çünkü güvenilirlik işlevi yalnızca uçlarda değil, ara sistemlerde de yürütülmektedir.

Uçtan uca argümanın söylediği şey, birçok durumda bu tür işlevlerin **uçlarda gerçekleştirilmesinin daha doğru ve daha temiz bir tasarım** olduğudur. Bunun temel nedeni şudur: Eğer son uygulama gerçekten tam güvenilirlik istiyorsa, ağın içindeki ara düğümler ne kadar iş yaparsa yapsın, en son doğrulama yine uç sistemlerde yapılmak

zorundadır. Çünkü verinin gerçekten eksiksiz, doğru sırada ve hatasız ulaşım ulaşıldığını en iyi bilen yer iletişimin uçlarıdır. Bu nedenle aynı işlevi ağın içinde tekrar tekrar yapmak bazen gereksiz karmaşıklık yaratabilir.

Bu argüman, internetin önceki slaytta gördüğümüz mimari ilkeleriyle de doğrudan ilişkilidir. İnternetin temel yaklaşımı, ağın çekirdeğini mümkün olduğunca sade tutmak ve zekâyı uçlara yerleştirmektir. Yani yönlendiriciler esas olarak paket taşımaya odaklanmalı; daha karmaşık işlevler ise uç sistemlerde çözülmelidir. Böylece çekirdek daha basit, daha ölçeklenebilir ve daha genel amaçlı kalır.

Elbette bu, ağ içinde hiçbir ek işlev olmayacağı anlamına gelmez. Bazı durumlarda yerel düzeyde hata denetimi, bağlantı katmanında yeniden iletim veya ağ içi trafik yönetimi faydalı olabilir. Ancak uçtan uca argüman şunu söyler: **Bir işlevin nihai doğruluğu zaten uçlarda garanti edilmek zorundaysa, bu işlevin asıl ve belirleyici yeri uç sistemlerdir.** Ağ içindeki uygulamalar yardımcı olabilir, ama temel güvenceyi tek başına onlar vermez.

Bu slayttan çıkarılması gereken ana fikir şudur: **İnternet mimarisinde birçok önemli işlev, özellikle güvenilirlik gibi uçtan uca anlam taşıyan işlevler, ağın içinde değil iletişimin uçlarında gerçekleştirilmelidir.** Bu yaklaşım, ağ çekirdeğini sade tutar ve internetin ölçeklenebilir, esnek bir sistem olarak çalışmasını destekler.

The end-end argument

- some network functionality (e.g., reliable data transfer, congestion) can be implemented **in network**, or at **network edge**

“The function in question can completely and correctly be implemented only with the knowledge and help of the application standing at the end points of the communication system. Therefore, providing that questioned function as a feature of the communication system itself is not possible. (Sometimes an incomplete version of the function provided by the communication system may be useful as a performance enhancement.)

We call this line of reasoning against low-level function implementation the “end-to-end argument.”

— Saltzer, Reed, Clark 1981 —

Bu slayt, **uçtan uca argümanın** en klasik ve en net ifadesini veriyor. Buradaki temel düşünce şudur: Bir iletişim işlevi, ancak iletişimin iki ucundaki uygulamaların bilgisi ve

desteęiyle **tam ve doęru biimde** gerekleřtirilebiliyorsa, bu iřlevi aęın alt seviyelerine yerleřtirmek mimari olarak yeterli deęildir. Aęın ii bu iřlevin bazı paralarını kısmen yapabilir; ancak son ve eksiksiz doęrulama yine ularda yapılmak zorundadır.

Slayttaki alıntının özü tam olarak budur. Eęer bir iřlevin gerekten doęru alıřıp alıřmadıęını yalnızca u noktalar anlayabiliyorsa, o iřlevin asıl yeri ulardır. Aęın kendisi bu iřlevi tamamen üstlenemez. En fazla, performansı artırmak iin eksik ya da yardımcı bir sürümünü saęlayabilir. Yani aę iindeki mekanizmalar faydasız deęildir; ancak nihai doęruluk garantisi vermezler.

Bunu güvenilir veri aktarımı üzerinden düşünmek kolaydır. Aę iindeki her baęlantı kendi yerel düzeyinde paketi dikkatle iletebilir, hatta bazı yeniden iletimler yapabilir. Fakat sonuçta verinin gerekten doęru uygulamaya, eksiksiz ve istenen anlamda ulařıp ulaşmadıęını en iyi bilen yine gönderici ve alıcı u uygulamalarıdır. Bu yüzden uçtan uca güvenilirlik, aę iinde desteklense bile sonunda yine ularda tamamlanmak zorundadır.

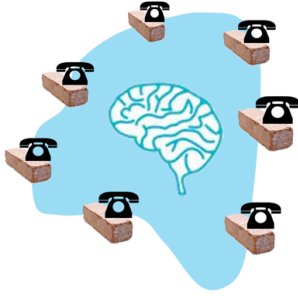
Aynı mantık başka iřlevler iin de geerlidir. Tıkanıklık kontrolü, güvenlik, veri bütünlüęü ya da uygulamaya özgü doęrulamalar gibi birçok konuda aę ii mekanizmalar yardımcı olabilir. Ancak bu mekanizmalar, uygulamanın ihtiya duyduęu kesin davranıřı her zaman tek başına garanti edemez. ünkü uygulamanın neyi “bařarılı iletiřim” saydıęını bilen yer iletiřimin u noktalarıdır.

Slayttaki önemli ifade řudur: Aę iindeki bir iřlevin **tam sürümü** mümkün olmayabilir, ama **eksik bir sürümü performans artırıcı olarak yararlı olabilir**. Bu ok dengeli bir görüřtür. Uçtan uca argüman, “aęın iinde hiçbir řey yapılmamalıdır” demez. Bunun yerine řunu söyler: Aę iindeki ek mekanizmalar faydalı olabilir, fakat mimari olarak asıl güvenceyi uçlar saęlamalıdır. Yani aę ii özümle yardımcıdır; belirleyici olan uçtan uca kontroldür.

Bu düşünce, internetin genel mimari felsefesiyle doğrudan uyumludur. Aęın ekirdeęi mümkün olduęunca sade tutulur, karmařıklık ve uygulama bilgisi uçlarda yer alır. Böylece aę daha genel amaçlı, daha öleklenebilir ve farklı uygulamalara daha açık hale gelir. Uçtan uca argüman da bu yaklaşımın teorik temelini oluřturur.

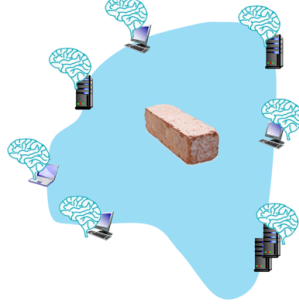
Bu slayttan ıkarılması gereken ana fikir řudur: **Bir aę iřlevi ancak u uygulamaların bilgisiyle tam ve doęru biimde gerekleřtirilebiliyorsa, o iřlevin asıl uygulanma yeri aęın ii deęil, iletiřimin uçlarıdır**. Aę iindeki destek mekanizmaları yararlı olabilir; ancak onlar nihai özüm deęil, en fazla tamamlayıcı performans iyileřtirmeleridir.

Akıl nerede?



20. yüzyıl telefon ağı :

- Ağ anahtarlarında zeka/işlem



İnternet (2005 öncesi))

- yapay zeka, uç bilgi işlem



İnternet (2005 sonrası)

- Programlanabilir Ağ Cihazları
- Zeka, bilgi işlem, uçta devasa uygulama düzeyinde altyapı

Bu slayt, internet mimarisindeki en temel tartışmalardan birini çok görsel bir biçimde soruyor: **Akıl, yani zeka ve işlem mantığı ağın neresinde olmalı?** Ağın ortasında mı, uçlarda mı, yoksa her iki tarafta birden mi? Slayt bu soruya tarihsel bir karşılaştırma üzerinden yaklaşıyor ve üç farklı dönemi yan yana koyuyor.

Soldaki ilk şekil **20. yüzyıl telefon ağını** temsil ediyor. Burada ağın iç kısmında büyük bir beyin çizilmiş, uçlarda ise oldukça basit telefonlar yer alıyor. Verilmek istenen mesaj açık: klasik telefon şebekesinde asıl zeka ağın içindedir. Anahtarlama, çağrı kurma, denetim ve işleme mantığı ağın merkezinde yer alır; uç cihazlar ise görece basittir. Yani kullanıcı cihazı çok akıllı değildir, asıl karar verici yapı ağın çekirdeğidir. Bu yaklaşım, kapalı ve merkezi denetimli telekom dünyasının tipik mimarisidir.

Ortadaki şekil ise **internetin 2005 öncesi** anlayışını özetliyor. Burada ağın ortasında yalnızca sade bir “tuğla” çizilmiş, beyinler ise uçlardaki bilgisayar ve sunucuların üzerine yerleştirilmiş. Bu, internetin klasik mimari ilkesini temsil eder: **ağ çekirdeği sade olsun, zeka uçlarda bulunsun**. Önceki slaytlarda gördüğümüz uçtan uca argüman ve RFC 1958’deki “intelligence is end to end rather than hidden in the network” yaklaşımı burada doğrudan görselleştirilmektedir. Ağın içi temel bağlantı işini yapsın; güvenilirlik, uygulama mantığı, yenilik ve karmaşık hizmetler uç sistemlerde yer alsın.

Sağdaki şekil ise **internetin 2005 sonrası** durumunu göstermektedir. Bu kez ağın içinde yine bir tuğla var, ama artık etrafında ve içinde çok daha fazla beyin yer alıyor. Uçlarda da zeka devam ediyor; ayrıca büyük veri merkezi yapılarında ve ağın çeşitli ara noktalarında da ek zeka görülüyor. Bu, modern internetin yalnızca saf uçtan uca

modelle açıklanamayacağını anlatır. Çünkü artık ağın içinde **programlanabilir ağ cihazları, middlebox'lar, yük dengeleyiciler, güvenlik duvarları, önbellekler, uygulama farkında servis altyapıları** ve büyük ölçekte **bulut/veri merkezi mantığı** bulunmaktadır. Yani zeka tamamen çekirdeğe dönmemiştir; ama yalnızca uçlarda da kalmamıştır. Ağın çeşitli noktalarına dağılmıştır.

Bu nedenle slayt, bir mimari evrimi gösterir. İlk aşamada telekom mantığında ağın içi zekidir, uçlar basittir. İkinci aşamada klasik internet yaklaşımıyla zeka uçlara taşınır, çekirdek sadeleşir. Üçüncü aşamada ise modern internet, bu iki uç yaklaşım arasında daha karmaşık bir dengeye gelir. Uçlarda hâlâ büyük bir zeka vardır; fakat ağın içinde de artık önemli ölçüde işlem ve politika uygulama kapasitesi bulunmaktadır.

Burada özellikle dikkat edilmesi gereken nokta, sağdaki modelin “internet ilkelerini tamamen terk ettiğini” söylememesidir. Aslında uçlarda zeka hâlâ vardır ve büyük ölçüde baskındır. Web uygulamaları, bulut servisleri, video platformları, oyun altyapıları ve uç sistem yazılımları son derece karmaşıktır. Ancak bunun yanında, performans, güvenlik, ölçeklenebilirlik ve hizmet yönetimi gereksinimleri nedeniyle ağın içine de ek zeka yerleşmiştir. Yani modern internet, klasik uçtan uca ilkenin tamamen karşıtı değil; onun pratik gereksinimler nedeniyle genişletilmiş ve kısmen esnetilmiş halidir.

Slayttaki “programlanabilir ağ cihazları” ifadesi de bu dönüşümün anahtar noktalarından biridir. Önceki genelleştirilmiş iletme ve OpenFlow slaytlarında gördüğümüz gibi, ağ cihazları artık yalnızca paket ileten pasif kutular değildir. Başlık alanlarına bakabilir, kurallara göre karar verebilir, paketi değiştirebilir, düşürebilir ya da denetleyiciye gönderebilir. Bu da ağ içindeki zekânın yeniden artmasına yol açmıştır. Benzer şekilde middlebox'lar ve NFV yaklaşımları, ağ içi işlevlerin yazılımla daha esnek hale gelmesini sağlamıştır.

Bu slayttan çıkarılması gereken ana fikir şudur: **Ağın içinde aklın nerede bulunacağı sorusunun cevabı tarih boyunca değişmiştir.** Klasik telefon ağlarında zeka çekirdekteydi. Klasik internet mimarisi zekâyı uçlara taşıdı. Modern internet ise uçlardaki büyük zekâyı korurken, ağın içine de yeniden programlanabilir ve hizmet odaklı işlevler ekledi. Bu nedenle bugünkü interneti anlamak için ne yalnızca “çekirdek zekidir” ne de yalnızca “uçlar zekidir” demek yeterlidir; doğru ifade, **zekânın artık uçlarda yoğun olmakla birlikte ağın çeşitli noktalarına da yayılmış olduğudur.**

Chapter 4: done!

- Network layer: overview
- What's inside a router
- IP: the Internet Protocol
- Generalized Forwarding, SDN
- Middleboxes



Question: how are forwarding tables (destination-based forwarding) or flow tables (generalized forwarding) computed?

Answer: by the control plane (next chapter)

Bu slayt, Bölüm 4'ün genel bir kapanış özetini veriyor ve aynı zamanda bir sonraki bölüme doğal geçiş yapıyor. Şimdiye kadar ağ katmanının **veri düzlemi** tarafını ele aldık. Yani paketin yönlendirici içinde nasıl işlendiğini, hangi çıkışa gönderildiğini, IP datagram yapısını, adreslemeyi, IPv6'yı, genelleştirilmiş iletme yaklaşımını ve middlebox kavramını gördük. Başka bir ifadeyle bu bölüm boyunca temel soru şuydu:

Bir paket yönlendiriciye geldiğinde onunla ne olur?

Slayttaki başlıklar bu akışı özetliyor. Önce ağ katmanının genel görünümü incelendi; ardından yönlendiricinin iç yapısı, giriş portları, anahtarlama yapısı, çıkış portları, tamponlama ve zamanlama gibi veri düzlemi bileşenleri ele alındı. Sonra internetin temel ağ katmanı protokolü olan IP, datagram biçimi, adresleme, NAT ve IPv6 ile birlikte işlendi. Bunun ardından klasik hedefe dayalı iletmenin ötesine geçilerek **genelleştirilmiş iletme** ve **SDN** yaklaşımı tanıtıldı. Son olarak da modern internetin önemli gerçeklerinden biri olan **middlebox** yapıları tartışıldı.

Bu kapanış slaydının en önemli kısmı ise alttaki soru-cevap bölümüdür. Burada çok kritik bir ayırım yapılıyor. Veri düzlemi bize, **hazır bir iletme tablosu** ya da **hazır bir akış tablosu** verildiğinde paketin nasıl işleneceğini anlatır. Ancak şu soru henüz cevaplanmamıştır: Bu tabloları kim hesaplıyor? Yani bir yönlendirici, hangi hedef önek için hangi çıkış portunun kullanılacağını nasıl biliyor? Ya da bir OpenFlow/SDN cihazındaki akış kuralları hangi mantıkla oluşturuluyor?

Slaytın verdiği cevap nettir: **Bu tablolar kontrol düzlemi tarafından hesaplanır.** Yani veri düzlemi paketleri işler, fakat hangi kuralların uygulanacağını belirleyen taraf kontrol

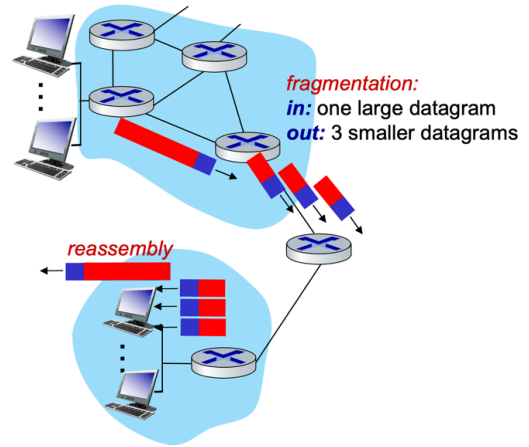
düzlemidir. Bu ayırım çok önemlidir. Veri düzlemi hızlı, yerel ve paket başına çalışan işlevleri yerine getirir. Kontrol düzlemi ise daha küresel bakışla yol hesaplama, tablo oluşturma, kural dağıtma ve genel ağ davranışını belirleme görevini üstlenir.

Bu yüzden Bölüm 4 aslında ağ katmanının yalnızca yarısını tamamlamış olur. Burada “paket geldiğinde nasıl iletilir?” sorusunu öğrendik. Bir sonraki bölümde ise “hangi kurallara göre iletileceğine kim karar verir?” sorusuna geçilecektir. İşte bu da bizi doğal olarak **Bölüm 5: Ağ Katmanı Kontrol Düzlemi** konusuna götürür.

Bu slayttan çıkarılması gereken ana mesaj şudur: **Bölüm 4, ağ katmanının veri düzlemi tarafını tamamlamıştır; ancak iletme ve akış tablolarının nasıl hesaplandığı konusu kontrol düzleminin işidir ve bir sonraki bölümde ele alınacaktır.** Yani veri düzlemi uygulayıcıdır, kontrol düzlemi ise karar vericidir.

IP fragmentation/reassembly

- network links have MTU (max. transfer size) - largest possible link-level frame
 - different link types, different MTUs
- large IP datagram divided (“fragmented”) within net
 - one datagram becomes several datagrams
 - “reassembled” only at *destination*
 - IP header bits used to identify, order related fragments



Network Layer: 4-101

Bu slayt, **IP parçalama (fragmentation)** ve **yeniden birleştirme (reassembly)** konusunun temel mantığını anlatıyor. Buradaki çıkış noktası şudur: Ağ üzerindeki her bağlantı aynı büyüklükte çerçeve taşıyamaz. Her bağlantının bir **MTU** değeri vardır. MTU, o bağlantı üzerinde taşınabilecek en büyük bağlantı katmanı çerçevesini ifade eder. Farklı bağlantı türleri farklı MTU değerlerine sahip olduğu için, ağ içinde ilerleyen bir IP datagramı bazen bir sonraki bağlantı için fazla büyük olabilir.

Böyle bir durumda büyük IP datagramı olduğu gibi geçirilemez. Bunun yerine ağ içinde **parçalanır**. Yani tek bir büyük datagram, birden fazla daha küçük IP datagramına dönüştürülür. Slayttaki şekilde de bu açıkça gösteriliyor: girişte tek bir büyük datagram

varken, çıkışta üç küçük datagram elde ediliyor. Bu parçaların her biri ayrı ayrı taşınır. Başka bir ifadeyle, ağ büyük paketi taşıyamıyorsa, onu taşıyabileceği boyutlara böler.

Burada dikkat edilmesi gereken önemli nokta, **bir datagramın ağ içinde parçalanabilmesi ama yeniden birleştirmenin yalnızca hedefte yapılmasıdır**. Yani ara yönlendiriciler parçaları tekrar bir araya getirmez. Parçalar ağ boyunca bağımsız IP datagramları gibi ilerler ve ancak son hedefe ulaştıklarında özgün büyük datagram yeniden oluşturulur. Bu tasarım, ara yönlendiricilerin iş yükünü azaltır; çünkü her ara noktada yeniden birleştirme yapıp sonra tekrar parçalama yapılması gerekmez.

Bu süreçte IP başlığındaki bazı alanlar çok kritik rol oynar. Slaytta da belirtildiği gibi, **IP başlık bitleri ilişkili parçaları tanımlamak ve sıralamak için kullanılır**. Yani hedef sistem hangi parçaların aynı özgün datagrama ait olduğunu ve bunların hangi sırayla birleşeceğini bu alanlar sayesinde anlar. Böylece farklı yollardan gelmiş olsalar bile uygun parçalar doğru sırayla birleştirilerek orijinal datagram yeniden elde edilir.

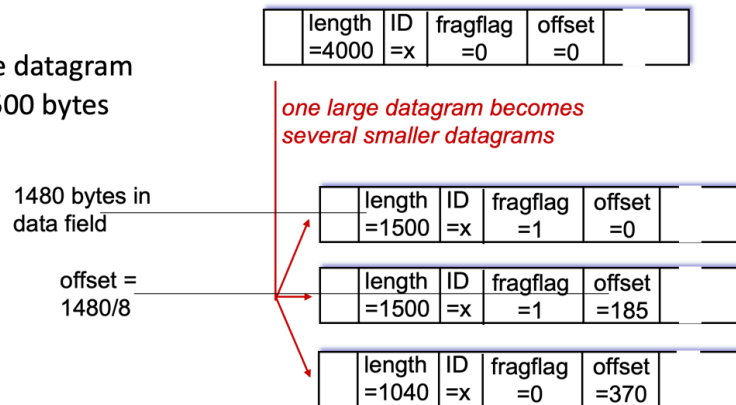
Bu konunun mantığını şöyle özetlemek faydalı olur: Ağ katmanı uçtan uca aynı büyüklükte çerçeveler sunmaz; alt taraftaki bağlantı teknolojileri farklı sınırlara sahiptir. IP de bu heterojenliği yönetmek için gerektiğinde büyük datagramları parçalar. Böylece üst katmanlar tek bir datagram üretmiş olsa bile, ağ bunu taşıyabilmek için kendi içinde birden çok küçük parçaya ayırabilir.

Bu slayttan çıkarılması gereken ana fikir şudur: **Farklı bağlantılardaki MTU sınırlamaları nedeniyle büyük bir IP datagramı ağ içinde birden fazla küçük parçaya bölünebilir; bu parçalar bağımsız olarak taşınır ve yalnızca nihai hedefte yeniden birleştirilir**. Bu, IP'nin farklı bağlantı teknolojileri arasında çalışabilmesini sağlayan önemli mekanizmalardan biridir.

IP fragmentation/reassembly

example:

- 4000 byte datagram
- MTU = 1500 bytes



Network Layer: 4-102

Bu slayt, IP parçalamanın sayısal olarak nasıl yapıldığını gösteriyor. Önceki slaytta genel mantık verilmişti; burada ise bir **4000 baytlık datagramın**, **MTU değeri 1500 bayt olan** bir bağlantı üzerinden geçerken nasıl üç parçaya ayrıldığı adım adım gösteriliyor.

İlk olarak şunu hatırlamak gerekir: IP datagramının toplam uzunluğu yalnızca veri kısmını değil, **IP başlığını da** içerir. IPv4'te temel başlık 20 bayttır. Bu nedenle MTU 1500 bayt ise, her parçanın taşıyabileceği veri miktarı en fazla **1480 bayt** olur. Çünkü $1500 - 20 = 1480$ baytlık alan veri için kalır. Slayttaki "1480 bytes in data field" notu tam olarak bunu anlatmaktadır.

Başlangıçta elimizde **length = 4000**, **ID = x**, **fragflag = 0**, **offset = 0** olan tek bir büyük datagram vardır. Bu datagram MTU'ya sığmadığı için üç parçaya ayrılır. Buradaki **ID alanının aynı kalması** çok önemlidir; çünkü hedef sistem bu parçaların aslında aynı özgün datagrama ait olduğunu bu kimlik sayesinde anlar.

Birinci parça için toplam uzunluk **1500 bayt** olur. Bunun 20 baytı başlık, 1480 baytı veridir. Bu ilk parça özgün verinin en başını taşıdığı için **offset = 0** olur. Ayrıca bunun son parça olmadığını göstermek için **fragflag = 1** olarak ayarlanır. Yani "benden sonra başka parçalar da geliyor" bilgisi verilir.

İkinci parça da yine **1500 bayt** uzunluğundadır ve yine 1480 bayt veri taşır. Ancak bu parça verinin başından değil, ilk 1480 bayttan sonraki bölümünden başlar. Bu yüzden **offset** değeri verinin başladığı konumu gösterecek şekilde ayarlanır. IPv4'te offset alanı **8 baytlık birimler** üzerinden tutulduğu için burada doğrudan 1480 yazılmaz; **1480 / 8 =**

185 yazılır. Slayttaki **offset = 185** ifadesi bundan gelir. Bu parçanın da son parça olmadığını göstermek için **fragflag = 1** kalır.

Üçüncü parça ise geri kalan kısmı taşır. İlk iki parça toplam $1480 + 1480 = 2960$ **bayt veri** taşımıştır. Başlangıçtaki toplam datagram uzunluğu 4000 bayt olduğuna göre, bunun 20 baytı başlıktır; yani özgün veri miktarı **3980 bayttır**. Geriye $3980 - 2960 = 1020$ **bayt veri** kalır. Bu son parçanın da kendi 20 baytlık başlığı olacağı için toplam uzunluğu **1040 bayt** olur. Slayttaki **length = 1040** değeri buradan gelir. Bu parçanın başlangıç konumu 2960 bayt olduğundan, offset alanı yine 8 bayt cinsinden yazılır: $2960 / 8 = 370$. Bu yüzden **offset = 370** olur. Son parça olduğu için burada **fragflag = 0** yazılır.

Bu örnekten çıkarılması gereken en önemli noktalar şunlardır. Birincisi, parçaların hepsi aynı **ID** değerini taşır; böylece hedef bunların aynı büyük datagrama ait olduğunu bilir. İkincisi, **offset** alanı her parçanın özgün veri içindeki başlangıç yerini gösterir ve bu değer 8 baytlık birimlerle ifade edilir. Üçüncüsü, **fragflag** alanı son parça olup olmadığını belirtir. Son olarak da her parça kendi IP başlığını taşıdığı için parçalama yalnızca veriyi bölmek değil, aynı zamanda her parça için yeni bir IP datagramı oluşturmaktır.

Bu slaytın ana mesajı şudur: **IP parçalama sırasında büyük datagram, MTU sınırına uyacak şekilde birden fazla bağımsız IP datagramına dönüştürülür; hedef sistem ise ID, fragflag ve offset alanlarını kullanarak bu parçaları doğru sırayla yeniden birleştirir**. Bu örnek, parçalamanın yalnızca kavramsal değil, sayısal olarak da nasıl çalıştığını açık biçimde göstermektedir.